

Computer Network Technologies and Services

IPv4 ADDRESSING AND ROUTING	1
IP-SPECIAL ADDRESSES	1
IP-ADDRESSING CLASSES	2
IP ADDRESSING: CIDR.....	2
IP ROUTING.....	3
IPv4 MULTICAST	5
IPv6	7
MOTIVATION AND HISTORY	7
IPv6 ADDRESSES	12
IPv6 ADDRESS TYPES.....	14
IPv6 MODIFIED PROTOCOLS	18
PACKET HEADER FORMAT	18
INTERFACING WITH LOWER LAYER	20
NEIGHBOR DISCOVERY AND ADDRESS RESOLUTION	21
ICMPv6	22
DEVICE CONFIGURATION IN IPv6	25
IPv4 TO IPv6 TRANSITION.....	30
DUAL STACK APPROACH	32
TUNNELING SOLUTIONS	32
SCALABLE, CARRIER-GRADE SOLUTIONS	35
WIRELESS AND CELLULAR NETWORKS	43
ELEMENTS OF A WIRELESS NETWORK	43
WIRELESS LINKS CHARACTERISTICS	44
CELLULAR NETWORK	47
CHANNEL ACCESS	47
FREQUENCY REUSE.....	48
FREQUENCY ALLOCATION	52
NETWORK ARCHITECTURE	52
CELLULAR NETWORK EVOLUTION	54
GSM: THE 2ND GENERATION	56
LTE: THE 4TH GENERATION.....	62
LTE DATA PLANE/CONTROL PLANE	67
MOBILITY	70
5G NETWORKS	75
WIRELESS LANs.....	79
PRINCIPLES OF MODERN LAN DESIGN AND OPERATION	86
HISTORY	86
REPEATER	86
MULTIPOINT REPEATERS: HUBS.....	87
BRIDGES	87
COLLISION AND BROADCAST DOMAINS	88
HALF AND FULL DUPLEX MODE	89
FULL DUPLEX AND SWITCHES.....	90

TRANSPARENT BRIDGES/SWITCHES	90
SMART FORWARDING PROCESS	91
NEW COMPONENTS IN “SMART” BRIDGES/SWITCHES	92
ROUTERS	98
ROUTING	105
INTRODUCTION AND TAXONOMY	105
ROUTING ALGORITHM CLASSIFICATION	105
INTERNET ROUTING ARCHITECTURE	111
INTRA-AS ROUTING	112
INTER-AS ROUTING	114
MULTIMEDIA NETWORKING AND QUALITY OF SERVICE.....	120
ENGINEERING QoS FOR MULTIMEDIA	120
STREAMING APPS REQUIREMENTS	120
COUNTERMEASURES IN THE NETWORK	121
CONTROL ON TRAFFIC ENTERING THE NETWORK	123
INTERNET’S AMBITIOUS SOLUTION: INTEGRATED SERVICES (INTSERV)	124
DIFFSERV TODAY	124
VIRTUAL PRIVATE NETWORK	124
MOTIVATION AND PURPOSE	125
MOTIVATIONS	125
HOW IT WORKS.....	125
VPNS IN MODERN WORK ENVIRONMENTS	126
KEY ELEMENTS	126
BASIC TERMINOLOGY AND SCENARIOS	126
VPN DEPLOYMENT SCENARIOS.....	130
INTERNET ACCESS VIA VPN	130
DEPLOYMENT MODELS, PROVISIONING MODELS & VPN PROTOCOL LAYERS.....	133
GRE – GENERIC ROUTING ENCAPSULATION	137
LAYER 2 FRAME, WITHIN AN IP PACKET	138
IP SEC VPN.....	142
THE OPTICAL INTERNET.....	143
WDM: WAVELENGTH DIVISION MULTIPLEXING.....	144
MOTIVATION FOR OPTICAL NETWORKING	144
WHY OPTICAL SWITCHING?.....	144
TYPES AND ARCHITECTURES OF OPTICAL SWITCHES.....	145
ELECTRICAL CORE	146
WAVELENGTH CONVERSION AND THE N^2 PROBLEM SWITCHING.....	146
COMMON ARCHITECTURES AND COMBINATIONS	147
DEPLYOMENT	148
CONTROL PLANE	149
ROUTING IN OPTICAL NETWORKS	149
CONTROL AND SIGNALING PROTOCOLS	149
DATA TRANSPORT AND PROTOCOL STACK	149
MPLS - MULTI-PROTOCOL LABEL SWITCHING	150
INTRODUCTION	150
MPLS HEADER	156
LSP SETUP: LABEL AND PATH SELECTION.....	156
MPLS ROUTING PROTOCOL.....	159

TRAFFIC ENGINEERING.....	162
FAULT RECOVERY.....	165
PEER VPN AND MPLS-BASED SOLUTIONS.....	166
TRANSPORT IPV6 TRAFFIC IN THE CORE OF THE (MPLS) NETWORK	171
CDN - CONTENT DELIVERY NETWORK.....	174
INTRODUCTION	174
HISTORICAL PERSPECTIVE	176
CONTENT DELIVERY NETWORKS	177
DNS-BASED CDN	180
PATTERN ANALYSIS.....	187
ANOMALY DETECTION	188
TRAFFIC MODELING	189
ARIMA – AUTOREGRESSIVE INTEGRATED MOVING AVERAGE.....	191

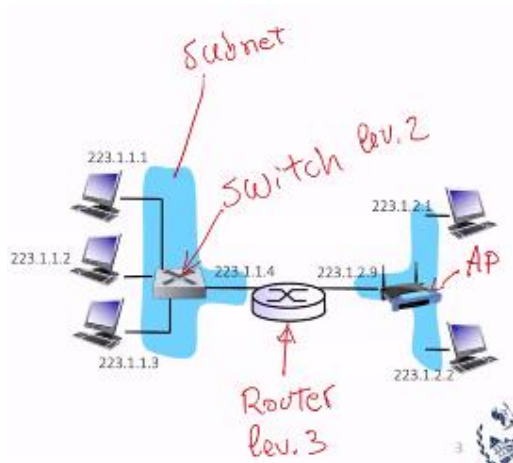
IPv4 addressing and routing

Interface: part of device which is capable to connect to the network. One or more IP addresses can be assigned to the same interface (depends from whether the network is subdivided into many subnetworks or not);

IP address: IP addresses are represented with the dotted decimal notation. The first part of the address identifies the network (network ID). The last part identifies the specific interface in that network instead (host ID). Switches and access points are layer-2 devices and do not have an IP address; → it identifies routers and hosts

A host can have multiple interfaces, but only one is active at a time. A router (layer 3 device) has multiple interfaces, and each interface has its own IP address.

Subnetwork: portion of the network characterized by interfaces which have the same high order bits (the same network ID) and share the same physical layer. The network ID has some “random” high orderbits (either 1 or 0) and only 0s in the lower part of its address.



In the middle, we have a router (a layer 3 device), and on the left, there is a switch (a layer 2 device).

The blue area represents a subnet, which is a layer 3 construct that allows grouping together interfaces that share something in common: the network ID. They are grouped together through a switch. A subnet is a logical construct.

On the right, there is a Wi-Fi access point, which acts as an aggregator. Here, we have another subnet, different from the previous one.

- **Network part:** the high-order bits of the IP address
- **Host part:** the low-order bits of the IP address

In the image, the IP addresses are represented in octet notation (dotted decimal form). 32 bit addresses divided into four part with 8 bit each.

An IP network is defined as the set of IP devices whose interfaces:

- Have the same network part of the IP address (there are some exceptions)
- Are connected to the same physical (link-layer) network

Example: 100110011 ... 011 ... 0111 → 155.73.45.7

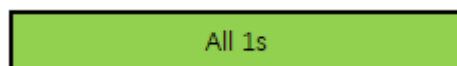
IP-special addresses

There are some special addresses that are never assigned to host/router interface:



the (sub)network ID

When all the host bits (the end part of the address) are set to 0s, the address represents the subnet itself.



limited broadcast (local net)

The address 255.255.255.255 is the limited broadcast address. If a packet has this as its destination address, it is delivered to all hosts on the same physical network.

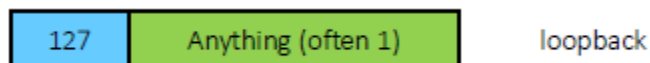
At layer 2, the hosts recognize this special address and perform a broadcast to all other hosts connected to the same router. It is called limited broadcast because it is stopped at the router—it does not go beyond that network.



directed broadcast for net

Example an host has as destination address 223.1.2.255.

The effect is as follows: the packet is sent from the host, then the switch forwards it directly to the router. The router looks up the network ID in its routing table and determines the correct interface where the packet should be sent. Within that subnet, it acts as a limited broadcast, so the packet is delivered to all hosts in that network.



Example: 127.0.0.1.

This address is never sent to a network. Instead, it is returned to the same host, where the IP layer immediately recognizes it. It is usually used to check if a local service (e.g., a web server) is working correctly.

IP-addressing classes

In the early days of the Internet, IPv4 addresses were allocated using classful addressing:

Class A → very few networks, each with millions of hosts

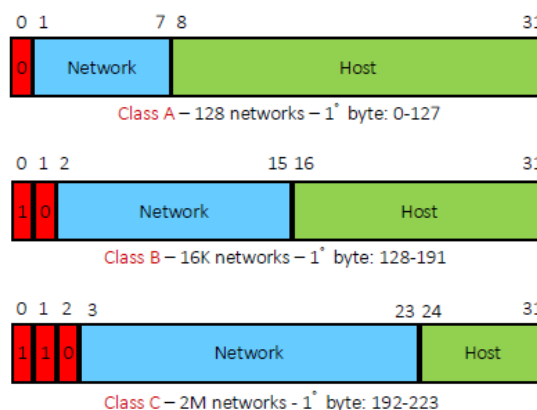
Class B → more networks, each with about 65k hosts

Class C → many networks, each with up to 254 hosts

This rigid division into fixed blocks soon became inefficient.

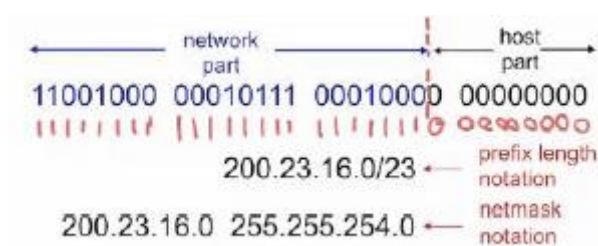
A typical organization didn't need 16 million host (Class A), nor was 254 (Class C) enough. They often requested a Class B, which gave 65k addresses, but maybe only used 5k → the rest was wasted.

This waste accelerated the depletion of IPv4 addresses. Many networks were underutilized, but the addresses were still consumed.



IP addressing: CIDR

It no longer relies just on the IP address to distinguish the network ID from the host ID. There's an extra piece of information: the netmask. All IP addresses now come with a prefix length shown within the IP addresses themselves in the format "/x" at the end of each address (where x is a decimal number).



When we want to write the netmask associated with the address shown in the figure, the procedure is as follows. We set all the bits in the network part to 1 and all the bits in the host part to 0. Then, we notice that the network part ends after 23 bits. Of course, it's not necessary to write the addresses in binary; it's enough to use dotted decimal notation.

For example:

200.23.16.0/23 → This is a valid network ID, since there are 9 zeros in the host part. Since the IP address contains more than 9 zeros, we could also define another, larger netmask that is still valid. For instance:

200.23.16.0/20 → This is also a valid network ID. However, if the address 200.23.16.0/23 has already been allocated, then 200.23.16.0/20 cannot be allocated, because this would create overlapping address spaces.

The netmask is a string of bits which has all 1s for the network ID part and all 0s for the host ID part. The netmask can be "built" with a sequence of x-ones followed by as many zeros as needed to reach the 32-bits length of the IP address. Performing a logical bitwise AND operation between the netmask and the IP address returns the network ID as result.

Example Let's consider the following IP address: 196.64.8.2/16

As we can see, we have x = 16. That means the netmask of this IP address has the first 16 bits equal to 1 and the others equal to 0. With the dotted decimal representation, we obtain: 255.255.0.0

Netmask: 11111111 11111111 00000000 00000000

Let's now convert our IP address from the dotted decimal representation to its binary representation.

IP address: 11000100 01000000 00001000 00000010

Let's perform a bitwise AND operation between the netmask and the IP address to extract the network ID. In other words, we can just copy the first 16 bits of the IP address and discard all the others (we assign to them the value 0). We obtain:

Network ID: 11000100 01000000 00000000 00000000 = 196.64.0.0

Valid netmasks

Considering all netmasks are formed by a sequence of 1s followed by a sequence of 0s, each byte in a generic netmask MUST assume a specific value within the range [0...255]. For example, let's consider the following netmask: 255.255.255.xxx; the 4th byte (xxx) of this netmask must assume one of the following values:

0	0000 0000	(256)
128	1000 0000	(128)
192	1100 0000	(64)
224	1110 0000	(32)
240	1111 0000	(16)
248	1111 1000	(8)
252	1111 1100	(4) smaller usable netmask

The last row of the previous table represents the smallest usable netmask. Assigning the value of 1 to the 2nd bit too would reduce the number of available host IDs to 2 (these host IDs would simply be, respectively, 1 and 0). That's not acceptable! Those IDs would be assigned to the Network ID 255.255.255.254 and the broadcast address 255.255.255.255, leaving us with 0 IP addresses for the hosts in our network!

254	1111 1110	(2) not valid in the 4° byte
255	1111 1111	(1) represents the single device

Valid Network ID	Invalid Network ID
▪ 130.192.1.4/30	▪ 130.192.1.1/30
▪ 130.192.1.16/30	▪ 130.192.1.4/29
▪ 130.192.1.16/29	▪ 130.192.1.24/28

IP routing

Routing table: it's the fundamental data structure used by routers to decide as quickly as possible where an incoming IP datagram must be sent (it depends from the datagram address);

Hops: number of routers encountered by a packet before reaching its final destination;

The general rules of IP routing:

- Given a destination IP address to reach, an IP device search its own routing table, looking for a match;
- In case of multiple matches, it selects the most specific one (longest prefix matching);

Let's consider an example. A router has the following routing table:

routing table	
destination	output link
200.23.16.0/20	1
200.23.18.0/23	2
199.31.0.0/16	2

An incoming datagram has the following IP address: 199.31.7.152;

The router goes through each row of the routing table (actually, it's not like that because it uses efficient algorithms to scan the table) and applies the netmask of each destination address to the incoming datagram address, looking for a

match between the destination IP address and the result of the bitwise AND operation between the netmask and the incoming datagram address. Let's do that:

destination IP	netmask length	result	match
200.23.16.0	20	199.31.0.0	X
200.23.18.0	23	199.31.6.0	X
199.31.0.0	16	199.31.0.0	✓

In other words, if the result is equal to the destination IP, a match has been found! The length of that match is equal to the length of the applied netmask. In our case we have a 16 bits long match (that means the first 16 bits of the datagram IP address are equal to the first 16 bits of the destination IP address).

Let's consider another example with the same routing table as before. This time, the incoming datagram has the following IP address: 200.23.17.53;

We obtain:

destination IP	netmask length	result	match
200.23.16.0	20	200.23.16.0	✓
200.23.18.0	23	200.23.18.0	✓
199.31.0.0	16	200.23.0.0	X

This time we have 2 matches! So, we choose the longest match (the 2nd one, because $23 > 20$). So, the datagram will be forwarded through the interface (output link) 2.

Two hosts can communicate directly (without a router) only if both consider each other inside the same subnet, based on their IP address and subnet mask.

Examples:

– **Case 1:**

- Host A: 130.192.1.1/25 (mask 255.255.255.128)
- Host B: 130.192.1.129/24 (mask 255.255.255.0)
- Host A's subnet: 130.192.1.0 – 130.192.1.127
- Host B's subnet: 130.192.1.0 – 130.192.1.255

Result:

- Host A sees Host B outside its subnet → needs a router to reach B
- Host B sees Host A inside its subnet → tries direct communication

– **Case 2:**

- Host A: 192.168.1.1/24 (mask 255.255.255.0)
- Host B: 192.168.1.129/25 (mask 255.255.255.128)
- Host A's subnet: 192.168.1.0 – 192.168.1.255
- Host B's subnet: 192.168.1.128 – 192.168.1.255

Result:

- Host A sees Host B inside its subnet → direct communication possible from A to B
- Host B sees Host A outside its subnet → will send packets to router to reach A

– **Case 3:**

- Host A: 130.192.1.1/25 (Subnet mask 255.255.255.128)
- Host B: 130.192.1.100/24 (Subnet mask 255.255.255.0)
- Host A's subnet: 130.192.1.0 – 130.192.1.127
- Host B's subnet: 130.192.1.0 – 130.192.1.255

Result: both hosts consider each other inside their subnet.

Two or more networks are aggregatable if they can be combined into a single larger network with a shorter subnet mask that includes all their addresses continuously.

1. Identify the original networks and their subnet masks.
2. Convert network addresses to binary.
3. Check if the networks are adjacent and if they share a common prefix when masked with a shorter subnet mask.
4. Determine the new subnet mask that covers both networks.

Example:

- Network 1: 192.168.1.0/24 (192.168.1.0 – 192.168.1.255)
- Network 2: 192.168.2.0/24 (192.168.2.0 – 192.168.2.255)

Are they aggregatable? Yes, because:

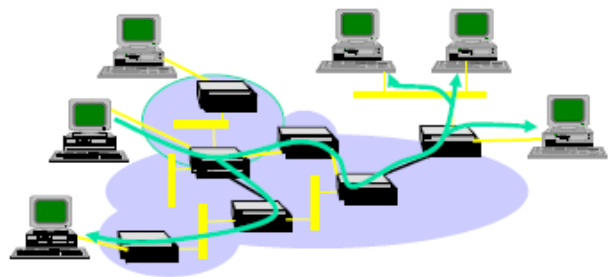
- They are contiguous (192.168.1.x ends at .255, 192.168.2.x starts at .0).
- They can be combined under the network 192.168.0.0/22, which covers the address range 192.168.0.0 to 192.168.3.255.

0.0.0.0/0 is the **default router**. A default router, also called a default gateway, is a network device (usually a router) that serves as the next hop for any IP packet whose destination address is not within the local subnet or for which the sending device does not have a more specific route.

IPv4 multicast

It's a communication model which can be used by IPv4 and IPv6. In IPv6 there's no broadcast, so broadcast communications will be handled by multicast.

Multicast is a form of communication where packets are routed from one single source to multiple destinations simultaneously. Instead of sending multiple copies of the same packet—each addressed to a different host—the source sends just one copy. This packet carries a special address known as a multicast address, which identifies a group of hosts rather than a single destination. In other words, it is not a unicast address for one host, but a collective address for many.



This mechanism is especially useful for group communications, such as videoconferencing or live video broadcasting. However, large-scale streaming services like Netflix or Amazon Prime Video do not rely on multicast; they typically use unicast connections, sending individual streams to each client. Multicast is more common in environments where the entire network is managed by a single operator or in local networks, since proper configuration of routers is essential.

The principle of multicast is straightforward: a packet is sent to a multicast address, and any host interested in receiving that traffic must explicitly subscribe to the group associated with that address. Membership is dynamic, hosts can join or leave groups at any time. Within a group, hosts can also decide whether they want to receive traffic from all senders or only from specific sources.

Routers play a crucial role in managing multicast groups. They keep track of which hosts are subscribed and which hosts are active in each of these groups and ensure that packets are forwarded appropriately. Each group has its own multicast address, but hosts still retain their individual unicast IP addresses. The multicast address simply acts as a placeholder for group membership. Delivery of packets relies on collaboration between Layer 3 (IP) and Layer 2 (typically Ethernet).

Multicast addresses belong to Class D that begin with 1110 → 224.0.0.0 - 239.255.255.255. You'll never find other IP addresses which start with 1110!

This address identifies a host group and a packet sent to such an address is delivered to all hosts in the group and anywhere in the network (across routers) even if some router, especially those that belong to operators and carriers, do not support multicast traffic.

IGMP (Internet Group Management Protocol): is a protocol used by routers to announce the existence of multicast groups on local networks. It doesn't work outside the Internet service provider domain. Routers build a distribution tree for each host group;

The IGMP protocol (incapsulated in IPv4 datagrams) handles membership; it is used by hosts and routers. IGMP does not carry the multicast data itself; instead, it manages signaling between hosts and routers.

When a host wants to join a group, it sends an IGMP Membership Report to its local router (not to other hosts). The router then knows there is at least one active member for that group, and forwards multicast packets accordingly because he knows that there is at least one member of this multicast group. It's carries inside the normal IPv4 packets with a multicast address in the destination address. It works only for local network.

When a host joins a multicast group, its network interface card (NIC) is configured accordingly to receive frames that use the corresponding multicast MAC address. This happens when the host sends an IGMP membership report: at that moment, the MAC layer of the host is also set up to accept traffic addressed to the multicast MAC address corresponding to the multicast IP group.

In multicast, it is the recipients, not the source, that determine who receives the traffic. In unicast communication, the source explicitly decides the destination; the source doesn't even know who are the possible recipients! Only recipients know they'll have to receive a packet (by checking the multicast address). With multicast, however, control is more complex because group membership is dynamic and decided by the hosts themselves.

IEEE 802: group of standards which cover local networks;

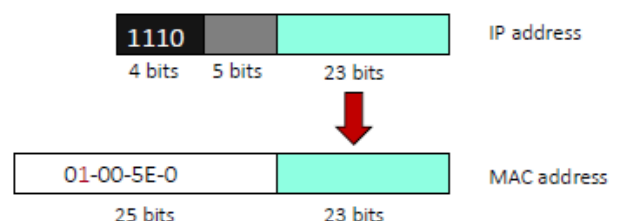
Delivery inside the local network requires mapping the multicast IP address to a reserved multicast MAC address, defined by IEEE, where multicast delivery is delegated to the lower layer (the MAC layer).

MAC layer (layer 2) multicasting is used to associate an IP multicast address to multiple MAC addresses in order to deliver a packet (with a certain destination IP multicast address) just to the legitimate recipients (whose addresses match the IP multicast address). This means that when an IP multicast packet is transmitted, the corresponding Ethernet frame must use a multicast MAC address as its destination. To achieve this, there is a mapping between IP multicast addresses and MAC multicast addresses:

- Each MAC multicast address must start with these 25 bits: 01-00-5E-0 ← 1 bit
 - o for simplicity, the first 6 digits above have been represented as hex digits (4 bits per digit) but the last digit represents a single bit (the 25th bit);
- The last 23 bits of the MAC multicast address are the least significant bits of IP multicast address;

As shown in the figure:

- An IP multicast address begins with the prefix 1110 (Class D), followed by 28 bits.
- Out of these, the last 23 bits are taken and mapped into the MAC address.
- The MAC address format is therefore: 01-00-5E + (23 least significant IP bits).



Because only 23 bits are preserved in the mapping, the 5 most significant bits of the multicast IP address are discarded. This creates the possibility of address collisions: multiple different IP multicast addresses can map to the same MAC multicast address. This collision is usually harmless: if a host receives a multicast frame for a group it did not join, the packet will be discarded at Layer 3 once the IP header is inspected.

Switches handle multicast in two main ways. Simpler switches, which do not analyze IGMP messages, treat multicast MAC addresses just like broadcast addresses. They forward multicast packets out of all ports, and it is up to the interface of the end hosts to drop traffic for groups they are not subscribed to.

More advanced switches implement IGMP snooping. These switches inspect IGMP messages, even though IGMP is encapsulated in IPv4 packets, which switches normally ignore. By “snooping” into Layer 3, the switch learns which ports have hosts that are members of a multicast group. This allows the switch to forward multicast traffic only where it is needed, instead of flooding it across the entire LAN. This is a cross-layer technique because a Layer 2 device (the switch) must interpret Layer 3 control traffic.

When a switch receives an IGMP membership report, it becomes aware that the host on that port has joined a multicast group. As a result, the switch activates filtering rules so that the multicast MAC address associated with the group is recognized and forwarded only to that host. This process requires a cross-layer approach: the switch must “snoop” into the IPv4 packet to identify IGMP control traffic and detect which multicast group is being requested.

However, IGMP operates only within a local network segment. Membership reports never cross routers; they are delivered only to the nearest router, which records the local group membership. This raises a key challenge: how can a router learn which other routers are responsible for delivering the same multicast traffic, especially when the traffic originates from a remote part of the network?

To solve this, **routers use Protocol Independent Multicast (PIM), a routing protocol designed specifically for multicast.** While IGMP manages host membership locally, PIM allows routers to coordinate globally. Routers use these protocols to build a distribution tree for each multicast group. The tree ensures that packets are delivered only to LANs with active group members, while avoiding unnecessary duplication of traffic. Routers exchange messages to build a multicast distribution tree:

- Join messages are used to add branches, allowing a router to become part of the distribution tree for a multicast group.
- Prune messages are used to cut branches when no downstream receivers remain, preventing unnecessary traffic.

In addition, routers periodically send IGMP general query messages on their local networks to verify which hosts are still active members of multicast groups. Combined with PIM, this ensures that multicast traffic is efficiently delivered only where it is needed, minimizing wasted bandwidth.

Despite its efficiency, multicast is not widely supported on the global Internet. It introduces challenges for traffic engineering, since operators lose fine-grained control over traffic flows, and it can raise security concerns. As a result, multicast is mostly confined to controlled environments, such as IPTV or video broadcasting within an ISP's network, where the operator manages both the infrastructure and the multicast configuration.

Finally, applications are responsible for letting hosts know which multicast groups to join. Typically, an app queries a server to learn about active multicast groups and then subscribes to the appropriate one on behalf of the user. This ensures that membership management remains transparent to end users.

IPv6

Motivation and History

IPv6 is the most recent version of the Internet Protocol, and today most devices are equipped with both an IPv4 and an IPv6 address.

The primary motivation for IPv6 was the need for a larger address space. IPv4's 32-bit address space was running out, and IPv6 expanded this dramatically with 128-bit addresses.

There were also other motivations, some of which were later partially addressed within IPv4 itself:

1. More efficient on LANs→ IPv6 was designed to allow more cooperation between Layer 2 and Layer 3, improving the efficiency of local area network
2. Multicast and anycast→ Native support for more advanced addressing and communication models.
3. Security → IPv6 aimed to provide built-in mechanisms for securing transmitted information, something IPv4 originally lacked.
4. Policy routing → IPv4 routing was mainly based on address aggregation to simplify routing tables, but it didn't consider the preferences of carriers. Modern providers need finer control over routing paths, not only to decide the destination but also to control which networks and regions traffic crosses. This matters for

reasons like censorship, surveillance, or geopolitical concerns. IPv6 promised to enable this type of policy-based routing.

5. Plug-and-play addressing → In early IPv4, devices had to be manually configured. IPv6 introduced automatic configuration. Over time, IPv4 adapted and also implemented DHCP and automatic configuration, but originally this was an IPv6 innovation.
6. Traffic differentiation
7. Quality of service support → IPv6 aimed at offering better quality of service support.
8. Mobility → IPv6 supported mobile and portable devices more effectively, which was not initially a consideration in IPv4.

IPv4 was defined in the 1980s, while IPv6 was conceived in the mid-1990s. However, its actual deployment was extremely slow. By the 2000s, some implementations started appearing, but widespread adoption took a very long time.

Because IPv6 adoption was so gradual, many of the features it introduced were eventually ported back into IPv4 or workarounds were created (e.g., NAT – Network Address Translation). As a result, IPv4 remained “good enough” for many years, and networks could continue operating without an urgent need for IPv6.

When IPv6 was accepted as a standard, the IPv4 user base was already massive — hundreds of millions of hosts worldwide. A complete, immediate switchover (“flag day”) was impossible. Migrating all hosts and routers simultaneously would have required enormous global coordination and risked bringing the internet to a halt.

Such a disruption would have caused catastrophic economic losses. By the early 2000s, the internet was already at the center of a booming economy, particularly during the dot-com bubble. Many companies and startups depended entirely on internet connectivity. A sudden transition could have led to enormous instability.

Therefore, the deployment of IPv6 had to be gradual, ensuring compatibility with IPv4 during the transition.

Why are IPv4 addresses scarce?

IPv4 addresses are 32 bits long, which means there are about 4.3 billion possible addresses in total. At first glance, that number looks large, but in practice it turned out to be insufficient.

Not all of those 4 billion addresses are actually available for end hosts. Addresses were divided into classes (A, B, C) for hierarchical management. While this made it easier to interpret addresses, it also led to significant waste of address space. Some ranges (e.g., those starting with 111) were reserved for multicast or other special purposes. As a result, the number of usable IPv4 addresses is closer to 3.5 billion.

IPv4 uses hierarchical allocation. Each physical network must have a unique prefix, which cannot be reused elsewhere. This system improves organization but fragments the available space.

The subnetting system works in powers of two. If a network requires, for example, 1,048 hosts, you must allocate a block that supports at least that many — the next available size might be 4,096 addresses. That means more than 3,000 addresses are wasted.

This inefficiency was unavoidable in the IPv4 design, and it led to many addresses being left unused, even though the global demand for addresses kept increasing.

The demand for IP addresses quickly outgrew IPv4’s capacity:

- The world population is about 8 billion, and at least half of them have internet access at home.
- There are over 5.5 billion mobile phones worldwide that need connectivity.
- Many individuals own multiple devices (e.g., phone, laptop, tablet, smartwatch). This pushes the demand far beyond 8 billion addresses, reaching an estimated 20 billion devices.
- On top of that, the rise of the Internet of Things (IoT) is expected to add tens of billions of additional connected devices.

IPv4 simply cannot keep up. The combination of:

- Limited usable space (due to classes, reserved ranges, hierarchical allocation), and
- Explosive growth in connected devices

led to address exhaustion. No matter how efficient subnetting becomes, the waste is inherent to the system. This is why IPv4 is fundamentally inadequate for today’s internet and why IPv6 was necessary.

Interim solutions to the saturation of address space

As IPv4 addresses began running out, several temporary solutions were introduced to extend its life. These were not permanent fixes but workarounds that allowed the internet to continue functioning until IPv6 could be deployed.

One early improvement was the introduction of netmasks, which allowed networks to be assigned more flexible sizes rather than being locked into rigid classes (A, B, C). This helped, but the waste caused by hierarchical addressing was still a problem.

One of the most effective interim solutions was the introduction of private addresses (defined in RFC 1918). These are addresses that are not routable on the public internet and are only visible within local networks. Routers on the global internet do not recognize them, which means they can be freely reused by different organizations. This was a major reason why IPv4 managed to survive for so long.

To make private addressing work in practice, it had to be combined with *Network Address Translation* (NAT). NAT is a mechanism that maps many private addresses inside a local network to one (or a few) public IPv4 addresses owned by the organization. It often uses a combination of IP addresses and port numbers to distinguish individual connections.

While NAT was extremely popular and very effective in conserving IPv4 addresses, it came with important drawbacks:

- It breaks end-to-end connectivity, since a NAT device must sit between private hosts and the public internet.
- It adds latency and processing overhead at gateways.
- It increases complexity, especially for applications that expect direct IP visibility.

In other words, NAT worked well as a stopgap, but it was never an ideal solution.

For applications that had trouble functioning behind NAT, additional mechanisms such as *Application Layer Gateways* (ALGs) were introduced. ALGs act as intermediaries that translate application-level traffic between private and public networks. There were also proposals like RSIP (Realm Specific IP), which attempted to provide alternatives to NAT, though they never became widely adopted.

Together, these measures — netmasks for more flexible subnetting, private addresses with NAT, and ALGs — allowed IPv4 to stretch far beyond its original limits. But they also highlighted the need for a more sustainable long-term solution: IPv6.

Who assigns IP addresses?

The authority for IP address assignment is IANA (Internet Assigned Numbers Authority), which allocates large blocks (/8 prefixes) to Regional Internet Registries (RIRs):

- AFRINIC – Africa
- APNIC – East Asia, Australia and Oceania
- ARIN – USA, Canada and some Caribbean islands
- LACNIC – South America, Mexico and some Caribbean islands
- RIPE NCC – Europe, Middle East and Central Asia

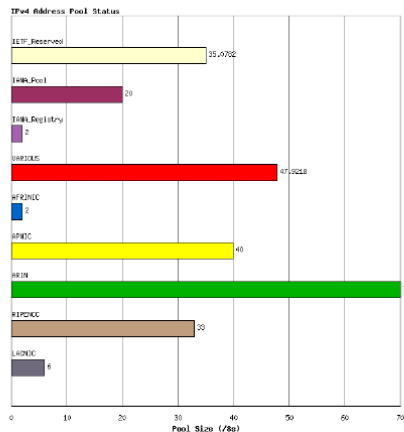


RIRs then split these into smaller blocks, which are assigned to National or Local Internet Registries (LIRs), usually ISPs or large organizations.

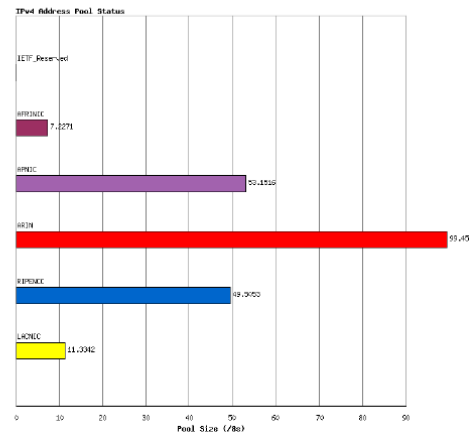
To get an allocation, organizations must justify their request — documenting their needs (e.g., carriers, multinational networks). Ordinary users cannot request directly.

IPv4 Address Pool status

2010

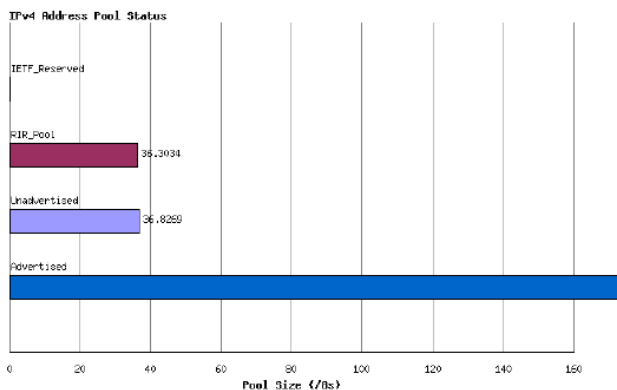


Today



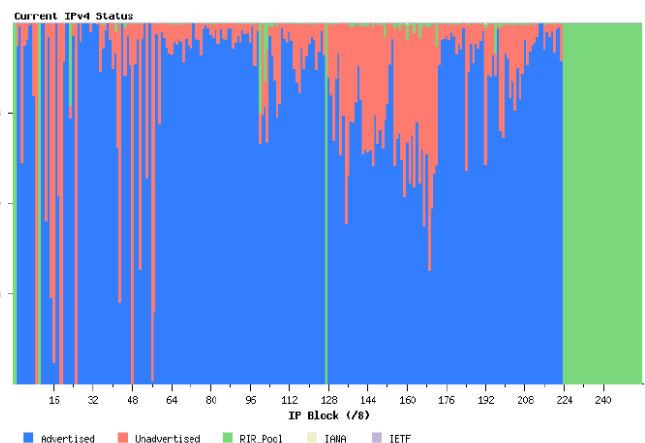
An IPv4 address can be in different states:

- Reserved for special use
- Unallocated (still in IANA pool — very few left today)
- Unassigned (held by RIRs but not distributed yet)
- Assigned but not advertised (allocated to a user, but not in BGP tables → wasted)
- Assigned and advertised (in use and visible in routing tables)



Some IPv4 address blocks remain unadvertised because they were either over-allocated in the past or left unused after old assignments. This situation contributes to overall inefficiency in address usage.

A significant portion of these addresses — particularly in ranges like 128.x.x.x to 200.x.x.x — are technically assigned but remain unused. They are mostly leftovers from earlier allocations, especially from the era of Class C addresses. At the time, many small Class C networks were distributed, later collected and aggregated, but eventually abandoned because their size was too limited to be useful. As a result, these addresses simply remain unused, sitting idle in the global pool.



Routing Scalability Issues

Another side effect of IPv4 exhaustion is the explosion in routing table size. Each subnetwork must be advertised separately in BGP. Because address assignments are fragmented and scattered across regions, prefixes that belong to the same larger block may end up in different countries.

For example:

- 150.183.0.0/16 assigned to networks in France
- 150.183.8.0/24 assigned to a network in Greece

Although both belong to the same block, routers cannot aggregate them into a single route. Instead, they must maintain two distinct entries, one for France and one for Greece.

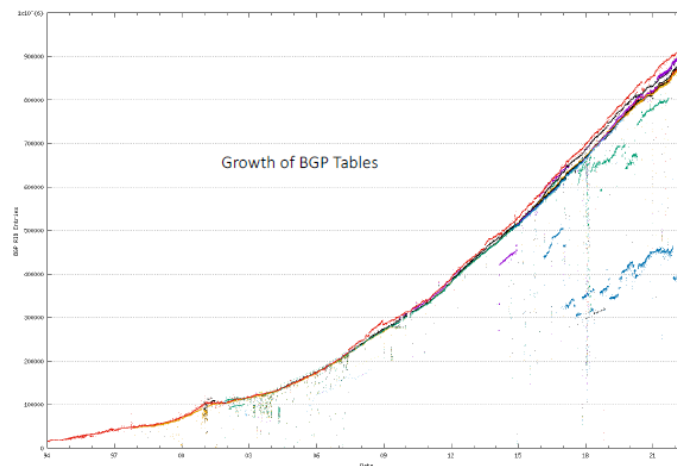
This fragmentation leads to:

- Larger routing tables (now approaching 1 million entries)
- Higher router resource consumption (memory and CPU)
- Increased lookup delays, since routers must perform a longest prefix match for each packet
- More frequent route changes, especially at the backbone level

Routers can mitigate the cost of lookups using algorithms like balanced trees or specialized data structures, but the fundamental problem remains: route aggregation is severely limited.

Aggregation is only possible when prefixes are both numerically adjacent and geographically consistent. For instance, if two contiguous /24 networks are located in the same region, they can be aggregated into a single /23 route. But if those same networks are assigned to organizations in different countries, aggregation is impossible—routers must still keep them as separate entries.

This inefficient and fragmented assignment of IPv4 prefixes is what drives the uncontrolled growth of routing tables, making Internet routing more complex, slower, and resource-intensive.



Possible solution with IPv4

One of the proposed interim solutions to address the growth of routing tables in IPv4 was route aggregation. The idea was to combine multiple contiguous networks into a single, shorter prefix. For example:

Instead of advertising 1.2.1.0/24 and 1.2.2.0/24 separately, both could be aggregated into 1.2.0.0/16.

This approach was formalized as CIDR (Classless Inter-Domain Routing), which eliminated the rigid boundaries of classful addressing (Class A, B, C). CIDR significantly improved efficiency by allowing flexible prefix lengths and route aggregation.

However, CIDR's effectiveness was limited by the historical, non-rational assignment of IPv4 prefixes. Since addresses had already been allocated in a fragmented and inconsistent way, many blocks could not be aggregated, leaving the routing scalability issue only partially mitigated.

Another measure was the introduction of *Regional Internet Registries* (RIRs). These organizations began distributing addresses in larger, contiguous blocks only to major ISPs and organizations. For example, instead of handing out small /24s, they would allocate a /20 block (≈4096 addresses) to allow better aggregation.

Even with these measures, routing scalability remained a major problem. Routing protocols could not indefinitely keep up with the exponential growth of entries. This limitation was one of the driving factors behind the design of IPv6, although the problem is not yet fully solved.

Transition to IPv6

In the early 1990s, the *Internet Engineering Task Force* (IETF) launched discussions on the “*next generation IP protocol*” (IPng). A working group was formed, and several proposals were debated. Among them was an experimental protocol called IPv5, but it was eventually discarded.

Another proposal, SIPP (Simple Internet Protocol Plus), was chosen and rebranded as IPv6. The jump from version 4 to version 6 (skipping 5) was intentional: since IPv5 was already associated with an abandoned proposal, the working group decided to avoid confusion by moving directly to version 6.

IPv6 addresses

The transition to IPv6 began at the IETF Boston Meeting in 1992, when a “Call for Proposals for IPng” (IP next generation) was issued. Dedicated working groups were appointed, and several competing proposals emerged:

- TUBA (TCP and UDP with Bigger Addresses): proposed adopting the OSI CLNP as the new IP layer.
- CATNIP: aimed at integrating multiple existing network protocols (IP, CLNP, IPX) and transport protocols (TP4, SPX, TCP, UDP).
- SIPP (Simple Internet Protocol Plus): designed as an incremental improvement over IPv4, keeping protocol changes minimal while extending the address size and eliminating unused fields.

The winning proposal was SIPP, which introduced 128-bit addresses while preserving backward compatibility concepts and minimizing disruptions to existing architecture.

How many addresses for IPv6?

The decision to adopt 128-bit addresses was not arbitrary but based on a scientific approach (RFC 1715, later refined in RFC 3194). Researchers considered the concept of addressing efficiency, measured through an H-index, which expresses how efficiently the available address space is used in practice. varies between 0.14 (pessimistic) and 0.26 (optimistic)

$$H = \frac{\log_{10}(\text{number of addresses})}{\text{number of bits}}$$

32 bits	3 E+4	2 E+8
64 bits	9 E+8	4 E+16
80 bits	1.6 E+11	2.6 E+27
128 bits	8 E+17	2 E+33

- In IPv4, the efficiency index (H) was never higher than about 0.86, and often much lower (0.75 or even 0.14 in pessimistic cases).
- When projecting this inefficiency to larger address sizes, the results were still staggering:
 - o 32 bits → insufficient (only a few billion usable addresses)
 - o 64 bits → already provided $\sim 10^{14}$ – 10^{15} usable addresses, an enormous number
 - o 80 bits or more → practically inexhaustible for foreseeable needs
 - o 128 bits → overwhelmingly abundant, but chosen as a safe margin

The final decision: adopt 128-bit addressing.

With 128-bit addresses, the space corresponds to about 6.55×10^{29} addresses per square meter of Earth’s surface. Even if we assume 1 million billion (10^{15}) networked devices, the efficiency index would still be around 0.23, meaning we can afford very sparse, hierarchical allocations and still have an effectively unlimited supply of addresses. The choice of 128 bits also avoided impractical extremes: for instance, a 1000-bit address space would have been

Number Name	Scientific Notation	Number of Zeros
1 Thousand	10^3	1,000
1 Million	10^6	1,000,000
1 Billion	10^9	1,000,000,000
1 Trillion	10^{12}	1,000,000,000,000
1 Quadrillion	10^{15}	1,000,000,000,000,000
1 Quintillion	10^{18}	1,000,000,000,000,000,000
1 Sextillion	10^{21}	1,000,000,000,000,000,000,000
1 Septillion	10^{24}	1,000,000,000,000,000,000,000,000
1 Octillion	10^{27}	1,000,000,000,000,000,000,000,000,000
1 Nonillion	10^{30}	1,000,000,000,000,000,000,000,000,000,000
1 Decillion	10^{33}	1,000,000,000,000,000,000,000,000,000,000,000
1 Undecillion	10^{36}	1,000,000,000,000,000,000,000,000,000,000,000,000
IPv4 4.29 Billion		
IPv6 340 Undecillion		340,282,366,920,938,463,463,374,607,431,768,211,456

mathematically infinite for human purposes, but packet headers would become unreasonably large.

Thus, IPv6 struck a balance: large enough to ensure scalability and inefficiency tolerance, but not so large as to make packet handling wasteful.

Notation

IPv6 addresses are 128 bits long, so their dotted decimal notation would consist in 16 decimal numbers. That's not really the best, so we use a different notation instead: 8 hexadecimal numbers separated by ":"

X:X:X:X:X:X:X

Examples:

- FEDC:0000:0876:45FA:0562:0000:3DAF:BB01
- 1080:0000:0000:0007:0200:A00C:3423:A089

To make them more compact, two shortcut rules apply:

1. Leading zeros in a field can be omitted.
 - a. 0007 → 7
 - b. 1080:0000:0000:0007... → 1080:0:0:7...
2. Consecutive groups of zeros can be replaced with ::, but this can only be done once per address to avoid ambiguity

FEDC:0000:0876:45FA:0562:0000:3DAF:BB01 →

FEDC::0876:45FA:0562::3DAF:BB01

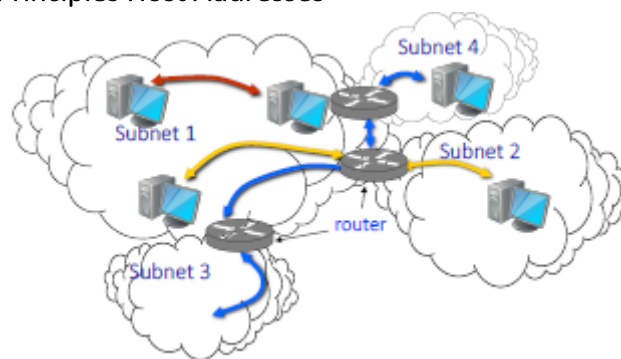
X **X**

Example:

- Initial address: 1080:0000:0000:0007:0200:A00C:3423:A089
- Applying rule 1: 1080:0000:0000:0007:0200:A00C:3423:A089 → 1080:0:0:7:200:A00C:3423:A089
- Applying rule 2: 1080:0000:0000:0007:0200:A00C:3423:A089 → 1080::0007:0200:A00C:3423:A089
- Applying both rules: 1080::7:200:A00C:3423:A089

N.B. The "::" stand for "you have to add as many 0s as it's needed to reach the total 128 bits length".

Routing and Addressing Principles Host Addresses



IPv6 maintains the same routing principles as IPv4. If the source and destination share the same prefix, delivery is direct using the local MAC address. If the prefixes differ, the packet is sent through a router.

There are some significant differences between IPv6 and IPv4 addresses structure:

- IPv6 doesn't adopt any netmask;
- IPv6 has no address classes (unlike IPv4 A/B/C/D/E).
- The address/netmask pair is substituted by a prefix notation: Address/N where N is the prefix length in bits.
- No address classes!

Example:

- Address: FEDC:0123:8700::3/48
- Prefix (also called *site prefix*): 111111011011100000000010010001110000111100000000

- [illegible]



Example

FEDC:0123:8700::100/40

The first 40 bits of this address are its prefix. The remaining 88 bits are the interface ID:

- Prefix: 1111 1110 1101 1100 0000 0001 0010 0011 1000 0111 (hex: FEDC 0123 87);
- Interface ID: 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0001 0000 0000 (hex: 00 0000 0000 0000 0000 0100);

Address assignment principles are the same as IPv4 but the terminology is different:

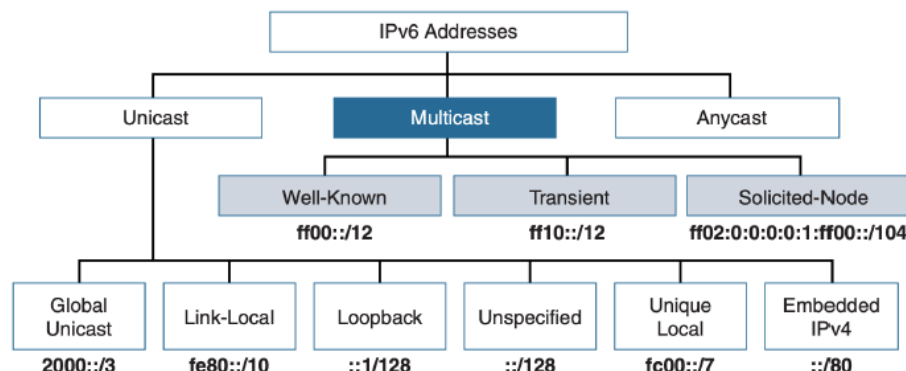
- Subnetwork: set of hosts with same prefix
- Link: physical network (in IPv6, subnet = link).
- On-link hosts have the same prefix → Direct communication
- Off-link hosts have different prefix → Communication through a router

IPv6 Address Types

IPv6 defines three main categories of addresses:

1. Unicast – one-to-one communication.
2. Multicast – one-to-many communication.
3. Anycast – one-to-nearest communication (delivered to the closest host based on routing).

Multicast



IPv6 relies much more heavily on multicast than IPv4, since it replaces some IPv4 functions such as ARP or ICMP-based discovery.

In IPv6, multicast addresses start with a sequence of eight 1s. Using IPv6 notation, we can write multicast addresses range as:

FF00::/8

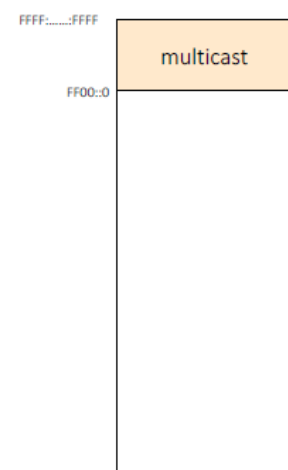
which includes all addresses whose prefix is FF

FF00:: → FFFF:FFFF:FFFF:FFFF:FFFF:FFFF:FFFF:FFFF

that corresponds to 2120 multicast addresses! (equivalent to the IPv4 multicast address 224.0.0.0/4 (1111 1111...))

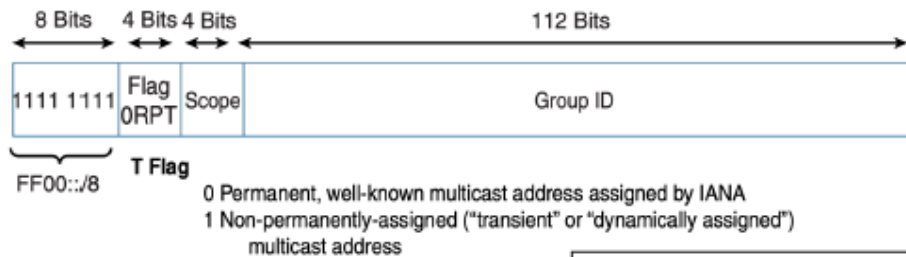
This huge range of addresses is further divided into:

- Well-known multicast addresses (**FF00::/12**) : predefined or reserved multicast addresses for assigned groups of devices (e.g., all routers).
- Transient multicast addresses (**FF10::/12**): dynamically assigned by multicast apps



- Solicited-node multicast addresses (**FF02:0:0:0:1:FF00::/104**): special range (/104) used to replace ARP in IPv6. These allow efficient discovery of neighbors without flooding the network.

Multicast addresses have a predefined structure:

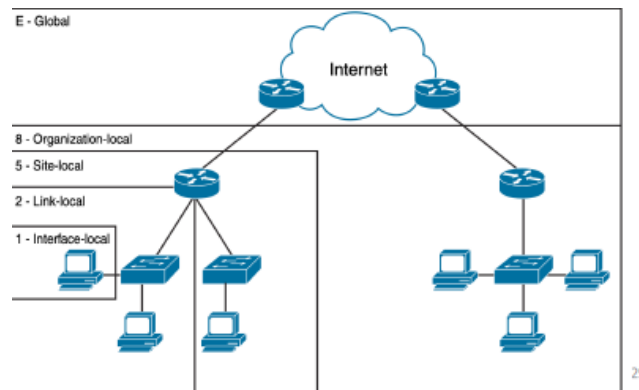


The “flag” tag can assume one of the following values:

- 0: permanent, well-known multicast address assigned by IANA;
- 1: non-permanently-assigned (“transient” or “dynamically assigned”) multicast address;

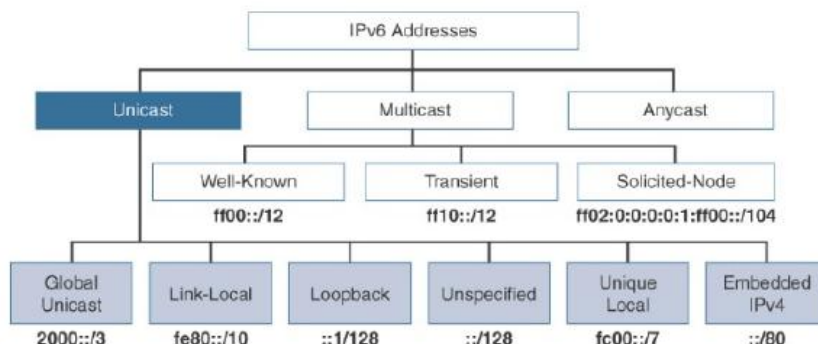
The “scope” flag can assume one of the following values:

- 1 (Interface-local): the address is reserved to interfaces;
- 2 (Link-local): the address must not be routed by the router. It will just remain inside the local network (e.g., solicited-node multicast addresses);
- 5 (Site-local): the address may be forwarded by the router to other links (subnetworks) which the router itself is connected to BUT it can’t be forwarded to other routers;
- 8 (Organization-local): the address may be forwarded to other routers belonging to the same organization (typically, by “organization” it is meant “Internet service provider”);
- E (Global): the address can be forwarded everywhere;

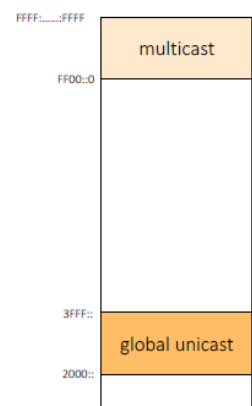


This allows fine control over multicast propagation, solving a limitation of IPv4 multicast.

Unicast

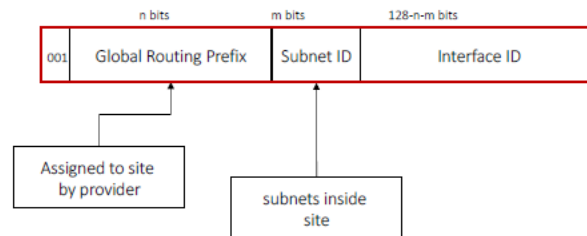


Also known as aggregatable global unicast addresses, they are the IPv6 equivalent of public IPv4 addresses. They are globally routable and reachable, meaning that packets sent to these addresses can traverse the Internet and reach their destination without any special configuration, making them essentially plug-and-play. One of the key advantages of these addresses is that their assignment is topology-based: addresses are allocated according to the structure of the network, which allows for efficient route aggregation. This means that multiple networks that are geographically or logically close can share a common prefix, reducing the size of global routing tables and simplifying routing decisions. In practice, this design makes the Internet more scalable, while still providing each host with a unique, globally reachable IP address.

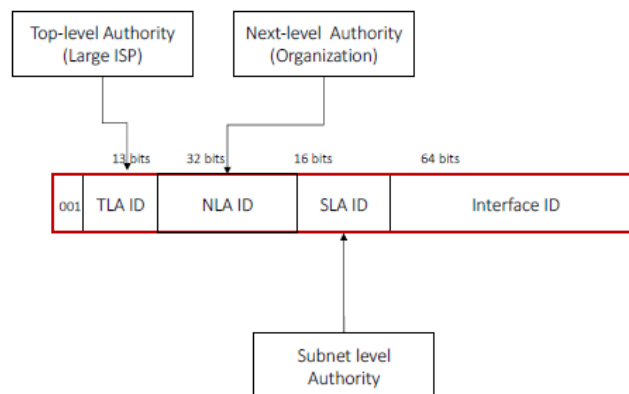


They start by **001**;

Usually, the first 48 bits (after 001) form the Global Routing Prefix. In the past, those bits were assigned depending on the Internet Service Provider. The top level authority, IANA, assigned the first 13 bits to different areas, the regional level authority assigned the next 32 bits and the subnet level authority assigned the last 16 bits. However, this guide line was soon abandoned because of its complexity. So, because we have so many addresses, a much more straightforward approach consists in assigning chunks of 48 bits.



Today's Global Routing prefix was formerly assigned by multi-level authorities:



IPv6 global unicast assignment (deprecated)

Link local addresses

Link/site local: 1111 1110 1....

Link local addresses: usually, they have all 0s from the 10th up to the 64th bit; they're addresses which must never be routed outside a local network. They're similar to private addresses but they're not the same (they also belong to different groups);

Link local is used for addresses on a single link for auto-address configuration, neighbor discovery, or when no routers are present. Packets sent to these addresses are not supposed to be forwarded across local links

Their range is **FE80::/64** (1111 1110 10....)

Site local addresses

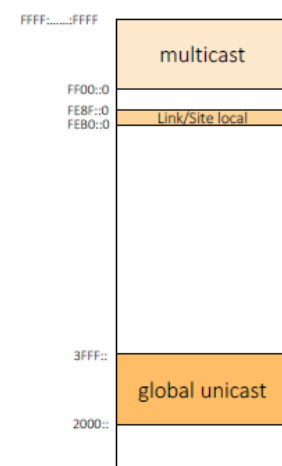
Site local addresses were meant to be confined inside a site but no one could quite agree what exactly the "site" was (a company? a branch of a company?), so they've been deprecated.

Their address range was **FEC0::/10**

Unique Local addresses

They're the equivalent of private IPv4 addresses;

Similar to global unicast addresses but just for private use, so they must be routed only in private links (e.g., inside an organization) and not be routed in the global Internet . They must be unique inside the same organization but could



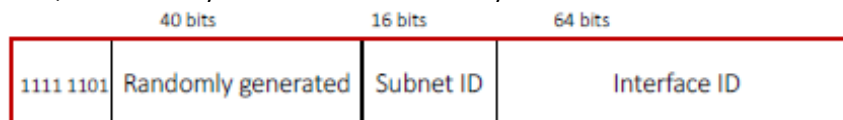
be duplicate within different companies; It is used for devices that never need access to the Internet and never need to be accessible from the Internet.

They're separate from link local addresses;

Their range is **FC00::/7**

The 8th bit is the "Local (L) Flag", dividing the range in:

- fc00::/8 (1111 1100): If L flag is 0, may be defined in the future
- fd00::/8 (1111 1101): If L flag is 1, the address is locally assigned
- fd00::/8 are the only valid ULA addresses today!

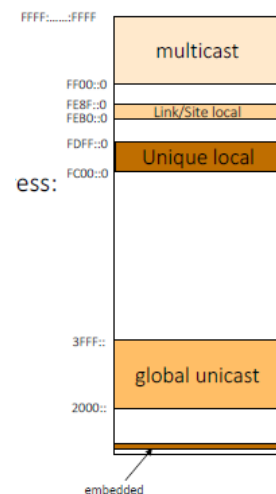
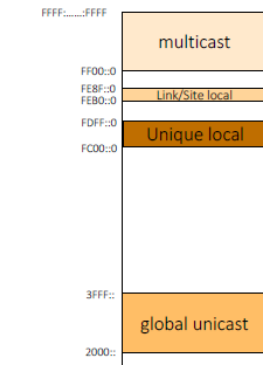
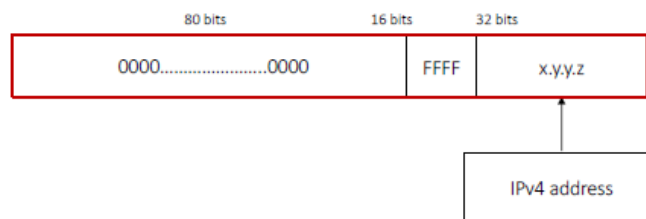


IPv4 embedded addresses

When you must route packets through a network which includes both IPv4 and IPv6, you may want to preserve your IPv4 address. To do so, IPv6 is chosen to mimic an IPv4 address. The IPv6 address includes the imprint of the IPv4 address! These addresses will never interfere with other IPv6 addresses, because they don't overlap on the Global unicast IPv6 addresses range (they don't start with 001).

They are used to represent an IPv4 address inside an IPv6 address → aid the transition from IPv4 to IPv6. The IPv4 address is carried in the low-order 32 bits

e.g., IPv6 compressed format of IPv4-mapped 20.10.4.3 → address: ::ffff:20.10.4.3



Loopback addresses

Used by a node to send an IPv6 packet to itself for testing reasons

They are identified by the address ::1 (000...1), equivalent to IPv4 address: 127.0.0.1

Same rules as IPv4:

- cannot be assigned to a physical interface.
- packets with a loopback address should never be sent beyond the device.
- routers can never forward packets with a destination address that is a loopback address.
- device must drop a packet received on an interface if the destination address is a loopback address.

Unspecified addresses

An unspecified unicast address is an all-0s address. It is used as a source address to indicate the absence of an address. It cannot be assigned to an interface. They're used in Duplicate Address Detection (DAD) in ICMPv6;

Anycast addresses

Anycast addresses are assigned to more than one interface (typically different devices). A packet sent to an anycast address is routed to the "nearest" interface having that address, according to the router's routing table

- Requires cooperation of BGP (*Border Gateway Protocol*) to configure routing tables properly.
- The user has no direct awareness: the same IP may resolve to different servers depending on location.

Initially designed for DNS but still in the experimental stages (although first RFC dates back to 1993!)

Examples:

- DNS Anycast: 8.8.8.8 (Google DNS) routes you to the closest Google server.

- Content Delivery Networks (CDNs): e.g., netflix.com or youtube.com resolve to the nearest cache server to optimize performance.

Anycast is therefore a critical enabler of scalable, low-latency internet services.

→ A border router (i.e., a route giving access to the Internet) receiving a packet with source address fd00:1::3 discards it, because it can only work on local link.

IPv6 modified protocols

IPv6 introduced several modifications to existing protocols, while others were upgraded but largely unchanged:

- IP
- ICMP: updated to ICMPv6
- ARP Integrated in ICMPv6
- IGMP Integrated in ICMPv6

Upgraded but not changed:

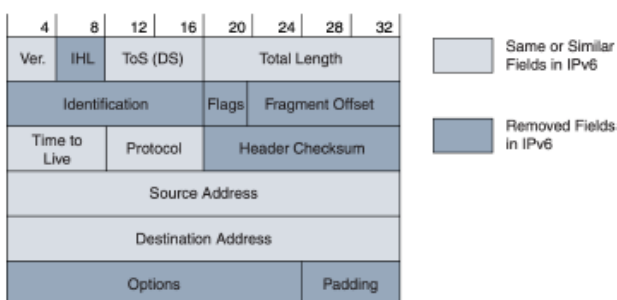
- DNS: Supports AAAA records for IPv6 addresses.
- Routing protocols: RIP, OSPF, BGP, and IDRP.
- Transport layer: TCP and UDP remain the same.
- Socket interface: No major changes; applications continue to use familiar APIs.

Packet Header Format

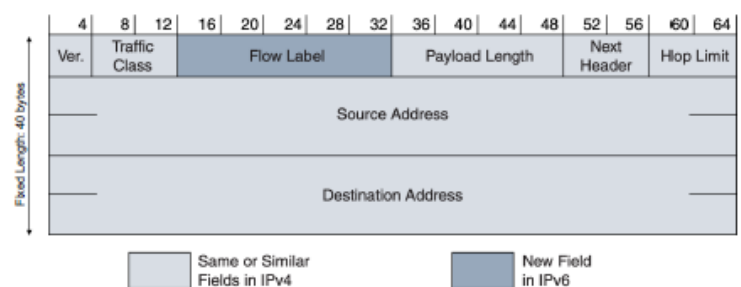
IPv6 headers are more streamlined and efficient compared to IPv4. Key points include:

1. No Fragmentation by Routers: IPv6 routers do not fragment a packet unless they are the source of the packet. If a packet exceeds the *Maximum Transmission Unit* (MTU), the router discards it and sends an ICMPv6 Packet Too Big Message back to the sender asking to send a smaller packet. Fragmentation is not handled by intermediate routers, reducing processing overhead.
2. No Header Checksum: checksums are redundant because error detection is already performed at Layer 2 and by TCP/UDP. Many IPv4 routers already ignored header checksums due to performance concerns, so IPv6 formalized this optimization. Note: UDP checksum becomes mandatory in IPv6
3. Fixed Header Length: IPv6 headers are always 40 bytes, eliminating the need for a "Header Length" field.
4. Extension Headers: IPv6 replaces the IPv4 Options and Protocol fields with Next Header fields. These allow optional headers to be chained after the main header to extend the functionality of the base header. The Next Header field specifies the type of payload (TCP, UDP, ICMPv6, or extension headers such as Routing, Fragment, Security). It specifies the protocol carried in the data portion of the IPv6 packet. Protocols are coded using same values as IPv4 (0:Hop by Hop option, 6:TCP, 17:UDP, 43:Routing extension, 44:Fragment header, 50:Encapsulating Security Payload, 51:Authentication, 60:Destination option, 59:no next header...);
5. Flow Label Field: Used for labeling packets belonging to the same flow to support traffic classification and *Quality of Service* (QoS).
6. Payload Length Field: Specifies the length of the payload (excluding the 40-byte main header).

Recall the IPv4 header



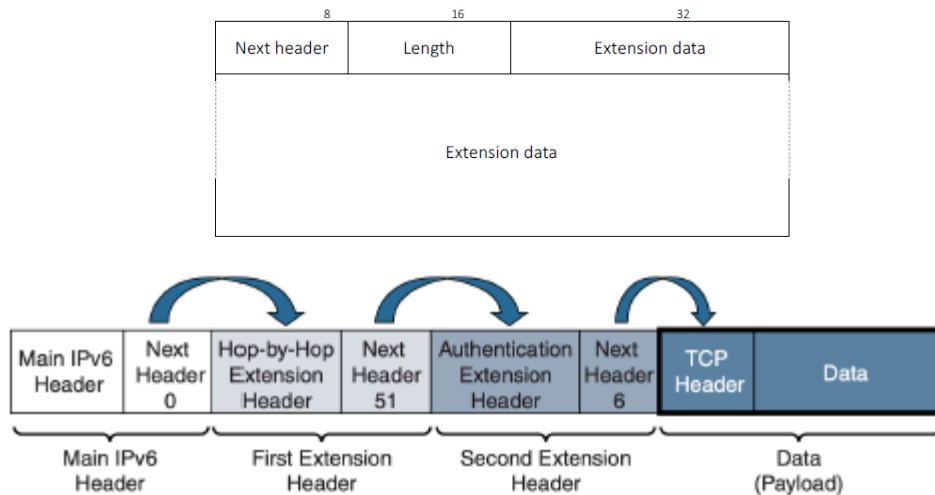
IPv6 Header



IPv4 options were often unstructured, requiring routers to parse each option individually. IPv6 streamlines this: the main header is fixed and simple, optional extension headers are only included if needed, the Next Header field points directly to the payload or the next extension header, improving packet processing efficiency.

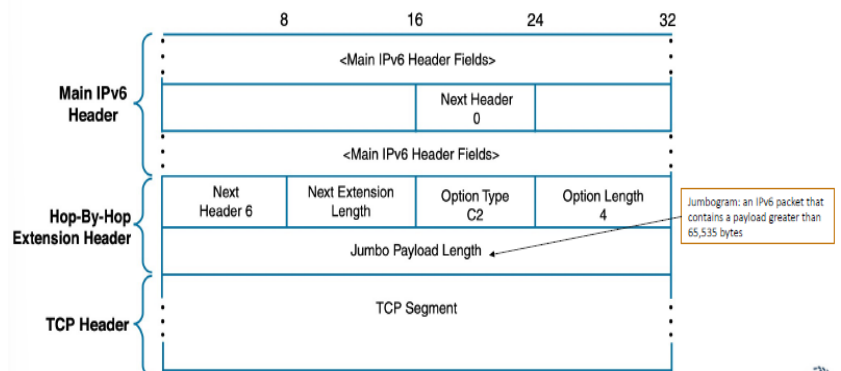
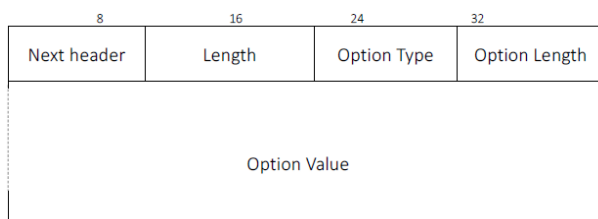
TCP and UDP payloads are handled the same way as in IPv4, but the streamlined header reduces overhead and simplifies forwarding.

Extension Header Format



Type pf extension headers:

- Hop-by-Hop Extension Header: Used to carry optional information that should be examined by every router along the path of the packet. Each option contains a set of Options Type, Options Length, and Options Value fields (TLV triplets). If present, it immediately follows the IPv6 header; for ex. Jumbo Payload, which allows packets larger than 65,535 bytes (jumbograms).



- Routing Extension Header: used to support source routing allows the source of the packet to specify the path to the destination by listing one or more intermediate routers. That means it ignores routing tables. This is useful in case crackers poison routing tables! It is used for mobility support in IPv6;
- Fragment Extension Header: used when the source needs to divide the packet into fragments, each with its own main IPv6 header and a Fragment extension header. The recipient of the packet reassembles the fragments,
Recall: unlike in IPv4, an IPv6 router does not fragment a packet unless it is the source of the packet
- Authentication and Encapsulating Security Payload Extension Headers: used by IPsec, a suite of protocols for securing delivery of packets in IP networks. *The Authentication Header (AH)* is used to guarantee the authenticity and integrity of a packet. *The Encapsulating Security Payload (ESP)* provides authentication, integrity, and encryption.

→ The use to the Next Header field in the IPv6 header allows the elimination of the IPv4 “Protocol” field.

Interfacing with Lower Layer

Encapsulation

IPv6 packets are encapsulated into Layer 2 frames, just like IPv4 packets (for Ethernet, Type: 86DD). However, IPv6 is treated as a completely new protocol at the data link layer. This is what enables the dual-stack approach, where IPv4 and IPv6 can coexist: IPv4 traffic continues to work without modification, while IPv6 can be deployed alongside it.

How does a Layer 2 frame know that it is carrying an IPv6 packet? This is specified in the frame header. For example, in Ethernet frames, there is a field called EtherType that indicates the protocol inside the payload. For IPv6, this value is 86DD, distinct from IPv4, which has its own EtherType value. This separation allows IPv6 packets to be recognized and processed independently, making dual-stack networking possible without interfering with IPv4.

How is the destination MAC address set when an IPv6 packet is encapsulated? There are two different cases:

- IP unicast address is selected using :
 - o Procedural (protocol-based) discovery, involve ICMPv6
 - o Neighbor Discovery (see later)
- IP multicast address: determined algorithmically by mapping the IPv6 multicast address to a MAC multicast address.

IPv6 Multicast transmission

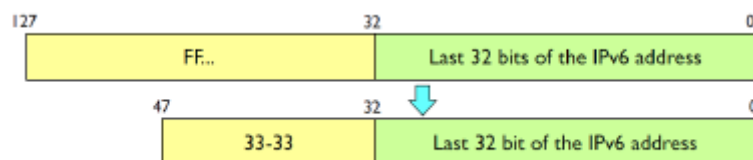
IPv6 multicast addresses allow hosts to join multicast groups and listen to specific traffic flows. Unlike unicast addresses, multicast addresses are not permanently assigned to an interface. Instead, the interface is instructed to listen for packets sent to those addresses.

To allow multicast packets to travel over Ethernet (Layer 2), each IPv6 multicast address must be mapped to a multicast MAC address → based on Ethernet multicast, but unlike an Ethernet broadcast, an Ethernet multicast can be filtered by the NIC.

This mapping ensures that Ethernet interfaces can recognize and handle IPv6 multicast traffic correctly. According to RFC 7042, the mapping between IPv6 multicast and Ethernet multicast addresses follows these rules:

- Every IPv6 multicast packet is carried using a destination MAC address reserved for IPv6 multicast.
- multicast address mapped to MAC address:

33-33-xx-xx-xx-xx
- The last 32 bits (xx-xx-xx-xx) of the IPv6 multicast address are copied directly into the last 32 bits of the MAC address.



Example: The IPv6 multicast address FF0C::89:AABB:CCDD maps to the MAC address 33-33-AA-BB-CC-DD.

Ethernet interfaces handle multicast and broadcast traffic differently:

- Broadcast addresses (FF:FF:FF:FF:FF:FF) are always received by all interfaces on the link. Every NIC (Network Interface Card) reads broadcast frames and passes them to the upper layers (IPv4/IPv6, depending on the frame type).
- Unlike an Ethernet broadcast, an Ethernet multicast can be filtered by the NIC. An Ethernet interface will only accept multicast frames if it has explicitly subscribed to the corresponding multicast group. Otherwise, the NIC will silently discard the packet without forwarding it to the IP layer.

This filtering mechanism helps to reduce unnecessary CPU load and prevents devices from processing irrelevant multicast traffic

Because only the last 32 bits of the IPv6 multicast address are used in the Layer 2 mapping, there is a small chance that two different IPv6 multicast groups could map to the same Ethernet multicast address. However, this is not a problem in practice. The IPv6 layer performs an additional check: when a packet reaches the IP layer, the full 128-bit

IPv6 multicast address is verified. If the address does not match any of the groups the host has joined, the packet is discarded. This layered verification ensures correctness even in the rare case of mapping collisions.

Neighbor Discovery and Address Resolution

New function in ICMPv6

- It substitutes ARP
- Based on multicast: Leverages the Solicited-node Multicast Address
- Given the way multicast solicited address are constructed, most likely only one station gets involved

In IPv4, address resolution between an IP address and a MAC address was handled by ARP. IPv6 removes ARP entirely. Instead, this functionality is integrated into ICMPv6 through the Neighbor Discovery Protocol (NDP).

NDP allows a node to determine the MAC address corresponding to an IPv6 unicast address. It is also used to check whether a node is reachable, to find default gateways, and to maintain neighbor caches. Crucially, NDP uses multicast-based queries instead of broadcasting like ARP.

Solicited Node Multicast Address

One of the key mechanisms that enables IPv6's *Neighbor Discovery Protocol* (NDP) to replace ARP is the Solicited-Node Multicast Address. For every unicast or anycast IPv6 address assigned to an interface, the system automatically creates a Solicited-Node Multicast Address. This special multicast address allows efficient, targeted communication between nodes on the same local link without flooding the entire network as ARP does in IPv4.

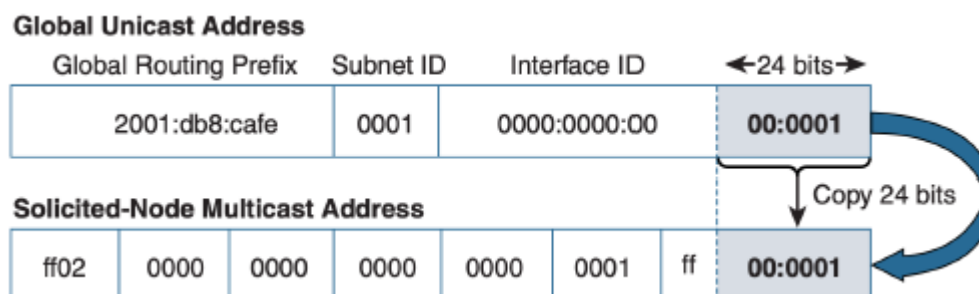
→ The Solicited Node Multicast Address in IPv6 plays a role in the resolution of an IPv6 address

The structure of the solicited-node multicast address is:

- o Prefix: **FF02::1:FF/104**
- o Suffix: last 24 bits of the unicast/anycast address

So, the resulting address always starts with FF02::1:FF followed by the last 24 bits of the host's IPv6 address.

Example: If a host has a unicast address: 2001:DB8:ACAD::1234:5678; its solicited-node multicast address is generated by copying the last 24 bits into the multicast address space: FF02::1:FF34:5678



Each interface subscribes to the solicited-node multicast groups corresponding to its own IPv6 addresses. This means that when another device on the same link wants to reach a specific IPv6 address, it doesn't have to broadcast to everyone, it just sends the query to the solicited-node multicast group that the target host belongs to.

This mechanism makes neighbor discovery: Localized (only relevant nodes receive the packet), Efficient (no need for network-wide broadcasts), Scalable (reduces unnecessary traffic).

Because the last 24 bits of IPv6 addresses rarely collide, it's very likely that each group has only one member. In the rare case where two addresses share the same 24-bit suffix, multiple hosts might join the same multicast group, but the IPv6 layer can still correctly filter packets based on the full 128-bit destination address.

Address resolution

The process for resolving an IPv6 address to a MAC address works as follows:

1. Neighbor Solicitation (NS) → Requester sends frame to Solicited Node Multicast Address of target IPv6 address

When a host (e.g., H1) needs the MAC address corresponding to another IPv6 address (H2), it sends an ICMPv6 Neighbor Solicitation message. This NS message includes the target IPv6 address in its payload (the one for which it wants the MAC mapping). The NS message is encapsulated in an IPv6 packet addressed to the Solicited-Node Multicast Address derived from the target IPv6 address.

This ensures that only the host owning that specific IPv6 address (or, at most, a few others with matching suffixes) receives the solicitation — unlike ARP, which would broadcast to every device on the subnet.

2. Neighbor Advertisement (NA) → Sent back to requester's unicast address

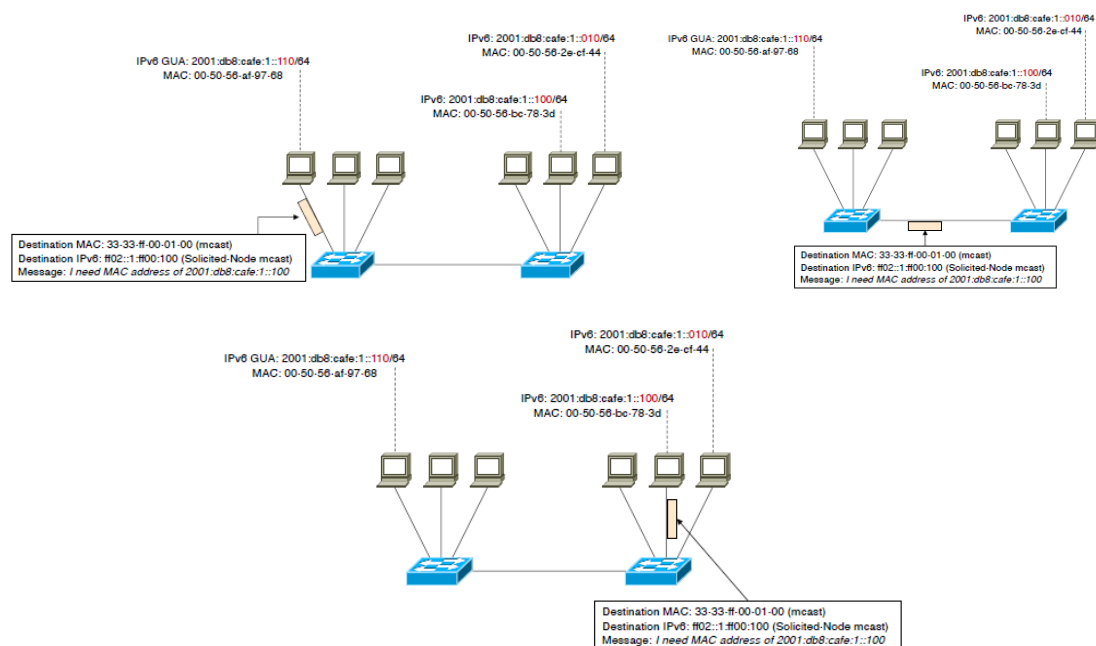
The target host (H2) replies with an ICMP Neighbor Advertisement message. This NA is sent back directly to the unicast address of the requester, not to the multicast group. Inside the NA message, the replying host includes its own MAC address (the link-layer address). This is necessary because once the packet is decapsulated, the original MAC header is discarded, so the ICMPv6 message must carry the sender's MAC address explicitly. The requester can now store the mapping (IPv6 ↔ MAC) in its local Neighbor Cache

In IPv6, the mapping between IPv6 addresses and their corresponding MAC (link-layer) addresses is stored in the Neighbor Cache, which plays the same role as the ARP cache in IPv4. This cache keeps track of all the IPv6-to-MAC address associations that a host has learned for other nodes on the same local link. Each entry is created and updated through the ICMPv6 Neighbor Solicitation and Neighbor Advertisement messages, which together perform the address resolution process. This allows for faster communication in the future, since the host doesn't need to perform another solicitation every time it needs to send a packet to that same node.

This mechanism allows a host to:

- Determine whether another node on the same link is reachable.
- Resolve IPv6 → MAC address mappings efficiently.
- Avoid flooding the link with broadcast frames.

Thus, NDP with Solicited-Node Multicast addresses makes IPv6 more scalable, efficient, and secure than IPv4's ARP-based system.



ICMPv6

What is used for?

- Diagnostics (e.g., echo request/reply = ping).
- Neighbor Discovery, replacing ARP
- Multicast group management, (replacing IGMP in IPv4).
- Issue notification (similar to ICMPv4).

Includes functions that in IPv4 were delegated to

- ARP (Address Resolution Protocol)
- IGMP (Internet Group Membership Protocol)

Message Format

ICMPv6 messages are encapsulated in IPv6 packets with:

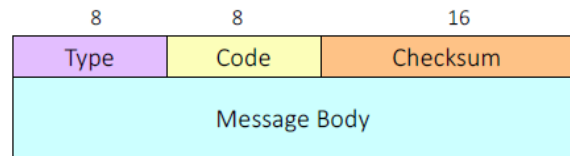
- Next Header = 58
- Maximum length = 65,495 bytes

Each ICMPv6 message has the following structure:

- Type field (1 byte): identifies the message type.
 - o Values <128 = error messages.
 - o Values ≥128 = informational messages.
- Code field (1 byte): gives additional details about the type.
- Checksum: verifies integrity.
- Message body: depends on the message type.

When an ICMPv6 error message is generated, it typically includes the IPv6 header and sometimes part of the original payload, so that the sender can identify which packet caused the error.

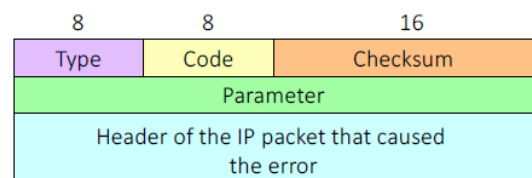
This message structure closely resembles that of ICMPv4. The Type and Code fields together uniquely identify each ICMPv6 message. When a receiver encounters inconsistencies in the IPv6 header chain (for example, an invalid Next Header field), it may send back an ICMPv6 error message indicating a Parameter Problem so that the sender can correct and retransmit the packet.



Error messages (<128)	1	Destination Unreachable
	2	Packet too big
	3	Time exceeded
	4	Parameter Problem
Informational messages (≥128)	128	Echo Request
	129	Echo Reply
	130	Multicast Listener Query
	131	Multicast Listener Report
	132	Multicast Listener Done
	133	Router Solicitation
	134	Router Advertisement
	135	Neighbor Solicitation
	136	Neighbor Advertisement
	137	Redirect

Error Messages

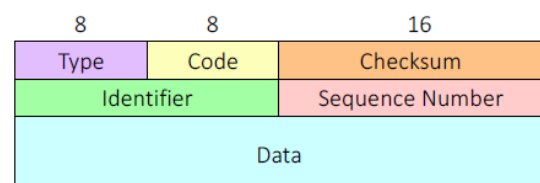
1. Destination Unreachable (type = 1)
Usually generated by router or firewall. Code field provides reason (such as no route destination, wrong scope, address/port unreachable)
2. Packet too big (type = 2)
Sent by router when a packet exceeds the MTU of the outgoing link.
Note: IPv6 routers do not fragment packets; this message informs the sender to reduce packet size.
3. Time exceeded (type = 3)
If router receives a packet with Hop Limit = 0
4. Parameter Problem (type = 4)
Generated when a device finds a problem with a field in the main IPv6 header or an extension header; example: invalid value in the Next Header field.



Informational Messages

1. Echo request (type= 128)
2. Echo reply (type= 129)

Used by the *ping* utility to test connectivity between two IPv6 nodes.

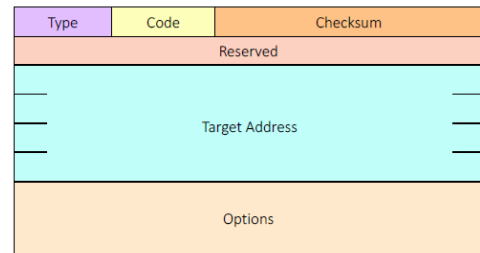


When a host receives an Echo Request, it replies with an Echo Reply, confirming that its IP interface is operational. If no reply is received within a timeout period, the destination is considered unreachable.

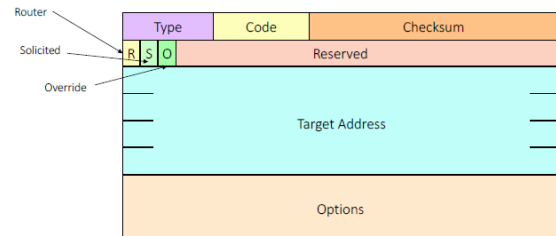
Ping only tests *network reachability*, it does not verify whether specific applications or services are running.

These ICMPv6 messages replace the role of ARP in IPv4 and are essential for discovering other nodes on the same link.

1. Neighbor Solicitation (Type = 135) → Sent by a host to determine the MAC address corresponding to a known IPv6 address. Contains a Target Address field (the IPv6 address being resolved). May include an Option with the sender's own link-layer (MAC) address.



2. Neighbor Advertisement (Type = 136) → Sent in response to a Neighbor Solicitation. Confirms the sender's link-layer address and updates neighbor caches. Includes flags and the Target Address field. Also carries an Option with the MAC address of the sender.



Multicast Group Management using ICMPv6

ICMPv6 also manages multicast groups within a local link and across routers.

- Within a link, IPv6 multicast relies on data link layer multicast (Ethernet frames starting with 33-33).
- Across links, routers use ICMPv6 messages to learn which hosts are members of each multicast group and to forward multicast traffic appropriately.

Each IPv6 multicast address includes a scope field (4 bits) that defines how far multicast packets can travel: Link-local (stay within one subnet), Site-local, Organization-local, Global

If the multicast scope is limited to the link, the traffic will not cross routers.

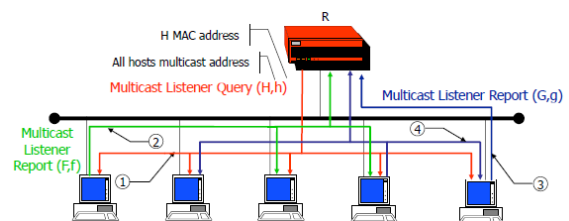
Host Membership Discovery

Within a single link, IPv6 relies on the multicasting service provided by the data link layer. This requires an appropriate mapping between IPv6 multicast addresses and MAC-layer multicast addresses, ensuring that multicast traffic is efficiently delivered to all intended receivers on the local network segment.

When communication extends beyond the local link, multicast packets must be routed across multiple links by routers. To manage this, ICMPv6 plays a crucial role in identifying which hosts are members of a given multicast group on the local link.

Hosts explicitly signal their interest in receiving multicast traffic, allowing the network to keep track of group memberships. For members located off-link, multicast routing protocols are responsible for determining where these receivers are and for forwarding multicast packets appropriately across the wider network.

Host Membership Discovery



- R to announce multicast groups G and F
- R to forward packets to G and F

1. A router sends a Multicast Listener Query to all hosts to determine which multicast groups they want to join.
2. Each host replies with a Multicast Listener Report indicating its interest in a specific group (e.g., G or F).
3. The router records these memberships and then forwards multicast packets only to the hosts that requested them.

Informational Messages: Group Management

- Multicast Listener Query (Type = 130) → Sent by routers to discover which multicast groups have active listeners. Can be a general query or group-specific.
- Multicast Listener Report (Type = 131) → Sent by hosts (listeners) to the All-Routers Multicast Address (FF02::2) to report membership in one or more multicast groups.
- Multicast Listener Done (Type = 132) → Sent by a host to the All-Routers Multicast Address when it leaves a multicast group, informing routers to stop forwarding that group's traffic to the link.

These messages correspond to IGMPv2 messages in IPv4 networks.

Type	Code	Checksum
Maximum Response Delay		Unused
Multicast Address		

Device Configuration in IPv6

What information is needed? Address prefix, Interface identifier, Default gateway, DNS server, Hostname, Domain name, MTU (Maximum Transmission Unit)

Configuration Methods:

- Manual Configuration → The user or administrator manually assigns all parameters.
- Stateful Configuration (DHCPv6) → All information, including IP address and prefix, is provided by a DHCPv6 server. It is called stateful because the server keeps track of which addresses are assigned to which hosts. Returning, clients may often receive the same address they had previously.
- Stateless Configuration (SLAAC) → The host automatically generates its own address using:
 - o A prefix advertised by a router
 - o An interface identifier, often derived from its MAC address (EUI-64 format). Useful in networks without DHCP servers, such as IoT deployments or sensor networks.
- Hybrid (Stateless DHCPv6) → The host uses SLAAC to obtain its address but queries a DHCPv6 server for additional information (e.g., DNS servers, domain name).

Interface Identifier

It can be manually configured, obtained through DHCPv6, automatically generated (from EUI-64 MAC address or privacy aware). Typically, an IPv6 address is structured as:

[64-bit prefix] + [64-bit interface identifier]

The 64-bit prefix is usually assigned by the ISP or local router, while the IID uniquely identifies the interface on that link. Automatic generation is especially useful in networks that lack centralized configuration services.

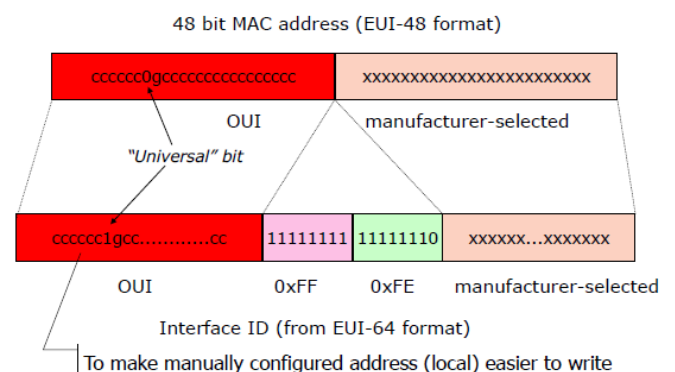


EUI-48 to EUI-64 mapping

EUI stands for *Extended Unique Identifier*.

With IPv4, IEEE defined a standard for MAC addresses:

- 48-bit hardware identifier that uniquely identifies a network interface card (NIC).
- Their 7th bit assumes different values based on the following rules:
 - o 0: the MAC address is guaranteed to be universally unique;
 - o 1: the MAC address is NOT guaranteed to be universally unique;
- The first 24 bits represent the *Organizationally Unique Identifier* (OUI), which is assigned by the IEEE to manufacturers of devices that have MAC layer communication interfaces.



- the least significant 24 bits are like a serial number assigned by the manufacturer each time he produces a new interface.

As a result, every Ethernet or Wi-Fi interface produced by a specific manufacturer has a globally unique MAC address.

To generate an Interface Identifier (IID) for IPv6 from this 48-bit MAC address, we must expand it to 64 bits, creating what's known as an EUI-64 identifier. The process is as follows:

1. Split the 48-bit MAC address into two halves (24 bits each).
2. the most significant 24 bits (1st half) of EUI-48 will become the most significant 24 bits of EUI-64 and so they'll be the OUI of EUI-64;
3. Insert the fixed 16-bit sequence 0xFFFE (11111111 11111110) after the EUI-64 OUI. This expands the address to 64 bits.
4. add the last 24 bits of the EUI-64 by copying the least significant 24 bits (2nd half) of the EUI-48;
5. set the 7th bit of EUI-64 to the opposite value of the 7th bit of EUI-48 (in EUI-64 the value 1 indicates a unique Interface ID and the value 0 indicates a non-unique Interface ID, just the opposite of what was established for EUI-48).

The resulting 64-bit EUI-64 value becomes the Interface ID portion of the IPv6 address. Combined with a 64-bit network prefix, it forms a full 128-bit IPv6 unicast address.

Since MAC addresses are designed to be globally unique, the corresponding EUI-64-derived Interface IDs are also unique, minimizing the risk of address collisions on the network.

→ The purpose of EUI-48 to Eui-64 mapping is to define an interface identifier

Privacy Concerns

By using always the same unique MAC address we could potentially be traced!

- Traceability: the least significant 64 bits of the IPv6 address of an interface never change when MAC address is used;
- RFC 4941, "Privacy Extensions for Stateless Address Autoconfiguration in IPv6";

When we enable our Wi-Fi interface, our device starts offering access to the network and simultaneously searching for available networks. That's why, when we open the Wi-Fi menu, we immediately see a list of nearby access points.

Similarly, when we walk into our homes, our smartphones automatically connect to our home Wi-Fi. This happens because the phone actively sends a probe request message, essentially saying, "Hey, I'm here! Is my network (SSID) available?" The access point that recognizes that SSID responds, allowing the device to connect.

However, this probing process introduces a privacy issue. Each probe request traditionally includes the device's MAC address, which is a unique hardware identifier. If your phone constantly broadcasts its real MAC address while moving through public spaces, it becomes possible to track your movements. Anyone with a Wi-Fi sniffer could see your MAC address and know that you were in a specific location.

To address this, modern smartphones use MAC address randomization. Instead of sending the real MAC address, the device periodically generates a temporary, random MAC address for probe messages. These randomized addresses change frequently, typically every 1 to 5 minutes, depending on the operating system, to make long-term tracking much harder.

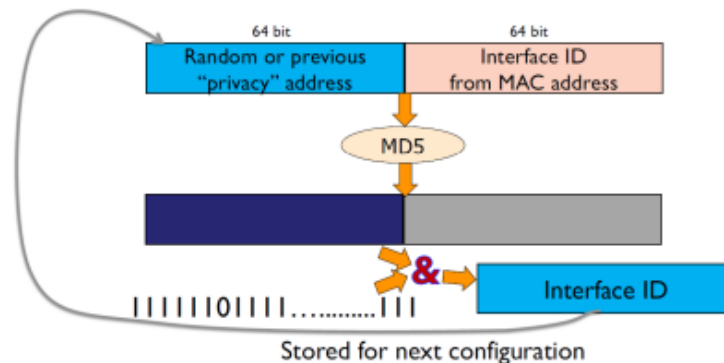
Once the phone actually connects to a network, it may revert to a consistent address for that specific SSID, but during the discovery phase, it keeps rotating temporary MAC addresses while sending probe requests.

Privacy Extension Algorithm

The process works roughly like this:

1. Concatenate the Interface ID to a 64 random bit long string;
2. Hash the result with MD5 function;
3. Perform a bitwise AND operation between the most significant 64 bits of the hashed result and the mask 0xFFFFFFFFFFFFFFF to force the 7th bit to the value 0 because MD5 can produce duplicate results (because of the collisions), so you no longer have a unique Interface ID!
4. Store the resulting Interface ID for the next configuration (it will be used in the same way we've use the 64 random bits long string at step 1);

This hashed result becomes the temporary IPv6 Interface ID, meaning the device effectively generates a new IPv6 address every time it connects or after a certain period. That grants privacy! Obviously, the first time it's compulsory to use a 64 random bit long sequence to initialize the algorithm, otherwise an attacker could trace the device just by generating all possible interface IDs!



You might ask: Why not just use a completely random sequence each time? The reason is traceability and debugging. The device keeps a small amount of internal state, so if necessary (for example, for logging or diagnostics), it can recreate the sequence of temporary addresses it used in the past, but to outsiders, these addresses appear fully random and unlinkable.

In summary, these mechanisms, MAC address randomization at the Wi-Fi layer and privacy extensions at the IPv6 layer, work together to prevent third parties from tracking users across different networks while still allowing legitimate traceability within the device.

Address usage

As soon as a host is powered on, it can automatically generate a unique IPv6 address without relying on DHCP. This process, known as Stateless Address Autoconfiguration (SLAAC), allows the host to become immediately addressable on the network.

Each network interface of the host also generates a link-local IPv6 address — a special address valid only within the local subnet (the “link”). Link-local addresses are not routable beyond the link and are used for local communication such as discovering routers and neighbors.

A host can maintain multiple IPv6 addresses simultaneously for each interface, including:

- a default address, typically derived from the network prefix advertised by a router;
- a privacy-aware (temporary) address, used to enhance anonymity when initiating or receiving connections.

Applications or even the user may select which address to use when multiple are available.

Address prefix

An IPv6 address prefix — the network portion of the address — can be:

- manually configured by an administrator;
- obtained from a DHCPv6 server;
- automatically generated, as in link-local addresses; or
- learned from a router through Router Advertisement (RA) messages.

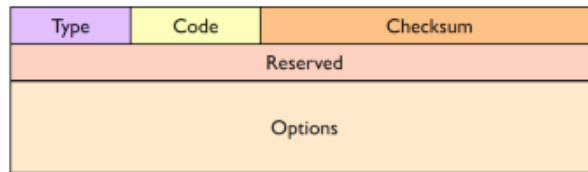
ICMPv6 Router Discovery

Routers periodically send Router Advertisement (RA) messages to inform hosts about the network configuration. These messages belong to the ICMPv6 protocol family and can be:

- Solicited, sent in response to a Router Solicitation (RS) message from a host that wants to discover routers on its link.
- Unsolicited, sent periodically by routers to update all hosts on the link.

If this is a link local address that only work on the link; they are not routed outside the subnet.

Router solicitation is an ICMP message sent by a host who wants to discover if there's a router in its network and which gateway to use in order to reach it. The Router Solicitation message is sent to the all-routers multicast address **FF02::2**.



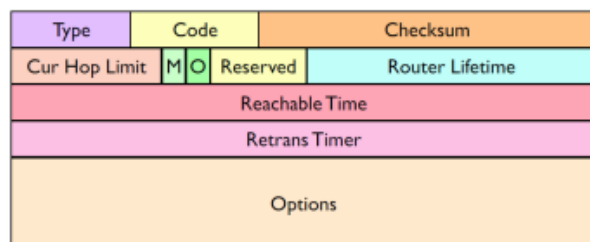
When a router receives an RS, it replies with a Router Advertisement, containing information such as:

- Router Lifetime: specifies how long will the router stay in the network;
- Reachable Time: specifies how long will the router be reachable;
- Retransmission Timer: specifies how frequently will the router send the advertisements;
- M (*Managed Address Configuration*): if set to 1: "Don't ask me! Address available through DHCP!";
- O (*Other configuration*): if set to 1, other configuration details (like DNS servers) are available through DHCPv6;
- Options: they have general format and length in multiple of 8 bytes; includes additional data such as prefixes, MTU, and link-layer addresses.

Router Advertisement Message Structure

Each RA message includes:

- Type, Code, and Checksum fields (standard ICMPv6 header);
- A series of options, each aligned to 8 bytes, that specify network parameters.



These advertisements may be:

- sent to the all-hosts multicast address FF02::1 (for periodic RAs), or
- sent unicast directly to the host that made a Router Solicitation (for solicited RAs).

Among the information carried in an RA, we can find:

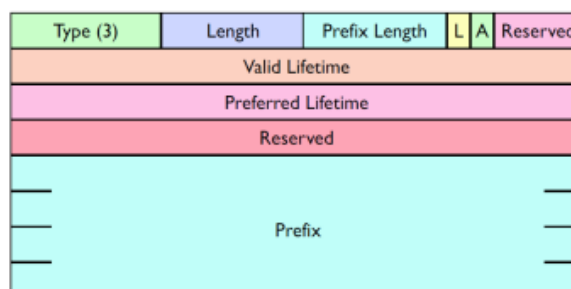
- The current hop limit that hosts should use in IPv6 packets;
- The M and O flags (indicating DHCPv6 use or additional configuration);
- The router's MAC address, stored in the Link-Layer Address option for caching purposes.

When the host receives this information, it stores the router's MAC address in its Neighbor Cache so that, in the future, it can contact the router directly (unicast) instead of broadcasting to the all-routers multicast address.

Prefix Information Option

The router specifies what's the host prefix:

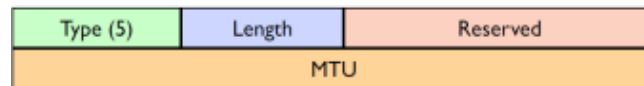
- L (on-link flag):
 - 0: the prefix is not used on this link but it's used on other links that the router has access to;
 - 1: the prefix is used on this link;



- A (Autonomous flag): 1 → prefix can be used for autonomous configuration;

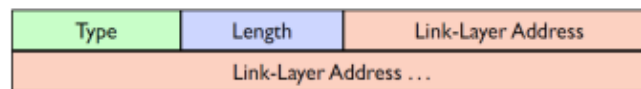
MTU option

The MTU (*Maximum Transmittable Unit*) option ensures all hosts on-link use the same MTU value.



Link Layer Address Option

Used by routers to communicate their MAC address to the hosts, so they can easily update their cache without performing the Neighbor Discovery (similar to ARP in IPv4)..

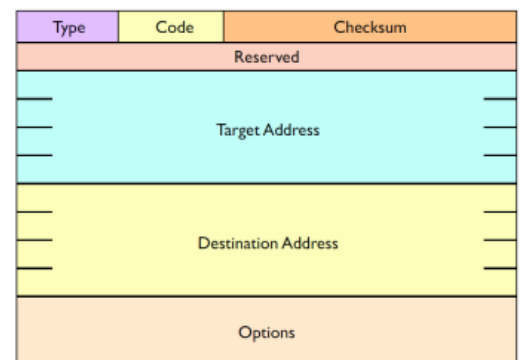


ICMP Redirect

→ Sent by a router to advise a host about a best first-hop. The first-hop is always on-link, irrespective of prefix;

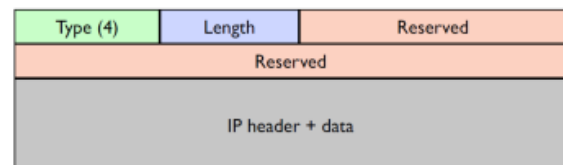
If multiple routers exist on a link, one router can send an ICMP Redirect message to inform a host that another router is a better first-hop toward a given destination. This helps optimize routing on the local network. The redirect is always on-link — it only applies within the local subnet.

The Redirect Header Option within the message provides information about the packet that triggered the redirect, including the target address (the original host), the destination router's address (the new first-hop) and the router's MAC address.



Redirect Header Option

Information about the packet being redirected.



Duplicate Address Detection (DAD)

Duplicate Address Detection (DAD) is a crucial step in IPv6 address configuration. Its main goal is to ensure that an IPv6 address is unique on a given link before a host starts using it.

Here's how it works step-by-step:

1. The host probes the uniqueness of its tentative IPv6 address.
2. It sends a *Neighbor Solicitation* (NS) message, where the target address is the IPv6 address being tested. This solicitation is sent to the corresponding Solicited-Node Multicast Address at the IPv6 layer. At the MAC layer, this maps to the corresponding multicast MAC address.
3. The host then waits for at least one second for any response:
 - If no Neighbor Advertisement is received, the address is considered unique and valid.
 - If a response is received, it means another host is already using that address, and the configuration must stop or retry with a new address.

During this process, the Neighbor Solicitation is not sent with the host's own address as the source. That's because the host isn't yet sure that its address is valid—it's testing it! If it used that same address as the source, it could create confusion or even loops on the network.

So, instead, the source address field in the IPv6 packet is set to the unspecified address (::), which means "no valid address yet". However, the MAC layer frame still carries the host's real MAC address within the Options field, so that any potential responder can know who sent the solicitation.

If another device on the network already uses that IPv6 address, it will reply with a Neighbor Advertisement, effectively signaling a conflict.

Stateless Configuration

There are two main scenarios for address configuration in IPv6: (1) Local (link-local) configuration, and (2) Configuration with a router.

1. Stateless Configuration – Link-Local Address: For link-local communication, a host follows this sequence:
 - a. Generate a link-local address, usually by combining the FE80::/10 prefix with an interface ID (derived from its MAC or generated randomly).
 - b. Perform Duplicate Address Detection (DAD) to ensure the address is unique on the link.
 - c. Subscribe to the Solicited-Node Multicast Address corresponding to that IPv6 address:
 - i. Configure the reception of the corresponding multicast MAC address.
 - ii. Send an ICMPv6 Multicast Listener Report to announce its subscription.
 - d. Once the address is verified as unique, on-link communication is enabled.

This method doesn't require any router and works only within the local link.

2. Stateless Configuration – With Router (Global Unicast Address): When a router is present, the host can also obtain a global unicast address:
 - a. Optionally send a Router Solicitation to request configuration information.
 - b. Listen for Router Advertisements, which provide the network prefix.
 - c. Generate its full address by combining the prefix with a locally generated interface ID.
 - d. Perform DAD to verify uniqueness.
 - e. Subscribe to the Solicited-Node Multicast Address, as before:
 - i. Configure multicast MAC reception.
 - ii. Send a Multicast Listener Report.

The router provides only the prefix; the host locally generates the interface ID, ensuring privacy and avoiding conflicts. If a conflict is detected, the host simply regenerates a new address and repeats the DAD process.

IPv4 to IPv6 transition

The fundamental assumption behind the IPv4-to-IPv6 transition is that both protocols will coexist for a significant period of time. The migration can't happen overnight, IPv4 is too deeply embedded into the global Internet infrastructure. Therefore, the goal is to allow both to work side by side, gradually shifting the balance toward IPv6.

The transition from IPv4 to IPv6 is one of the most complex and gradual evolutions in Internet history.

Ideally, this process should be:

- Incremental (and indeed, it still is — the transition is ongoing);
- Seamless, so that users and applications are not affected;
- Smooth, since a large number of devices still rely on IPv4.

One of the most common strategies is the dual-stack approach, where devices are configured to run both IPv4 and IPv6 simultaneously.

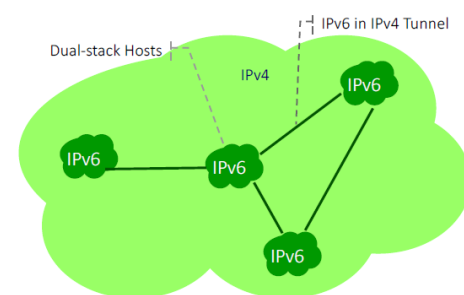
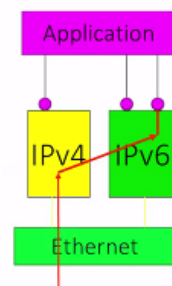
In this setup devices can communicate using either protocol depending on what's available. IPv6 is typically used when connecting to mobile networks, while Wi-Fi connections often still assign IPv4 addresses.

This allows applications to choose automatically between IPv4 and IPv6 based on connectivity and configuration, ensuring a transparent and user-friendly transition. Because IPv6 is a completely new Layer 3 protocol, coexistence requires specific techniques such as:

- Address mapping between IPv4 and IPv6;
- Tunneling, where an IPv6 packet is encapsulated inside an IPv4 packet to travel across an IPv4-only network;
- Translation mechanisms, which convert packets between IPv4 and IPv6 formats when necessary.

STEP 1: isolated IPv6 networks

At the beginning, the entire Internet was purely IPv4. Then, around 20 years ago, the first IPv6 "islands" started to appear — isolated IPv6 networks embedded within the global IPv4 infrastructure.

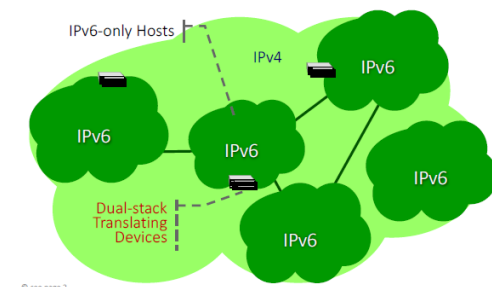


Hosts inside these IPv6 islands were typically dual-stack devices, meaning they supported both IPv4 and IPv6. These IPv6 islands were interconnected using tunnels: logical connections that allowed IPv6 packets to travel across IPv4 networks.

Essentially, an IPv6 packet would be encapsulated inside an IPv4 packet before being sent across the IPv4 Internet. When it reached another IPv6 island, the IPv4 header was removed (decapsulation), and the original IPv6 packet continued its journey natively.

However, this raised a major challenge: how to map a large pool of IPv6 addresses into a much smaller IPv4 address space. This mismatch made the process complex and inefficient, but at the time, it worked — since IPv6 deployment was still limited and scattered.

STEP 2: IPv6 islands grow



Over time, IPv6 networks began to grow and expand. These islands now included not only dual-stack hosts but also IPv6-only devices. Despite the internal use of IPv6, inter-island connectivity was still achieved through IPv4 tunnelling. To handle the translation between the two worlds, dual-stack translating devices were placed at the edges of IPv6 networks.

These devices could speak both languages, encapsulating or translating packets as needed, ensuring compatibility and reachability across the hybrid IPv4/IPv6 Internet. However, as networks

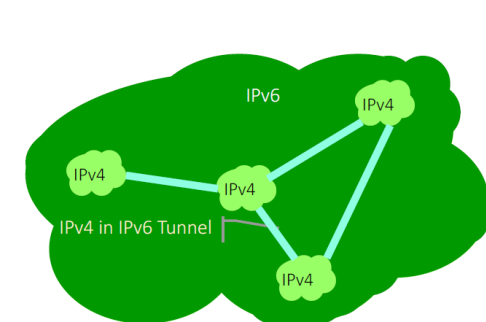
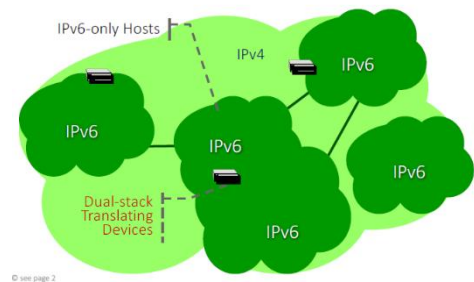
expanded, tunnelling alone became insufficient — mainly due to scalability and the complexity of address mapping between the two address spaces.

STEP 3: Native IPv6 connectivity

As IPv6 adoption grew, the isolated islands started to merge into larger native IPv6 networks.

At this point, routing and address aggregation became important issues — just like they had been for IPv4 decades earlier.

Today, we're roughly in this Step 3 stage. Most major ISPs, routers, and operating systems already support IPv6 natively. Modern routers maintain both IPv4 and IPv6 routing tables, and virtually all modern systems — Windows, Linux, macOS, Android, iOS — come with full IPv6 support out of the box.



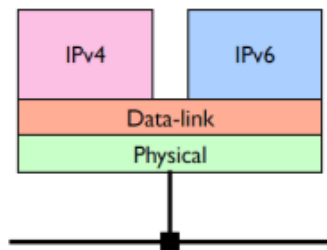
STEP 4: IPv6 takes over

Eventually, IPv6 will become the dominant protocol, and IPv4 will be the exception — a reversal of the situation in Step 1. At that point, IPv4 packets will need to be encapsulated inside IPv6 packets, rather than the other way around. This stage will actually be easier to manage, because it involves mapping a smaller IPv4 space into the massive IPv6 address pool, eliminating most of the collision and scarcity problems.

Are we ready? Quite so! IPv6 has been fully standardized since 1996, and most network devices, routers, and switches already include IPv6 implementations — though in some cases, still less optimized than IPv4.

IPv6 is fully implemented on routers, although its support is sometimes considered less mature and stable compared to IPv4. In certain cases, some functionalities may still be missing or not as fully developed. IPv6 is also available in hardware-based implementations, such as Layer-3 switches, which integrate routing capabilities directly into the switching hardware.

Only one true motivation: [address space depletion](#).



Dual Stack Approach

The Dual Stack approach is the most common and straightforward strategy to ensure coexistence. It consists of equipping every device, hosts and routers alike, with both IPv4 and IPv6 capabilities.

This means:

- Devices can communicate natively using either protocol, depending on what the destination supports.
- IPv4 and IPv6 stacks operate independently but in parallel.
- Over time, IPv4 support can be phased out as all hosts and routers become fully IPv6-capable.

To make this work, a complete duplication of the network stack is required:

- Separate routing protocols for IPv4 and IPv6;
- Separate routing tables;
- Separate access lists, which define which IP addresses or subnets are allowed to access resources — must be duplicated. That means administrators must maintain both IPv4 and IPv6 rulesets, increasing management overhead and potential for misconfigurations.
- Separate security rules (since filtering must be defined for both IPv4 and IPv6 traffic).

Dual Stack Limitations

While practical, the Dual Stack approach has several limitations:

1. It doesn't solve IPv4 address exhaustion. Each dual-stack host still requires an IPv4 address to communicate over IPv4. Therefore, the demand for IPv4 addresses remains unchanged.
2. It adds complexity. Applications must decide whether to use IPv4 or IPv6, often based on DNS responses or local configuration.
3. Not all hosts are dual stack. Some devices may be IPv6-only, especially in newer or experimental environments, while others remain IPv4-only.

Because of the mixed environment, several scenarios must be supported:

- IPv6 hosts communicating through an IPv4 network
→ Requires tunneling mechanisms to encapsulate IPv6 packets inside IPv4 packets.
- IPv4 hosts communicating through an IPv6 network
→ Requires similar tunneling or translation in the opposite direction.
- IPv6 hosts communicating with IPv4 hosts
→ Requires translation mechanisms, since the two protocols are incompatible at the IP layer.
However, mapping the massive IPv6 address space onto the limited IPv4 space is challenging — making this translation asymmetric (usually only IPv6 → IPv4 is feasible).

Moreover, since IPv4 addresses remain scarce, dual-stack systems do not reduce the pressure on IPv4 allocation. They merely delay the problem, allowing a smoother transition period where both can coexist. So, while dual-stack is necessary right now, it's not a long-term solution. It's a temporary coexistence mechanism that makes the Internet bilingual, but also more complex.

Tunneling Solutions

Dual stack and native IPv6 hosts exchange IPv6 packets through an IPv4 network.

When IPv6 hosts (or networks) need to exchange traffic through an IPv4-only infrastructure, the solution is tunneling. This technique allows IPv6 packets to travel "inside" IPv4 packets as payloads, effectively emulating a direct IPv6 connection.

IPv6 packets are encapsulated inside IPv4 packets and this involves adding an IPv4 header before an IPv6 one. The resulting IPv4 packet is then routed normally through the IPv4 Internet. At the destination, the IPv4 header is stripped off (decapsulation), revealing the original IPv6 packet. Emulates a "direct" link among IPv6 devices.

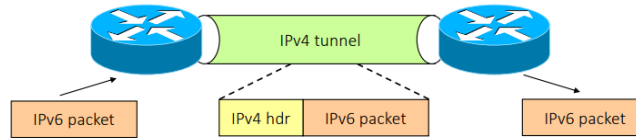
Technical Details:

- Endpoints: tunnels can be set up between hosts or routers.
- Protocols:

- GRE (Generic Routing Encapsulation)
- IPv6-in-IPv4, identified by **Protocol Type 41** in the IPv4 header.
- Setup: can be manual or automatic.

Common tunneling mechanisms include: 6over4 (RFC 2529), 6to4, ISATAP, Teredo, Tunnel Broker (RFC 3053).

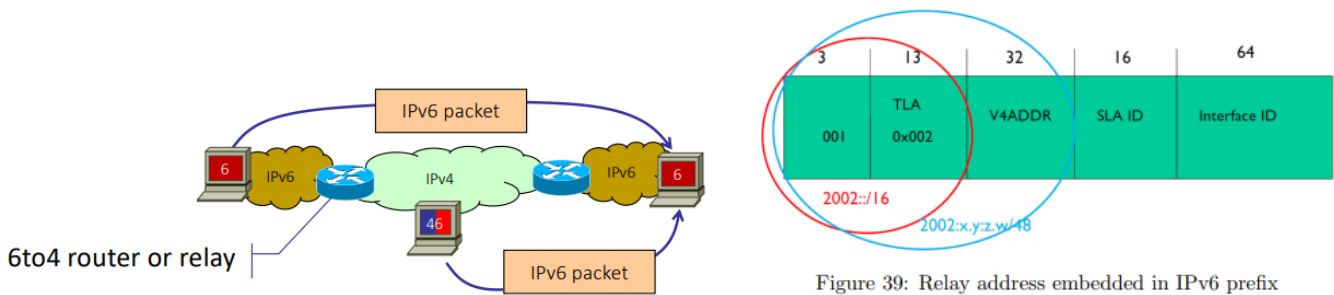
Note: Some older methods (like IPv4-compatible addresses, 6over4, and ISATAP) are now deprecated and experimental. The most widely used (until recently) has been 6to4.



Today, instead of tunneling, the transition mainly relies on a mechanism called NAT (Network Address Translation).

6to4

The idea is to write in the prefix of the IPv6 packet the address of the IPv4 router that I want to use to reach the IPv6 network. To do so, we reserve a specific 16 bits long prefix **2002::/16** to the addresses which should be supported by 6to4. Whenever a router receives an address which has that prefix, it knows how to deal with it.



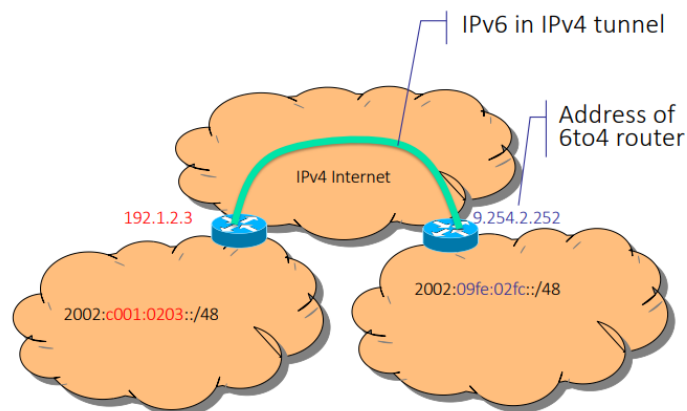
- Each 6to4 router is assigned a unique IPv6 address that begins with the prefix "2002::/16" followed by its 32-bit IPv4 address
- Packets sent by an IPv6 host are encapsulated in IPv4 by a 6to4 router and routed through IPv4 Internet to another 6to4 router
- The 6to4 router decapsulates it to reveal the original IPv6 packet and routes it to the destination

This approach allows IPv6 "islands" to communicate across an IPv4 backbone without requiring direct IPv6 routing.

Basic 6to4 Scenario

Not meant for communication between an IPv4 host and an IPv6 host

Let's consider the following scenario:



How can an host H1 belonging to the 2002:c001:0203::/48 network deliver an IPv6 packet to an host H2 belonging to the 2002:09fe:02fc::/48 network through the IPv4 Internet?

First of all, let's notice that the routers sitting at the edge of both networks are Dual-stack devices. Indeed, they have an IPv6 address for the interface facing the IPv6 network and an IPv4 address for the interface facing the IPv4 network. Having said that, let's remind that each host belonging to a native IPv6 network has assigned, from the router, an IPv6 address which contains in its prefix 32 bits which come from the IPv4 address of the interface of the router which sits at the border of both networks.

Let's assign some addresses to clarify this example:

Device	Network	IPv6	IPv4
H1	2002:c001:0203::/48	2002:c001:0203::3/48	NONE
R1	EDGE	2002:c001:0203::1/48	192.1.2.3
H2	2002:09fe:02fc::/48	2002:09fe:02fc::4/48	NONE
R2	EDGE	2002:09fe:02fc::2/48	9.254.2.252

As we've just mentioned earlier, each device in an IPv6 network has an IPv6 address that has included in its prefix the IPv4 address of the corresponding Dual-stack router:

c001:0203 = 192.1.2.3

09fe:02fc = 9.254.2.252

If H1 needs to send an IPv6 packet to H2, this is what happens:

- H1 sends the IPv6 packet to R1 with the following fields:
 - Source address = 2002:c001:0203::3/48;
 - Destination address = 2002:09fe:02fc::4/48;
- R1 skips the first 16 bits of the destination address (the prefix 2002::/16 assigned by IANA) and looks the following 32 bits up (IPv4 address of the network 9.254.2.252). It compares them to the IPv4 address of its own network (192.1.2.3) and deduces H1 and H2 belong to different networks;
- R1 checks its routing table and sees what's the next hop to reach the network 9.254.2.252;
- R1 creates a tunnel between the networks by encapsulating the IPv6 packet in the payload of an IPv4 packet with the following fields:
 - Protocol type = 41;
 - Source address = 192.1.2.3;
 - Destination address = 9.254.2.252;
 and sends the packet through the IPv4 Internet;
- R2 receives the packet, decapsulates it and sees it contains a destination IPv6 address which belongs to its own network (the address of H2, 2002:09fe:02fc::4/48);
- R2 checks its routing table and sends the IPv6 packet through the proper interface, performing a direct delivery to H2;
- H2 receives the packet;

6to4 problems

6to4 was progressively phased out starting from 2010 because it introduced complexity into the network configuration

- address conflicts and misconfigurations (large IPv6 address pool mapped onto a smaller IPv4 pool)
- not all networks had globally routable IPv4 addresses available
- NAT and firewalls interfered with 6to4 deployment
- security vulnerabilities, particularly related to routing and address spoofing

Mixed 6to4 Scenario

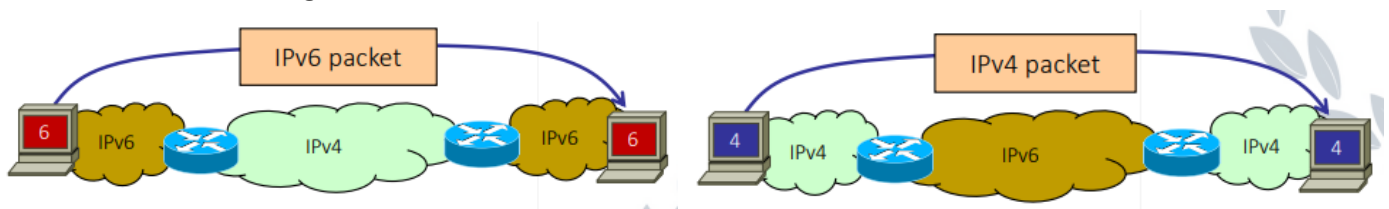
6to4 works fine with small IPv6 islands that use 2002::/16 as prefix. However, using just that prefix reduces a lot our addressing space. In a scenario where there's a bigger island which uses a different prefix (e.g. 3001::/64), 6to4 falls apart because the addresses with that prefix are not meant to be mapped to an IPv4 address. If an host tries to send an IPv6 packet to the Global IPv6 Internet through the IPv4 Internet, the border routers of both networks won't know how to address packets with prefixes different from 2002::/16 (the packet couldn't even enter the IPv4 Internet). So,

to deal with this problem, a 6to4 Relay is introduced at the edge of the target IPv6 island. The 6to4 Relay is a special router which is reachable through a predefined IPv6 anycast address. The other routers in the network will have that address, as the final entry of their routing table, configured as the default destination. So, whenever an address doesn't match any of the entries of a routing table (e.g. because it has a prefix different from 2002::/16), the router sends the corresponding packet to the default destination. Basically, the 6to4 Relay must be the default gateway of 6to4 routers.

However, this approach prevents us from using a different default destination, because we've already reserved it to the 6to4 Relay. So, in the end, 6to4 is useful when we have to deal with small IPv6 islands which can use just the addresses with the prefix 2002::/16, but it's not a great solution when we have bigger islands whose addresses have a different prefix (mixed 6to4 scenario).

As a result, modern deployments now rely more on native IPv6 connectivity and managed tunnels rather than 6to4. Today, tunneling and dual-stack environments still coexist. Most networks are in the hybrid phase, where IPv6 is increasingly dominant, but IPv4 remains necessary for backward compatibility. Over time, as IPv6 becomes universal, the need for tunneling and dual-stack mechanisms will disappear, and IPv6 will take over as the Internet's default protocol.

Scalable, Carrier-grade Solutions



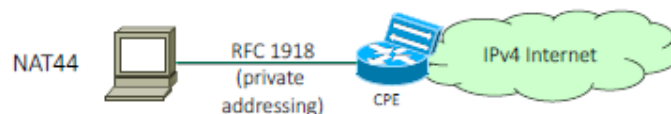
There remains a need to support various coexistence scenarios during the transition to IPv6. In particular, IPv4 servers may still need to communicate with IPv6 hosts, and many IPv4 clients continue to operate in mixed networks. To address these situations, several transition options and mechanisms are available, ensuring interoperability between the two protocol versions.

NAT

NAT44

NAT stands for Network Address Solution.

The general idea of NAT is to map an address that cannot be routed on the global Internet onto an address that can be routed globally. But what exactly is a non-routable address? These are private IP addresses, defined by RFC 1918.



According to this RFC, private IPv4 address ranges include:

- 10.0.0.0/8
- 172.16.0.0/12
- 192.168.0.0/16

These private addresses are intended for local networks, for example, within a home or corporate LAN. They are not valid on the public Internet: if a packet using one of these addresses leaves the private network, routers on the Internet will have no way to return the traffic to the sender, because these addresses are not globally unique or routable. To solve this, we use an additional component: NAT, typically implemented inside your **home router**, also called the **CPE** (*Customer Premises Equipment*).

Let's imagine your host in a private network has the address 10.0.0.7. You open a browser and request a website, for instance, politico.it. Your browser creates an IP packet with:

- Destination address: 130.192.1.1 (the public address of the web server)

- Source address: 10.0.0.7 (your private local address)

Since this is a web request, it's transported by the TCP protocol. Within the TCP header, there are two 16-bit numbers: source port and destination port. These identify the application endpoints: the source port corresponds to the port used by the client application (your browser) and the destination port corresponds to the port on which the server application (the web server) is listening. For example, source port 7600 (randomly chosen from the range 1024–65535) and destination **port 80** (the default port for HTTP web servers). Ports below 1024 are called well-known ports, they are reserved for standardized applications (e.g., port 80 for HTTP, 25 for SMTP, etc.).

Ports are essential for TCP multiplexing: while the IP address tells the packet which host it should reach, the port number tells it which application on that host should receive it.

When your packet reaches the NAT in your home router:

- The source IP address (10.0.0.7) is replaced with the public IP address assigned to your router (for example, 90.4.4.77).
- The destination IP address (130.192.1.1) remains unchanged.
- The TCP header (including source and destination ports) is preserved.

The packet now looks like this:

```
Source IP: 90.4.4.77
Destination IP: 130.192.1.1
Source port: 7600
Destination port: 80
```

This modified packet is sent to the Internet and reaches the web server. The server processes your request and sends back a response containing the requested webpage.

The reply packet has:

```
Source IP: 130.192.1.1
Destination IP: 90.4.4.77
Source port: 80
Destination port: 7600
```

At this point, the packet arrives back at your home router, the CPE.

Now, the destination IP address (90.4.4.77) belongs to your router, not directly to your computer. How can the router know which internal host should receive the response, especially if multiple devices are connected (e.g., phones, laptops, TVs)? This is where the NAT translation table comes into play. When a reply packet arrives, the router checks its NAT table using the destination port number (7600 in this example). From the table, it determines that this port corresponds to the internal host 10.0.0.7 and forwards the packet there. In this way, the NAT performs a bi-directional translation:

- Outgoing traffic: private → public
- Incoming traffic: public → private

This mechanism ensures that even though multiple hosts share a single public IP address, each session remains uniquely identifiable by its port mapping.

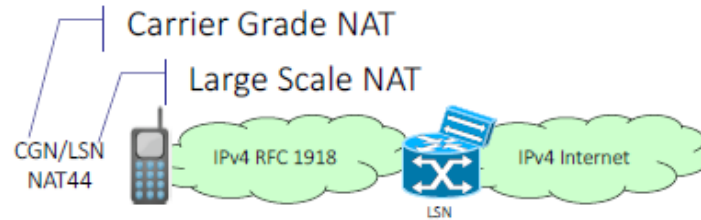
The primary reason for using NAT is the shortage of IPv4 addresses. By allowing multiple private hosts to share one public IP address, NAT makes it possible for millions of users to connect to the Internet without requiring globally unique addresses for every device. The process just described is called NAT44, because it translates IPv4 addresses into IPv4 addresses, both on the private and public sides of the network.

Large-Scale NAT

In the second example, we look at what is commonly called *Large-Scale NAT* (LSN) or *Carrier-Grade NAT* (CGN). This solution is typically adopted by mobile network operators.

A mobile operator has two main options:

1. Use IPv6 directly – If the operator’s infrastructure is already IPv6-native, there is no issue. IPv6 provides an enormous address space, so every device can easily receive a unique address.
2. Continue using IPv4 – If the operator has not yet transitioned to IPv6 and still operates on IPv4, it faces the problem of address scarcity. The only viable way to handle millions of users is by using private IPv4 addressing internally and relying on NAT.



In this setup, the NAT function resides on the operator’s edge router, positioned at the interface between the operator’s private network and the global Internet. On this router, the NAT module translates private IPv4 addresses into public IPv4 addresses and vice versa.

Since a mobile operator serves millions of customers, it possesses multiple public IPv4 addresses — not just one. That’s why the solution is called large-scale NAT: it must manage mappings for millions of private hosts concurrently, as opposed to a typical home NAT that might only handle a handful of devices.

Each public IPv4 address can serve approximately 65,000 private connections (one per TCP/UDP port). So, for example, to support 10 million users, the operator would need roughly a thousand public IPv4 addresses. This large address-pool and mapping infrastructure justifies the term Carrier-Grade NAT (CGN).

Double NAT (NAT444)



A third scenario occurs when customers, typically those of fixed network operators rather than mobile ones, are not assigned a public IP address at all. Instead, they receive only a private IP address from their Internet Service Provider (ISP). In this case, there are two layers of NAT:

1. A Carrier-Grade NAT at the provider’s network edge (between the operator’s private network and the public Internet).
2. A Home NAT within the customer’s own router (the CPE).

Here’s how it works:

- The provider network itself operates entirely on private IP addressing.
- Each customer’s home router is assigned a private IP address within the provider’s private network, not a public one.
- Within the home network, the router further assigns private addresses (RFC1918) to its internal devices.

So, traffic from a home device goes through two translations:

Private (home) → Private (provider) → Public (Internet)

This design is called NAT444, meaning “NAT IPv4 → NAT IPv4 → NAT IPv4”.

Why does this approach exist? Because the provider cannot easily manage the routing of individual private addresses for every host inside each customer’s LAN. Assigning one private address per home router is simpler than assigning one per device.

Although NAT444 is technically functional, it’s rarely used today, since most customers want to host services (e.g., home automation, game servers, cameras) that require direct inbound connections. NAT (Network Address Translation) has been super useful for extending the life of IPv4, but it also introduces several limitations and headaches, especially when it comes to connectivity and scalability.

NAT444 and CGN introduces complexity and limitations:

- It breaks the end-to-end connectivity principle.
- Hosting servers at home becomes problematic because inbound connections cannot easily reach devices behind multiple NAT layers.
- Port forwarding becomes either impossible or extremely difficult because it needs a public address, but with NAT444 the router does not have a public IP address anymore.

Problems with NAT:

- Inbound sessions are problematic → NAT works great for outgoing connections (like when a client initiates traffic), but it struggles with incoming sessions. For example, if a device behind a NAT wants to act as a server, external clients can't easily reach it because NAT hides the private IP behind a public one.
- Peer-to-peer applications need workarounds → NAT breaks direct device-to-device communication. That's why protocols like STUN (Session Traversal Utilities for NAT) and TURN (Traversal Using Relays around NAT) exist — they help establish peer-to-peer sessions by discovering or bypassing NAT restrictions, but they are complex and computationally expensive.
- It becomes a bottleneck and a single point of failure → Since all traffic passes through the NAT device, it can limit throughput and create dependency; if the NAT fails, all traffic behind it fails too.

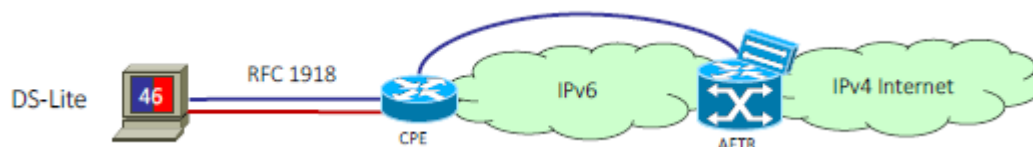
Despite all these issues, NAT is still extremely common today. Multiple layers of NAT (cascaded NATs) are now the norm, for example, a home router performs NAT for your devices, your ISP might also perform Carrier-Grade NAT (CGN) for many customers and even virtual machines or containers often have their own internal NAT layers. So packets might go through NAT inside NAT inside NAT.

We keep using NAT because IPv4 addresses are scarce. There simply aren't enough public IPv4 addresses to give one to every device, so NAT remains a practical necessity — even though it complicates networking.

Now, let's apply these NAT concepts to the IPv4-to-IPv6 transition, focusing on cases where we may have both IPv4 and IPv6 hosts.

All three solutions address the same situation, where the operator's internal network runs on IPv6, while the global Internet still primarily uses IPv4. The key difference between these approaches lies in where the NAT function is placed within the network architecture.

1. DS-Lite (Dual-Stack Lite)



In the DS-Lite solution, the NAT function is located on a special router at the boundary between the IPv6 and IPv4 networks. This router is called the AFTR (*Address Family Transition Router*). This router allows IPv4 host to communicate with IPv4 hosts over an IPv6 carrier infrastructure.

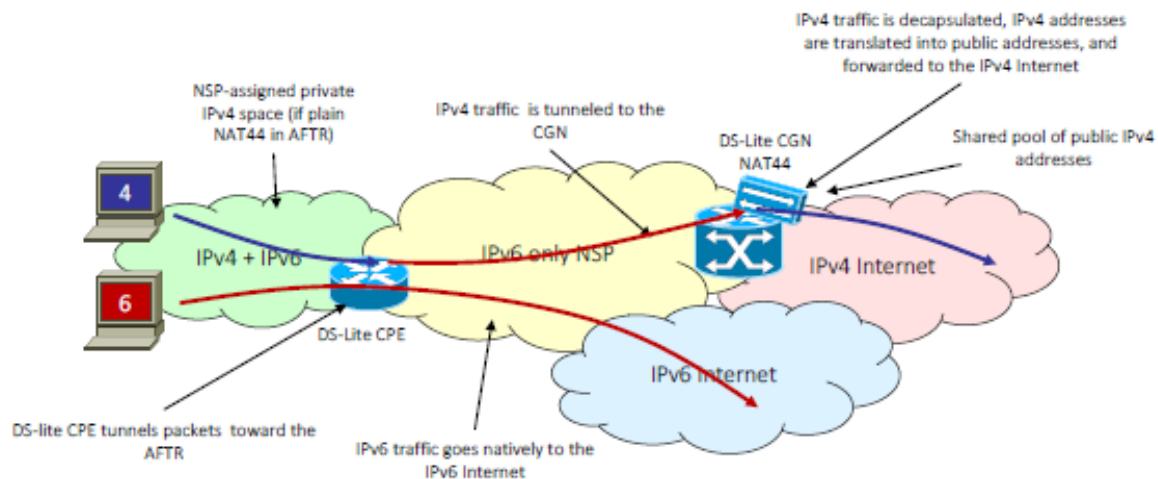
At the network edge, a dual-stack approach is typically implemented, allowing devices to communicate using both IPv4 and IPv6 protocols seamlessly. This setup may also include a large-scale Network Address Translation (NAT) system to manage the limited availability of IPv4 addresses. Meanwhile, the core service provider backbone increasingly operates as an IPv6-only environment, reflecting the shift toward native IPv6 deployment for improved scalability and performance.

The AFTR acts as a gateway that accepts IPv6 traffic from customers, decapsulates and translates it into IPv4 traffic, and forwards it to the public IPv4 Internet. Conversely, when IPv4 traffic comes back from the Internet, the AFTR encapsulates it into IPv6 to send it across the operator's IPv6 network. So, in essence:

IPv4 host → (encapsulated into IPv6) → IPv6 network → AFTR → IPv4 Internet

This process enables IPv4-only hosts within the customer's network to communicate with IPv4 servers over an IPv6-only service provider infrastructure. This setup also includes tunnelling, but it's the opposite of older 6to4 tunnels.

Previously, 6to4 encapsulated IPv6 packets inside IPv4 packets; DS-Lite instead encapsulates IPv4 packets inside IPv6 packets.



In this first solution, we deal with a situation where the customer network (not necessarily a small home network—it could be any customer network) contains both IPv4-only and IPv6-only hosts. Inside this network, the Customer Premises Equipment (CPE) — basically, the customer’s router — does more than just normal routing. It also acts as a tunneling point for IPv4-in-IPv6 traffic. It handles both IPv4 and IPv6 traffic.

IPv4 traffic path → when an IPv4-only host inside the network sends a packet:

1. The packet reaches the CPE, which encapsulates the IPv4 packet inside an IPv6 packet.
2. This encapsulated packet travels through the IPv6-only network of the Internet Service Provider (ISP).
3. At the edge of the IPv4 Internet, it reaches a special router called the AFTR (Address Family Transition Router).
4. The AFTR decapsulates the packet (removes the IPv6 wrapper), recovering the original IPv4 packet.
5. Then, the AFTR applies NAT44 translation, converting the private IPv4 address into a public IPv4 address before forwarding the packet to the IPv4 Internet.

In this way, even though the ISP network only supports IPv6, native IPv4 hosts can still communicate over it.

IPv6 Traffic Path → Meanwhile, traffic from native IPv6 hosts is simpler:

1. It moves directly through the IPv6 network without encapsulation.
2. If the destination is in the IPv6 Internet, it continues natively.
3. If the destination is in the IPv4 Internet, the packet passes through an Address Family Translator (AFT), which translates between IPv6 and IPv4. The AFT assigns a temporary IPv4 address for communication, which then goes through NAT44 before reaching the IPv4 Internet.

This mechanism is known as indirect delivery because packets may be encapsulated or translated along the way instead of traveling natively end-to-end.

This setup is known as Dual-Stack Lite (DS-Lite). It reduces the need for IPv4 addresses because IPv6 hosts do not require one at all. Only the IPv4 hosts need translation through NAT, which helps conserve IPv4 address space. However, this requires an extended form of NAT, one that includes both IPv6 and IPv4 address mappings in its translation table. So, an IPv6 address and port pair gets mapped to a public IPv4 address and port pair.

By employing an extended NAT system, customers can use overlapping private address spaces, since the IPv6 address of the Customer Premises Equipment (CPE) is stored in the NAT table to maintain proper mapping.

One key limitation is that the NAT is controlled by the ISP, not by the customer. That means users cannot configure port forwarding on their own. Port forwarding is the mechanism that lets incoming traffic be directed to a specific host inside your private network, typically used when hosting a server, like a web server or a game server, at home. Without control over the NAT, customers cannot host servers, because inbound connections cannot be mapped to internal devices.

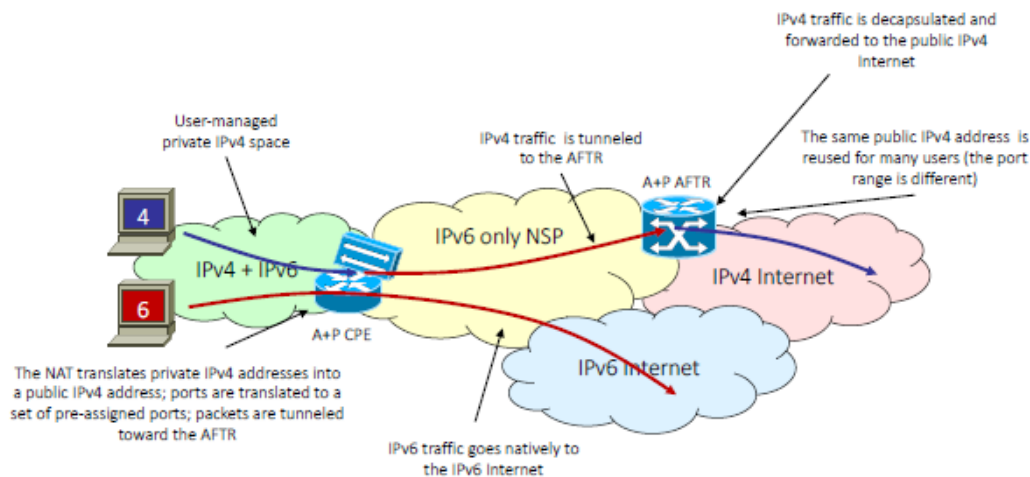
2. A+P (Address plus Port)

The A+P (Address plus Port) approach addresses the limitations of DS-Lite by giving customers control over NAT. Instead of relying solely on the provider's NAT, specific ranges of TCP and UDP ports are allocated to each customer. This means that the NAT operates only on the assigned port ranges for each user, enabling more granular control over network resources.



The general setup remains similar:

- The provider's core network is still IPv6-only.
- The AFTR is still present at the boundary between IPv6 and IPv4 domains.
- Customers may have both IPv4 and IPv6 hosts.



However, the key difference is that the NAT function is moved to the customer's CPE, rather than being controlled by the ISP. This allows the customer to manage NAT features such as port forwarding locally.

But this setup introduces a new challenge. When traffic leaves the home network, each outgoing IPv4 packet needs to be mapped to a specific port number. The problem arises because many home networks could be sending traffic through the ISP at the same time, and they might be using the same private IPv4 addresses and even the same port numbers. So, how can we avoid collisions and still identify which traffic belongs to which customer?

To make this feasible, each customer is assigned a shared public IPv4 address, and a specific range of TCP/UDP ports on that address. Only these specific ports are utilized by the NAT on the external side. This means every CPE has its own restricted port range for NAT translation.

This combination, Address + Port range, uniquely identifies each customer, hence the name A+P.

While this solves the port-forwarding problem, it introduces new complexity in port allocation and management, as every customer's device must stay within its assigned port range. For example:

- Customer A may use ports 7700–7999
- Customer B may use ports 6000–6999

In some cases, the range can be even smaller depending on the applications' needs. In this setup, each home NAT operates normally but must restrict its mappings to the allocated port range.

Suppose a packet originates from a private IPv4 host with source IP 10.0.0.7 and destination IP 130.192.1.1, carrying a TCP segment with source port 15701 and destination port 80. The customer's NAT device will translate this packet using a mapping table that considers both the IP address and the port.

The translated packet might look like this:

Source IP: 140.7.4.8
Source Port: 7500
Destination IP: 130.192.1.1
Destination Port: 80

To manage this, the device keeps a mapping table that records both IP addresses and ports. This mapping table ensures that even if multiple customers share the same public IPv4 address, their port ranges keep the flows distinguishable.

The IPv4 packet is then encapsulated inside an IPv6 packet to travel across an IPv6-only network. When it reaches the other end, it's decapsulated and sent to the IPv4 Internet as a normal routable packet.

When the return packet arrives, the router examines the destination port (which was the source port of the original connection). Even if multiple CPEs share the same public IPv4 address (140.7.4.8), the system can still determine which customer should receive the response based on the port number.

This system is quite complex because it requires careful coordination of addresses and port ranges, which is one reason why most mobile operators have already migrated to IPv6. In mobile networks, users don't need direct control over their NATs, so IPv6-based solutions are easier to manage.

Many home Internet Service Providers, however, still rely on IPv4 and use solutions like Dual Stack Lite (DSLite) or NAT444 instead of going fully IPv6. These mixed environments introduce extra complexity, since both IPv4 and IPv6 worlds must coexist and interoperate. The advantage of A+P is that it eliminates the issue of overlapping private IPv4 addresses, because each customer is uniquely identified by their assigned port range.

There's also a protocol called *Port Control Protocol (PCP)* that automatically assigns port ranges to CPEs. If a customer needs additional ports, their CPE can negotiate extra ranges with the AFTR (Address Family Transition Router) dynamically using PCP.

With A+P, you no longer need a NAT44 within the AFTR, because the translation into a public IPv4 address happens directly at the customer side, the AFTR only handles tunneling across the IPv6 network. When packets emerge from the tunnel, they are already routable IPv4 packets. This differs from the DSLite solution, where the packet that comes out of the tunnel is not routable until it passes through a NAT inside the AFTR. A+P simplifies that process by performing the NAT translation earlier.

→ The main motivation to have the NAT deployed at the CPE in the A+P transition solution is to let customers configure their own port mappings.

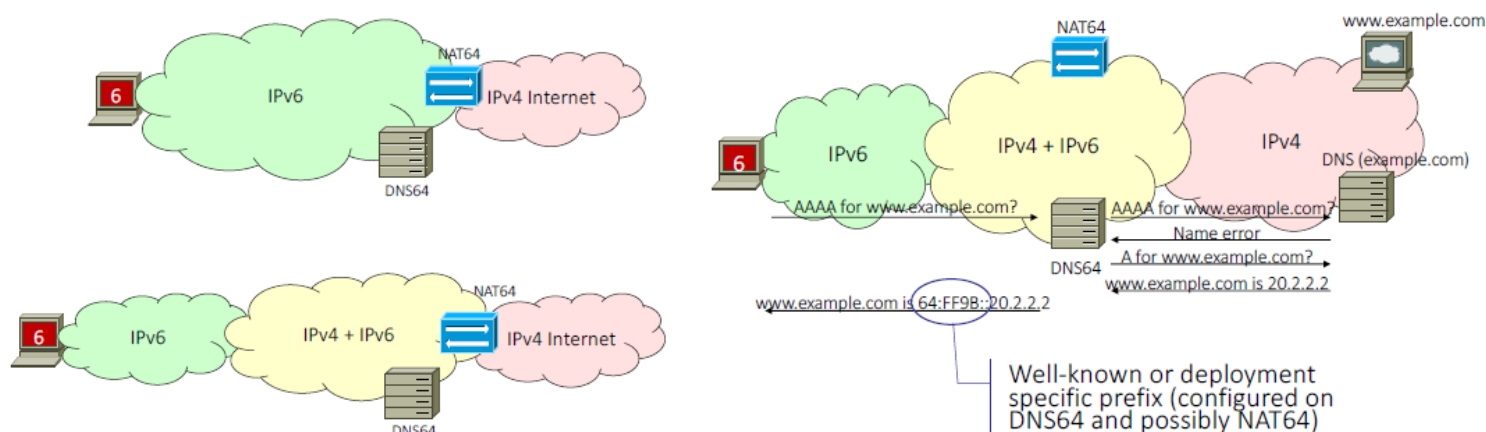
3. NAT64 and DNS64



Another common mechanism for IPv6–IPv4 coexistence is NAT64 combined with DNS64. To understand this, recall that:

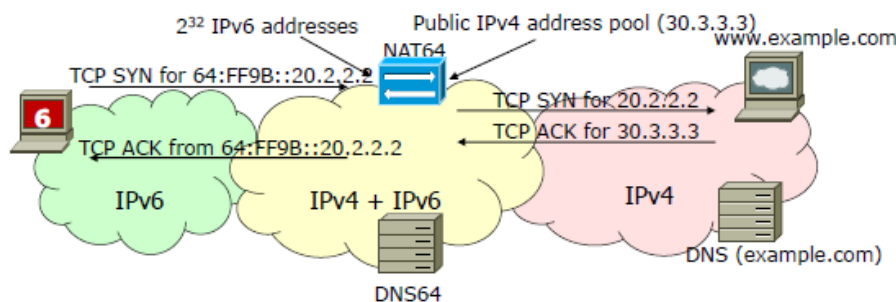
- A DNS A record maps a hostname (e.g., `www.polito.it`) to an IPv4 address.
- A DNS AAAA record (quadruple-A) maps a hostname to an IPv6 address.

A DNS64 server can generate synthetic AAAA records from existing A records, allowing IPv6-only hosts to reach IPv4-only servers.



Here's how it works:

1. A native IPv6 host wants to connect to www.example.com. It queries its DNS64 server for the IPv6 address of that hostname.
2. The DNS64 server forwards the query to the authoritative DNS for example.com. Since the domain is IPv4-only, it replies with an A record (e.g., 20.2.2.2) and no AAAA record.
3. The DNS64 server constructs a synthetic IPv6 address using a special well-known prefix (defined by RFCs) — typically **64:FF9B::/96**. The last 32 bits of this IPv6 address correspond to the IPv4 address, so the IPv6 synthetic address will be 64:FF9B::20.2.2.2
4. The IPv6 host then sends packets to this address, unaware that the destination is actually an IPv4-only host. The NAT64 gateway recognizes the 64:FF9B::/96 prefix, extracts the last 32 bits, and creates a corresponding IPv4 packet.
5. When the server replies, NAT64 reverses the mapping and delivers the response back to the original IPv6 host.



NAT64/DNS64 works only when the connection is initiated through DNS queries. If an application tries to connect directly using an IPv4 literal address (e.g. ping 1.2.3.4), the translation won't happen unless the IPv6 address has been pre-mapped.

A further issue arises with **DNSSEC** (*DNS Security Extensions*). DNSSEC ensures data integrity by signing DNS records with cryptographic certificates. However, DNS64 modifies the original DNS replies by generating synthetic AAAA records. This breaks the DNSSEC chain of trust, since the DNS64's response no longer matches the signature from the authoritative server.

These transition mechanisms, A+P, DSLite, NAT64/DNS64, are complex and temporary solutions designed to bridge the gap between IPv4 and IPv6 worlds. As native IPv6 traffic continues to grow, the hope is that one day we'll reach a fully IPv6 Internet and no longer need such intricate workarounds. Unfortunately, while NAT is far from ideal, it remains a necessary part of today's Internet, despite adding significant operational complexity to network management.

Wireless and Cellular Networks

In recent years, a new family of technologies has become absolutely essential for modern communication: wireless and cellular networks. Today, most of the world's internet traffic originates from mobile devices, making it increasingly important to understand how these systems function and how they differ from traditional wired communication.

By 2019, the number of wireless phone subscribers had already exceeded wired subscribers by roughly 10 to 1, and the number of mobile broadband connections surpassed fixed broadband connections by about 5 to 1. Modern 4G and 5G cellular networks have now fully adopted the Internet Protocol (IP) stack, integrating technologies like Software-Defined Networking (SDN) to enhance flexibility and scalability.

Wireless devices communicate via a wireless link, meaning no physical cables are required. Current wireless technologies can reach speeds comparable to fixed broadband connections, making them a viable and often preferable alternative to wired access.

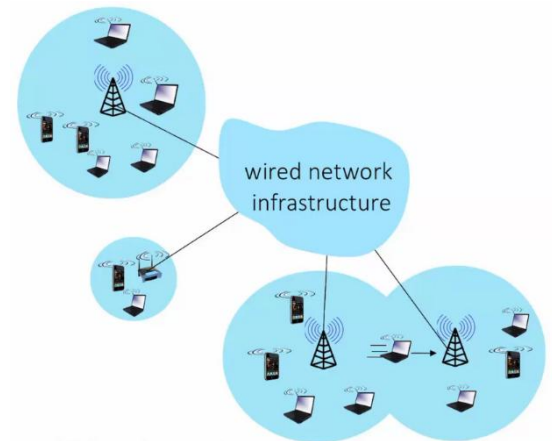
One of the defining characteristics of wireless networks is mobility. While it provides great flexibility, it also introduces complexity. As a user moves, their point of attachment to the network changes — for instance, data being transmitted to one antenna must be rerouted to another as the device moves across coverage areas. Managing this constant change of attachment points is one of the main challenges in wireless communication.

In short, we can distinguish between two fundamental but related aspects:

- Wireless: communication over wireless link
- Mobility: handling the mobile user who changes point of attachment to network

Elements of a wireless network

- Wireless Network Coverage → Each wireless network has a coverage area, represented as a blue circle. In the center there is an antenna or base station that provides connectivity.
- Connection to Wired Infrastructure → Although they are called “wireless,” these networks rely on a wired infrastructure in the background. The antennas are wired together and connected to the Internet backbone. Wireless communication only covers the “last hop”, the link between mobile devices and the network. Beyond that, traffic travels over wired connections to other antennas, mobile users, or fixed servers.
- Wireless Hosts → These are devices with wireless interfaces: laptops, desktops (using USB modems in the past), tablets, smartphones, smartwatches, etc. They can be stationary or mobile, for instance, a tablet may connect to Wi-Fi in a classroom, then to another Wi-Fi in an office, or switch to mobile data when taken to a café (wireless not always means mobility).
- Base Stations → These are the access points connecting wireless hosts to the wired network (in its area). Examples include:
 - o Cell towers (for mobile networks)
 - o 802.11 access points (for Wi-Fi networks)



The term 802.11 refers to the IEEE standard that defines Wi-Fi technology.

- Wireless Links → typically used to connect mobile(s) to base station, also used as backbone link. Multiple access protocol coordinates link access. These links transmit bits via radio waves. To send digital data over electromagnetic waves, devices use a process called modulation (various transmission rates and distances, frequency bands).

Modulation — Translating Bits into Waves

Modulation is the process of altering a sinusoidal electromagnetic wave to represent digital information (bits). The modification can occur in three main ways:

- Amplitude Modulation (AM or ASK): change the wave's amplitude.

- Frequency Modulation (FM or FSK): change its frequency.
- Phase Modulation (PM or PSK): change its phase.

Sometimes combinations are used (e.g., amplitude + phase modulation). Each bit or group of bits corresponds to a specific change in the wave. For example:

- A particular amplitude could represent 101
- A different amplitude might represent 011

The transmitter and receiver agree beforehand on what each change means. At the receiver end, detecting a certain amplitude, frequency, or phase lets the system decode whether a 1, 0, or a group of bits was sent.

It's also possible to encode multiple bits per symbol, e.g., groups of 3 bits → 8 possible values → 8 distinct waveform variations. The bitrate depends on:

- How many bits are encoded per symbol.
- The symbol duration (longer or shorter).
- How distinct the possible waveforms are (to prevent decoding errors).

This mechanism defines different transmission rates and coverage ranges depending on the modulation complexity.

Wireless systems operate within specific frequency bands, which are ranges of frequencies used for transmission:

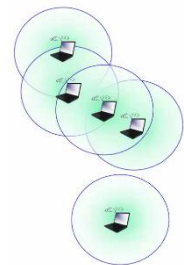
- Wi-Fi: typically uses 2.4 GHz, 5 GHz, and newer versions also 6 GHz bands. (The number refers to the center frequency, not a single frequency.)
- Cellular networks: generally operate between 1 and 3 GHz.

These frequencies determine how signals propagate and how much data they can carry.

Communication Modes

There are two main communication modes in wireless systems:

1. **Infrastructure Mode** → The standard mode for Wi-Fi and cellular networks.
 - a. Devices connect through a base station (access point or cell tower) into wired network
 - b. Supports handoff (or handover) when a moving device switches from one base station to another.
 - c. During handoff, the network must detect weaker signal strength, identify a stronger nearby tower, and redirect ongoing data flows seamlessly.
2. **Ad Hoc Mode** → “Ad hoc” (Latin: for this purpose) refers to a self-contained network without infrastructure.
 - a. Devices communicate directly with each other, so there is not base station
 - b. nodes can only transmit to other nodes within link coverage
 - c. nodes organize themselves into a network: route among themselves
 - d. Example: Bluetooth connections between a laptop and wireless mouse, or headphones, a small, isolated “island” network where traffic never leaves the local devices.



Wireless Links Characteristics

Wireless communication behaves very differently from wired communication. The key characteristics include:

1. **Path Loss** → Signal strength decreases with distance. Radio signal attenuates as it propagates through matter (path loss). In free space, power falls proportionally to the square of the distance.
2. **Interference** → Wireless signals are broadcast, not confined to cables. This causes interference when multiple transmitters use the same or nearby frequencies. Especially common in unlicensed bands like 2.4 GHz and 5 GHz, shared by many devices. For example, at 2.4 GHz, many devices coexist: Wi-Fi, Bluetooth, security cameras, motion sensors, and even microwave ovens (which don't transmit data but use 2.4 GHz electromagnetic radiation to heat water and food).
3. **Multipath propagation** → Occurs when a signal takes multiple paths (reflections, diffractions) to reach the receiver, leading to signal distortion and interference between copies of the same signal. Radio signal reflects off objects ground, arriving at destination at slightly different times.

These make communication across (even a point to point) wireless link much more difficult.

Multipath propagation occurs every time you transmit a signal and it starts bouncing off walls and other obstacles before reaching the receiver. You can think of it as an “echo” effect. For example, imagine you are speaking inside a

large hall or gym. Gyms are not soundproof, so when you shout from one end to the other, your voice bounces and echoes off the walls, creating a confusing, resonating sound that makes it hard to understand speech. That phenomenon — sound waves reflecting and overlapping — is a perfect analogy for multipath propagation in wireless systems.

In communication, this is one of the most devastating problems, because the receiver does not only get the direct signal from the transmitter, but also multiple delayed copies of that same signal. Each signal carries a bit that lasts for a very short period of time, so even a few milliseconds of delay between copies is enough to make them overlap and interfere with each other.

Imagine a cell tower transmitting a wave with some modulation. Modern antennas are quite precise — not exactly like laser beams, but still capable of focusing their energy toward a specific area. However, in urban environments, there are always obstacles — buildings, vehicles, trees, even leaves — that cause the radio waves to diffract, reflect, or scatter. As a result, your mobile device receives multiple copies of the same transmission, each arriving with different amplitudes, phases, and delays. These act like echoes of the original signal: the receiver might first get the main signal, and then a slightly delayed version of it that overlaps with the new data being received. This overlap makes it difficult to correctly interpret the transmitted bits — just like trying to listen to a voice that keeps echoing in a gym.

What makes multipath propagation even worse in urban areas is that obstacles move. Cars, buses, and even people constantly change positions, so the reflected paths keep changing dynamically. You might receive an echo, try to compensate for it, and then a few milliseconds later it disappears, replaced by another echo coming from a different object. This continuous variation makes wireless communication extremely challenging, and it's honestly remarkable that our devices manage to reconstruct the transmitted signal so reliably despite all this interference.

Signal-to-Noise Ratio (SNR)

To understand how communication quality is affected, we use the concept of Signal-to-Noise Ratio (SNR).

SNR is the ratio between:

- the power of the useful signal (the one carrying your information), and
- the power of the noise, meaning everything that interferes with it — echoes, other signals, environmental disturbances, etc.

If the noise is too strong compared to the signal, the receiver cannot distinguish information clearly.

- A high SNR means the signal is much stronger than the noise → good communication quality.
- A low SNR means noise dominates → poor communication quality or total loss.

Mathematically, SNR is often expressed in decibels (dB), and higher dB values indicate cleaner, more reliable communication.

Bit Error Rate (BER)

The Bit Error Rate (BER) measures how many bits are received incorrectly. It's the probability that a bit is received as "1" when it was actually transmitted as "0", or vice versa.

- In wired or optical networks, BER can be as low as 10^{-12} , meaning one error per trillion bits.
- In wireless communication, due to noise and interference, BER is usually much higher — sometimes 10^{-3} , 10^{-2} , or even 10^{-1} , which makes reliable communication impossible.

There's a direct relationship between SNR and BER. If you increase the transmission power, you increase the SNR, which in turn reduces the BER. In other words, a stronger signal relative to the noise allows bits to be received more accurately.

For a given SNR, you can choose the physical layer parameters, such as the modulation scheme, that achieve the required BER while providing the highest possible throughput.

For instance, if the channel quality is good (high SNR), you can use a more complex modulation like 256-QAM to send more bits per symbol and maximize data rate. However, SNR is not constant. It can change dynamically with user mobility, interference, or obstacles.

Therefore, modern systems adapt the physical layer in real time — adjusting the modulation technique and transmission rate according to current SNR conditions.

This process is known as **Adaptive Modulation And Coding (AMC)**, and it ensures that the connection remains reliable (low BER) while still maintaining the highest possible throughput for the given channel quality.

- As SNR increases, BER decreases — you get fewer bit errors.
- As SNR decreases, errors skyrocket.

This relationship is often visualized as a curve, showing that higher SNR levels result in exponentially lower error rates.

Let's look, for example, at this dotted green curve. It corresponds to a modulation called QAM-256 (Quadrature Amplitude Modulation, 256 levels).

QAM is a technique that combines amplitude and phase modulation to encode bits into symbols. In QAM-256, each transmitted symbol represents 8 bits, since $2^8 = 256$ possible combinations of amplitude and phase exist. This makes it a very aggressive modulation scheme — during each symbol period, the signal can take one of 256 distinct values.

However, this also means that the channel must be very clean and quiet, with minimal noise and interference. Even a small echo or distortion can shift the amplitude or phase slightly, causing the receiver to misinterpret one symbol as another. So, a small disturbance might make the receiver decode the wrong bit pattern — for instance, mistaking 1100110 for 1101010.

If the channel quality is very good — for example, with a Signal-to-Noise Ratio (SNR) of around 30 dB — QAM-256 performs extremely well, achieving a Bit Error Rate (BER) as low as 10^{-7} , which means only one bit error every 10 million bits.

But as the channel becomes noisier, due to interference or movement that causes multipath reflections, the SNR decreases. Consequently, the BER increases until the modulation becomes unusable: the receiver can no longer reliably distinguish between the 256 symbols.

When that happens, the system must adapt and switch to a less complex modulation scheme, such as QAM-16. In QAM-16, each symbol encodes 4 bits (since $2^4 = 16$), so there are only 16 possible signal states. This makes each signal point more distinct and easier to tell apart, even in a noisier channel — though, of course, fewer bits are transmitted per symbol. If the channel quality drops even further and the SNR becomes very low — say around 10 dB or less — even QAM-16 becomes unreliable.

At that point, we can switch to **BPSK (Binary Phase Shift Keying)**, one of the simplest and most robust modulation techniques. In BPSK, there are only two possible signals: one with phase 0° (representing bit '0') and one with phase 180° (representing bit '1'). Because these two states are maximally distinct, BPSK performs well even in very noisy environments. You would need the SNR to be almost zero — meaning that noise power equals signal power — for the communication to fail completely.

So, these are the essential trade-offs between SNR and BER:

- High SNR → more aggressive modulation (higher data rate, e.g., QAM-256).
- Low SNR → simpler modulation (lower data rate, e.g., BPSK).

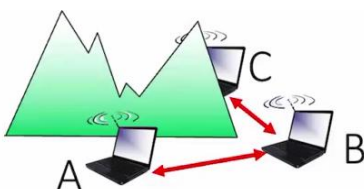
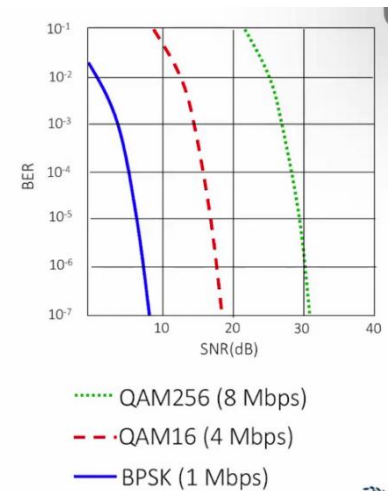
Modern communication systems use adaptive modulation to switch automatically between these schemes based on the current SNR, balancing throughput and reliability in real time.

Hidden terminal problem

Another issue that often occurs in wireless communication is the Hidden Terminal Problem.

Let's imagine this simple scenario: There are three devices — A, B, and C. Device A wants to transmit data to B, and at the same time, device C also wants to send data to B. In most wireless communication systems, before transmitting, each device performs a procedure called **carrier sensing** — it listens to the channel to check if it is free. If the channel is idle (no one else transmitting), the device starts sending data. This is the basic idea behind **CSMA (Carrier Sense Multiple Access)**, the same principle used in Wi-Fi.

Now, here's the problem: both A and C can communicate with B, but A and C cannot hear each other. They might be separated by an obstacle — a thick wall, a metal door, or simply located far

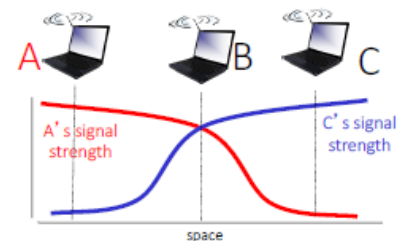


enough apart that their signals don't reach one another. Because they can't detect each other's presence, A and C both think the channel is free. They start transmitting simultaneously to B — and their signals collide at B. As a result, B receives a corrupted transmission, similar to two people talking to you at the same time without realizing the other is speaking.

Signal attenuation

Sometimes this doesn't even require an obstacle — A and C might simply be too far apart for their signals to overlap, while B, positioned between them, receives both.

This situation, where two transmitters cannot sense each other but interfere at a common receiver, is called the **Hidden Terminal Problem**.

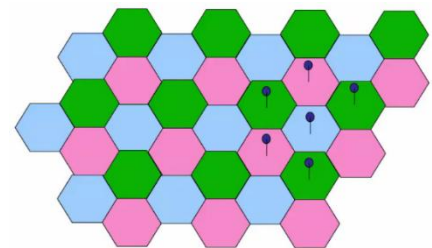


Cellular network

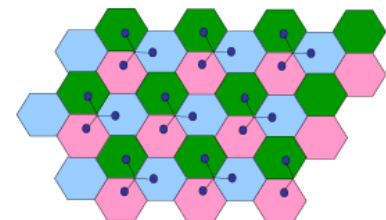
Why are they called cellular? The term actually comes from the honeycomb structure of beehives 🐝. If you look inside a beehive, you see it's made of many hexagonal cells, all fitting together perfectly to cover an area without gaps.

Early engineers designing mobile networks used this as inspiration. They realized that covering a geographical area with hexagonal cells was the most efficient way to organize radio coverage — like a tessellation, where each hexagonal “cell” represents the area covered by one base station.

In a cellular network, the entire region is divided into these adjacent or slightly overlapping hexagonal areas, called **cells**. Each cell has its own base station (or antenna), and users can move from one cell to another without losing connection. This process of transferring an active connection between cells is known as a **handover** (or handoff).



In theory, the layout of a cellular network looks like a perfect hexagonal grid, each hexagon representing a cell. At the centre of each cell, there's usually an antenna, sometimes an isotropic antenna (which transmits equally in all directions), or, in other designs, a tower placed at the intersection of three cells, equipped with three directional antennas covering 120° each, located at one extreme of each cell. This setup reduces cost, since fewer towers are needed, each one serves three sectors instead of just one.



However, in reality, cell coverage is far from perfectly hexagonal, and they do not have the same size. Real-world factors such as terrain, buildings, antenna height, emitted power, and environmental conditions make the coverage areas irregular and uneven.

This irregularity depends on factors such as: power emitted at the antenna, antenna height and antenna gain (how far the signal can reach), obstacles that block signal propagation, topography and urban structures, atmospheric conditions; Engineers often say that “cells breathe”: when it rains or it's foggy, the path loss increases, so the effective cell radius shrinks; on sunny days, the same cell might appear larger. Network operators can even intentionally “make cells breathe” by increasing transmit power to temporarily expand coverage in a specific area.

Modern networks use a mix of *macro cells* (covering 1–10 km or more) and *micro cells* (covering less than 1 km, typically used in urban areas) so, different size, depending on the specific needs.

Channel access

Multiple user channel access: within a single cell, many users need to share the same wireless channel to communicate with the base station. This is handled by multiple access techniques, which allow several users to transmit data without interference. The most common approaches are:

- **FDMA** (*Frequency Division Multiple Access*) → Different users are assigned different frequency bands. For example, one user communicates using frequency F1, another uses F2, and so on. As long as the frequencies don't overlap, the users won't interfere if they communicate at the same time. Each cell is assigned a specific set of frequencies, and neighbouring cells use different frequency ranges to avoid cross-interference at the edges of coverage. For instance, Cell 1 might use frequencies from 3.50 to 3.53 GHz and Cell 2 might use 3.51 to 3.54 GHz, and so on.
- **TDMA** (*Time Division Multiple Access*) → All users share the same frequency, but they transmit at different time slots. For example: during one time slot, the base station communicates with User 1, in the next time slot, with User 2 and then back to User 1, and so on. This prevents overlap, at any given time, only one transmission occurs on that frequency.
Typically, two separate frequencies are used, one for the downlink (from antenna to user), one for the uplink (from user to antenna).
- **CDMA** (*Code Division Multiple Access*)
- **SDMA** (*Space Division Multiple Access*) → at the same time and at the same frequencies, but separated by obstacles

Modern systems combine both techniques. For example, groups of users are assigned different frequency bands (FDMA), and within each frequency band, users take turns transmitting at different time slots (TDMA). This hybrid approach allows networks to support thousands of simultaneous users, efficiently sharing spectrum and time resources.

Frequency reuse

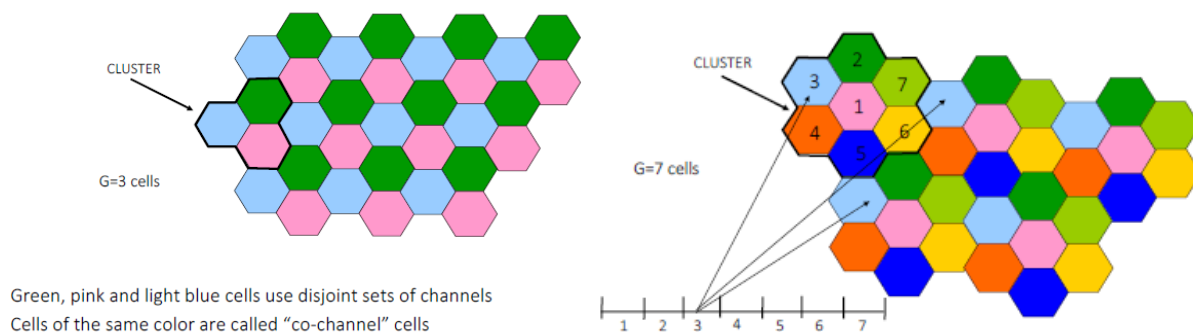
Given the limited number of available resources, we need to ensure coverage and serve a high number of users → FDMA with frequency reuse (SDMA) = reuse the same frequencies at different locations.

In a cellular network, the coverage area is divided into cells, each served by a base station. A **cluster** is a group of adjacent cells, denoted by G , where each cell within the cluster operates on a different set of frequencies.

Together, these G cells use the entire set of frequencies allocated to one operator. Each operator owns a certain range of frequencies, which they use to run their cellular network. These frequencies are not free, they are licensed by the government through periodic frequency auctions (typically every 10 years). For example, in Italy, the Ministry for Telecommunications owns and manages the national frequency spectrum. Operators must purchase licenses to use specific frequency bands, and these licenses are extremely expensive. Naturally, this cost is indirectly passed on to customers through mobile plan fees.

Not all frequencies require a license, though. For example, Wi-Fi frequencies (like 2.4 GHz and 5 GHz) are unlicensed and can be used freely. But cellular network frequencies are always licensed.

3-cell cluster and 7-cell cluster

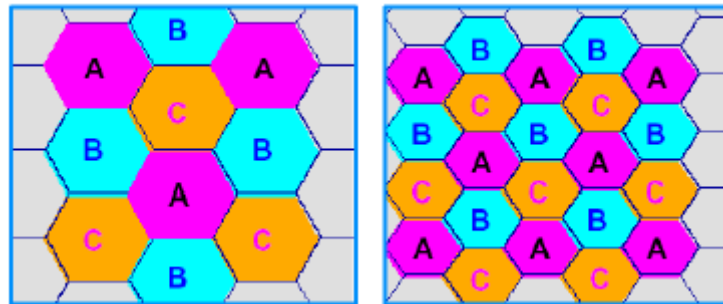


Imagine an operator owns the frequency band 3.65 GHz to 3.85 GHz. Cells that are far enough apart can reuse the same portion of this band without causing interference — that's the principle of frequency reuse.

When we talk about cluster size (G), we're referring to how many adjacent cells use distinct frequency sets before reuse occurs. For example:

- A 3-cell cluster uses three distinct frequency bands.
- A 7-cell cluster uses seven distinct frequency bands.

Fixing G and varying the cell size R



Let's fix the cluster size and vary the cell radius (R). If we decrease the cell size, meaning the radius R becomes smaller, system capacity increases. Why? Because smaller cells serve fewer users, so each user gets a larger share of the total available bandwidth.

Assume each base station (antenna) can handle up to 1 Gbps, sometimes 10 Gbps, total. In the larger cells, this capacity might be shared among 500 users, while in smaller cells, it may be shared among only 100 users. Clearly, users in smaller cells experience better service quality, since they have more bandwidth available per person.

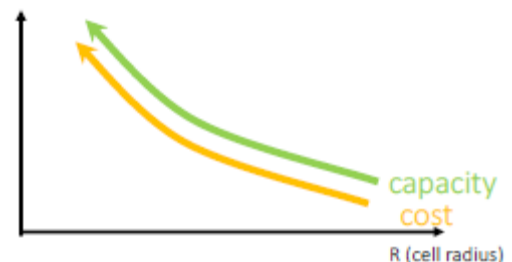
Therefore:

- Smaller cells → higher capacity and better performance
- Larger cells → lower capacity but fewer antennas required

The trade-off is cost: smaller cells mean you need more antennas to cover the same area. Each antenna adds installation and maintenance expenses. So, while capacity improves, costs increase.

Given the cluster size G:

- **The smaller the R, the larger the capacity**
- **The smaller the R, the larger the number of antennas needed to ensure same total coverage**



Now let's fix the cell radius and change the cluster size. If we make the cluster smaller (e.g., 3 cells instead of 7), frequencies are reused more frequently, meaning co-channel cells are closer together, increasing the risk of interference, especially for users near cell edges. In contrast, larger clusters place co-channel cells farther apart, reducing interference and improving signal quality.

For example:

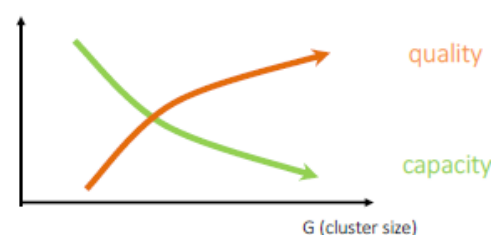
- In a 3-cell cluster, the available 200 MHz might be split into three frequency blocks — about 70 MHz per cell. Each cell can then serve roughly 70 users simultaneously.
- In a 7-cell cluster, the same 200 MHz is divided into seven smaller chunks, about 30 MHz per cell, allowing fewer simultaneous users per cell.

So there's a trade-off:

- Small G (small cluster) → Higher capacity, more interference, lower quality
- Large G (large cluster) → Lower capacity, less interference, higher quality

Given the cell size R:

- **The larger the G, the smaller the number of channels per cell, the lower the system capacity**



- **The larger the G , the larger the distance between co-channel cells, the less the interference, the better the quality**

As mobile networks evolved — from the early cellular systems in the 1990s, through 3G and 4G, and now into 5G — operators have had to constantly rethink coverage design. They’ve had to decide how large each cell should be, how to group them into clusters, and how to adapt to population density, geography, and traffic demand.

Designing an efficient cellular network means balancing capacity, interference, and cost — and several key techniques have been developed to improve frequency reuse and overall performance. These include:

- Cell Splitting
- Sectoring
- Antenna Tilting
- Creating femtocells (small cells)

Let’s go through each of them.

1. **Cell Splitting:** it means dividing a large cell into several smaller ones. Coexistence between
 - a. Microcells: small cells in highly populated areas with high traffic density (large cities). Transmit power at the base station ≈ 3 W
 - b. Macrocells: large cells in areas with low traffic density (rural areas). Transmit power at the base station ≈ 20 -60 W



This is typically done in areas with a high density of users, like city centers, where the available capacity of one large cell isn’t enough to serve everyone efficiently. By splitting the coverage area, operators can install more base stations, each serving a smaller number of users. This increases the overall network capacity, since every new, smaller cell gets its own frequency allocation (following the reuse pattern).

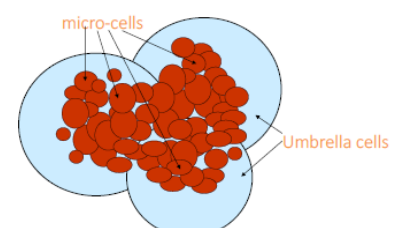
2. **Sectoring:** Instead of one omnidirectional antenna serving the entire cell, the base station divides the cell into several sectors, typically 3 or 6. Each sector has its own directional antenna, covering only a portion of the cell (for example, 120° for three-sector cells). By doing this, the same frequencies can be reused more efficiently across different sectors, and interference between users in opposite directions is reduced. Sectoring effectively multiplies capacity within the same geographical area without changing the overall cell radius.
3. **Antenna Tilting:** Tilting refers to adjusting the angle of the antenna — usually downward — to control how far its signal reaches.
 - a. Down-tilting the antenna reduces its coverage range, limiting interference with neighboring cells and focusing power on nearby users.
 - b. Up-tilting, conversely, extends the coverage area, useful in rural or low-density regions.

By carefully tuning the tilt angle, operators can balance between wide coverage and reduced interference, adapting dynamically to the environment.

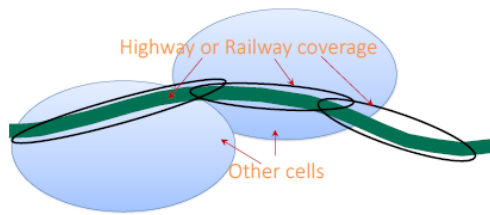
4. **Femtocells** (Small cells) → In addition to macro and microcells, modern networks often use femtocells (or small cells) extremely small base stations that cover very limited areas, such as a home, office, or café. These small cells are deployed to fill coverage gaps between larger cells (called **umbrella cells**) and to provide extra capacity where traffic demand is high. For example, in dense urban areas or inside buildings where macro signals struggle to penetrate, femtocells ensure strong, localized coverage and offload users from the main network.

Cell shaping

Directional antennas enable cells with customized shapes and sizes. This allows for multi-layer coverage, where large “umbrella” cells overlap smaller microcells. Microcells can dynamically follow users, ensuring consistent signal strength as they move.



In reality, cell coverage areas are not perfect hexagons — that's just a theoretical simplification. Cells often vary in shape and size, intentionally designed to match geographical features or traffic patterns.



For instance, along highways or railways passing through rural areas, operators deploy directional antennas that focus coverage along a narrow strip of land following the transport route. These antennas don't need to support many users (since the area is sparsely populated), but they must provide continuous coverage along the track for moving vehicles and trains. Meanwhile, surrounding areas are covered by broader cells that serve local

residents.

Power control

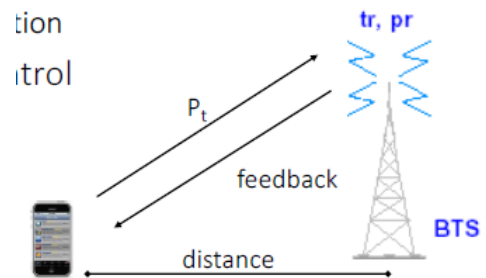
Another crucial concept in cellular networks is power control, the mechanism that adjusts the transmission power of both the base station and the mobile device.

Your smartphone doesn't always transmit with the same power. You may have noticed that your battery drains faster when you're far from a cell tower. That's because your phone must transmit at higher power to maintain a reliable link. Conversely, when you're in a city, surrounded by antennas, your phone uses much lower power because the nearest tower is close and easy to reach.

This leads to an interesting (and often misunderstood) conclusion: more antennas mean lower exposure to electromagnetic radiation.

When there are many nearby antennas, both your phone and the base stations operate at lower power levels, which is actually safer for human exposure. In contrast, having fewer antennas forces each tower to cover larger areas, requiring higher transmission power, which means stronger electromagnetic fields, not weaker ones.

Power control is implemented as a closed feedback loop between the mobile device and the base station. If a phone transmits with too much power, it could overwhelm the antenna and interfere with other users on the same channel. If it transmits with too little, the connection could drop. So, the base station continuously monitors the signal strength and instructs the phone to increase or decrease its transmission power as needed. This ensures efficient spectrum use, reduced interference, lower energy consumption, safer and more stable connections.



Why Power Control is Needed - Power control is essential to reduce interference, lower energy consumption, and manage open or closed loop control mechanisms within wireless networks.

Power Control Strategies:

- Uplink:
 - o Open loop power control
 - o Closed loop power control
 - o Outer loop power control
- Downlink:
 - o Downlink power control

Open Loop Power Control (Uplink)

In open loop power control, the user terminal independently adjusts its transmit power without direct feedback from the base station. This is a two-step process:

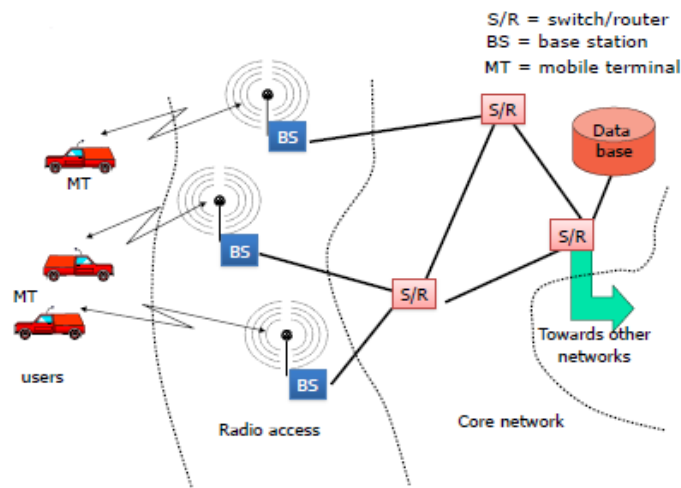
1. The user measures the quality of the downlink signal received from the base station.
2. Based on this measurement, the terminal runs an algorithm to set its uplink transmit power so that the estimated *Signal-to-Interference-plus-Noise Ratio (SINR)* stays above a predefined threshold.

This method is called “open loop” because it lacks feedback from the base station, making it less accurate, especially since uplink and downlink transmissions often use different channels. More precise but complex techniques, like closed loop power control, use feedback to improve accuracy.

Frequency allocation

- **Fixed Channel Allocation (FCA)**
 - o Based on the concept of cluster
 - o Frequencies are assigned in a static way
 - o Frequency plan is changed only rarely to improve performance and adapt to slow variations in user traffic
- **Dynamic Channel Allocation (DCA)**
 - o Resources assigned to cells by a central controller when needed
 - o Frequency plan changes over time to adapt to the system status
- **Hybrid Channel allocation Scheme (HCS)**
 - o One portion is statically allocated (FCA)
 - o One portion is dynamically allocated (DCA)

Network architecture



The network architecture of a cellular system can be broadly divided into three main parts:

1. Mobile terminals (the users)
2. The radio access network (RAN)
3. The core network

Mobile Terminals: these are the devices used by end users — smartphones, tablets, laptops with SIM cards, etc. The term mobile does not necessarily mean “in motion” (like being in a car); it simply refers to devices that connect wirelessly to the network.

Radio Access Network (RAN): the radio access part is the portion of the cellular network that we can physically see — the cell towers placed on rooftops, towers, or hills. These base stations are responsible for establishing wireless communication with our mobile devices. They receive the signals sent by our phones and transmit data back to them, enabling voice calls, text messages, and data connections.

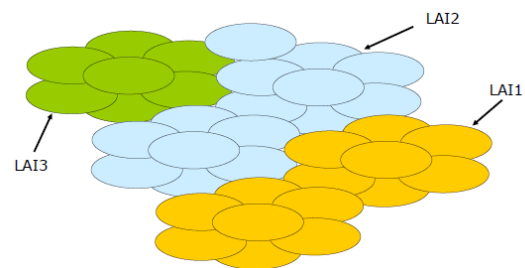
Core Network: the core network is the backbone of the operator’s infrastructure. It manages and routes traffic between mobile users and external networks such as the Internet or other mobile users. The core network must constantly be aware of which base station each user is currently connected to. When you move, the system needs to seamlessly transfer your connection from one cell tower to another without dropping the call or data session. This process is called handover, and it ensures continuous connectivity as users move through different coverage areas.

Basic procedure

1. **Registration** → It allows a mobile terminal to connect with the network, to identify and authenticate itself. It is performed:
 - When the device is switched on and needs to associate with the network
 - Whenever a user requests access to a service (e.g., starting a phone call — even if already registered, the network still verifies service permissions)
 - Periodically, to maintain up-to-date informationDuring registration, the mobile terminal sends a special request to the nearest base station. The operator authenticates this request and registers the user in the system.
2. **Mobility management** → Mobility is a defining feature of cellular networks — users move, and the system must keep track of them. To handle this, several procedures exist.

Roaming → works hand in hand with the **location updating process**. The operator must always know the approximate position of each user — not the exact geographical coordinates, but rather the specific cell or group of cells where the user's mobile device is currently located.

This means that the user's position needs to be traceable at all times, even when they move across the network's coverage area. To make this possible, the system stores user locations in a dedicated database, to locate users whenever needed. The entire network area is divided into *Location Areas (LAs)*, each consisting of multiple cells. Every location area has its own identifier, known as the *Location Area Identifier (LAI)*.



Location updating → it is the procedure through which the user position is updated. A control channel is periodically broadcasted in each cell of an LA.

The operator keeps this information up to date thanks to the **location updating messages** periodically sent by your phone. Whenever your mobile terminal detects that it has entered a new location, meaning it receives a different LAI from the one previously stored, it triggers a **location update procedure**, in order to update her position in the database.

Why does the operator need to maintain this information? Because when someone tries to contact you, the network must first determine where you are in order to deliver the incoming call or data. This is done through a process called **paging**, which relies on the latest location data stored in the operator's system.

Paging → is the procedure through which the network notifies a mobile terminal about an incoming call or data transmission. When someone calls your phone, the operator receives a request directed to your number. However, before the call can reach you, the network needs to determine where you currently are. To do so, it checks the roaming database, which contains information about the last known location area (LA) associated with your device.

Once the system identifies the correct LA, it sends a paging message, a broadcast notification transmitted across all the cells within that area. This message includes an identifier that corresponds to your mobile terminal. Your device, which is constantly listening to control channels, recognizes the paging message addressed to it. As soon as it detects this signal, it alerts you, typically by ringing or showing an incoming call notification.

This mechanism ensures that you can receive calls or data seamlessly, even while moving across the network's coverage area.

Handover (or handoff) → is the procedure that allows an active connection, such as an ongoing call, video stream, or data session, to be transferred from one cell to another as the user moves through the network. It is a complex process that requires tight coordination between multiple components of the network architecture, including the base stations and the core network, protocols and signaling.

For example, imagine you're on a call or streaming a video while walking or driving. As you move away from the coverage area of your current base station, the signal weakens, while another nearby base station provides a stronger

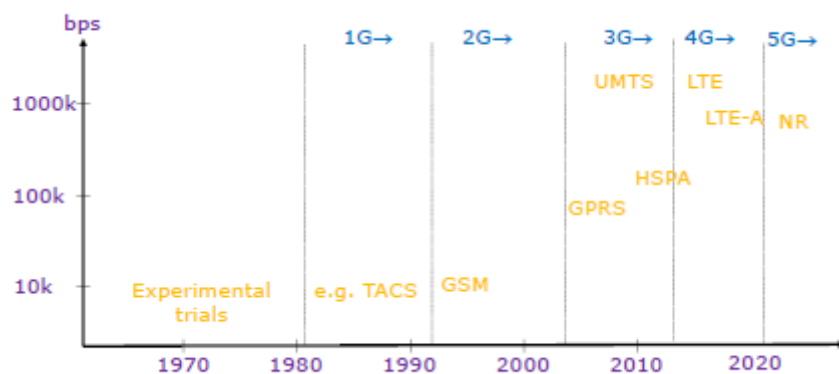
signal. At this point, the network automatically triggers a handover. This involves redirecting all communication traffic, which was previously handled by the old base station, to the new one. The process must occur seamlessly, without interrupting your service or causing noticeable delays.

Handover management is therefore essential to maintaining continuous connectivity and quality of service, especially in environments where users are constantly on the move.

Handover classification

TYPE	DESCRIPTION
Intra- vs. Inter-Cell	Determines whether the handover occurs between frequencies or base stations within the same cell (intra-cell) or between different cells (inter-cell).
Soft vs. Hard	In a soft handover, both radio channels (old and new) are active simultaneously, ensuring a smooth transition. In a hard handover, only one channel is active at a time — the connection with the old cell is broken before the new one is established.
MT- vs. BS-Initiated	Refers to which entity triggers the handover. If the mobile terminal (MT) starts it (based on signal strength measurements), it's MT-initiated. If the base station (BS) starts it, it's BS-initiated; i.e., which entity performs measurements to understand where and when a handover has to be executed
Backward vs. Forward	Describes whether handover signaling is routed through the origin base station (backward) or directly through the destination base station (forward).

Cellular network evolution



1G: First generation

The history of cellular networks dates back to the 1970s, when the first experimental trials of mobile communication began. The first commercial service was called TACS (*Total Access Communication System*), known as 1G, or first generation.

1G networks used analog technology (meaning there was no digital signal processing) and Frequency Division Multiple Access (FDMA) to support voice traffic only. There was no data transmission, no messaging, nothing beyond simple phone calls. They suffered from low service quality, inefficient frequency reuse, and limited network capacity. For example, TACS operated at 900 MHz with a channel bandwidth of 25 kHz. In Italy, this service was discontinued by TIM on December 31, 2005.

GSM – The First Successful Digital Network

The first truly successful cellular network was **GSM** (*Global System for Mobile Communications*), introduced in the 1990s. GSM marked the beginning of the digital era in mobile communications and is often referred to as 2G, or second generation.

Surprisingly, GSM is still in use today. Even modern smartphones, iPhones, Samsungs, and others — still rely on GSM for traditional voice services. When you make a normal phone call (by dialing a number, not using WhatsApp or FaceTime), you are still very likely using GSM. On the other hand, data services, internet browsing, video streaming, app usage — are handled by 4G (LTE) and 5G networks.

2G: Second generation

The major shift in 2G was the move from analog to digital technology. This transition enabled several advantages:

- Integration of new services such as SMS.
- Enhanced security through cryptography
- Advanced voice coding techniques that reduced bandwidth usage

In Europe, 2G used FDMA/TDMA, while the U.S. adopted CDMA. Common frequency bands included 850, 900, 1800, and 1900 MHz. Key active networks were GSM in Europe, IS-95 in the U.S., and PDC in Japan.

2.5G: Extended second generation

This phase introduced packet-switched data services, with technologies like GPRS and EDGE in Europe, and IS-95B in the U.S. Data rates improved up to 170 kb/s for GPRS and 384 kb/s for EDGE. Billing shifted from time-based to traffic-based models, reflecting the new data-centric usage.

3G: Third generation

The third generation (3G) of mobile networks has essentially disappeared. In Italy, it was discontinued around three years ago. The reason is simple: 3G didn't offer significant advantages once 4G became widespread.

4G and 5G can deliver faster, more stable data and better voice quality, so operators had no reason to maintain 3G infrastructure. Meanwhile, GSM still has a niche purpose, voice, which justifies keeping it operational.

Note: The "G" in 3G, 4G, 5G stands for generation, not gigabit. The difference between 4G and 5G mainly lies in connection speed and capacity, with 5G capable of achieving gigabit-per-second data rates

3.5G: Extended third generation

This includes evolutions of UMTS such as HSPA (High-Speed Packet Access), combining HSDPA and HSUPA for improved downlink and uplink speeds, and HSPA+, which enhanced the physical layer. Data rates could reach up to 56 Mb/s downstream and 22 Mb/s upstream.

4G: Fourth generation

Known as **LTE** (*Long Term Evolution*), 4G brought significant performance gains:

- Data rates up to 250 Mb/s.
- Use of **MIMO** (*Multiple Input Multiple Output*) and advanced modulation techniques.
- An all-IP architecture, initially supporting data only but later including voice services via **VoLTE**.

Official "4G" standards by ITU require extremely high spectral efficiency and minimum speeds of 1 Gb/s for fixed users and 100 Mb/s for mobile users. Early LTE did not meet these, sometimes called "3.9G," but LTE-Advanced, standardized in 2011, fulfills these requirements.

5G: Fifth generation

5G integrates multiple wireless technologies, evolving from LTE-Advanced, WiFi, and millimeter-wave (mmWave) communications. Its transmission technology is called **New Radio** (NR). It heavily leverages *Software Defined Networking* (**SDN**) for dynamic network control and *Network Function Virtualization* (**NFV**) for flexible service deployment. This architecture enables ultra-fast, low-latency, and highly scalable connectivity, supporting diverse applications from IoT to enhanced mobile broadband.

Emergency Calls and Coverage Obligations

When a mobile operator obtains a license from the government, certain coverage requirements are imposed. Typically, the operator must ensure:

- At least 98% of the country's land area is covered
- At least 95% of the population is reachable

However, in mountainous or remote areas, coverage may still be limited due to the terrain.

To handle emergencies in such regions, civil protection authorities use a separate system called **TETRA** (*Terrestrial Trunked Radio*). TETRA is an independent emergency communication network, not managed by mobile operators, and is primarily used by rescue and emergency services, for example, in mountain rescues or disaster response.

In theory, if you are lost in the mountains and have no mobile coverage, your phone should be able to place an emergency call through the TETRA network. In practice, however, this doesn't always work reliably, depending on the region and network configuration

GSM: the 2nd Generation

During the 1990s, operators made massive investments to deploy the GSM infrastructure, which has proven remarkably durable. Even though the technology is now over 30 years old, it remains viable for voice communication and helps free up valuable frequency spectrum for newer data technologies like 4G and 5G.

Main GSM Services

1. Voice → GSM provides two standard rates for voice:
 - Full Rate (13 kbit/s) – normal call quality
 - Half Rate (6.5 kbit/s) – used when the cell is congested, reducing quality to fit more usersSo, in crowded areas, you may notice a small drop in voice clarity.
2. SMS (Short Message Service) → the famous text message system. While rarely used for personal communication today, SMS remains important for banking and authentication (e.g., receiving verification codes, balance alerts, or transaction notifications).
3. Supplementary Services → Additional features such as call forwarding, call waiting, and recall on busy tone are also handled within the GSM framework.

Why Does My Phone Drop from 4G to 2G During a Call?

You might have noticed that when you make a traditional phone call, your phone's status icon switches from 4G/5G to 2G/GSM, or sometimes just shows signal bars. This happens because your device temporarily disconnects from the data network and uses GSM for the voice call. While some operators support simultaneous voice and data, most traditional GSM calls take over the entire channel. That's why you can't browse the internet or use data-heavy apps during a normal phone call — your device is "locked" to the voice service until the call ends.

(WhatsApp, FaceTime, and Telegram calls are exceptions since they use data, not GSM.)

Voice over LTE (VoLTE)

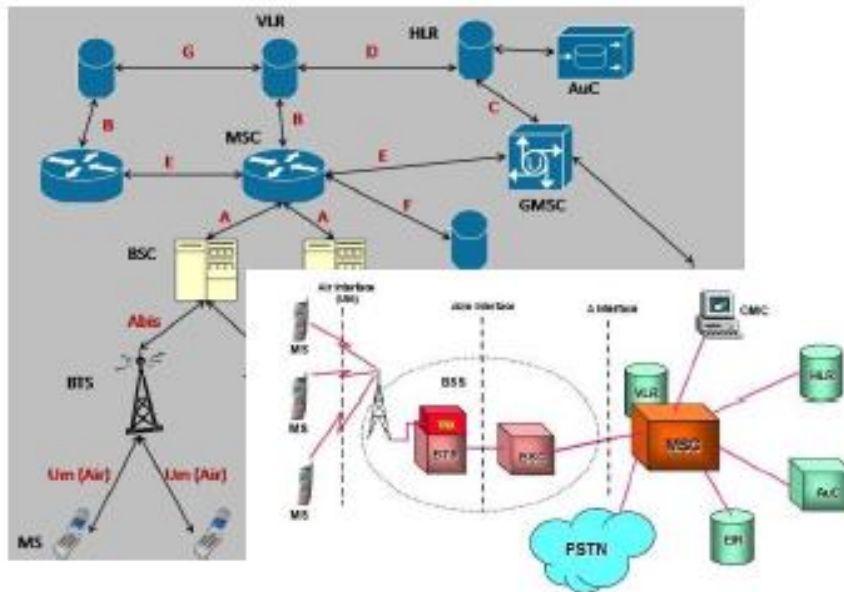
Although GSM is still used for voice, many operators are transitioning to Voice over LTE (VoLTE) — a technology that transmits voice using the 4G data network instead of GSM. VoLTE provides much higher audio quality, often called HD Voice. You can usually tell if your operator is using VoLTE by the clarity of the call. If the voice sounds crystal clear, you're probably on VoLTE. However, not all operators use VoLTE all the time. If network resources are limited, the system may fall back to GSM for voice calls, freeing LTE capacity for data users.

If you are using VoLTE you can still use your data.

GSM network architecture

GSM is primarily designed for voice communication, so when you make traditional phone calls, the GSM network handles them. Data, on the other hand, typically travels over a different network, which we will cover later.

One important point about GSM is that it has relatively few core building blocks, but each is highly complex. At a high level, a cellular network consists of cellular towers, which are integrated into the GSM architecture, along with several key components such as the BSC, MSC, VLR, and HLR.



The GSM network follows the same overall structure as the general cellular architecture:

- Mobile Users – the devices (phones, tablets, etc.)
- Access Network – the radio base stations (cell towers) that connect devices to the network
- Core Network – the operator’s central infrastructure that routes calls and data between users and external networks (like the Internet or other carriers)

Although the layout in GSM diagrams may vary slightly from modern architectures, the functional components remain essentially the same.

The mobile station

In GSM, the mobile device is called a **mobile station (MS)**. Note that earlier generations did not have smartphones—they were simple “dumb phones” capable of making calls and sending messages. Today, the official term used in standards is **User Equipment (UE)**, which can include smartphones, tablets, or smartwatches with a SIM card, mobile devices, mobile phones, antennas on vehicles, so several types depending on the application.

Subscriber Identity Module (SIM)

Every device connecting to a cellular network requires identification to authenticate with the operator, so to connect with the network.

SIM=Smart card with processor and memory (to be inserted in the SIM reader)



Traditionally, this is done via a SIM card, but more recently, eSIMs, virtual SIMs implemented in software, are also common. The SIM/eSIM stores the user’s identity, phone number, subscription details, and security credentials, allowing secure access to the operator’s network.

Base Station Subsystem (BSS)

The BSS manages communication between the mobile station and the network. It includes two main elements:

1. **Base Transceiver Station (BTS)**
 - This is the antenna you see, along with the equipment controlling it.
 - Physical interface in charge of transmission and reception
 - Point of access for the MT
 - It does not broadcast but it handles transmissions and receptions at specific frequencies.
 - Unlike other signal sources (e.g. radio, TV), BTSs transmit signal only towards users that are active, ensuring efficient frequency usage.

- Each BTS supports up to 32 frequency channels: 16 for half uplink (from mobile to network) and 16 for half downlink (from network to mobile).
- 2. **Base Station Controller (BSC)**
 - Resource control on the radio interface: BSCs and BTSs communicate over a wired link
 - A BSC controls a high number of BTSs: from tens to hundreds
 - Typically, BSCs are collocated with one MSC, instead of being located near the BTSs
 - Main functions
 - o It decides frequency assignments (for the BTSs), manages encryption, measures signal quality, and controls handover when a mobile user moves between cells
 - o The BSC also handles voice transcoding, converting analogic voice into digital form at 13 kbps (compared to 64 kbps for traditional landlines) to save bandwidth, though this reduces audio quality slightly.
 - o Paging, frequency allocation, and handover management are also responsibilities of the BSC
 - o Radio resource control: dynamic frequency assignment to BTSs
 - o Signal quality measurement
 - o Management of handover between BTSs controlled by the same BSC

Network and Switching Subsystem (NSS)

The core network includes the Network and Switching Subsystem, which manages call handling, service support, mobility support, and authentication.

The main components are:

1. **Mobile Switching Center (MSC)**
 - There are typically few MSCs per operator; each covers a large area.
 - The MSC routes calls from the mobile terminal to other users, whether within the same operator or to external networks.
 - It keeps track of mobile terminal locations and coordinates with VLR and HLR.
 - Mobility support
 - Call routing between MT
 - GMSC: Interface between the GSM network and other networks
2. **Home Location Register (HLR)**
 - A central database storing the static identity of all subscribers: phone number, enabled services, subscription, billing information, and security parameters (that is not the SIM).
 - The HLR works with the Authentication Center (AuC) to store cryptographic keys for secure communication and all the Dynamic data to handle user mobility (e.g., VLR identifier)
3. **Authentication Center (AuC)**
 - Authentication based on challenge & response protocol
 - Generations of encryption keys for over-the-air communication
4. **Visitor Location Register (VLR)**
 - A temporary database that tracks the current location of mobile devices within the MSC's area: IDs, on/off status, LAI, routing information, security
 - The VLR ensures that calls and services reach the user wherever they are within the network.
5. **Equipment Identity Register**
 - Database of stolen devices, not SIM cards, but the actual device.
 - Each phone has a unique identifier. If your phone is stolen, you can report it to the police, and with the report, the operator can block the device's identifier. This means that even if someone replaces the SIM card, the stolen device cannot access the network for calls or data. Every time your phone connects to a network, it transmits both your SIM ID and your device ID, allowing the network to enforce these blocks.

Call Procedure Overview

When a user makes a call:

1. The mobile station dials a number and sends a request to the BTS.
2. The BSC manages the communication and queries the MSC to locate the called user.

3. The MSC consults the HLR to retrieve the subscriber's identity and subscription info, then checks the VLR to find the user's current location.
4. The MSC switches the call through the correct BSC/BTS path to reach the receiver

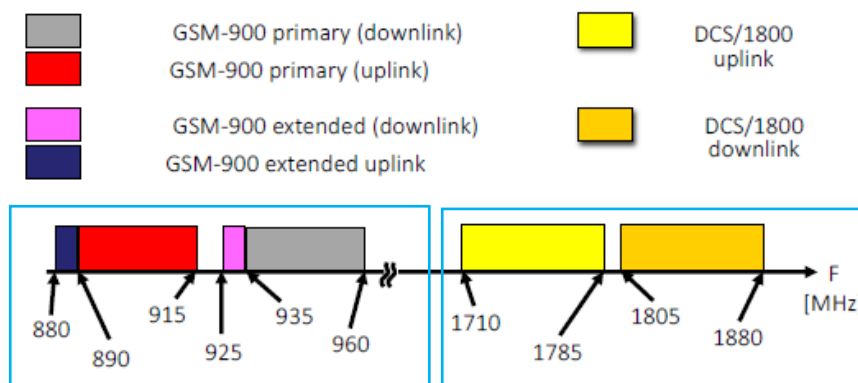
This division ensures:

- The HLR keeps track of who the user is (static identity).
- The VLR tracks where the user currently is (dynamic location).

Even when a mobile is idle, it continuously updates its location, allowing the network to route incoming calls efficiently. Your phone can always be located. Anytime you receive a call, your approximate location is known by the network.

When a phone moves out of coverage, say you spend a day in the mountains, the last cell your phone was connected to becomes your "last known location." If the phone doesn't respond, the network marks it as **out of coverage**. If your phone doesn't update its location for a while (typically 1–2 days), the VLR clears this cached information. This also applies when traveling abroad: if your phone remains off, the system will eventually purge its last known location.

GSM frequencies (Europe)



Allocated frequencies: 850, 900, 1800, 1900 MHz.

GSM uses a **Frequency Division Duplex (FDD)** system: part of the spectrum is for **downlink** (network → device) and part for **uplink** (device → network). The uplink frequencies are deliberately lower to save battery power, since transmitting requires energy from the mobile device, whereas the downlink comes from the operator's antenna, which can use more power.

GSM physical channels

Transmission in GSM is organized using both Frequency Division Multiple Access (FDMA) and Time Division Multiple Access (TDMA). Each frequency channel is **200 kHz wide**. Each FDM channel is divided into TDM frames. Each TDM frame is divided into 8 slots.

→ Frequency + time slot = **physical channel**

For instance, a Base Transceiver Station (BTS) can allocate 16 uplink and 16 downlink frequencies. Each frequency is further divided over time into frames, and each frame contains 8 time slots.

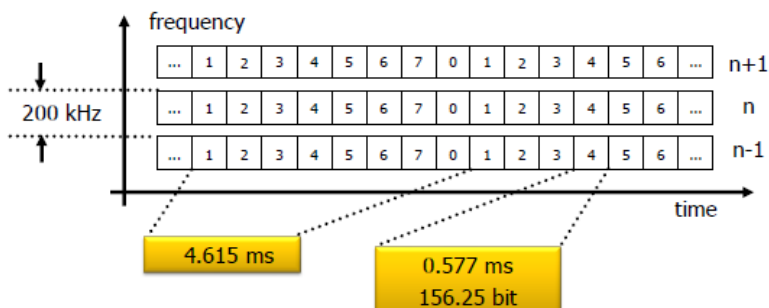
When you make a call, your mobile is assigned one frequency for uplink (F1) and another for downlink (F2). Your voice is converted from analog to digital and transmitted in bits within a **single slot in each frame**. The other 7 slots in the frame are used by other users in the same cell. The network ensures that you always have your slot for the entire call. This is called **circuit switching**, which reserves resources for the call whether you are actively speaking or not.

Transmission is organized into burst, i.e. blocks of data that are transmitted on a physical channel. Bursts are similar to packets but it is still circuit switching: we are setting aside resources from a communication, whether we are using it or not.

The raw transmission speed within a slot is **271 kbps**, but because you only transmit during your assigned time slot (one out of eight), the effective throughput is much lower. Additional overhead from channel coding, encryption, and other features further reduces the actual voice data rate to around **13 kbps**.

Channel access - summary of FDM/TDM schema

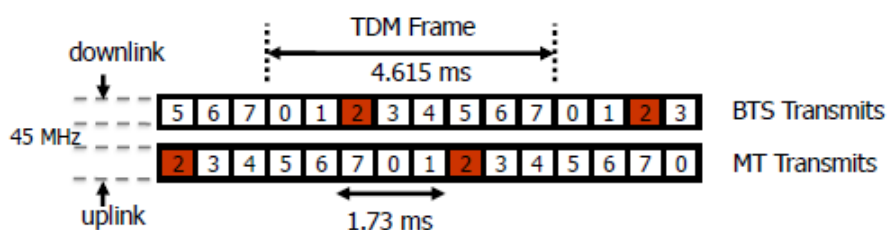
- Slot time = 0.577 ms - 1 slot carries 156.25 bits (i.e., 0.577×271 kbps)
- Each time slot carries 1 transmission burst
- Slots are grouped into TDM frames, of 8 slots each
- Frame = 4.615 ms



Key parameters to note: each frame lasts about 4.6 milliseconds, and each frame contains 8 slots. The channel bandwidth of 200 kHz limits the modulation options, which is why later generations like 4G and 5G use much wider channels for higher data rates.

Channel access – GSM frame

- Circuit-switched network each call full rate (13kb/s) is assigned 1 slot per frame in UL and 1 slot per frame in DL
- Each MT transmits on a carrier for one time slot, i.e., one burst of data, and remains silent during the other 7 slots
- Frames on UL and DL are synchronized on a time slot basis and shifted by 3 slots
- Separating UL/DL transmissions in frequency and time allows the use of one transceiver only

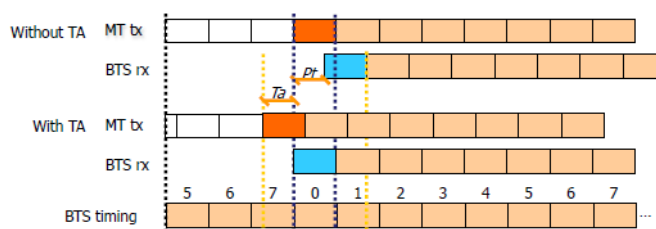


In GSM, each communication structure allocates resources in both frequency and time. Imagine a checkerboard where each square represents a small time-frequency slot. Each user gets one of these squares for transmission in the uplink and downlink. If the system runs out of slots, say, at a crowded stadium, your call might not go through until another user ends their call, freeing up a slot. This is the essence of circuit switching: slots are reserved for the entire duration of a call, even if you don't fully use them, which can be inefficient.

Each mobile terminal transmits on a carrier frequency, which is essentially a sinusoidal wave that gets modulated in amplitude, frequency, or phase. Importantly, our devices only have one transceiver, so they constantly switch between transmitting and receiving. In GSM, uplink and downlink slots are slightly offset to allow this switch, typically happening every 2 milliseconds, so fast that the user doesn't notice it. This design efficiently uses hardware by requiring only a single circuit to handle both transmission and reception.

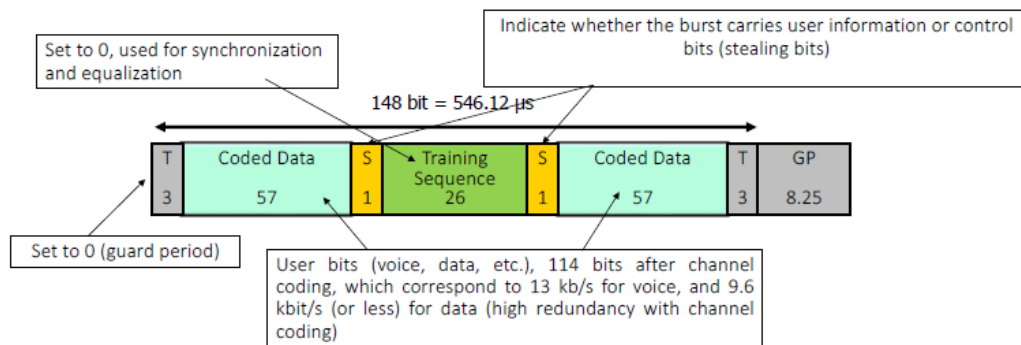
Channel access – Timing advance

A crucial concept in GSM is **timing advance**, which solves the problem of different distances between mobile terminals and the base station (BTS). For example, one terminal might be 100 meters away, while another is 15 kilometers distant. Signals from the far terminal take longer to reach the BTS. If both terminals transmit based on the same timing, their signals could overlap, causing interference: bursts transmitted by MTs might arrive at the BTS when the slot is already finished. Timing advance allows the BTS to calculate the



distance based on signal delay and instruct terminals to transmit slightly earlier, ensuring all signals arrive within their assigned time slots → Transmission at the MT starts before the real timeslot beginning.

Regular burst structure



GSM Logical channels

There's a distinction between physical channels and logical channels.

- **Physical channels** → it is the combination of timeslot and carrier, so are the actual frequencies and time slots used for transmission. For instance, a terminal communicates on a specific carrier at a particular time slot, possibly adjusted by timing advance. 8 physical channels per carrier: timeslots 0 - 7
- **Logical channels** describe the information or purpose carried by those physical resources. For example, some slots on a downlink frequency (F0) carry the operator's identity, the cell's identity, or are reserved for new terminals to register with the network. Logical channels are mapped into physical channels according to proper criteria, such as "slot 2 of F0 carries the operator identity."

A physical channel is defined by three main components:

1. Frequency – the carrier frequency used for transmission.
2. Time slot – the specific time interval in which a transmission occurs.
3. Transmission method – how the data is transmitted, i.e., the portion of the channel reserved for your communication.

In simple terms, when you want to communicate, your terminal transmits on a specific frequency at a specific time slot, and that combination is your physical channel.

Logical channels, on the other hand, define the meaning or purpose of a specific physical channel. They map specific types of information onto the physical resources.

For example, consider a base station (BTS) with 16 frequencies in a cell. One of these downlink frequencies—called F0—is not used for normal voice transmission to phones. Instead, it serves a special purpose:

- F0 is the main broadcast frequency in the cell, making it easy for new devices to detect.
- When your phone enters a cell, it detects F0 because it is the strongest signal.
- The base station then broadcasts the identity of the network operator (e.g., Vodafone or TIM) and the identity of the cell on this frequency.
- Your phone uses this information to identify the cell and register itself with the operator.

Even though F0 is a physical frequency, its individual time slots carry specific information. For instance:

- Slot 2 may carry the operator's identity.
- Slot 4 may carry the cell identity.

This mapping of a purpose or meaning to a frequency and time slot is what defines a logical channel.

There's a similar mechanism for the uplink F0 frequency. This frequency is used when a new device first enters a cell and wants to register. Since the device has not yet been assigned a specific slot. The uplink F0 frequency has unallocated slots that any new user can use to transmit their identity. The device randomly picks a slot to announce itself: "Hey, I'm here, and this is my ID."

Logical channels are divided in two types:

- **Control channels**, carry control information (related to user or network) , so handle signaling, user registration, handovers, network parameters, and signal quality. They tell the mobile terminal how well the network receives its signal and whether a handover is necessary.
 - o Network signaling: cell parameters, synchronization, receiver tuning
 - o User signaling: call control, signal quality control (measurement delivery)
- **Traffic channels**, carry user information like voice calls.

A key point is that new terminals use unallocated slots in the F0 frequency to register. If multiple terminals try to register at the same slot simultaneously, collisions can occur. This is reminiscent of the ALOHA protocol, a classic technique from the 1970s still relevant for managing random access in shared channels.

The reality is that 5G adoption isn't as widespread as it might seem. Many network operators have not yet fully migrated the architecture of their networks from 4G to 5G. So, even though when you look at your phone today you might see a 5G icon in the corner of the screen, you are actually connected to a 5G radio access network. However, the core of the network—the part managed by operators, at least in Italy and many other Western countries—still relies on the 4G architecture. We are essentially looking at a fourth-generation network with a glimpse into the fifth.

LTE: the 4th Generation

Today, when we talk about mobile data, we mostly rely on fourth-generation networks, or LTE (Long-Term Evolution), which is still widely in use. Of course, 5G is the new generation, but as we'll see later, the basic architecture of 5G is very similar to LTE.

LTE (Long-Term Evolution) came after 3G, which, despite its ambitions, didn't fully deliver on its promises. The third generation aimed to provide video calls and high-speed data, but in practice, speeds were limited and video calls were rarely adopted due to cost constraints. LTE, starting from Release 10, marked a real step forward. This release signified a significant point in the evolution of the standardization process, as it introduced the first LTE standard.

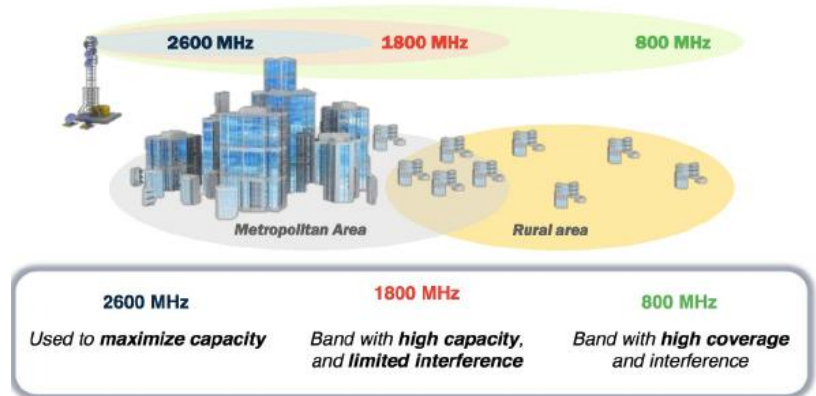
LTE (Long-Term Evolution) is a 4G standard standardized by 3GPP, the international body that defines cellular standards. 3GPP releases standards in waves, called *releases*, updating the specifications every few years. For LTE, the defining standard was Release 8, finalized around 2010, with commercial LTE networks appearing around 2012. There is typically a two-year gap between a 3GPP release being frozen and the first commercial devices becoming available, as manufacturers and network operators need time to implement the new standards.

- LTE supports high speeds—up to 300 Mbps downlink and 50 Mbps uplink using 64-QAM modulation, which combines amplitude and phase modulation. 64 indicates $2^6 = 64$ possible combinations meaning each signal represents 6 bits of information.
- Latency is low, typically around 10 milliseconds, meaning data from a server reaches your device very quickly. It refers to the time it takes for data to travel from the operator's network to your device. For example, when you load a new picture on Instagram, it takes about 10 milliseconds for the data to be delivered to your phone from your operator's network.
- Replaces WCDMA with OFDMA (DL) and SC-FDM (UL)
- Leverages high-order MIMO
- LTE channels are up to 20 MHz wide, significantly larger than GSM channels, reflecting the higher data rates and more complex modulation. In GSM, channels were narrower because the modulation only carried 1–2 bits per symbol, whereas LTE carries 6 bits per symbol. This allows LTE to transmit more data simultaneously, which is crucial for high-speed mobile broadband.
- Scalable frequency bands from 1.4MHz to 20 MHz

- The core network in LTE is fully IP-based (HSDPA), supporting modern internet traffic, meaning that all communications—including voice, video, and data—are transmitted over IP. This is a major difference compared to previous generations, where voice and data were often handled separately.

Usage of Frequency Bands

LTE operates on specific frequency bands. The 800 MHz and 1.8 GHz bands were already in use with GSM, but LTE introduced the 2.6 GHz band, which has been used for over a decade. Since higher frequency bands experience higher path loss, cells in the 2.6 GHz range cover smaller areas compared to 1.8 GHz cells.

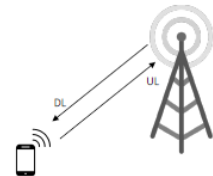


LTE Terminology

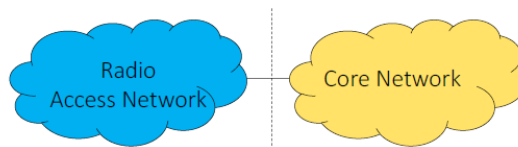
- Downlink (DL): data going from the network to the user device
- Uplink (UL): data going from the user device to the network

LTE and future generations distinguish between:

- **User plane:** all the operations related to the transport of user data in DL and UL – **ACCESS STRATUM**
- **Control plane:** all operations related to the set-up, control and maintenance of communications between user and network – **NON ACCESS STRATUM**

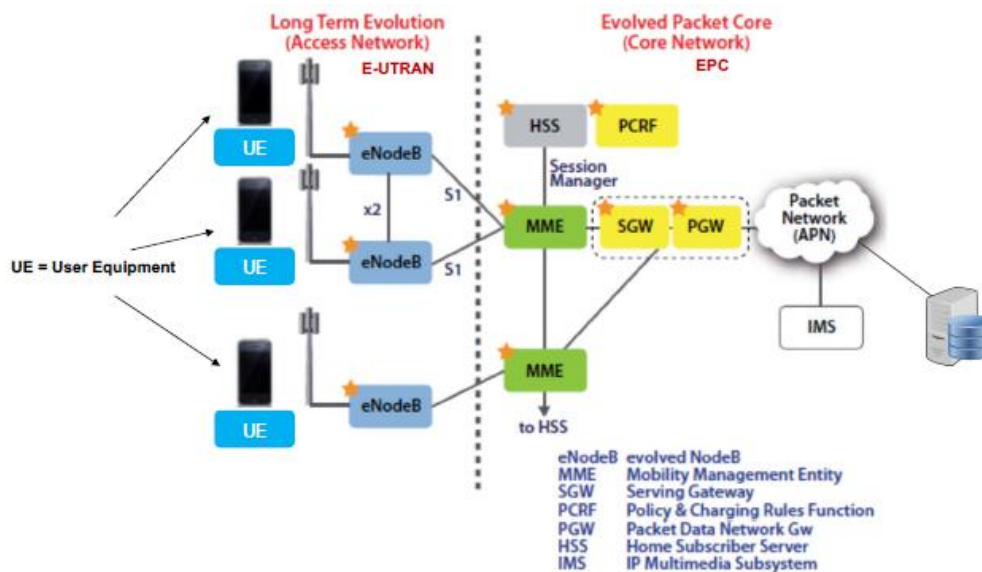


LTE Architecture



Radio Access Network (RAN): it includes all the devices involved with the direct interaction with user devices (such as UEs, antennas and the equipment controlling them). In LTE, it is called the **E-UTRAN (Evolved Universal Terrestrial Radio Access Network)**.

Core Network (CN): it includes all the devices responsible for the transport of data to/from the Internet or toward other user devices (routing, connections to other network and access to the Internet). In LTE, it is called **EPC (Evolved Packet Core)**.



Starting with E-UTRAN, your devices connect to eNodeBs (evolved Node B). The term “evolved” comes from historical naming conventions: in 3G, the equivalent component was called Node B to differentiate it from GSM’s Node A. LTE simply improved upon this and named it “**evolved Node B.**” The eNodeB controls the antennas that your devices connect to and is part of the operator’s access network. The interfaces between these network components—like S1 between an eNodeB and the MME, or X2 between eNodeBs—refer to the communication protocols used, not physical connections.

- S1 is the protocol used between an eNodeB and the MME (Mobility Management Entity)
- X2 is the protocol used for communication between different eNodeBs.

Now, let’s move to the core network, the EPC. The term “packet core” originates from 3G, which was the first generation to handle data. GSM (2G) was voice-only, relying on circuit switching. LTE, as the evolution of 3G, inherited this packet-based architecture—hence the name **Evolved Packet Core**. LTE is all about “evolved” components: eNodeB, evolved packet core, etc.

Design Principles

The LTE architecture was built around a clean-slate design philosophy, meaning it was developed from the ground up, without being constrained by the legacy circuit-switched concepts of earlier generations. The idea was to create a fully IP-based network capable of efficiently handling all types of traffic, regardless of their Quality of Service (QoS) requirements.

In this architecture, packet-switched transport is used for all types of communication, including conversational, streaming, real-time, non-real-time, and background traffic. This unification simplifies the network structure and enables more flexible management of different services over a common framework.

A crucial element of LTE design is **radio resource management (RRM)**. RRM ensures that radio resources are dynamically allocated and optimized to guarantee end-to-end QoS, provide efficient transport for higher layers, and enable load sharing and balancing among different cells or access technologies. It also includes mechanisms for policy management and enforcement, especially in heterogeneous networks that integrate multiple radio access technologies (RATs), such as LTE, UMTS, and GSM.

Another fundamental design goal was seamless integration with existing 3GPP 2G and 3G networks. This backward compatibility allows users and operators to transition smoothly to LTE without losing access to older systems or services.

Functions

The LTE system supports a wide range of network functions, each designed to handle a specific aspect of communication and management:

- Network access control, including network selection, authentication, authorization, admission control, policy and charging enforcement, and lawful interception
- Packet routing and transfer
- Security, including ciphering, integrity protection, and network interface physical link protection
- Mobility management to keep track of the current location of the UE
- Radio resource management to assign, reassign, and release radio resources taking into account single and multi-cell aspects
- Network management to support operation and maintenance
- IP networking functions, connections of eNodeBs, E-UTRAN sharing, emergency session support, among others

Components

- **Mobility Management Entity (MME)** → This is part of the control plane. It doesn't handle packet routing but manages connections, mobility, and key control functions. The MME tracks device mobility, manages handovers when a device moves between cells, locates devices for incoming calls or data, and supports user equipment context, identity, authentication, and authorization in coordination with the **Home Subscriber Server (HSS)**. Every operator may have multiple MMEs distributed across data centers, depending on the network size. For example, a national operator might have two main data centers (e.g., Milan and Rome) with five or six MMEs in each. Note that an eNodeB can control several antennas, sometimes up to ten. They mainly handle Non-Access Stratum (NAS) procedures, which can be divided into two broad categories. The first category includes all functions related to **bearer** management, where a bearer can be understood as a logical communication tunnel that carries user data with specific quality-of-service characteristics from the device through the EPC. The MME manages these bearers—setting them up, modifying them during mobility events, and tearing them down. The second category involves connection and mobility management, covering the procedures that establish, maintain, and update the user's connection as the device moves or changes state within the network.
- **Serving Gateway (S-GW)** → This belongs to the user plane and handles the routing of data. Receives and sends packets between the eNodeB and the core network. Since the SGW does not know user locations on its own, it works closely with the MME to know where a user is located and update routing tables accordingly. The SGW also serves as an anchor point for intra LTE-mobility, forwarding data when a user moves between cells. Incoming traffic always flows through the SGW, which then forwards it to the appropriate eNodeB based on the user's current location.

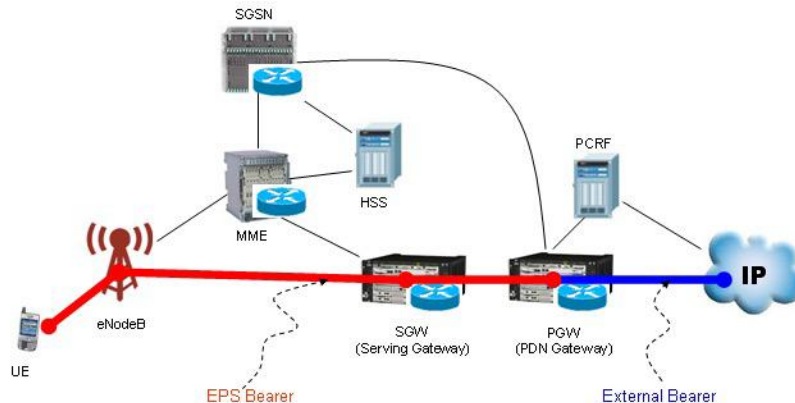
The SGW is also a point where lawful intercept can occur. Mobile communications are encrypted between the UE and the operator, and in addition, most traffic is encrypted end-to-end (e.g., via HTTPS). Still, the operator can log metadata such as which servers were contacted, timestamps, and cell locations. When ordered by a judge, the operator can log and provide data associated with a given user. This process is strictly regulated and requires a subpoena.

- **Packet Data Network Gateway (P-GW)** → Also part of the user plane, the PGW connects the LTE network (EPC) to external networks, including the internet. It acts like a **border router** that performs UE IP assignment, per user packet filtering or per-flow policies, NAT services and QoS enforcement. Often, the SGW and PGW reside on the same physical machine but perform distinct functions. The PGW serves as the mobility anchor whenever the user moves outside the home network, whether this involves roaming through a visited operator's network or transitioning via non-3GPP access technologies. As traffic to and from the Internet passes through the PGW, lawful intercept may also occur here.
- **Home Subscriber Server (HSS)** → This is the database where subscriber and user information is stored, including personal and accounting details. It works with the MME for authentication and authorization whenever a new user connects to the network. Similar to HLR (Home Location Register) in GSM architecture.

The LTE Bearers

A bearer in LTE is essentially a tunnel, very similar to an encapsulation tunnel such as those used in IPv4/IPv6 migration. All user traffic travels within such tunnels from the moment it leaves the device, across the entire operator network, and often until it reaches the Internet.

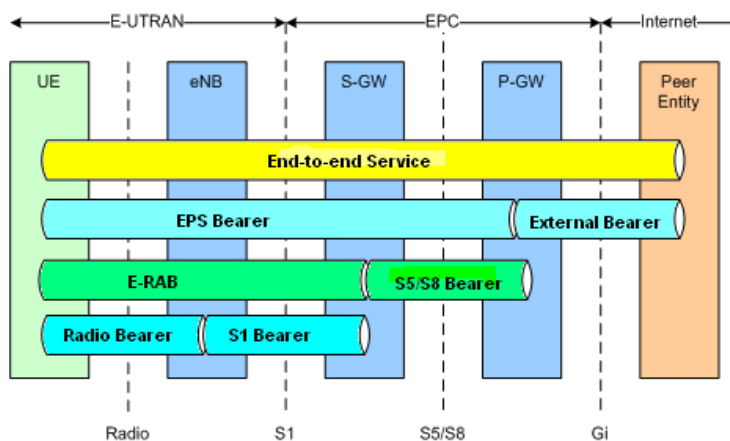
When a UE powers on and attaches to the network, the EPC creates a default bearer between the UE and the PGW. You can think of it as a pipeline with joints that allow the data path to be shifted without breaking continuity. Some operators may also create dedicated bearers for specific traffic types based on quality-of-service (QoS) requirements, but not all operators allow this, instead the default bearer is always present e.g., viewing a streaming video over the Internet can be done over a dedicated bearer. Dedicated bearers can use a bandwidth guarantee (a guaranteed bit rate, or GBR) tunnel.



The bearer is a concatenated tunnel consisting of three portions established in the following order:

- The **radio bearer** connects the UE to the eNodeB. This bearer follows the mobile user under the direction of the MME as the radio network performs handovers when the user moves from one cell to another
- The **S5 bearer** connects the Serving Gateway (S-GW) to the P-GW.
The tunnel can extend from P-GW to the Internet, called **S8 bearer** in this case; it is used when the SGW and PGW are in different networks, e.g. during roaming
- The **S1 bearer** connects the eNodeB with the S-GW. Handover establishes a new S1 bearer for end-to-end connectivity.

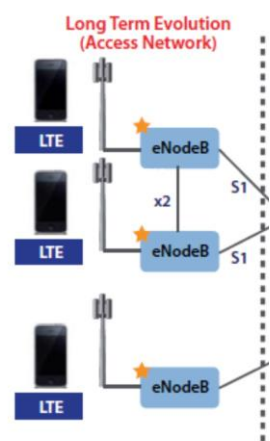
The idea of a “pipeline with joints” captures how LTE bearers support mobility. As the UE moves from one cell to another, the network only needs to update the radio bearer and the S1 bearer, since these are the segments directly tied to the serving eNodeB. The upper parts of the end-to-end tunnel remain unchanged, allowing the system to preserve its structure without rebuilding it at every handover. This modular design makes mobility efficient and ensures seamless data transport across the network, while also supporting routing needs and enabling lawful interception when required.



E-UTRAN

The radio access network in LTE consists of eNodeBs communicate through the X2 interface, which is used to coordinate the handover of radio bearers and the exchange of user-related data between eNodeBs. Consists of two types : **X2 control** and **X2 user**.

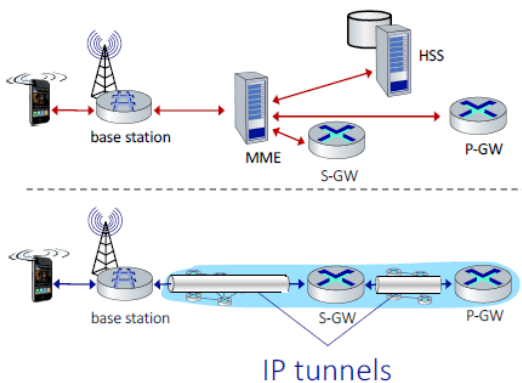
The main functions of the eNodeB include:



- Radio resource management: related to the radio bearer such as radio bearer control, radio mobility control, scheduling and dynamic allocation of radio resource at uplink and downlink.
 - o Radio resources refer to specific frequencies at specific times that a user can use to send or receive data.
- Header compression
- Security
- Connectivity to EPC

LTE Data Plane/Control Plane

Separation



The **control plane** runs protocols for mobility, security, and authentication, involving mainly the MME and the HSS. The MME communicates with the Serving Gateway, sending it control-plane information, especially during mobility events such as handovers. When a user moves, the MME instructs the S-GW to update its routing tables accordingly.

The **data plane** handles the actual transmission of user data through IP tunnels. On both the UE and the eNodeB, data originating from the application and transport layers is passed down through the lower layers of the network stack. New protocols at the link and physical layers, combined with extensive use of tunneling, enable efficient mobility support.

Data plane protocol stack

First hop

In the radio access network, the application and transport layers run on the user's device, while lower layers operate both on the UE and on the eNodeB. In the downlink direction—network to UE—data transmission uses a mix of frequency-division and time-division multiplexing.

This approach differs from GSM. In GSM, each user received a dedicated frequency for the duration of a call, and within that frequency the user was assigned a periodic time slot for transmitting or receiving data. LTE works differently: in both uplink and downlink, each user is dynamically assigned a specific frequency for a very short amount of time, and this frequency may change at the next transmission opportunity.

You can imagine multiple available frequencies over time, arranged like a grid of time–frequency resources. LTE assigns one frequency at one time to a user—say, frequency F1 at time T1—meaning the user must tune to F1 at T1 to receive its data. The device knows when and where to listen because the eNodeB periodically broadcasts scheduling information. At the start of every scheduling period, the eNodeB tells each user which frequencies and time intervals it will use to receive or transmit. The eNodeB essentially fills a “chessboard” of time–frequency resources: user 5 gets certain slots, user 7 gets others, user 11 gets others, and so on.

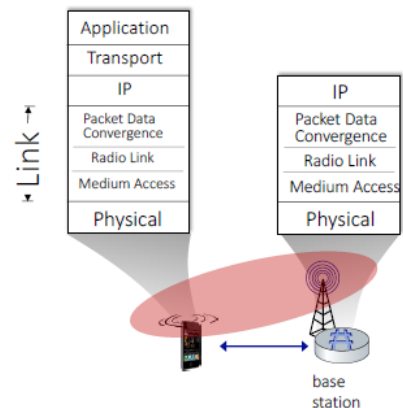
In the uplink direction, the mechanism is similar: at the beginning of each period, the eNodeB tells every device exactly when and on which frequency it may transmit. This scheme combines frequency-division and time-division multiplexing, but in a way that is fundamentally different from GSM. GSM allocated fixed uplink and downlink frequencies for the entire duration of a call; LTE continuously changes frequency assignments. This enables finer-grained and more efficient use of radio bandwidth. LTE allocates resources only when needed, unlike GSM where time slots remain reserved whether they are used or not.

The algorithms that operators use to fill this scheduling “chessboard” are proprietary. A well-designed algorithm results in a faster, more responsive data network because it efficiently fills unused gaps; a poorly designed one wastes radio resources. This is what “radio resource management” refers to: deciding which user gets which frequency at which time, something entirely handled by the eNodeB.

The combination of TDM and FDM used in LTE is formally called **OFDM (Orthogonal Frequency Division Multiplexing)**. “Orthogonal” means that although the frequencies are close to each other, they do not interfere. The allocation typically repeats every 10 milliseconds, at which point a new time–frequency grid is announced, and devices are told where to listen or transmit.

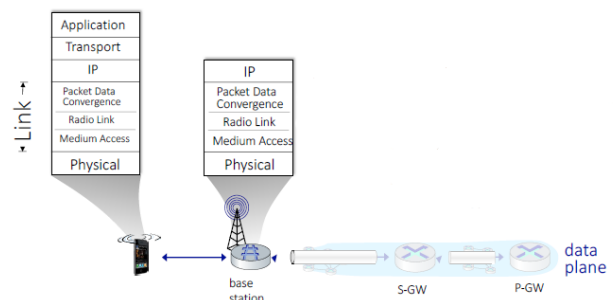
To resume → LTE radio access network:

- downstream channel: FDM, TDM within frequency channel (OFDM – orthogona frequency division multiplexing)
 - o “orthogonal”: minimal interference between channels
- upstream: FDM, TDM similar to OFDM
 - o each active mobile device is allocated two or more 0.5-ms time slots over 12 frequencies
 - o scheduling algorithm not standardized, up to operator
 - o 100’s Mbps per device possible



Above the physical layer, LTE includes several additional layers:

- the **Medium Access Control (MAC)** layer. While the physical layer decides the allocation of the “checkerboard” (the frequency-time slots), the MAC layer provides information on how much data must be sent to each user; requesting, use of radio transmission slots
- Above the MAC layer is the **Radio Link Control (RLC)** protocol, which handles fragmentation and reassembly of data, reliable data transfer
- On top of that, the **Packet Data Convergence Protocol (PDCP)** manages header compression and encryption, because remember, all data over the air is encrypted to ensure security.



In LTE, just like in GSM, we distinguish between physical channels and logical channels, and the meaning of each channel depends on what it carries. In GSM, for example, some frequencies were allocated for voice while others were reserved for network operations, such as user registration. LTE works similarly: some frequencies are not part of the periodic 10-ms allocation because they are reserved for specific purposes, such as allowing unregistered users to access the network.

Physical Channels

- Transmitted bits fit into the frame structure according to predefined subdivisions called Physical Channels
- Each channel conveys specific information related to: user data, TX/RX parameters, eNB identity, network control etc., as well as to the format of the channel itself (i.e., FDD or TDD)
- Each physical channel is mapped into a portion of the LTE subframe
- Physical channels are divided into Downlink and Uplink Channels. Each U/D channel is further divided into Data and Control

When you turn on your device, it first listens to the radio waves to detect the cell of your operator. Then it tunes to a specific frequency to register. At this point, your device cannot use pre-allocated frequencies or time slots, because the network has not yet recognized it; it relies on these special free channels. Other frequencies, instead, belong to the scheduled grid and are updated every 10 milliseconds.

Both uplink and downlink have control-plane channels and user-plane channels:

- Control plane channels carry network-related information, such as which operator a cell belongs to, or which frequency the user should use to register.
- Data plane channels are allocated every 10 ms for actual user data transmission. Uplink channels follow the same principle, including acknowledgment channels where the user confirms that data was received.

For example there are channels for acknowledgments as well—specific frequencies where the user checks whether its data has been received, and vice versa.

LTE Physical Downlink Channels

- Physical Broadcast Channel (PBCH)
 - o MIB (Master Information Block) and RACH parameters (Radio Access Channel)
- Physical Control Format Indicator Channel (PCFICH)
 - o Details on PDCCH format (e.g., number of symbols used)
- Physical Hybrid ARQ Indicator Channel (PHICH)
 - o Ack/Nack to uplink frame transmission
- Physical Downlink Control Channel (PDCCH)
 - o Downlink resource scheduling
 - o Uplink power control instructions
 - o Uplink resource grant
 - o Indication for paging or system information
- Physical Downlink Shared Channel (PDSCH)
 - o User data and SIB (System Information Block) for cell information

} control
} data

LTE Physical Uplink Channels (control)

- Physical Random Access Channel (PRACH)
 - o It carries the random access preamble a UE sends to access the network in nonsynchronized mode
 - o used to allow the UE to synchronize timing with the eNodeB
- Physical Uplink Control Channel (PUCCH): Signaling includes HARQ ACK/NACK, channel quality indicators (CQI), MIMO feedback and scheduling requests for uplink transmission

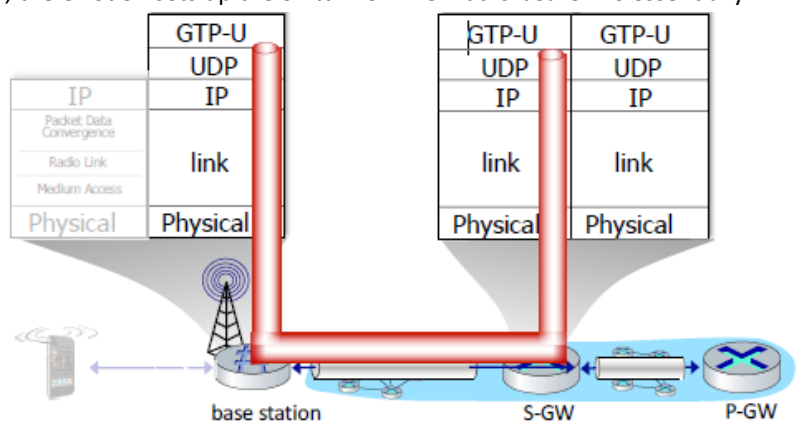
LTE Physical Uplink Channels (data)

Physical Uplink Shared Channel (PUSCH)

- It carries user data
- The uplink scheduling TTI is 1 ms; it is possible to ‘bundle’ a group of 4 TTIs to improve performance at cell edge and reduce higherlayer protocol overhead

Packet core

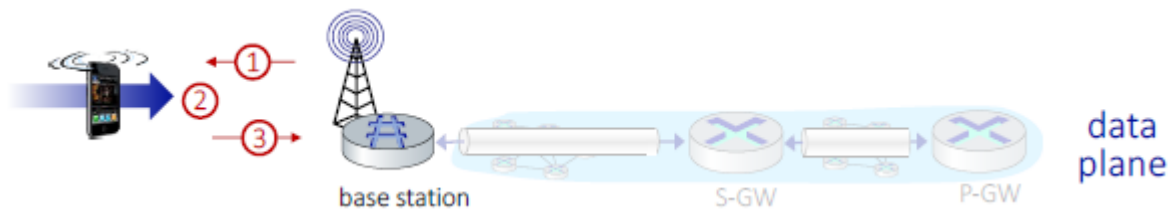
In the packet core, once data reaches the eNodeB, the eNodeB sets up the S1 tunnel. The “radio bearer” is essentially the virtual abstraction of the radio resources allocated every 10 milliseconds. Between the eNodeB and the S-GW, communication typically takes place over optical fiber. The S1 tunnel uses the GPRS Tunneling Protocol (GTP), encapsulating user data into UDP packets transported from the eNodeB to the S-GW. At the S-GW, data is decapsulated and then re-encapsulated to form the S5 bearer. S-GW re-tunnels datagrams to P-GW. During mobility, only the tunnel endpoints change as the mobile user moves.



LTE data plane: associating with a BS

When a user powers on their device, it first listens for a Primary Synchronization Signal (PSS) broadcast by the eNodeB every 5 ms across all frequencies. The PSS indicates the operator of the cell and points to another frequency where all the cell information can be found: frequencies for data transmission, locations of other channels, and the overall organization of the cell.

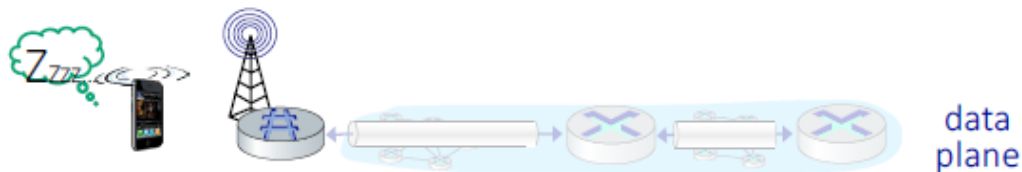
A device may detect multiple PSS signals from various operators but selects only the one from its own operator. If there are multiple nearby cells of the same operator, the device typically selects the cell with the strongest signal, which is likely the nearest one.



1. BS broadcasts primary synch signal every 5 ms on all frequencies
 - a. BSs from multiple carriers may be broadcasting synch signals
2. mobile finds a primary synch signal, then locates 2nd synch signal on this freq.
 - a. mobile then finds info broadcast by BS: channel bandwidth, configurations; BS's cellular carrier info
 - b. mobile may get info from multiple base stations, multiple cellular networks
3. mobile selects which BS to associate with (e.g., preference for home carrier)
4. more steps still needed to authenticate, establish state, set up data plane

LTE mobiles: sleep modes

The control plane also manages **device sleep modes**. The eNodeB can detect inactive users and put their devices into a light sleep mode, where the transceiver is turned off periodically (e.g., every 100 ms) and then wakes up to check for new messages. Deeper sleep modes may turn the device on less frequently, such as every 10 seconds, requiring re-association if messages are waiting. This is handled through a state machine in which the device transitions from active, to light sleep, to deep sleep after increasing periods of inactivity. When waking from deep sleep, the device may need to re-establish association.



As in WiFi, Bluetooth LTE mobile may put radio to “sleep” to conserve battery:

- light sleep: after 100's msec of inactivity
 - o wake up periodically (100's msec) to check for downstream transmissions
- deep sleep: after 5-10 secs of inactivity
 - o mobile may change cells while deep sleeping – need to re-establish association

Mobility

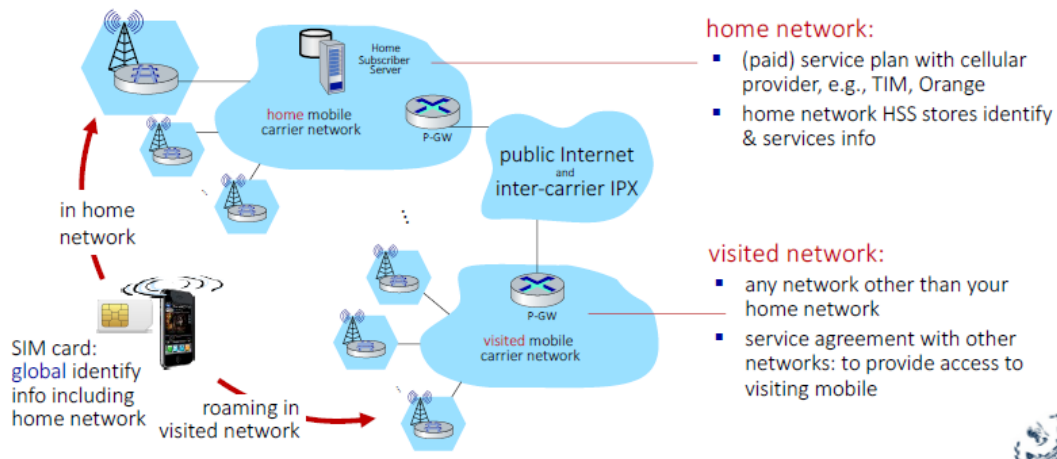
All these LTE features are necessary because mobility is one of the biggest challenges in cellular networks. When a user moves, the network must keep track of their location. Unlike contacting someone at a fixed address or number, mobile devices change position constantly, and the network must track and update their location in real time.

In 4G and 5G networks, this is handled by the home network, which knows the user's identity and subscription details (via the HSS), and the MME, which tracks the user's movements. When traveling abroad, the device connects to a visitor network, which coordinates with the home network to ensure the user can still access data, receive emails, and get messages. The visitor network knows who the user is and relays messages to their device seamlessly.

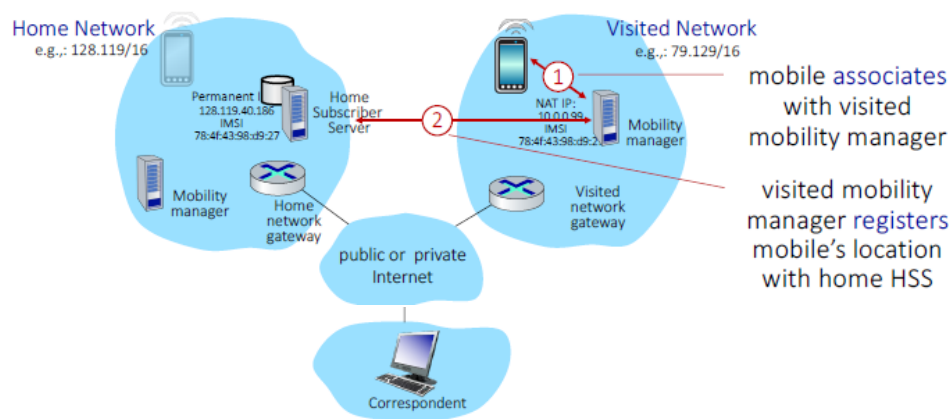
The interaction between the home network and visitor network is what makes international roaming and mobility support in LTE possible.

The main question is: how do we manage mobility between a user's home network and a visited network? In other words, how can a user's context and information be carried from their home operator to the operator of the country they are currently visiting or temporarily residing in? Of course, this situation is temporary, not permanent.

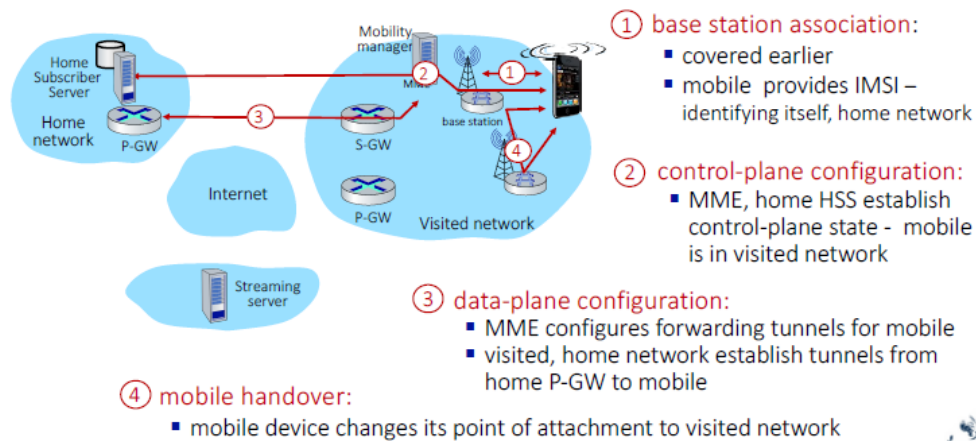
Home network, visited network: 4G/5G



Home network, visited network: generic



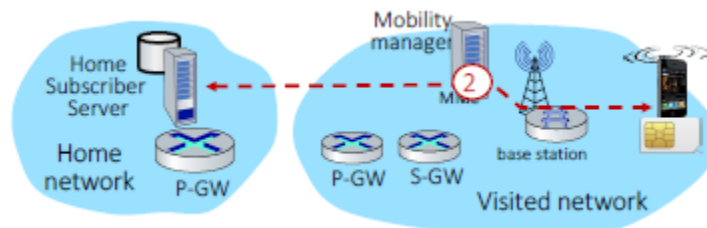
- Step 1: Registration in the Visited Network** → The first step is that the home network must know where you are. This happens the moment your device registers with the local (visited) network. When your device connects, the MME (Mobility Management Entity) in the visited network takes note of your registration. At this point, you're just a guest, you haven't yet been authorized to use the network's resources fully. Registration only means that your device has been assigned a set of frequencies to communicate with the network. However, your data cannot yet travel beyond the local cell, because the visited network still needs to verify your identity and authorization status.
- Step 2: Identification via IMSI** → So how does the MME in the visited network find out who you are and where you come from? When you register, your device transmits an identifier called the IMSI (International Mobile Subscriber Identity) — a bit string that uniquely represents your identity within the cellular system. The IMSI is composed of several fields, one of which identifies your home country and your operator. For example, suppose your IMSI reveals that you are from Italy and your operator is Vodafone. The visited network's MME can read this and say, "Alright, this user belongs to Vodafone Italy." The MME then contacts the Vodafone Home Subscriber Server (HSS) to verify your identity. As a result of this exchange:
 - The visited MME now knows who you are.
 - The home HSS now knows where you are (i.e., that you're currently connected to the visited network).
 At this point, communication between the two networks can begin.



3. Step 3: Control Plane Configuration → This initial procedure is called **association**, it's the step in which the device identifies itself and establishes registration. Once associated, the MME of the visited network contacts your home HSS and configures the control plane, preparing the system so that your data can be delivered even though you're connected through a foreign network. After this, the HSS knows your location, but the rest of the internet doesn't.

For example, when someone sends you a WhatsApp message, it doesn't go directly to the visited network. Instead, the message first reaches your home network, because, as far as the rest of the world is concerned, your number still belongs to your home operator.

The Vodafone HSS (in our example) then recognizes that this message is meant for one of its subscribers who is currently roaming. It therefore sets up a data-plane configuration that allows the incoming data (the WhatsApp message) to reach your device through the visited network.



- Mobile communicates with local MME via BS control-plane channel
- MME uses mobile's IMSI info to contact mobile's home HSS
 - o retrieve authentication, encryption, network service information
 - o home HSS knows mobile now resident in visited network
- BS, mobile select parameters for BS-mobile data-plane radio channel

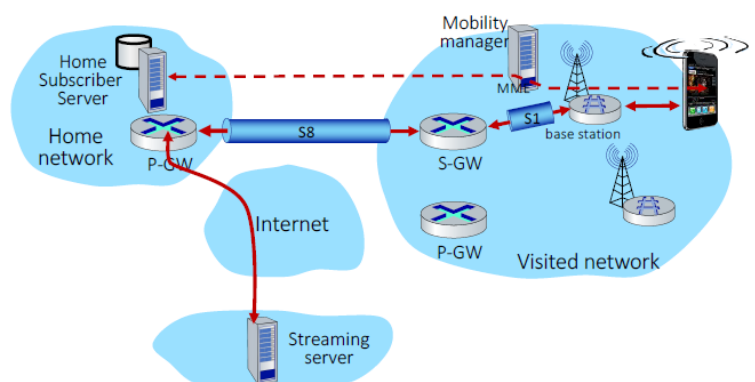
4. Step 4: Data Plane Configuration → This data-plane setup involves packet gateways (PGWs) and serving gateways (SGWs), which handle tunneling and data routing.

Let's visualize this:

- Your device is connected to the local eNodeB in the visited network.
- The MME in the visited network informs the home HSS of your presence.
- The home HSS then establishes a data-plane communication channel with your device.

There are two ways to configure the data plane. In the first approach the visited network provides radio and transport network resources. Actual data session is anchored back at the home PGW.

The S1 tunnel connects the local eNodeB to the Serving Gateway (SGW) in the visited network. From there, the tunnel extends to the Packet Gateway (PGW), but in your home network. All data destined for your device travels across the internet to your home PGW, which then routes it



back through the tunnel to the SGW in the visited network.

EU connects to SGW and MME in visited network, which routes traffic back to the home PGW using the S8 bearer.

When you travel abroad, you might have tried a simple trick without fully realizing what lies behind it. Imagine you connect to a Wi-Fi network in another country and want to watch a streaming service you normally use at home. For instance, the Italian national broadcaster RAI has its own app. But when you access it from a foreign Wi-Fi network, you immediately get a message saying that the content cannot be streamed outside Italy.

A common reaction is to turn off Wi-Fi and switch to the local mobile network of the country you're visiting. Maybe you bought a travel data plan that allows you to use something like 15 GB in 15 days. So you decide to use part of this allowance to watch RAI's streaming service. Surprisingly, when you connect using mobile data, the content becomes available—even if you are in the US, Sweden, or anywhere else.

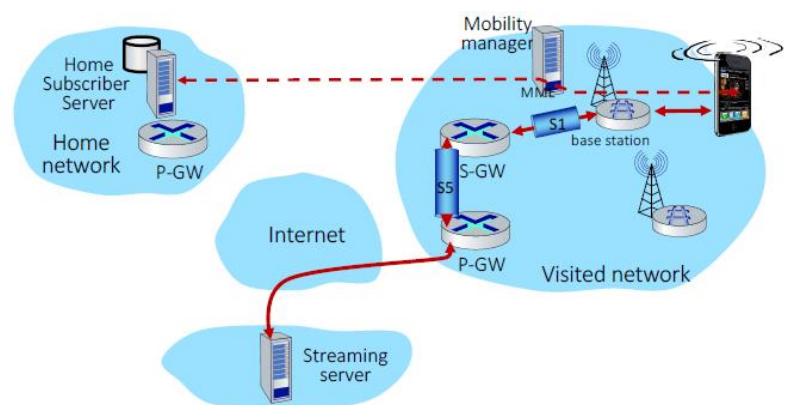
The reason this works is straightforward: from the streaming service's point of view, your request comes from an IP address located in Italy. This happens because the packet gateway (PGW) in your home network assigns you an IP address that is recognized as Italian. The streaming service only sees that it is sending data to an Italian IP; what happens afterward is completely opaque to it. In reality, your data is being tunneled all the way to your actual location abroad, but this is entirely hidden from the streaming server.

This mechanism relies on an S8 tunnel between the packet gateway in your home network and the serving gateway (SGW) in the visitor network. In this configuration, the PGW and SGW are no longer co-located, as they normally are inside the same network. Instead, they span different countries and even continents.

This solution is known as **home-routed roaming**, and it is widely used because it allows your home operator to accurately track your data usage and enforce your travel plan limits. Home network manages user authentication, data billing, and IP address allocation. User's data session and policies are consistent with their home subscription.

From a logical standpoint, your device still has an IP address belonging to your home network, even though your data physically travels through the visited network. This is because the packets are encapsulated (tunneled) inside the visited network's IP infrastructure. The inner packet remains untouched, it carries your home IP address. For example, if your home network uses the address range 179.10.0.0/16, your device might have IP 179.10.7.50. When you contact the RAI streaming server, it sees a request from 179.10.7.50 and recognizes it as Italian, allowing the connection.

A second solution exists, called **local breakout: the serving gateway in the visited network establishes a tunnel to its own packet gateway, and your traffic exits locally within the visited network.** The UE's data traffic exits locally within the visited network. Visited network handles both the control and user planes, including IP address allocation and data traffic, through own PGW and S5 bearer.



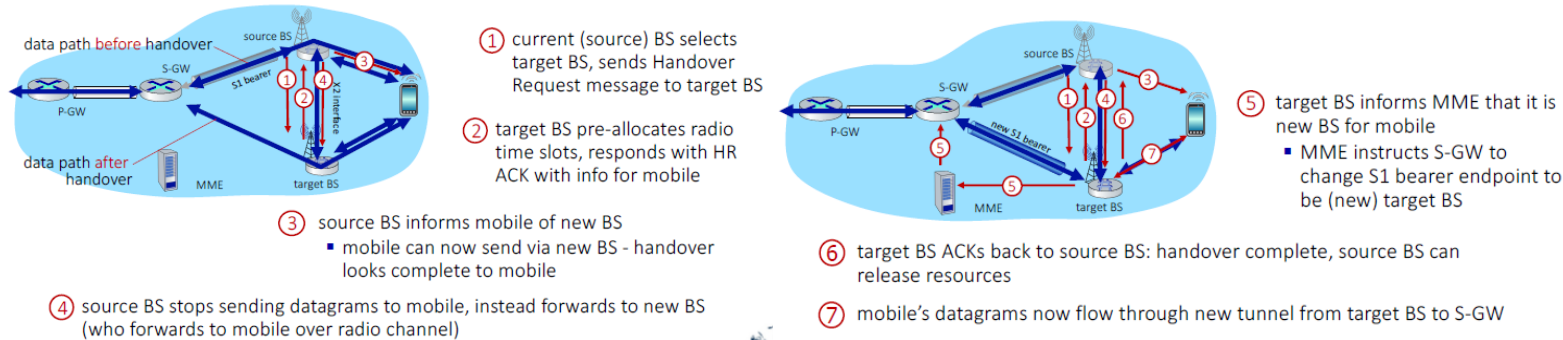
If a streaming service uses geographic restrictions, access will fail because your device appears to belong to the foreign network. Furthermore, the home subscriber server (HSS) loses visibility over your data usage, meaning it cannot track how much of your travel allowance you have consumed. For this reason, operators rarely use or accept local breakout, and they may even forbid it entirely.

However, there are specific situations where local breakout can be useful or even necessary: when applications require extremely low latency or for offloading data traffic to reduce network load on home operator. This is the case for scenarios such as: remote control of automated vehicles or operations close to international borders, where devices might switch networks frequently and still need real-time communication. In these cases, it's more efficient

for the data to stay within the visited network, minimizing latency and avoiding long routing paths through the home country. Home network might lose some control over QoS policies or billing.

So, while local breakout exists and has clear advantages for certain low-latency applications, in most practical situations home-routed roaming remains the preferred and default solution — since it ensures proper user authentication, billing visibility, and seamless integration with the home operator’s services.

Handover between BSs in same cellular network



Now, let's look at what happens when a handover occurs between two eNodeBs (Evolved NodeBs) within the same cellular network — that is, without changing operators or roaming to another country. In this case, we're still inside the home network.

Initially, the user device (the UE, or User Equipment) is connected to a specific eNodeB.

- There is a radio bearer between the device and that eNodeB.
- From the eNodeB, an S1 bearer connects to the Serving Gateway (SGW).
- Then, an S5 bearer connects the SGW to the Packet Gateway (PGW).

This is the situation before the handover begins, everything is stable and data is flowing through the established bearers.

Now imagine that the device begins to move, for example, the user is walking or driving, and starts getting closer to another cell. At some point, the device (or rather, the network) detects that the quality of the connection with the current eNodeB is deteriorating, while the signal from another eNodeB is becoming stronger. This triggers the need for a handover to the new cell.

The device cannot communicate directly with the target eNodeB yet, because it does not have any radio resources or frequencies allocated on that cell. Therefore, the handover request must go through the source eNodeB. The source eNodeB sends a handover request to the target eNodeB using the X2 interface, which is a direct logical connection between neighboring eNodeBs.

Upon receiving the request, the target eNodeB is informed about the incoming handover and starts preparing radio resources — frequencies and time slots — for the arriving device. You can think of this as preparing a space in the “checkerboard” of frequency and time slots we discussed earlier. Once the resources are ready, the target eNodeB confirms back to the source eNodeB, which in turn notifies the mobile device that the handover is feasible.

At this point, the source eNodeB tells the mobile device: “You are going to connect to the target eNodeB, and here are the radio resources (frequency and time slots) you can use.” These are exactly the resources that were reserved earlier by the target eNodeB. The mobile device now starts communicating directly with the target eNodeB, using those newly allocated resources. Meanwhile, the source eNodeB stops sending data, and all data that was previously routed from the Serving Gateway (SGW) to the source eNodeB is now redirected to the target eNodeB.

In other words, **the S1 bearer, which connects the eNodeB to the SGW, is switched from the source eNodeB to the target eNodeB.**

Once the handover is complete, the source eNodeB releases all the resources that were allocated to that mobile device and effectively “forgets” about it. From this point onward, the device is fully connected to the target eNodeB, and all communication continues seamlessly through the new path.

Recap of the Handover Procedure

To summarize, the handover procedure consists of the following steps:

1. **Detection:** The device (through measurement reports) or the source eNodeB detects that the signal from another eNodeB is stronger.
2. **Request:** The mobile device asks the source eNodeB to initiate a handover.
3. **Preparation:** The source eNodeB contacts the target eNodeB via the X2 interface and requests resource preparation.
4. **Resource Allocation:** The target eNodeB allocates radio resources and confirms the preparation to the source eNodeB.
5. **Switch:** The source eNodeB informs the MME that the S1 bearer should be switched from the source to the target eNodeB.
6. **Completion:** The data path is updated, the source eNodeB releases its resources, and the mobile device is now entirely managed by the target eNodeB.

In general, mobility management in cellular networks is a complex process involving multiple coordinated procedures. Most of these operations occur in the control plane, they involve signaling messages and synchronization among different network components. Everything must happen in perfect coordination:

- The new radio resources must be prepared on time,
- The bearers and tunnels within the Evolved Packet Core (EPC) must be updated,
- And all of this must happen in real time, so that the user experiences no interruption in service.

Despite the complexity, this process is completely transparent to the user — data continues to flow, and we usually don't even notice that our device has changed cells during the handover

5G Networks

5G is designed to meet the demands of a highly mobile and fully connected society. It aims to handle the massive proliferation of connected devices and objects, enabling a wide variety of new services and innovative business models. These include automation across multiple industries and vertical markets such as energy, e-health, smart cities, connected vehicles, and industrial manufacturing. Beyond this, 5G will support more pervasive human-centric applications like virtual and augmented reality, as well as high-quality 4K video streaming.

Importantly, 5G networks will accommodate the coexistence of two broad categories of applications: human-centric and machine-type communications. This diversity means 5G must support a wide range of functional requirements and performance targets (KPIs), which differ significantly across use cases.

Unlike previous generations, 5G will require not just improved networking technologies but also a sophisticated integration of massive computing and storage infrastructure, to support its complex service demands.

5G Ecosystem

To manage this complexity, operators will deploy **orchestrator functions** that allocate appropriate computing and network resources dynamically to services. These resources are organized into “**network slices**”—logical networks tailored with specific networking and computing functions to meet the performance needs of individual service providers.

5G will also enable cross-domain orchestration, coordinating services and resources across multiple administrative domains, potentially involving several operators. This requires flexible sharing schemes and seamless interworking between operators, even at the network function layer, such as coordinating *Software Defined Networking (SDN)* rules.

5G Use Cases

Three main categories define 5G applications:

- *Enhanced Mobile Broadband (eMBB)*: High data rate applications and enhanced user experience.
- *Massive Machine-Type Communication (mMTC)*: Support for an extremely large number of connected devices, such as IoT sensors.

- **Ultra-Reliable Low-Latency Communication (URLLC)**: Applications requiring minimal delay and high reliability, critical for industrial automation, autonomous driving, and remote surgery.

5G technologies in the Radio Access



Advanced waveforms

Alternatives to pure OFDM such as RBF-OFDM, FBMC, GFDM, and UPMC.



Advanced MIMO

Both co-located and distributed architectures are envisaged, with co-located massive MIMO particularly appealing to high frequency bands.



Millimeter Wave

Millimeter Wave spectrum (20-80GHz) offers large chunks (up to 2GHz) of contiguous bandwidth, for ultra-high throughput and low latency scenarios.

5G technologies in the Core Network



Software Defined Networking

SDN is an approach to networking in which routing control is centralized and decoupled from the physical infrastructure (data plane), which is distributed.



Network Function Virtualization

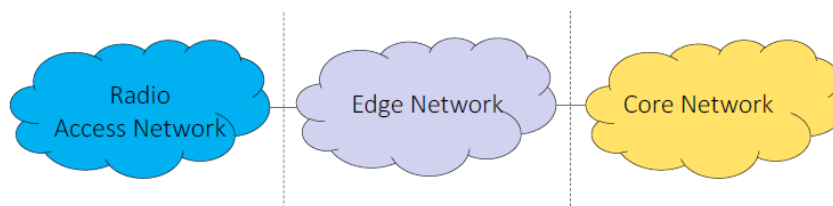
NFV moves network services (firewall, load balancing, access control) from hardware to software, creating virtual building blocks that can be easily connected (NFV chaining)



SDN/NFV Orchestration

Aka. the new network operating system: it manages function lifecycle, resources, policies and it provides system analytics

5G Architecture



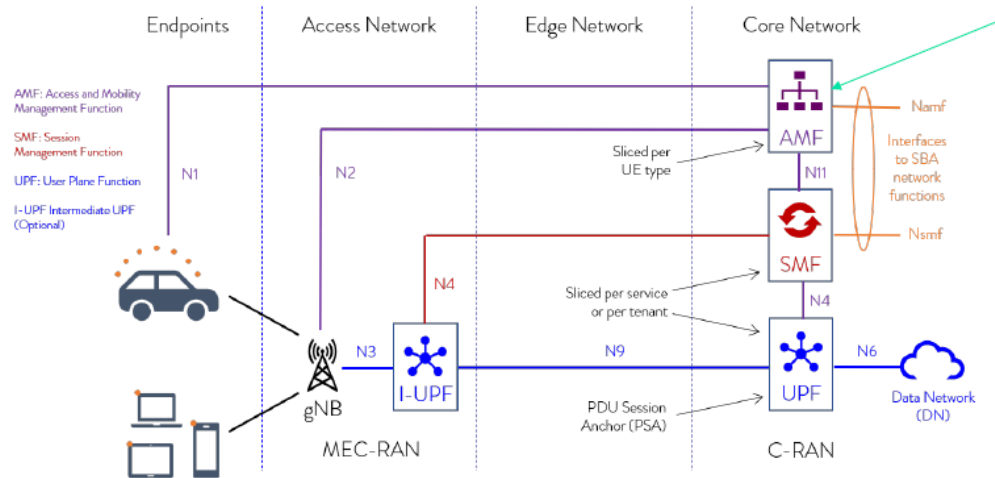
- **Radio Access Network (RAN)**: it includes all the devices involved with the direct interaction with user devices.
 - o In 5G, base stations are called gNodeB
- **Edge Network (MEC)**: it hosts computing and storage elements for local services
- **Core Network (CN)**: it includes all the devices responsible for the transport of data to/from the Internet or toward other user devices

Main differences between 4G and 5G networks

- **Nomenclature**: The radio interface changes from E-UTRA to New Radio (NR), and the radio access network from E-UTRAN to 5G-RAN. The core network evolves from EPC to 5GC.
- **Data and Control Plane**: 5G introduces a full separation of control and user planes, unlike 4G where these functions are mixed.
- **New Functionalities**: Network slicing enables tailored configurations of RAN for different services; a new QoS framework focuses on flows rather than end-to-end bearers; and the core network adopts a Service-Based Architecture (SBA) for modular, flexible service deployment.

5G CORE NETWORK

5G Service Based Architecture (SBA)



AMF (Access and Mobility Management Function): Evolves from the 4G MME to manage user access and mobility.

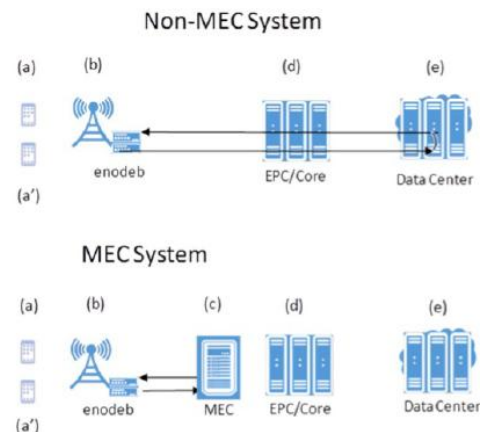
SMF (Session Management Function): Handles session control, evolving from 4G's control plane functions.

UPF (User Plane Function): Responsible for user data forwarding and traffic steering, an evolution of the 4G data plane.

5G EDGE NETWORK

Edge Network infrastructure providing IT service environment and cloud-computing capabilities at the edge of the mobile network, in close proximity to mobile subscribers. Standardization started in 2014, with the first specifications published in 2017.

Expected benefits: ultra-low latency, high bandwidth, real-time access to radio network, contextual information, location awareness, flexible and extendable framework for services.

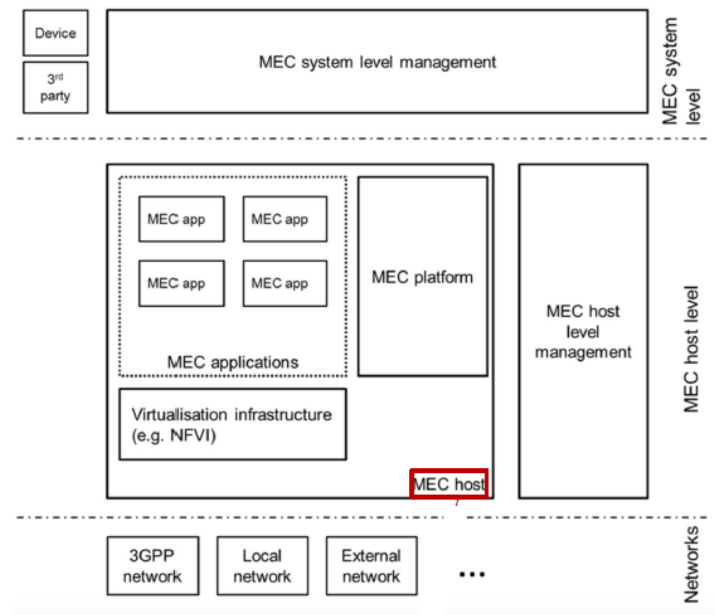


MEC Architecture

The **MEC host** contains the platform and virtualization infrastructure, which provides compute, storage, and networking for MEC applications.

The **MEC platform** offers an environment where the MEC applications can discover, advertise, consume and offer MEC services.

MEC applications are running as virtual machines (VM) on top of the virtualization infrastructure provided by the MEC host, and can interact with the MEC platform to consume and provide MEC services.

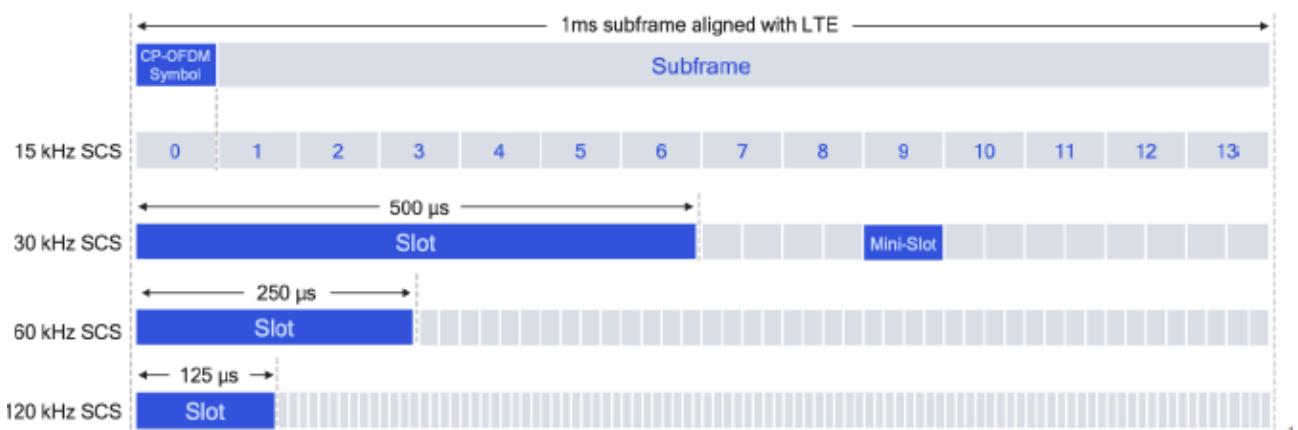


5G RADIO ACCESS NETWORK

5G New Radio - Overview

MIMO technology uses multiple antennas at both the base station and user equipment to transmit multiple data streams simultaneously over the same time/frequency resources. This doubles or quadruples throughput per user (e.g., 2x2 or 4x4 MIMO).

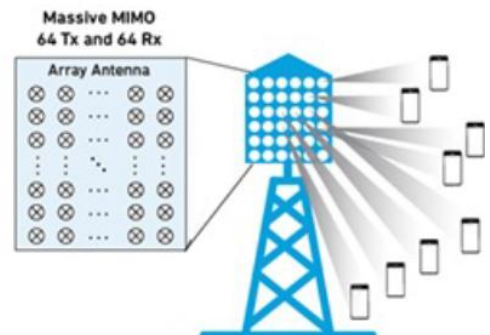
Massive MIMO extends this by deploying many more antennas than users, significantly boosting capacity and spectral efficiency. Future 5G releases will support distributed MIMO, where users can receive different parts of the same data stream from multiple transmission sites.



New Radio – Massive MIMO

MIMO (Multiple Input Multiple Output): both the base station and UE have multiple antenna ports and antennas, and multiple data streams are transmitted simultaneously to the UE using same time/frequency resources, doubling (2x2 MIMO), or quadrupling (4x4 MIMO) the peak throughput of a single user.

Massive MIMO (mMIMO) is a system where the number of antennas exceeds the number of users.



Although not part of the first release, 5G NR will support distributed-MIMO, where a user can receive different parts of the same data stream from multiple sites.

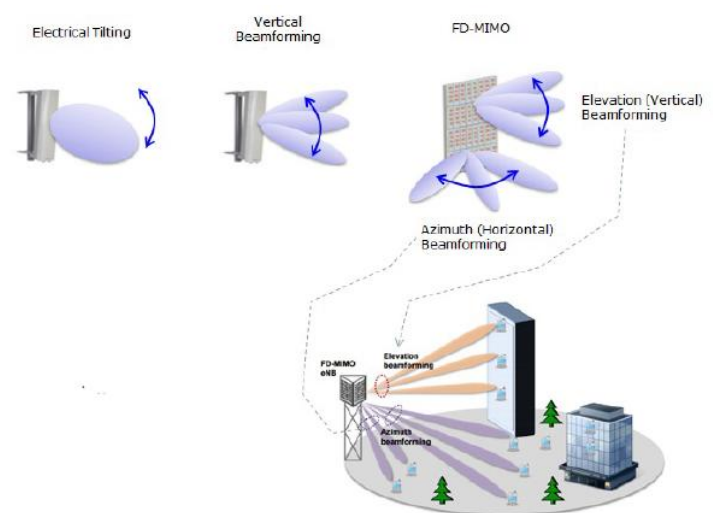
New Radio – Beamforming

Beamforming is the manipulation of the signals fed to and received from complex antennas to create beams in space that focus power in a particular direction.

LTE applies it only to data, 5G also to control.

At high frequencies beamforming will mostly be used to increase range by energy focus.

At the mid and low bands below 6GHz, where attenuation is less of a problem, beamforming will be a key part of MIMO.



Wireless LANs

Wi-Fi, short for Wireless Fidelity, is the common name for wireless local area networks (WLANs). Everyone uses Wi-Fi daily, at home, at school, or in public spaces, to connect devices to the internet through a service provider.

However, it's important to note that Wi-Fi itself is not a protocol. Instead, it is a **certification of compliance with the IEEE 802.11 family of standards**. These standards define how wireless communication occurs within local area networks.

IEEE 802.11 standard	Year	Max data rate	Range	Frequency
802.11b	1999	11 Mbps	30 m	2.4 Ghz
802.11g	2003	54 Mbps	30m	2.4 Ghz
802.11n (WiFi 4)	2009	600	70m	2.4, 5 Ghz
802.11ac (WiFi 5)	2013	3.47Gbps	70m	5 Ghz
802.11ax (WiFi 6)	2021	14 Gbps	70m	2.4, 5 Ghz
802.11af	2014	35 – 560 Mbps	1 Km	unused TV bands (54-790 MHz)
802.11ah	2017	347Mbps	1 Km	900 Mhz

All use CSMA/CA for multiple access, and have base-station and ad-hoc network versions.

The IEEE 802.11 standard was first introduced in 1999, at the end of the 20th century. Since then, many versions have been released — 802.11a, 802.11b, 802.11g, 802.11n, 802.11ac, 802.11ax, and so on. Each new version has brought improvements in bitrate, bandwidth, and coverage range. Modern Wi-Fi networks can now achieve tens of gigabits per second, while the earliest versions were limited to around 11 Mbps.

Most Wi-Fi versions use the 2.4 GHz and 5 GHz frequency bands. The most recent ones, such as Wi-Fi 6E and Wi-Fi 7, also operate in the 6 GHz band, providing even higher data rates and lower latency.

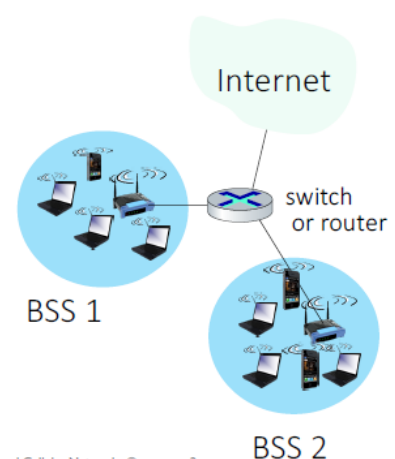
802.11 LAN architecture

Wireless host communicates with base station = access point (AP)

A typical 802.11 network is composed of:

- **Access Points (APs)** – the central devices that coordinate wireless communication;
- **Stations (STAs)** – the user devices such as laptops, smartphones, or tablets.

A wireless LAN typically consists of access points (APs) that function like Layer-2 switches. Their job is to collect data from individual devices and forward it



toward the internet, or distribute incoming data back to the devices. APs are usually connected to upstream switches or routers.

Each access point and its associated devices form a **Basic Service Set (BSS)**. It in infrastructure mode contains:

- wireless hosts
- access point (AP): base station
- ad hoc mode: hosts only

In a room with multiple APs, for instance 4, you have four BSSs. When you walk into the room, your device automatically scans the environment and attaches to the AP with the strongest signal. You usually don't know which one it is unless you use specialized tools. In any case, once the association happens, you become part of that BSS.

If no access point is present, devices may communicate directly using a feature known as **Wi-Fi Direct**, which is supported by many operating systems. For example, Apple's AirDrop uses this technology to exchange data directly between two nearby devices without needing an AP.

802.11: Channels, association

Unlike cellular networks, Wi-Fi does not use a centralized allocation mechanism. The available spectrum is divided into channels, each operating at a specific frequency. Each access point admin selects one of these channels to establish its BSS. To avoid interference, neighboring APs use different channels, while distant APs can reuse the same frequency. Network administrators typically configure these frequencies remotely to optimize performance.



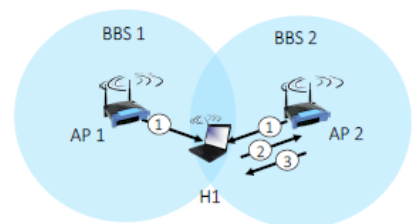
Access points operate by using different frequencies, and they periodically transmit **probe packets** that help devices discover them and decide which one to connect to. After this discovery phase, the device proceeds with authentication.

802.11: passive/active scanning

When a device enters an area covered by Wi-Fi, it must associate with an access point before exchanging data. This process begins with scanning — detecting which APs are available.

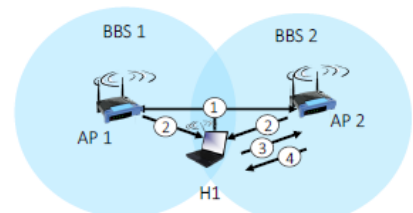
There are two types of scanning:

- **Passive Scanning**
 - o beacon frames sent from Aps - Every access point periodically transmits beacon frames (small packets) — roughly every 100 milliseconds. Each beacon frame contains the SSID (network name) and the MAC address of the AP.
 - o Association request frame sent from H1 to selected AP - The device listens to all Wi-Fi frequencies and collects these beacon frames. Once it identifies a known network (e.g., Polito or Eduroam), it sends an association request frame to the selected AP and begins the authentication process.
 - o Association Response frame sent from selected AP to the device



→ However, there is a drawback. Since the device continuously scans multiple frequencies, it may happen that when the access point sends a beacon frame, the device is currently listening to a different frequency. If this misalignment keeps happening, the device can repeatedly miss beacon frames, making passive scanning potentially slow.

- **Active Scanning**
 - o Probe Request frame broadcast from H1 - to reduce waiting time, devices can instead send probe requests across all frequencies. These requests contain a list of known SSIDs (e.g., myHomeWiFi, Polito, Eduroam), asking "Are you one of these networks?"
 - o Probe Response frames sent from Aps - Any AP that matches a listed SSID responds with a probe response, confirming its presence. This avoids the delays of passive scanning and speeds up the connection process.



- Association Request frame sent: H1 to selected AP
- Association Response frame sent from selected AP to H1

→ This mechanism, however, introduces a privacy issue. Originally, probe requests were transmitted using the device's real MAC address, and since probe requests must be sent in clear text (because you cannot encrypt a message to an access point you have not yet discovered), anyone could track users' locations by capturing these unencrypted packets. Modern operating systems address this by using randomized MAC addresses for probe requests. Depending on the OS, the MAC address may change with every probe request, every few seconds, or every minute.

Once a device chooses an access point, it must authenticate before joining the network. This ensures:

1. The user is authorized to access the AP;
2. The AP is genuine (not a fake hotspot trying to intercept traffic).

After successful authentication, the device receives an IP address via DHCP within the AP's subnet. Only then is the device fully associated with the network.

IEEE 802.11: multiple access

Once the device is connected and authenticated, communication within the Basic Service Set (BSS) follows the **CSMA** protocol, **Carrier Sense Multiple Access**. All devices connected to the same access point share the same frequency, so the key challenge is preventing overlapping transmissions.

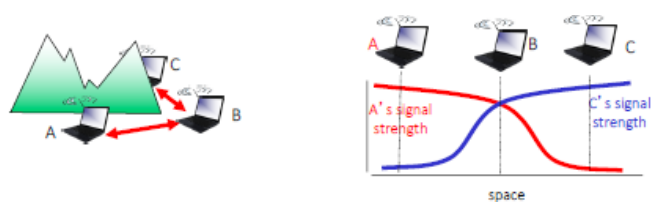
There is no pre-allocated time or frequency division, unlike in cellular networks. Instead, when a device wants to transmit a frame (or when the access point wants to send data to a device), it first listens to the channel. If the channel is idle, meaning no one else is transmitting, it sends the frame. The receiver must explicitly acknowledge the frame; otherwise, the sender cannot know whether the transmission succeeded or whether a collision occurred.

Collisions happen when two devices sense an idle channel at the same time and begin transmitting simultaneously. Since all devices connected to the same access point share the same frequency, this is always possible. Acknowledgments are therefore essential: without them, the sender has no way to detect a failed transmission.

In 802.11 networks, the medium access mechanism is based on CSMA, meaning that each device listens to the channel before transmitting. The idea is to avoid starting a transmission when another node is already sending data.

However, unlike wired Ethernet, 802.11 does not support collision detection. Detecting collisions in wireless environments is extremely difficult: a device's own transmitted signal is so strong that it overwhelms any incoming signal, and fading can further weaken what it receives. On top of that, a node cannot sense all potential collisions anyway, due to issues like the hidden terminal problem or variations caused by fading.

For these reasons, the protocol focuses on collision avoidance rather than collision detection. The goal is to minimize the likelihood of collisions through CSMA with Collision Avoidance (CSMA/CA).



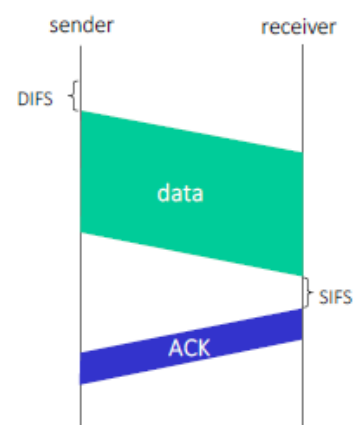
IEEE 802.11 MAC Protocol: CSMA/CA

After association, communication between stations and the AP is governed by the Carrier Sense Multiple Access (CSMA) protocol, specifically CSMA with Collision Avoidance (CSMA/CA).

Since all devices connected to an AP share the same frequency channel, transmissions must occur at different times to avoid interference.

Wi-Fi does not use centralized scheduling, instead, each device follows this decentralized process:

1. Listen before transmitting: the device senses the channel. If it is idle for a short interval (called **DIFS**, **Distributed Inter-Frame Space**), it may transmit.
2. If the channel is busy: The device waits and then chooses a random backoff time before trying again. This randomness prevents multiple stations from transmitting simultaneously right after the channel becomes free.



- a. start random backoff time
 - b. timer counts down while channel idle
 - c. transmit when timer expires
 - d. if no ACK, increase random backoff interval, repeat 2
3. Acknowledgment: When a receiver successfully gets a frame, it replies after a **SIFS (Short Inter-Frame Space)** with an ACK frame (ACK needed due to hidden terminal problem). If no ACK is received, the sender assumes a collision occurred and retries after another random backoff.

Avoiding collisions: : RTS-CTS exchange

Collisions are unavoidable because two stations may sense the channel as idle at the same time and transmit simultaneously. The acknowledgment mechanism is crucial to detect such collisions. The goal is to reduce the probability of collisions.

A particular issue in Wi-Fi networks is the **hidden terminal problem**. It occurs when two devices (say A and B) are too far apart to hear each other's transmissions but both can reach the same AP.

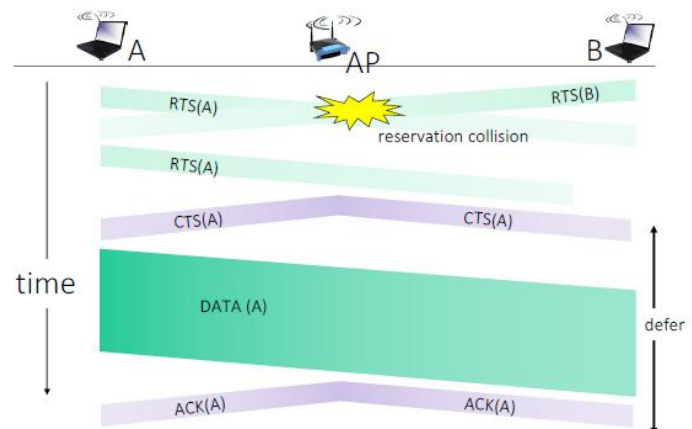
For example:

- Device A senses the channel idle and starts transmitting.
- Device B, being out of A's range, also senses the channel idle and starts transmitting.

Their signals collide at the access point, even though neither A nor B can detect the other's transmission.

To mitigate this, Wi-Fi uses the RTS/CTS mechanism (Ready to Send / Clear to Send). The basic idea is that the sender "reserves" the channel before transmitting its data by using small control packets. First, the sender sends a short Request-to-Send (RTS) message to the base station using CSMA. These RTS packets can still collide with one another, but since they are very short, the cost of a collision is minimal.

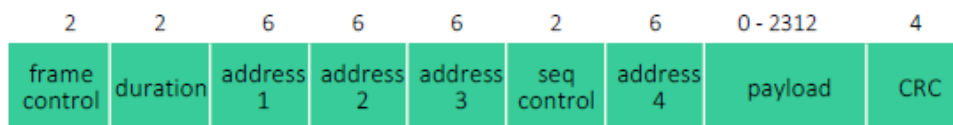
When the base station successfully receives an RTS, it responds by broadcasting a Clear-to-Send (CTS) packet. Every node in the area can hear this CTS, so they know that the channel has been reserved for the upcoming transmission. The sender then transmits its data frame, while all other stations defer their own transmissions until the ongoing exchange is finished.



RTS/CTS does not eliminate collisions entirely. If A and B both send RTS frames simultaneously, those RTS frames will collide. Collisions are therefore still unavoidable, but RTS/CTS reduces their likelihood by enabling a form of temporary channel reservation.

802.11 frame: addressing

The Ethernet frame is quite straightforward and simple. It has relatively few fields: it begins with a synchronization preamble to align the transmitter and receiver, followed by the destination MAC address and the source MAC address. Then, there's a field that can either specify the type of protocol or the length of the payload. After that comes the payload itself, and finally, the frame check sequence (FCS) to detect errors.



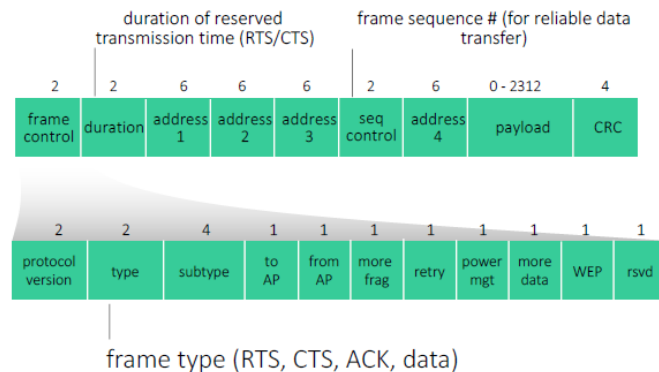
In contrast, the 802.11 frame is more complex for several reasons. Unlike Ethernet's star topology—where stations connect through switches that may have MAC addresses but are not explicitly addressed in frame transmissions—the Wi-Fi equivalent, the access point (AP), must be explicitly addressed. This is because 802.11 networks handle frames differently.

We need to specify where the frame is going because Wi-Fi supports two different communication modes:

1. Device-to-device communication through the access point, which is possible in home Wi-Fi networks if the AP allows it.
2. Device-to-AP-to-internet communication, which is the most common use case (about 99% of the time).

Unlike the Ethernet switch, which forwards frames without being explicitly addressed, the AP needs explicit addressing in the frame to properly route traffic. Due to these requirements, the 802.11 frame must contain more than just two MAC addresses—it often needs three, and sometimes four (although the fourth address is used only in specific ad hoc modes, which we won't cover here).

- The frame starts with the **Frame Control** field, which contains flags indicating protocol version and other important control information.

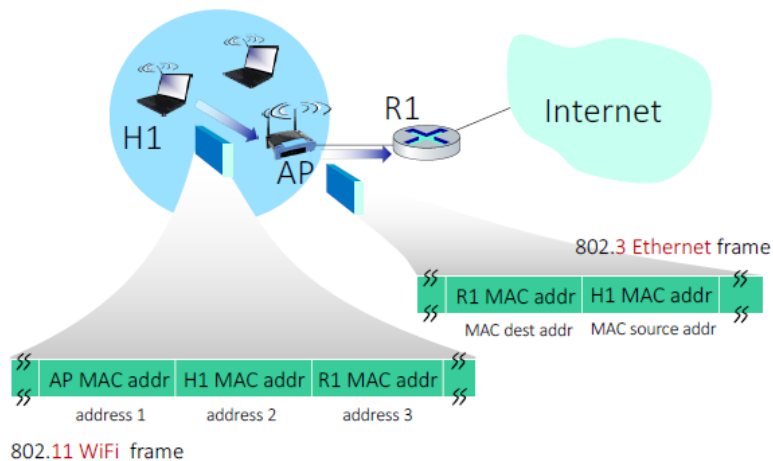


- The **Duration** field is crucial because, as we discussed in the MAC protocol behavior, when a device is not the recipient of a frame, it can go to sleep to save power. The Duration field tells the device how long the ongoing transmission will last, so it knows when it can safely wake up again.
- Next, the frame contains four address fields (Address 1, 2, 3, and sometimes 4). For typical infrastructure mode Wi-Fi, only three addresses are used:
 - o Address 1: MAC address of wireless host or AP to receive this frame
 - MAC address of the immediate destination of the frame, so the next station that has to receive the frame
 - o Address 2: MAC address of wireless host or AP transmitting this frame
 - MAC address of the station that was the source of this frame
 - o Address 3: MAC address of router interface to which AP is attached
 - MAC address of the router that is going to handle this traffic to the Internet
 - o Address 4: used only in ad hoc mode
- The **payload**, which carries Layer 3 data from upper layers
- The **CRC** (Cyclic Redundancy Check), similar to Ethernet and other Layer 2 protocols. This field checks the integrity of the frame to detect any transmission errors.

Address 3 depends on the traffic flow: if the frame is going from your device to the internet via the AP, Address 3 is the MAC address of the router that connects the AP to the internet. If the frame is device-to-device through the AP, Address 3 is the MAC address of the final destination device.

To visualize this, imagine a host device, H1, connected wirelessly to an access point, AP. The AP is connected via Ethernet cable to a router, R1, which provides internet access. When H1 sends a packet destined for the internet, the frame addresses will reflect this routing:

- Address 1 will be the AP's MAC (the immediate recipient),
- Address 2 will be H1's MAC (the source),
- Address 3 will be R1's MAC (the router providing internet access).

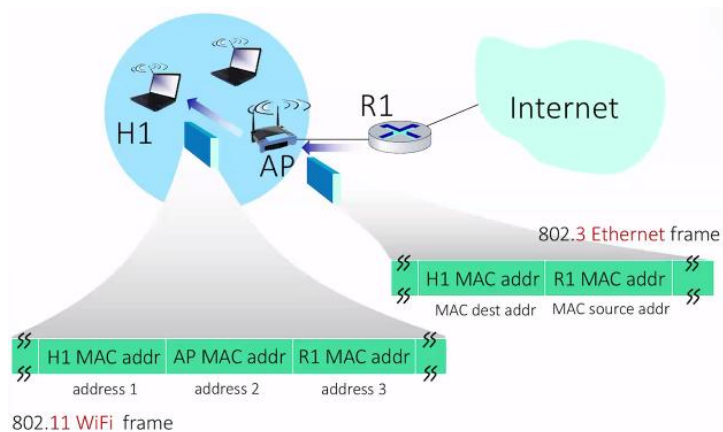


What happens to the frame when it reaches the access point? The access point takes the payload, the actual content inside the 802.11 frame, and extracts it. This payload is then placed inside an Ethernet frame. Since an Ethernet frame only has two addresses, the destination address of this Ethernet frame becomes address number 3 from the original 802.11 frame, that is, the router's MAC address, which is now the destination. For the source address of the Ethernet frame, the MAC address of the original sender (H1) is retained.

This situation creates an interesting mix of addressing. For example, on the Ethernet link between the access point and the router, the source MAC address (H1) doesn't actually belong to that link physically. Nevertheless, from the perspective of this communication, the access point behaves as if it were transparent, it essentially "disappears". In other words, it acts like a transparent switch on the Ethernet side. This means that, from the Ethernet link's point of view, it's as if the original sender (H1) were directly connected to the router (R1), bypassing the access point.

However, from the Wi-Fi perspective, the access point is very much present and visible. This is evident because its MAC address appears as the first address in the 802.11 frame.

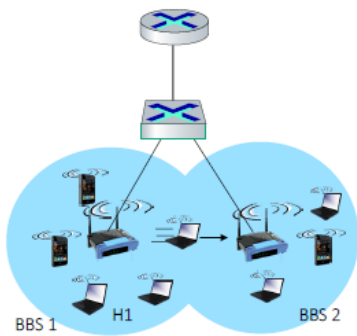
To understand this better, consider the opposite direction: traffic coming from the Internet destined for our station (H1).



An Ethernet frame travels from the router (R1) to the access point, with the destination MAC address set to H1's MAC address, even though H1 is not directly on this Ethernet link. Again, this is because the access point behaves as a transparent switch on the Ethernet side. The source MAC address remains that of the router, making the Ethernet frame valid. The access point then creates an 802.11 frame that will be sent over the wireless link to H1. In this 802.11 frame, the destination is H1, the immediate sender is the access point (address 2), and the third address is the router's MAC address.

It's important to recognize this duality in addressing: each address makes sense within its own domain. On the Ethernet side, addresses behave as in a switch network. On the 802.11 (Wi-Fi) side, the addressing scheme is adapted to include the access point explicitly.

802.11: mobility within same subnet



Moving on, let's consider a scenario with two Basic Service Sets (BSSs) — for example, two access points within the same classroom.

H1 remains in same IP subnet: IP address can remain the same

These access points are connected to a switch, which is a Layer 2 device, and this switch is connected to a router, a Layer 3 device. From the perspective of a device like H1 connected to either access point, all devices are in the same subnet because the subnet begins downstream of the router port. Therefore, H1 must keep the same IP address and subnet, regardless of which access point it connects to.

What happens if a device moves across the room and decides to associate with the other access point, which provides a better signal? Nothing changes at the IP level, but the physical path for data must change. The switch must now forward traffic destined for H1 through a different access point. This is managed through a process called **self-learning**: the switch listens to frames transmitted by H1 and updates its forwarding tables to reflect H1's new location, remember which switch port can be used to reach H1

Because communication at higher layers, like TCP, always involves two-way transmissions (data and acknowledgments), the switch eventually learns where H1 has moved based on these frames. This self-learning mechanism ensures seamless mobility within the same subnet across different access points.

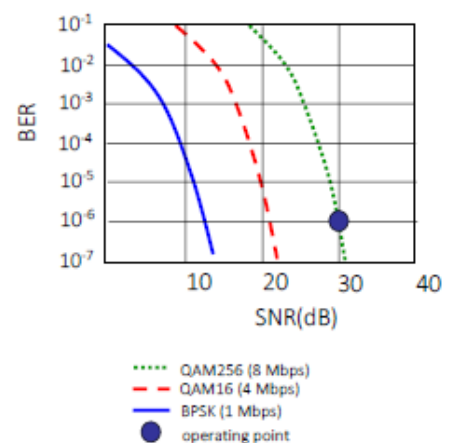
802.11: advanced capabilities

Rate adaptation

Another important point about 802.11 is that devices can use different data rates depending on the signal quality. You may have noticed that as you move farther from the access point, or if there are obstacles between your device and the access point, your connection speed decreases. This happens because the received signal power drops with distance — following roughly the inverse square law — and due to fading effects caused by interference and environmental factors.

Base station, mobile dynamically change transmission rate (physical layer modulation technique) as mobile moves, SNR varies

1. SNR (signal-to-noise ratio, the ratio between useful signal and noise/interference) decreases, BER increase as node moves away from base station
2. When BER becomes too high, switch to lower transmission rate but with lower BER



It does this by changing the modulation scheme: recall that in modulation, the number of distinct signals used determines how many bits each symbol carries. For example, with 16 different signals, each symbol represents 4 bits; with 64 signals, each symbol represents 6 bits. Higher modulation levels allow faster data rates but require better signal quality, so the device lowers the modulation order when the signal quality deteriorates to maintain reliable communication.

Power management

A mobile node can tell the access point that it is going to sleep until the next beacon frame. Once informed, the AP knows not to send any frames to that node during its sleep period. The node then wakes up shortly before the next beacon arrives.

Each beacon frame includes a list of mobile devices for which the AP has pending downlink frames. When the node receives the beacon, it checks whether there is data waiting for it. If there are frames to be delivered, the node stays awake to receive them; otherwise, it returns to sleep until the next beacon.

Principles of modern LAN design and operation

History

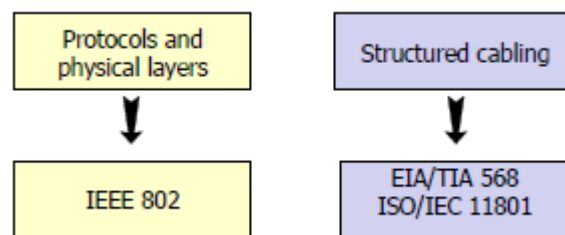
Local Area Networks (LANs) emerged after Wide Area Networks, which first appeared in the 1960s. At that time, only a few large and expensive mainframes existed, and organizations needed a way to access them from remote locations. Networking was driven largely by the need to share these rare and costly computational resources across different institutions.

In the 1970s, a significant shift occurred in computing technology. Instead of relying solely on mainframes, individual workstations began to emerge, particularly in research and data centers. This change introduced a new challenge: enabling communication between these workstations, not just between mainframes. By the late 1970s, alongside workstations and minicomputers, the first personal computers (PCs) appeared.

As PCs became more common, it became crucial to share communication resources efficiently within smaller groups, which led to the development of Local Area Networks (LANs). Cost was low enough so that it was no longer needed to access a remote mainframe.

These LANs, along with Wide Area Networks (WANs), began to evolve with distinct technologies and protocols. WANs primarily utilized optical fibers and other advanced methods, while LANs generally relied on copper wires. Mainframe still used, but for different purposes (e.g., scientific simulations) LAN.

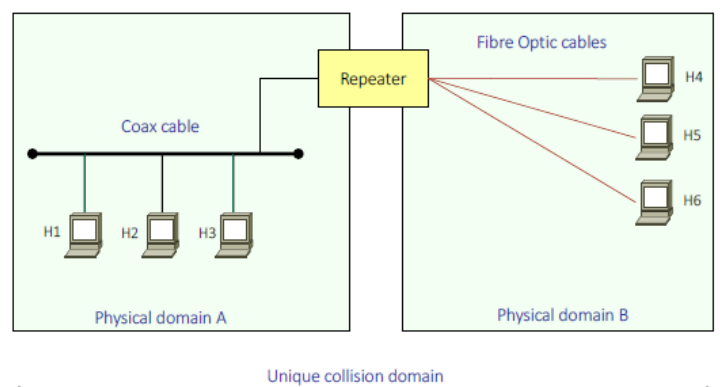
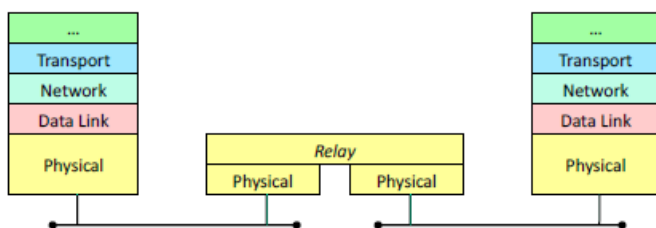
Early LANs experimented with multiple protocols, most of which eventually became obsolete: Decnet, SNA, IP, Novell, Banyan Vineis, NetBeui. The one protocol that endured was the Internet Protocol (IP), which became fundamental in network communication.



Focusing on LANs from an IP network perspective, devices are categorized based on the OSI model layers they operate on. Devices functioning at

- Layer 1 (the physical layer) are called **repeater** → Separate physical domains, same collision domain;
- Layer 2 are **bridges** or **switches** → Separate collision domains, same broadcast domain;
- Layer 3 are **routers** → Separate broadcast domain; Routers are not specifically LAN-oriented, though they can connect LANs.

Repeater



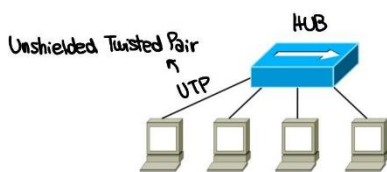
Used for Interconnecting networks having the same Medium Access Control (MAC), Same technology but different physical layer is possible, Recovering signal degradation (long cables), allowing larger distances.

A repeater is a Layer 1 device used to interconnect physical media. Its primary role is to extend the network's physical reach. It operates purely at the physical layer, detecting a raw bit sequence from one physical medium and regenerating them on another, without any understanding of the data structure or frames; it is completely unaware of frames, protocols, or structure, its sole job is to restore signal integrity.

Essentially, a repeater amplifies a weakening signal to prevent degradation over distance, thus maintaining transmission quality. For example, in older setups using coaxial cables, repeaters could link different physical media like coaxial and fiber optic cables by simply retransmitting the bit stream.

Multiport repeaters: Hubs

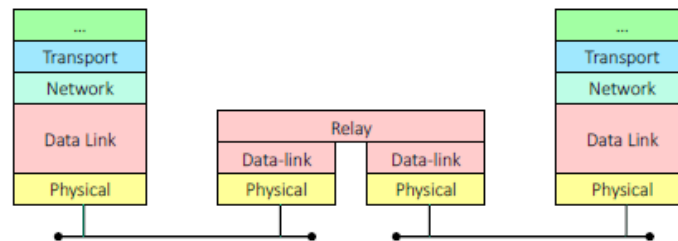
When repeaters have multiple ports, they are known as hubs. These hubs essentially act as multiport repeaters and became popular by the late 1990s, especially when networks began to adopt star topologies. Star topology was favored because it mirrored existing telephone wiring infrastructure in office buildings, where all extensions connect back to a central switch. Using hubs allowed the new network topology to fit seamlessly into this environment, typically using unshielded twisted pair (UTP) cables—the same kind of cables commonly found in homes for connecting wireless routers, and fiber cabling (hub-and-spoke topology).



However, repeaters and hubs had a major limitation: they only transmitted bits blindly, without error checking or awareness of data frames. This meant they forwarded all traffic indiscriminately, causing inefficiencies and potential congestion.

Bridges

To solve these issues, Layer 2 devices called bridges were introduced in 1983 by Digital Equipment Corporation (DEC) with the LANBridge 100.



Early bridges:

- Were implemented entirely in software,
- Typically had only two ports (for cost reasons),
- Operated at the data-link layer,
- Could interconnect different LAN types (e.g., Ethernet to Wi-Fi, Ethernet to Fast Ethernet),
- Could handle different MAC mechanisms and framing formats.

A bridge works in store-and-forward mode:

1. It receives and stores an entire frame.
2. It analyzes and may modify it (e.g., convert between Ethernet and Token Ring).
3. It forwards it only if needed.

Bridges understand frame boundaries, can verify integrity using the CRC field, and can decide forwarding based on MAC addresses. If the destination address is on the same segment where the frame arrived, the bridge suppresses unnecessary forwarding.

Modern Wi-Fi Access Points work exactly this way with wi-fi and Ethernet network: they act as bridges that extract the payload from one MAC frame format and encapsulate it into another.

In summary, the evolution from repeaters and hubs to bridges marked a move from simple bit-level signal repeaters to intelligent frame-aware devices. This development laid the foundation for modern LAN architecture, improving efficiency, scalability, and reliability of local networks.

Collision and broadcast domains

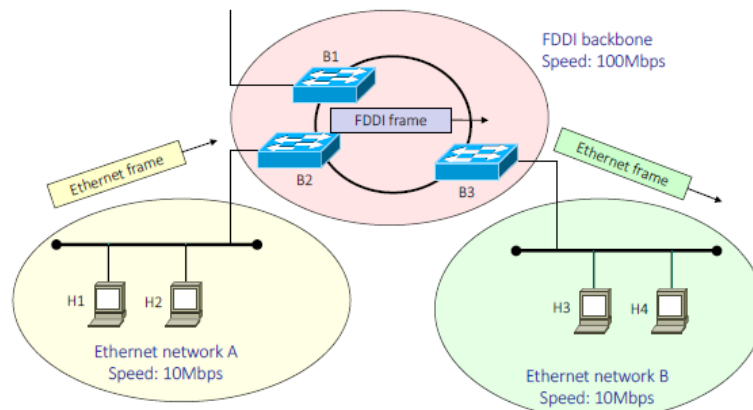
A **collision domain** is the area governed by a single medium-access control algorithm (such as CSMA/CD) I.e., the area covered by a single “physical” link; Frames are immediately propagated over all the links (possibly through repeaters); Also called **network segment**.

A **broadcast domain** consists of all devices that will receive a frame sent to the broadcast address (all ones) I.e., the area on which a LAN operates; can include several collision domain; Frames can be stored and later propagated over other collision domains.

- Repeaters and hubs do not separate these domains: the collision domain and broadcast domain are identical.
- Bridges keep the broadcast domain intact but separate collision domains.
- Routers separate both collision and broadcast domains.

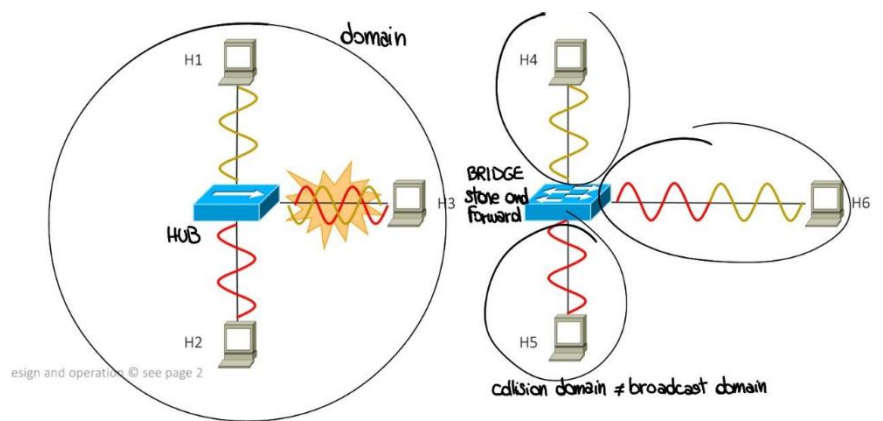
For example, if you send a broadcast packet from one device, that packet must be received by every device within the broadcast domain.

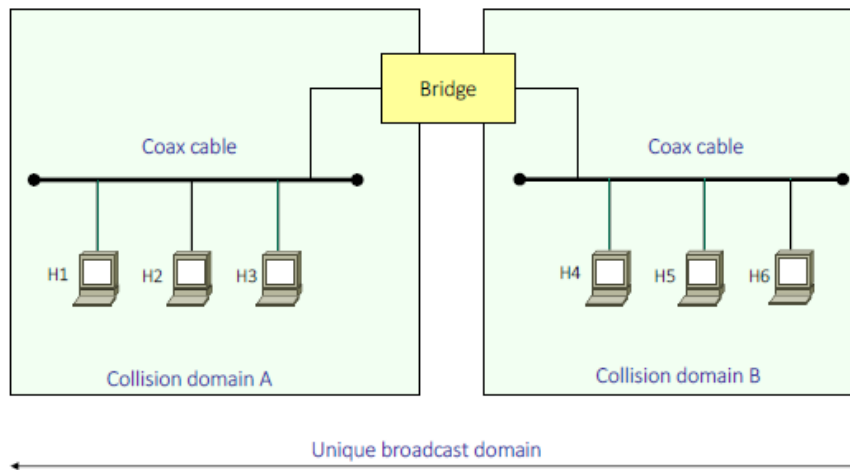
This is an example of interconnection popular in ‘80s and ‘90s. There are LANs interconnected using a FDDI backbone:



Picture two LANs connected by a bridge. Each LAN is its own collision domain. The bridge keeps the broadcast domain consistent across both, so broadcasts still reach all devices in both networks. But collisions are contained within each LAN, reducing collision size.

The key benefit of the bridge's **store-and-forward** mechanism is that it **decouples collision domains from broadcast domains**. Repeaters only extend the the collision domain to its theoretical limits by compensating for cable impairments, but bridges genuinely separate it while still propagating broadcasts.



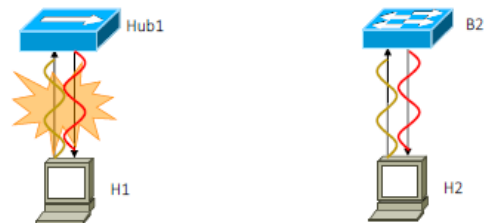


Half and Full duplex mode

The introduction of **twisted-pair cables** (especially UTP) brought a major advance: **full-duplex communication**, where sending and receiving can occur simultaneously over different wire pairs.

- Half-Duplex

- Traditional mode for older network interfaces (NICs).
- Transmission (TX) and reception (RX) cannot happen at the same time (e.g., in 802.11)
- Simultaneous TX/RX is considered a collision (as in Wi-Fi or classic Ethernet).



- Full-Duplex

- Introduced with Fast Ethernet (IEEE 802.3x)
- Available whenever the other party can temporarily store the frame, instead of repeating (immediately) the received bits on the other ports, like a repeater does
- Inside a UTP cable, there are multiple pairs of wires. Typically, two pairs are used to send data in one direction, and the other two pairs are used for data going the opposite way.

→Allows simultaneous TX and RX between any pair (host–host, host–bridge, bridge–bridge), eliminating collisions even on the same link.



Full duplex eliminated collisions entirely. As a result, the need for collision detection protocols like CSMA/CD (Carrier Sense Multiple Access with Collision Detection) disappeared. Initially, Ethernet used CSMA/CD to manage access to a shared medium and avoid collisions. But with full duplex links and bridges (now called switches), these multiple access protocols became obsolete. Links now operate point-to-point, and the Ethernet standard no longer requires collision detection beyond 1 Gigabit Ethernet.

This advancement doubled network capacity because nodes could communicate in both directions simultaneously, effectively increasing throughput by about 50%.

Advantages of full duplex include:

- Theoretical doubling of throughput due to simultaneous transmission and reception,
- Symmetric bandwidth, which is particularly beneficial for backbone links where equal upload and download speeds are required,
- No minimum Ethernet frame size constraints, because collisions are eliminated,
- No physical limitations imposed by collision domains, enabling larger and more flexible network topologies.

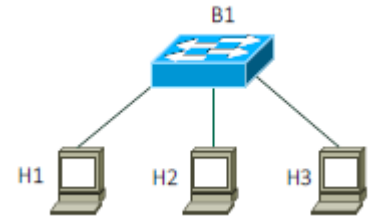
In practice, however, the benefits are somewhat limited for typical clients and servers:

- Clients usually saturate their download (downlink) capacity,
- Servers more often saturate their upload (uplink) capacity.

Full duplex and switches

Today's local area networks rely on full duplex communication managed by **Layer 2 devices** called **switches**. Switches are essentially **multiport bridges that operate in full duplex mode**, connecting hosts to the network with point-to-point links. Each switch port is a full-duplex ethernet interface. This architecture eliminates collision domains entirely.

In such topologies the network is shaped as a hub-and-spoke structure and each switch port is a full-duplex Ethernet interface. Same functions, different internal architecture.



Modern LANs: Switched Ethernet

Modern wired LANs rely on full-duplex communication, switches, and Ethernet technology—known as Switched Ethernet. Unlike older Ethernet versions, CSMA/CD (collision detection) is no longer used. This protocol was relevant only up to 1 Gigabit Ethernet (1GE), but newer standards don't require it anymore.

Today, most LANs use Gigabit Ethernet or faster speeds like 10GE and 100GE. The Ethernet standard still defines how frames are structured—with headers, payloads, and so on—but it no longer enforces collision detection or multiple access rules. Each switch port operates in full duplex, allowing simultaneous sending and receiving of data. Switches forward frames efficiently by using MAC addresses.

Wireless LANs work differently. Wi-Fi uses CSMA/CA (collision avoidance) instead of collision detection. Devices like Wi-Fi access points act like switches supporting multiple protocols—they typically combine several Ethernet ports (wired switching) with wireless interfaces.

Although switches dominate wired LANs, hubs still appear in some wireless network extenders. A Wi-Fi extender functions like a hub: it receives wireless signals from a distant access point and rebroadcasts them on a different channel to increase coverage. It does not process frames but simply retransmits the signal, similar to how a traditional hub works.

Transparent bridges/switches

Switches are transparent devices, meaning they operate almost invisibly in the network. You simply connect all the cables, power them on, and they work seamlessly without requiring any manual configuration. This plug-and-play nature is what makes switches so user-friendly and practical in Ethernet LANs.

Originally, not all bridges were transparent. Some early technologies, like Token Ring, used non-transparent bridges that required explicit configuration or imposed constraints on end systems. These designs have long since disappeared. The transparent bridging model was standardized by the IEEE under 802.1D, establishing the principles that modern Ethernet switches still follow.

→ Bridges (switches) should be plug-and-play and must not require any change in the configuration of end systems.

Performance (throughput, max distances) may be different from the original network, but functionalities are the same

Transparent bridges/switches and end hosts

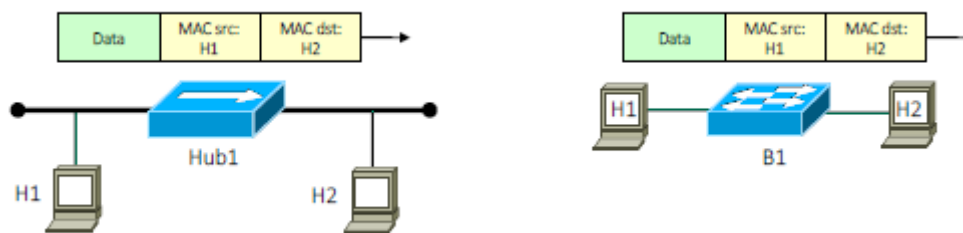
From the point of view of end systems, nothing changes when a switch is placed in the path. End hosts continue to use the same frame formats, the same source and destination MAC addresses, and the same protocols as if the switch were not there at all. There are no modifications to:

- The frame structure
- The MAC source and destination addresses
- The transmission behavior of hosts

However, their presence can slightly impact network performance. For example, when traffic passes between two hosts through a switch, you might notice a small delay. This happens because switches use a **"store-and-forward"** mechanism: they receive entire incoming frames, store them temporarily, and then forward them only when the output port is free. This process can cause a minor reduction in throughput or an increase in latency, but aside from that, switches are effectively invisible to end systems.

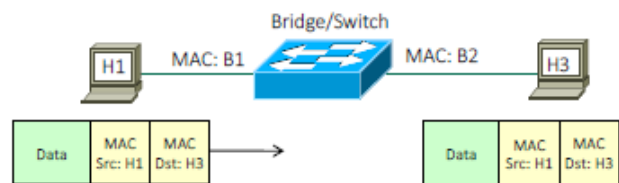
Switches can also filter out corrupted frames—frames that would otherwise reach end systems and be discarded there. They remove them early, improving overall efficiency, but the frame format is otherwise untouched.

Hubs are also “transparent” in the sense that they do not modify frames—they simply forward bits blindly. The difference is that hubs lack any intelligence: they repeat every bit on every port, while switches examine and forward entire frames selectively. Even features like VLANs only temporarily modify headers; these tags are removed before the frame reaches its final destination, so transparency at the end-system level is maintained.



Transparent bridges/switches and port addresses

Although switches aim to be invisible, each port of a bridge or switch does in fact have a MAC address at the MAC layer. These MAC addresses are not used for forwarding data frames between end systems. Instead, they are used for internally generated frames, such as management or control messages, and in protocols that switches use to coordinate with one another (e.g., spanning tree). These addresses are never explicitly targeted by end systems—they are invisible to normal network traffic. End hosts never address frames to a switch port.



Smart forwarding process

Since switches operate independently, they cannot rely on the source to indicate where to send frames. The source host sends a frame without specifying which port the switch should use to forward it. Therefore, switches must have their own logic and rules to decide the correct forwarding path.

Switches rely on a smarter forwarding process than hubs or repeaters. While broadcast and multicast frames are flooded out of all ports except the incoming one, unicast frames follow a more selective path. Multicast traffic can be handled more intelligently if the switch supports snooping, selectively forwarding to relevant ports only.

→ Broadcast and multicast traffic is handled through flooding

For unicast traffic, the switch consults its MAC forwarding table, also known as the **filtering database**. This table maps MAC addresses of destination devices to the specific ports where those devices are reachable. When a frame arrives, the switch looks up the destination MAC in this table and forwards the frame only to the corresponding port, reducing unnecessary traffic and increasing efficiency.

→ Unicast traffic uses destination-MAC-based forwarding

It's important to note that flooding doesn't mean simultaneous transmission on all ports. The switch transmits frames to each port when that port is free, respecting ongoing traffic and avoiding congestion. This staggered transmission ensures smooth handling of broadcast traffic even when some ports are busy.

Flooding in hubs was immediate bit-level propagation; switches handle it at frame-level timing

New components in “smart” bridges/switches

In order to operate successfully, a “smart” bridge/switch requires three additional components/features:

1. **Local forwarding table (filtering database):** a table that maps destination MAC addresses to switch ports.
2. **Stations auto-learning (backward learning):** switches dynamically learn which devices are connected to which ports by observing incoming traffic.
3. **Loop detection (spanning tree algorithm):** mechanisms such as the spanning tree protocol to prevent forwarding loops in the network.

These mechanisms ensure that a switch truly behaves as a plug-and-play device requiring no manual configuration.

While bridges and switches share many features, in Ethernet networks today we primarily use switches, so the focus shifts from bridging concepts to switching.

Filtering Database

The filtering database stores information about devices in the network:

- MAC address
- Corresponding output port
- Aging time, typically 300 seconds by default

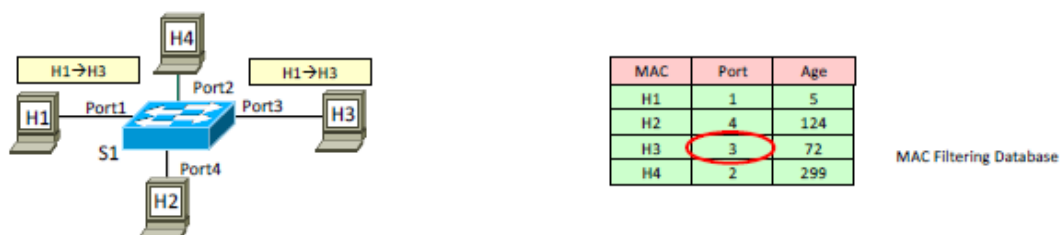
Historically, “filtering” referred to the switch’s ability to block traffic from unnecessary links, something hubs could never do.

There are two types of entries:

- **Dynamic entries**, learned automatically through backward learning, with capacities typically ranging from 2K to 64K entries
- **Static entries**, configured by the administrator, and not subject to aging (usually fewer than 1K)

Dynamic entries age out after the timer expires (**default 300 seconds**), which helps remove stale information for devices that have moved or disconnected.

The learning process in network switches operates on a principle similar to tracking where traffic comes from to know where to send responses. Imagine a switch connected to multiple access points, with devices (or hosts) moving between them. The switch needs to determine which port corresponds to which device so it can send traffic back correctly. The filtering database starts empty when a switch powers on. As devices send traffic, the switch learns the MAC addresses of connected devices and which ports they are reachable from, populating the table dynamically. This learning process allows switches to forward frames intelligently without prior configuration. Importantly, the switch ignores the destination address during this learning phase; it only cares about where the traffic comes from.



Forwarding process

If a destination MAC address is not in the filtering database, the switch temporarily behaves like a hub: it floods the frame out of every port except the incoming one. This is entirely normal and extremely common, especially right after startup: it is called *transient*. Only once backward learning populates the table does selective forwarding begin.

How do we populate the filtering database?

- By hand, possible on all modern devices, but not very handy
- By means of a proper algorithm

Switches are plug-and-play and have an algorithm to learn the location of the hosts. However, at the beginning, switches do not know where a host is located. In this case the “MAC Flooding” algorithm is the only way to go.

Backward learning

Backward learning is the algorithm that allows switches to discover network topology without any manual input:

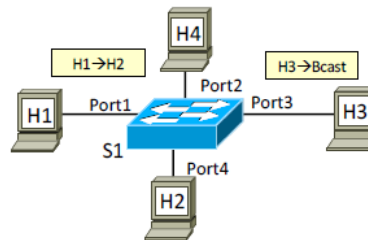
1. A frame arrives on port P1 with source MAC address H1.
2. The switch infers that H1 is reachable via P1.
3. It records or updates this entry in the filtering database.
4. The destination MAC address is ignored during the learning process.

Example with a simple network: Consider a network where the MAC filtering database starts empty.

Host H1 sends a frame to Host H2. The frame contains:

- Destination MAC: H2
- Source MAC: H1

The switch looks at the source MAC (H1) and notes it is reachable through the port where the frame arrived, say port 1. This information is stored in the database.



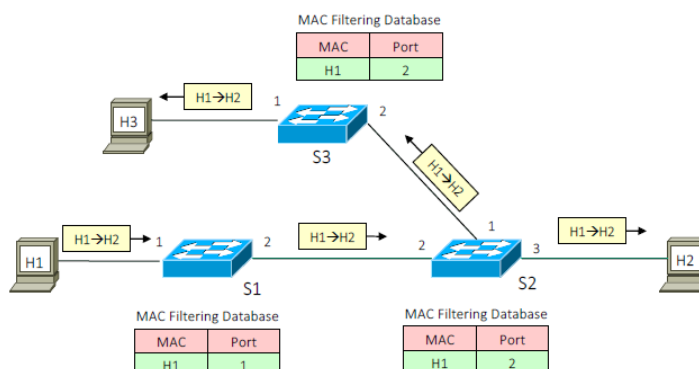
MAC	Port
H1	1
H3	3

Later, if Host H3 sends a broadcast frame (a frame sent to all devices), it's received by the switch and floods to all ports because the switch doesn't yet know where the destinations are. However, the switch records H3's source MAC as reachable on port 3, updating the database.

This process repeats across switches in a network, regardless of topology. Remote switches learn the position anyway, even if end-system not connected locally.

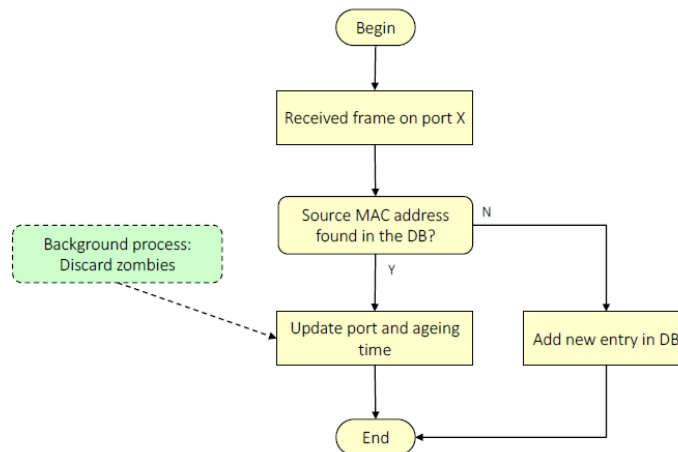
Propagation of Learning Across Switches (backward learning and frame forwarding taken together)

Suppose H1 sends a frame destined for H2. The frame travels from S1 to S2, with each switch learning that H1 is reachable through the port the frame entered on. Since initially no switch knows where H2 is, the frame is flooded to all other ports. This means it reaches H2 but also other switches and hosts like S3 and H3. Those switches then learn where H1 is as well.



Once this learning completes, all switches know how to reach H1, but no one yet knows the location of H2 (because destination addresses aren't learned). Next time a frame must be sent to H1, it will be sent on the specific port recorded in the filtering database, rather than flooded.

This mechanism allows switches to operate efficiently without any manual configuration, adapting dynamically as devices send traffic.



A flowchart of the learning logic looks like this:

- When a frame arrives on port X, check if the source MAC address is in the filtering database.
- If not, add a new entry associating that MAC with port X.
- If the MAC is already present but associated with a different port, update the port to the current one.
- If the MAC is found on the correct port, simply refresh the aging timer.

Switches use an aging timer to remove “zombie” entries that haven’t been seen for some time (typically 5 minutes). This avoids stale mappings if a device disconnects or moves.

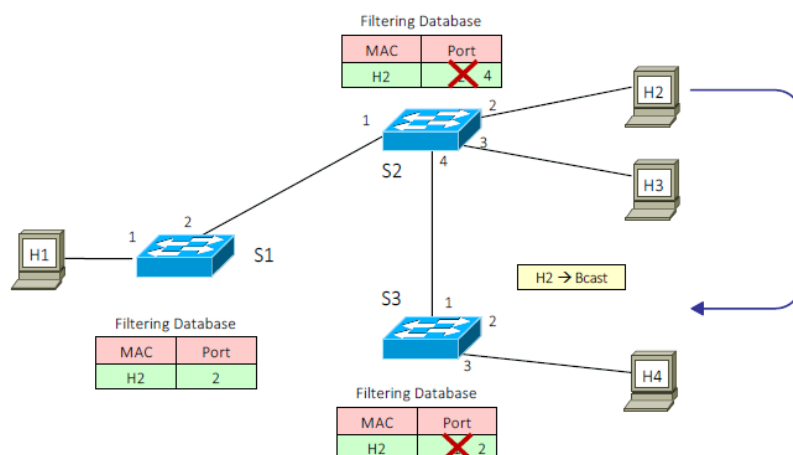
An important detail is that a device whose MAC address is not in the table is still always reachable. In that case, the switch simply floods frames destined for that address across all ports. Consequently, frames sent to a host that does not actually exist will also be flooded throughout the network.

On the other hand, a device whose MAC address is in the table may be temporarily unreachable—for example, if it moved—until the entry expires (typically after the aging time).

Handling Mobility

Consider devices moving around the network. For example, Host H2 moves from port 2 on Switch 2 to port 2 on Switch 3.

→ **If the end-system generates broadcast frame immediately no problem**



Initially, Switch 2’s filtering database says H2 is reachable on its port 2. When H2 reconnects on Switch 3, it begins sending broadcast or control traffic. This triggers switches to update their databases with H2’s new location.

If H2 immediately sends a broadcast or control frame after reconnecting, the network updates itself smoothly. Switches that receive the broadcast learn the new location and overwrite the old entry.

Switch 1 does not need to update its entry for H2 because, from its perspective, traffic for H2 still goes towards Switch 2, which handles forwarding to the correct port.

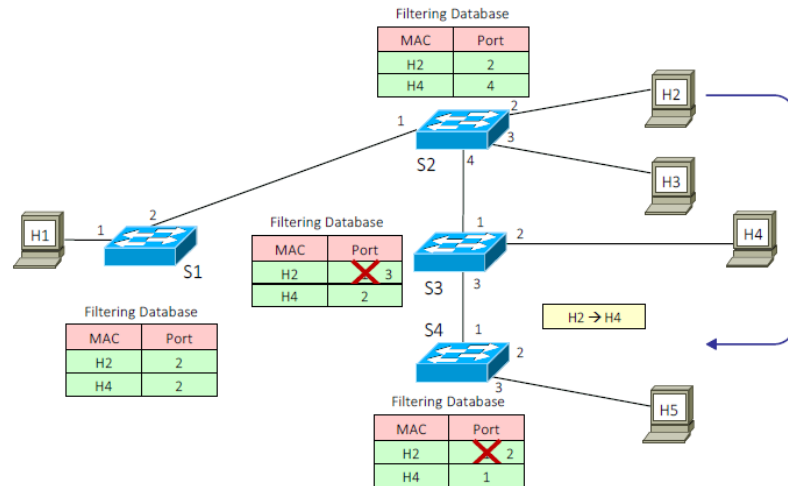
Switches S2 and S3 update their records to reflect H2's new location, ensuring that future frames destined for H2 will be forwarded correctly.

If the end-system generates unicast traffic immediately we may have forwarding errors

After H2→H4:

- H4→H2 is correctly delivered
- H3→H2 is lost

Unicast traffic is not flooded. Only switches on the unicast path learn the new location. Other switches still believe H2 is on its old port.



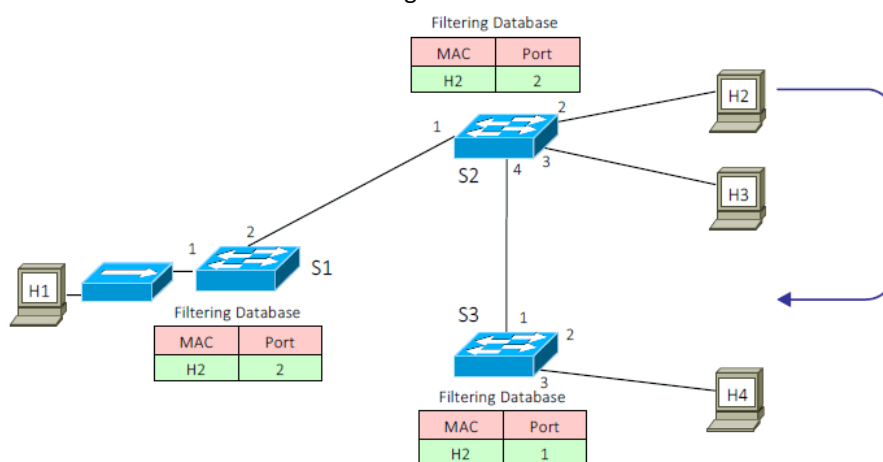
Some hosts (e.g., H4) may still reach H2 correctly if their traffic happens to pass through updated switches. Others (e.g., H3) may lose frames because parts of the network still believe H2 is located at its old port.

Broadcasts naturally solve this: they reach the entire network, so all switches update their tables. Unicasts do not, they only propagate where needed, leaving outdated entries elsewhere.

→ If the end-system does not generate traffic at all we may have forwarding troubles

No switch sees H2 on its new port. All MAC tables still contain the old location.

- H3→H2 is lost
- H4→H2 is correctly delivered “by chance” (due to flooding or aging)
 - o The frame is forwarded also to the original destination



In most everyday scenarios, this learning is fast enough. A laptop moving from one room to another will generate some traffic, allowing switches to update their entries. But in specialized environments—for instance, with fault-

tolerant NICs—faster convergence may be required. In such cases, the NIC driver may proactively send a broadcast frame to update the network immediately.

The same principles apply to larger networks: only the switches affected by the movement update their entries, and the updated information propagates gradually as traffic flows.

Possible attacks to the filtering database

When a switch's MAC address table (also called the filtering database) becomes full, it can no longer store legitimate addresses. When this happens, the switch won't stop working, but it will start flooding all traffic to every port.

What's the goal of this kind of attack? It's essentially to force the switch into flooding mode, all the ones whose destination address is not present in the DB.

Forces switches to operate like hubs, so that we can intercept traffic generated by other stations, but this slows down the network.

Imagine a switch with, say, 128 ports. Now picture a rogue workstation connected somewhere on the switch. Normally, this rogue device would not have access to all traffic exchanges between other devices because the switch forwards frames only to the correct destination ports. However, if this rogue device begins flooding the switch by generating frames with random, fake MAC addresses, it will fill up the switch's filtering database. This overflow pushes out legitimate MAC addresses. As a result, when traffic is sent from device A to device B, the switch no longer has B's address in its table, so it floods the frame out to all ports. This flooding not only disrupts the network by overwhelming it with unnecessary traffic, but it also allows the rogue device to intercept traffic not intended for it, simply by listening to the flooded frames.

To mitigate this, some vendors have implemented limits on how many MAC addresses a single port can learn. This makes sense because typically, you don't expect thousands of unique MAC addresses to appear on the same port. By restricting the number, even if someone tries to launch a flooding attack from a port, the damage is limited to that port's traffic only, and the overall MAC address database remains intact, preserving normal communication between legitimate devices.

Loop detection

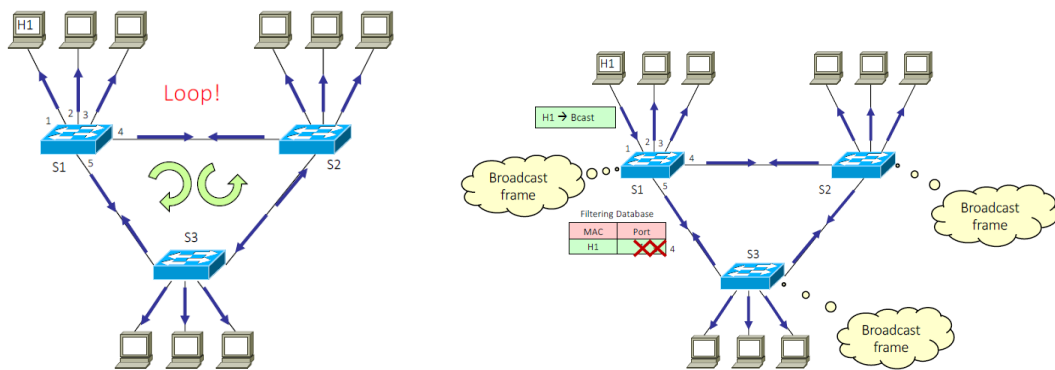
Loops in a switched network cause serious problems because frames can circulate endlessly. This breaks the logic behind MAC address learning (called backward learning).

Consider this example: three switches, S1, S2, and S3, connected with multiple links forming a loop. Suppose host H1 sends a broadcast packet. Switch S1 forwards this broadcast out on all its ports.

Switch	Host	Port
S1	H1	1
S2	H1	1
S3	H1	5

S3 receives it and also forwards it out on all its ports, including back toward S2. Since S2 and S3 are connected, the broadcast frame circulates between them continuously. Both switches update their MAC tables based on the source address of the frame, but due to the loop, they keep changing their knowledge about which port leads to H1, causing incorrect forwarding decisions.

Switch	Host	Port
S1	H1	1
S2	H1	1 → 5
S3	H1	5 → 1



If another host, say H7, tries to send a frame to H1, the frame bounces endlessly between S2 and S3 because both switches think H1 is reachable via the other switch. This is a classic loop causing frames to circulate forever, leading to a **broadcast storm** — an overwhelming number of transmissions that can congest or bring down the network.

A Layer-2 loop can behave like a cheap traffic generator, because frames circulate endlessly at line rate. This unintentionally produces continuous, high-volume traffic without any special equipment.

Which frames can generate a loop?

Several types of frames can trigger a forwarding loop in a switched Ethernet network. The most common ones are broadcast and multicast frames. Because these frames are intentionally flooded across the entire LAN, they naturally circulate everywhere—and if a physical loop exists, they will continue circulating indefinitely.

Another case involves frames sent to a non-existing station. If a destination MAC address is not present in the filtering database, the switch must flood the frame. This situation is rare under normal conditions, but it becomes more common during attacks that intentionally generate unknown destination addresses.

At the IP level, ARP normally avoids this problem: if an IP address does not exist, the ARP request receives no reply, and therefore the system never sends a MAC frame to a non-existent host. Still, the initial ARP broadcast itself will propagate everywhere, and in a looped topology it can circulate endlessly.

Heavy broadcast or multicast traffic can be particularly dangerous because Layer 2 has no built-in mechanism to stop looping frames. Ethernet does not carry a Time-To-Live field, so frames never expire. If a loop forms, the only immediate fix is to physically break the cycle—typically by unplugging a cable. This leaves network operators with almost no control once the storm begins.

Layer 3 networks behave differently: IP packets include a TTL (or Hop Limit in IPv6), which is decremented at every router. This prevents infinite circulation and allows the network to recover from transient routing loops.

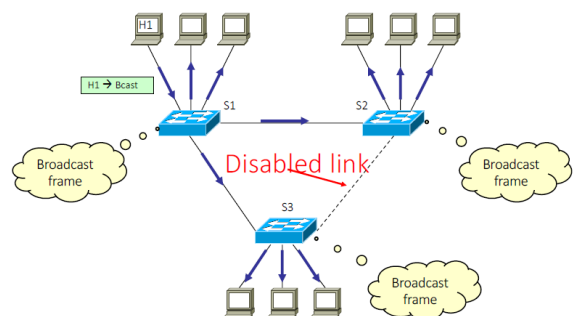
The backward learning problem

The backward learning problem occurs when switches build inconsistent MAC tables. A MAC entry may keep switching between ports indefinitely because looping frames make the switch “learn” the same address from different directions. As a result:

- entries in the filtering database continually change ports,
- some entries never settle into a stable state,
- and switches may propagate frames endlessly between each other (e.g., B1 → B2 → B1 → ...).

Larger loops—such as B1 → B2 → B3 → B1—can appear and persist as long as the topology remains cyclic.

Detecting this problem is tricky. Ethernet frames do not have sequence numbers or identifiers that allow switches to recognize if a broadcast frame is a duplicate. Without such markers, switches cannot tell whether they have already processed a particular broadcast frame.



The only ways to stop the storm are:

- Manually disconnecting cables to break the loop or create directly a loop-free network
- Having a protocol at the system level that automatically detects and disables redundant links.

This issue is specific to Layer 2 networks. Layer 3 networks (IP networks) don't suffer from this problem because IP headers include a Time To Live (TTL) field in IPv4 or a Hop Limit in IPv6. These fields decrement each time the packet passes through a router, eventually discarding the packet and preventing infinite loops.

Spanning tree

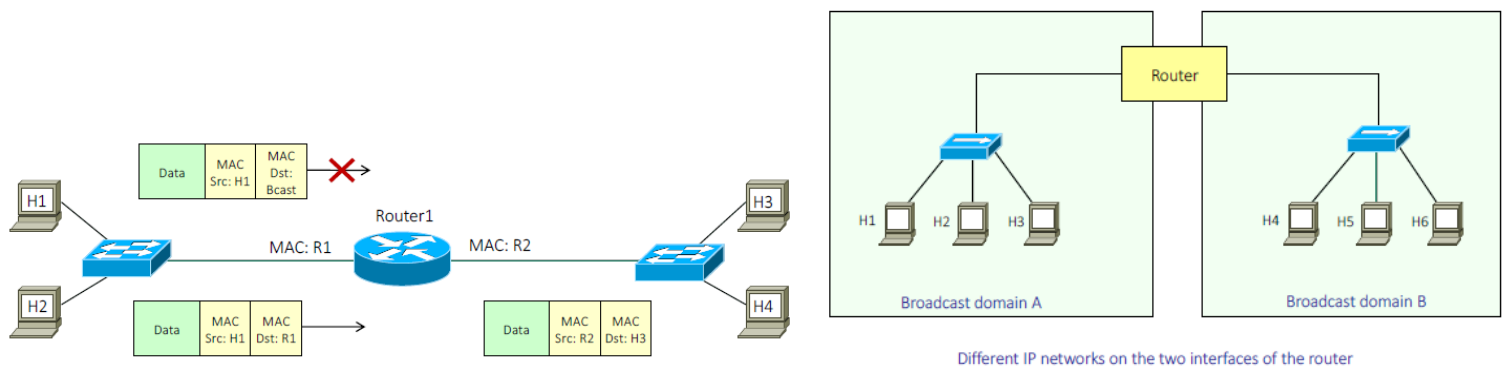
To solve the looping problem in Layer 2, switches implement the Spanning Tree Protocol (STP), developed by Radia Perlman in the 1980s. STP enables switches to communicate with each other, exchanging information based on their MAC addresses to detect loops in the network topology.

STP identifies redundant paths and blocks enough ports so that the overall topology becomes a tree: a structure with a single unique path between any two hosts.

When STP detects a loop, it disables one or more redundant links to eliminate it, effectively breaking the cycle and preventing broadcast storms. STP works by assigning costs (weights) to each link and computing the minimum-cost spanning tree that connects all switches without any loops. This algorithm is already built into every modern switch. As a network engineer or user, you don't need to manually manage STP, it runs automatically to keep the network stable and loop-free.

Routers

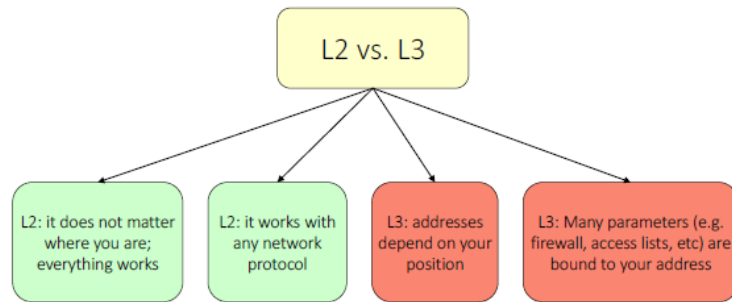
Routers are Layer-3 devices. They tend to be more expensive and complex than switches, and unlike switches, they are not self-configuring. They are not transparent with respect to MAC addresses. They separate broadcast domains.



Up to this point, our focus has mostly been on Layer-2. This raises a fundamental design question: should a network remain purely L2, or is it better to move toward L3? Turning to L3 becomes particularly appealing when we are concerned about MAC-based privacy attacks or about the excessive broadcast traffic associated with IPv4 ARP requests. In IPv4, ARP broadcasts propagate everywhere within the L2 domain. IPv6 provides mechanisms that mitigate these issues, but the underlying concerns remain relevant.

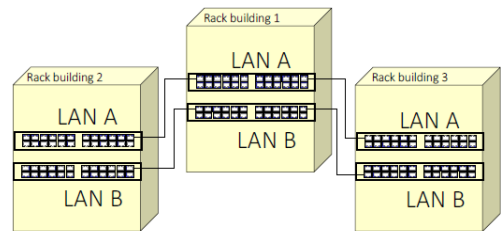
Layer-2 has some appealing properties: it is transparent, and it can carry multiple network protocols without modification. Layer-3, on the other hand, requires hosts to change their IP address whenever their point of attachment changes.

Can we obtain the benefits we normally associate with L3 while remaining within L2? One way is by controlling the range of broadcast domain in L2.



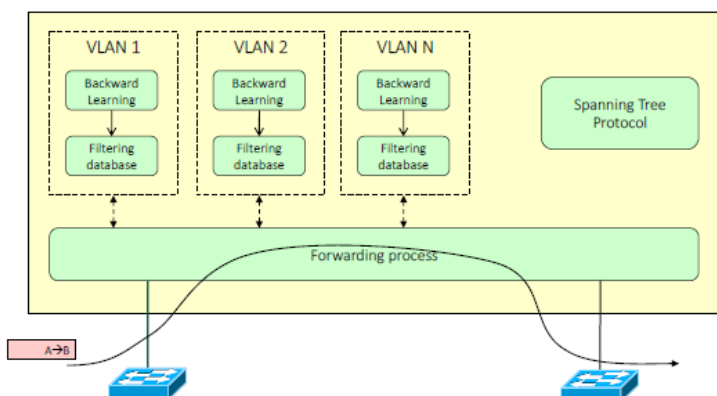
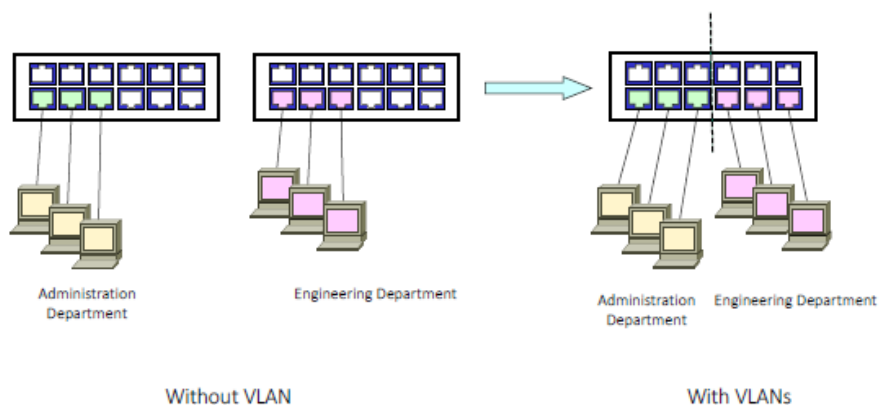
Virtual LANs

For example, in a campus environment, we must ask whether we want one large LAN or multiple LANs. In general, it is advantageous to keep L2 operation as long as possible—provided the network can still function as a single L2 system. But there are well-known scalability issues. A single, very large LAN suffers from excessive broadcast traffic, which switches do not filter. Flooded traffic, especially during frequent Spanning Tree Protocol (STP) reconfigurations, increases overhead. Privacy and security also become harder to guarantee: for instance, we may want to prevent stations from leaking information, such as through MAC flooding attacks. From a management perspective, smaller L2 domains allow simpler and more uniform policies. They also make it easier to separate different groups of users.



One option is to physically separate LANs by assigning a different switch to each building—or even giving a single user multiple switches across buildings—though this quickly becomes impractical and wasteful. A more elegant solution is to group users logically rather than physically.

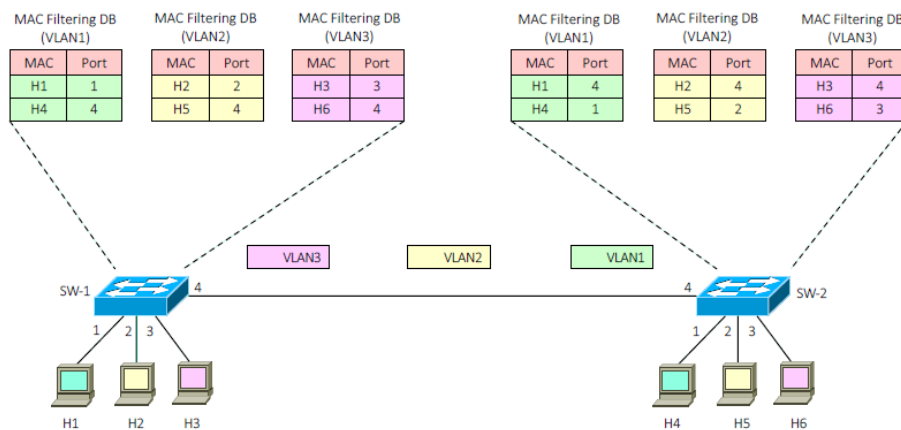
This is exactly what **Virtual LANs** (VLANs) provide. A VLAN is a logical partition of a switch in which some ports belong to one LAN and the rest belong to another. Multiple VLANs coexist on the same physical device.



The advantages are substantial. We maintain a single physical infrastructure with the same cabling and devices, avoiding switches where only a few ports are used. We also eliminate the need for multiple fibers in the backbone for different LANs. VLANs create distinct broadcast domains: Ethernet frames cannot cross VLAN boundaries, broadcasts stay confined within each VLAN, and attacks such as MAC flooding or ARP spoofing remain limited to a single VLAN.

Created through a proper (logic) separation on switches, it works inside a switch and it extends in multiple switching, supporting intra-switch as well as inter-switch operation.

Within each VLAN, the same spanning tree protocol is used. For each VLAN, the switch maintains backward-learning behavior and its own filtering database. Although conceptually each VLAN has a separate database, in practice switches store this information in a unified hardware structure, commonly a TCAM. The VLAN ID effectively becomes part of the lookup key. This allows the switch to know exactly which port corresponds to each user in each VLAN—but only within the same switch. For example, Switch 1 has no knowledge of a host connected to Switch 2.



Real implementations: unique filtering database (usually made with a TCAM, which is a single entity in the network device)

Interconnecting VLANs

Interconnecting VLANs requires Layer-3. L2 frames cannot cross VLAN boundaries, so a host in one VLAN cannot directly send a frame (Ethernet) to a host in another VLAN.

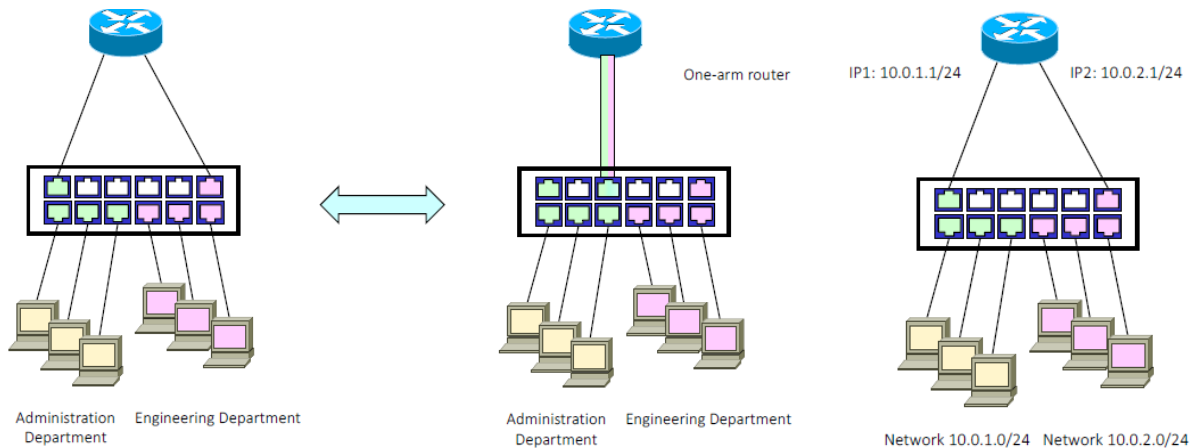
VLANs are different broadcast domains → router is needed to join one VLAN to another

Beware:

L2 data cannot cross VLANs!

A common mechanism is a “one-armed router,” where a single router interface is connected to a switch that carries multiple VLANs. The router can have one MAC address but several IP addresses, one per VLAN.

When routing between VLANs, the router performs an L3 lookup, discards the original L2 header, and constructs a new one with different source and destination MAC addresses. Because ARP broadcasts cannot cross VLAN boundaries, hosts on different VLANs must belong to different IP subnets.

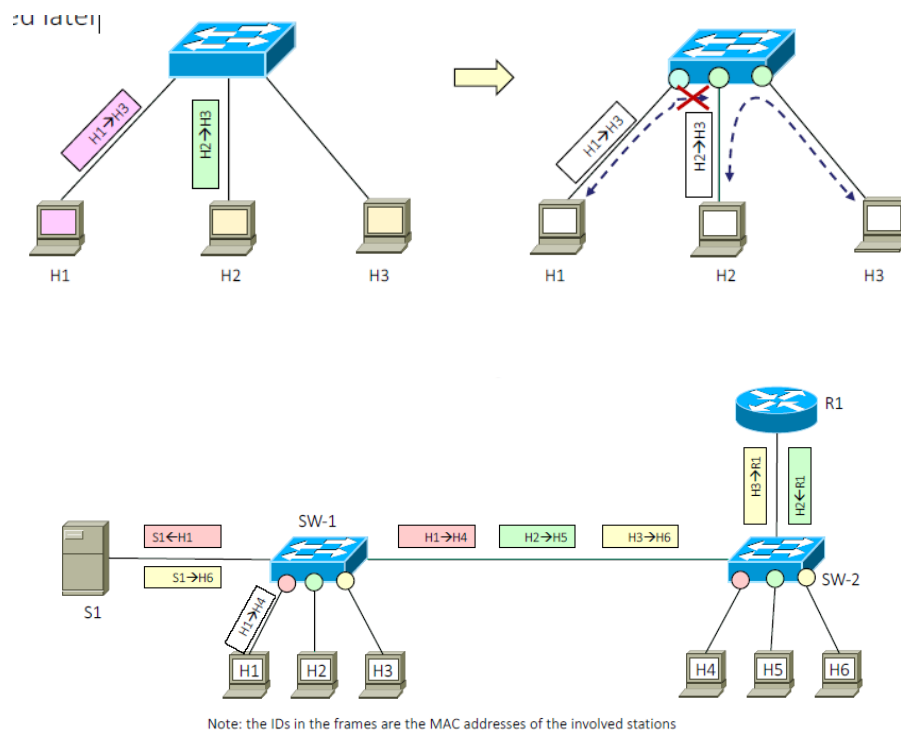


With VLANs, IP addressing follows specific rules. Broadcast traffic cannot cross VLAN boundaries, which also means that ARP cannot be used to resolve a MAC address belonging to a host in a different VLAN. As a result, hosts placed in different VLANs must be assigned to different IP subnets; otherwise, normal Layer-3 communication and address resolution would not function correctly.

IEEE 802.1Q Tag Encoding

We still need a way to associate frames with VLANs. On a single switch, the simplest method is to assign each port to a specific VLAN; then any frame received on that port is automatically associated with that VLAN.

However, when VLANs must span multiple switches, a single inter-switch link must carry traffic belonging to multiple VLANs. Same problem for devices that belong to different VLANs e.g., servers, routers

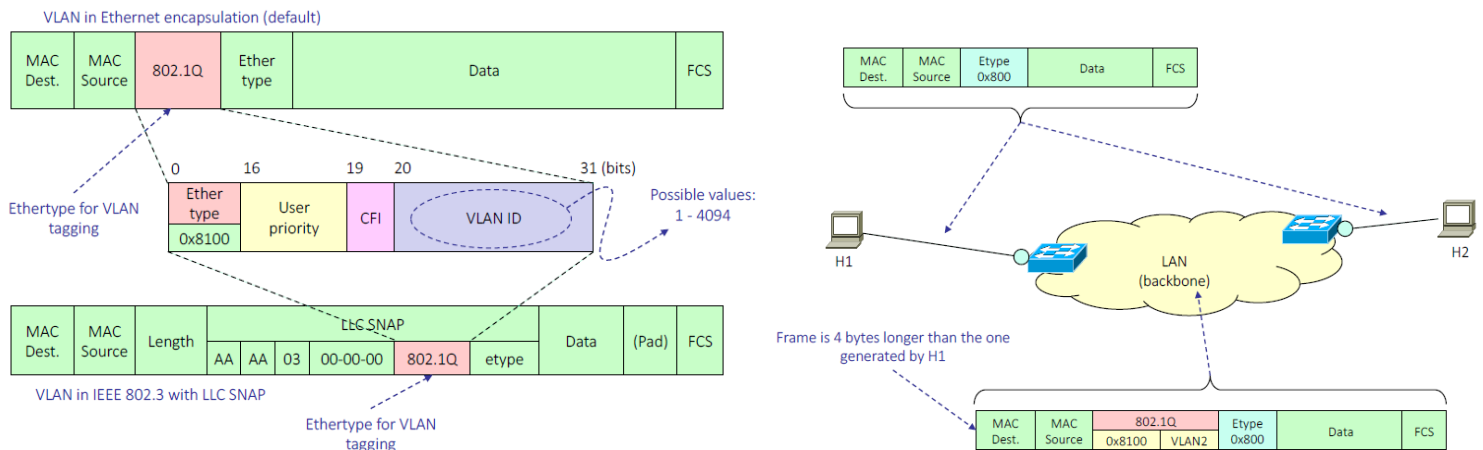


To distinguish the VLAN of each frame, tagging is required. Tagging is used only on links that carry traffic belonging to multiple VLANs. In this case, the switch inserts an additional header into the MAC frame. This mechanism is standardized by IEEE 802.1Q and adds 4 extra bytes to the original frame. The tag contains the VLAN ID along with several other control fields that help manage and classify the traffic.

Tagging inserts a VLAN tag in the Layer-2 header—this is the only situation in which switches are allowed to modify an Ethernet frame header. Tagging is required only on links that carry multiple VLANs, typically switch-to-switch or switch-to-router links. It operates independently of the underlying Medium Access Control technology.

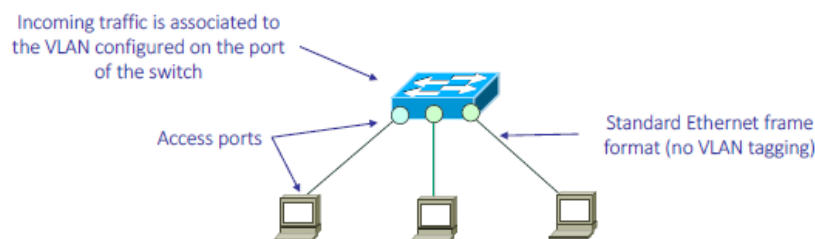
IEEE 802.1Q defines the VLAN tag format, adding 4 bytes between the source MAC address and the Ether type field. A special Ether type indicates that the next structure is the 802.1Q header, which in turn contains the actual Ether type of the encapsulated payload.

This also slightly increases the maximum Ethernet length frame from 1518 to 1522 bytes (4 bytes), while leaving the minimum size unchanged.



Ports are generally classified as either access or trunk ports. Ports that are directly connected to host are called **Access Ports**. Access Ports receive and transmit **untagged frames**, since the port's configuration already determines the VLAN.

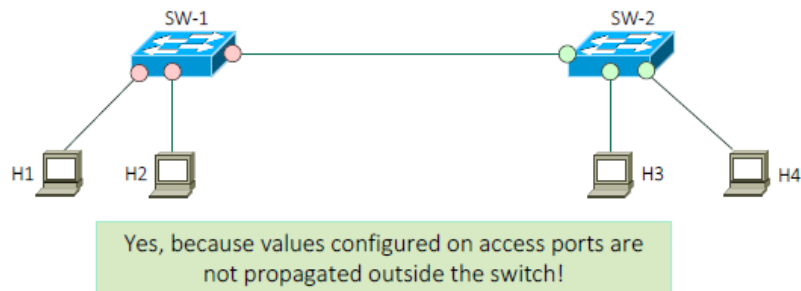
- Default configuration (on hosts, switches, servers, routers, etc)
- Usually used to connect end-stations to the network
- Hosts do not need to change their frame format (transparency)
- Administrator will configure the switch



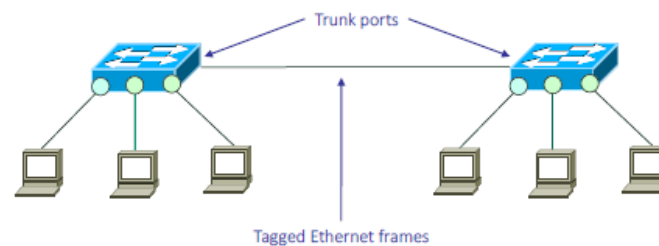
Given the following network:

- All ports are configured in "access mode"
- SW-1 is configured with the RED VLAN on all its ports
- SW-2 is configured with the GREEN VLAN on all its ports

Hosts connected to each switch can still communicate with each other because access-port VLAN settings do not propagate across switches.



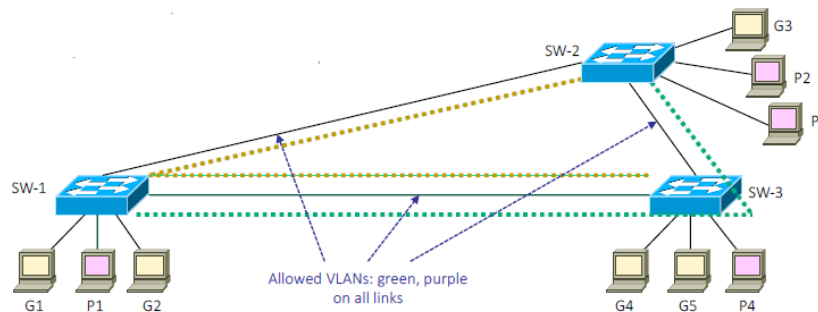
Trunk ports, by contrast, carry **tagged frames** and must be explicitly configured. They are used between switches or between switches and devices that participate in multiple VLANs, such as servers or routers.



Hosts can be assigned to VLANs in multiple ways:

- **Port-based** VLANs are the simplest.
- **Transparent assignment** mechanisms can associate VLANs with MAC addresses.
- **Per-user assignment** (802.1X) enables multiple users on a single workstation to be placed in different VLANs dynamically.
- **Cooperative assignment** allows hosts themselves to send tagged frames, which is common for servers or routers that participate in multiple VLANs; in these cases, trunk connections are required.

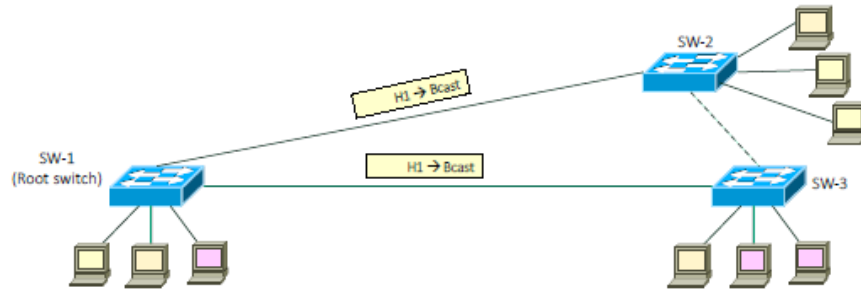
STP is computed first to eliminate loops, and VLANs operate on the resulting topology. Many vendors support Per-VLAN Spanning Tree, although some allow reverting to a common STP. Cisco, notably, always uses Per-VLAN STP.



Even with VLANs, network isolation is not absolute. Although frames are confined within their own VLAN and cannot cross into another, the physical links that carry this traffic are still shared. This means that a problem originating in one VLAN can indirectly affect others simply because they rely on the same underlying infrastructure.

A clear example is the case of a broadcast storm. VLANs do not inherently protect the network from this condition. Broadcast traffic is propagated across the entire switching fabric, except on access (edge) ports, which are tied to a specific VLAN. If a broadcast storm occurs within one VLAN, it can saturate a trunk link that is carrying traffic for multiple VLANs. Other VLANs do not receive the broadcast frames themselves, but they still suffer from the resulting congestion: the shared link may become overloaded and unable to transport traffic belonging to the other VLANs.

To mitigate this, it may be necessary to implement per-VLAN Quality of Service mechanisms. For example, a “round-robin” scheduling model based on VLAN IDs can allocate a minimum guaranteed amount of bandwidth to each VLAN, ensuring that traffic from one overloaded VLAN cannot starve the others.

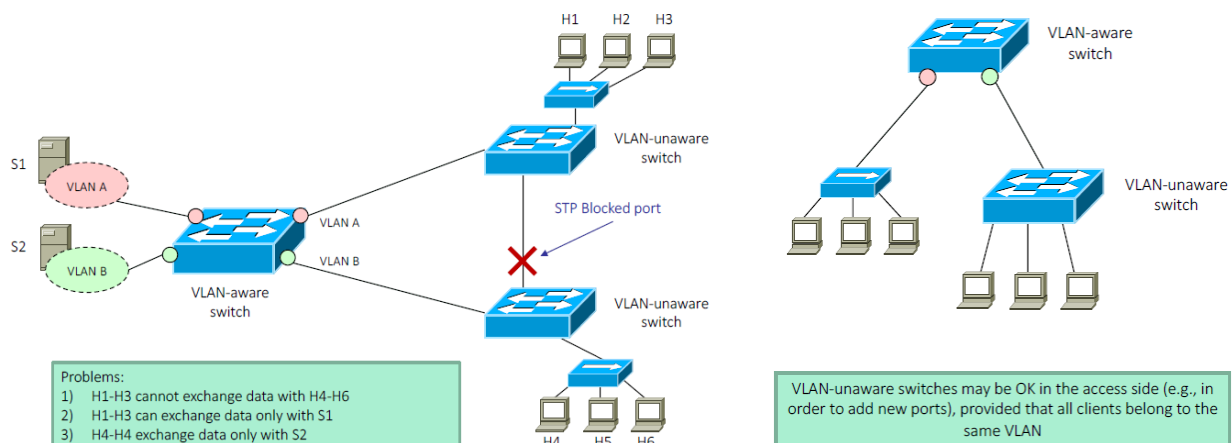


Two types of switches

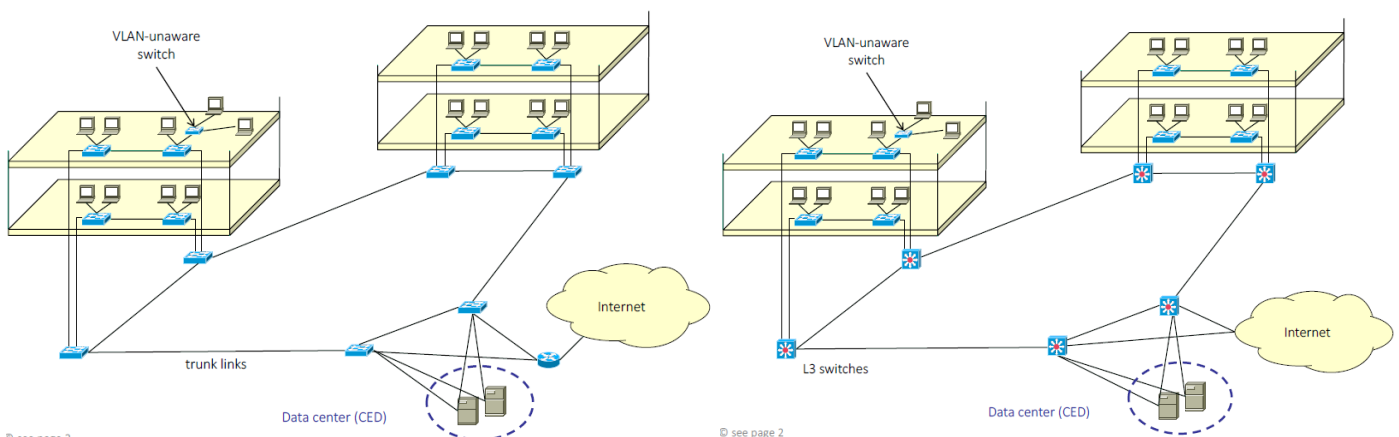
- **VLAN-Aware:** handle tagged and untagged frames
- **VLAN-Unaware:** do not accept tagged frames

Almost all professional switches support VLAN tagging, while most domestic (low-end) switches do not. VLANs are not a plug-and-play technology and require explicit configuration, unlike STP, which works automatically with some limitations. For this reason, VLAN support is usually absent in domestic devices, as typical users lack the skills needed to configure them correctly.

VLAN-aware and VLAN-unaware switches do not coexist well. A VLAN-unaware switch placed on a trunk link will drop tagged frames, unintentionally isolating hosts. However, such switches can still be used at the access layer, provided that all connected hosts belong to the same VLAN.



As a result, VLAN-unaware switches are common even in corporate networks. While network managers deploy professional, VLAN-capable switches, end users often extend their connections using cheap, unmanaged switches. Network design must therefore account for this possibility to avoid unexpected network misbehavior.



Conclusions

- Modern wired LANs are based on Switched Ethernet
 - Star topology with full-duplex links
 - Collision domain no longer exists
 - No need for CSMA/CD
 - Fault tolerance given by redundancy + Spanning Tree Protocol
- Wide adoption of VLANs
 - Traffic isolation
 - Broadcast domain size reduction
 - Routers required for both internal and external communications

Layer-3 functionality is increasingly pushed closer to the user, often via L3 switches.

Routing

Introduction and Taxonomy

Routing is the process of finding the best path for data to travel across a network. It involves two main parts: **routing algorithms** and **routing protocols**, which operate in the control plane.

The **control plane** is responsible for creating and updating routing tables that store information about network paths.

On the other hand, the **data plane** uses these routing tables to forward packets from one device to the next. Forwarding itself is straightforward—it involves looking up the next hop in the routing table and sending the packet along that path, without making decisions on the fly.

This clear separation allows routing to efficiently manage network paths while forwarding ensures smooth packet delivery.

Forwarding Procedure

You already encountered different forwarding methods when studying IPv4.

For example:

- Forwarding based on the network address (NetID).
- Label swapping in MPLS, which is another data-plane forwarding mechanism.
- Source routing, where the packet explicitly carries the path in its header. This is rarely used today due to significant security vulnerabilities.

Regardless of the specific mechanism, forwarding typically involves:

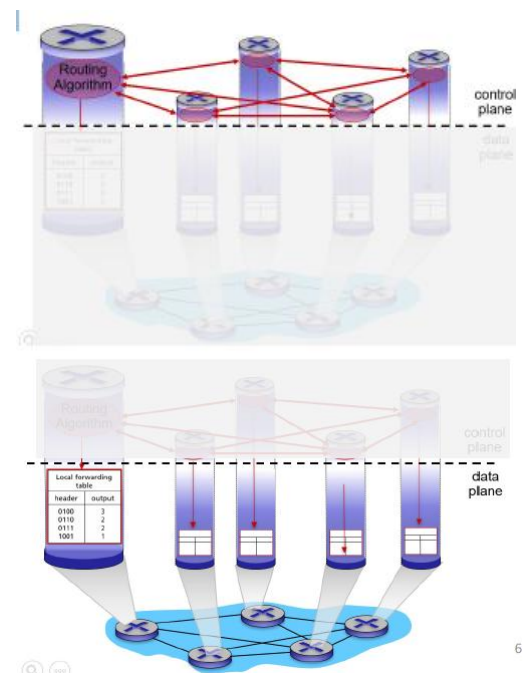
1. Next-hop/output port selection (using routing tables)
2. Switching: transfer the packet from input to output.
3. Transmitting the packet onward.

These operations belong to the data plane.

Routing Algorithm Classification

A routing algorithm defines the sequence of operations required to determine a suitable path—"optimal" with respect to some metric, such as: number of hops, delay, level of congestion, possibly other considerations, especially in large-scale networks.

Routing algorithms fall into two broad families: static (non-adaptive) and dynamic (adaptive).



Static (Non-Adaptive) Routing

Static routing algorithms choose paths without considering the current state of the network. Routing tables may be manually configured by an administrator and remain unchanged until explicitly updated, or they may follow simple rules that do not depend on topology or traffic conditions.

Examples include:

- **Random Routing** → Each router chooses an outgoing port at random for each incoming packet. Since decisions are independent of network conditions, this is considered a static approach.
- **Flooding** → The router forwards an incoming packet on all output ports except the one the packet arrived on. This mechanism is also independent of network state and is conceptually similar to Layer-2 flooding.
- **Hot-Potato (Deflection) Routing** → The router always selects the least congested output port at that moment. While this uses a minimal amount of local information, it still ignores the global network state, and is therefore generally categorized as non-adaptive.

Static routing has obvious drawbacks:

- It depends heavily on human administrators, making it highly error-prone.
- It does not react to changes such as link failures.
- It scales poorly and is impractical for large, dynamic networks.

For these reasons, static routing is rarely used except in very controlled environments.

Adaptive Routing

Adaptive routing algorithms account for the current state of the network, reacting to events such as topology changes, variable link costs, congestion, or failures. Depending on the design, routers may have partial or complete knowledge of the network state, and reactions may be fast or slow.

Centralized routing

In centralized adaptive routing, one special node—called the Routing Control Center (RCC)—collects information from all other routers in the network. It has full visibility of the topology and link states. It computes the optimal routing tables and distributes them to every router.

Advantages:

- The RCC has complete and consistent information from all nodes.
- It computes and disseminates globally optimal routing decisions, so decisions are globally consistent.
- Needs information from all nodes
- Because all logic is centralized, troubleshooting becomes significantly easier.
- Optimizes performance
- Simplifies troubleshooting, centralization prevents routing loops and inconsistent states

Disadvantages:

- A single point of failure: if the control center fails, routing halts.
- A major bottleneck, since all routers must send updates to it and receive results.
- Poor performance in highly dynamic networks due to heavy update traffic and potential congestion around the central node.

Isolated Routing

Each node makes decisions independently, based only on local information. There is no exchange of routing information.

This is how Layer-2 switches operate when building forwarding tables: each switch uses backward learning without consulting others. The routing is dynamic, tables evolve as hosts move, but isolated.

Distributed Routing

Routers cooperate in exchanging information to compute paths independently but cooperatively. There is no single control center. Decisions should ideally be coherent among routers, though inconsistencies can arise.

Distributed routing can rely on partial information, or global information.

Distributed Routing with Partial Information: Distance Vector Algorithm

Distance Vector Routing is based on a simple but powerful idea: each router learns about the network through its immediate neighbors.

Each router receives from its one-hop neighbors a list of reachable destinations and the cost to reach them, e.g., number of hops on the path. These neighbors in turn learned their information from their own neighbors, and so on. By successive updates, routing knowledge propagates gradually through the network.

Because information is only learned one “hop” at a time, convergence is not instantaneous. Transient states may produce suboptimal routing or even temporary loops. But once convergence occurs, all routers share a consistent view.

Bellman–Ford algorithm

The algorithm is commonly implemented using the **Bellman–Ford algorithm**, where each node maintains two data structures:

- **Distance Table (DT)**: lists costs to any known destination through each neighbor (next hop)
- **Routing Table (RT)**: lists costs and next hops of selected routes to each destination

A Distance Vector contains the same information as the RT, minus the next hop.

The distance table has one row per each possible destination and one column for each adjacent node. Each entry stores the cost from the router X to destination D_i via neighbor N_j , so it is the cost to of reaching D from X, using N_j as next hop.

The value in each table entry is:

$$\text{Cost}(X \rightarrow N_j) + \min_{\text{neighbors of } N_j} \{\text{Cost}(N_j \rightarrow D_i)\}$$

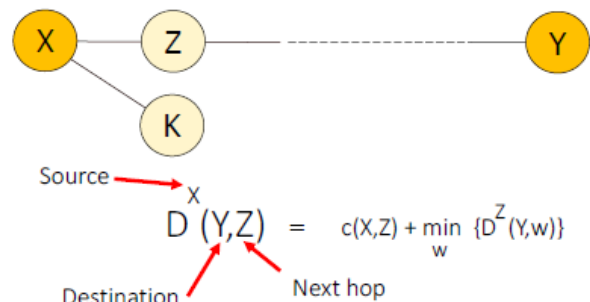
The first term is known locally by router X. The second term is learned from the distance vector sent by neighbor N_j

When router X receives a distance vector from neighbor N:

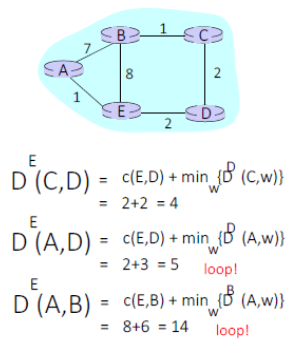
1. New destinations → If N announces a destination unknown to X, X adds it to its distance table and routing table.
2. Better path → If N provides a cheaper route to a destination than the one X is currently using, X updates both tables accordingly.
3. Increased cost → If X is currently routing through N and N reports that the cost to a destination has increased, X must increase its own cost—unless it knows of an alternative route.

The same method applies to all entries: the table exhaustively records the cost to each destination through each neighbor. Only afterward does the router extract the best next hop to form its routing table; the path with minimum cost is taken.

In this example E is the source. $D^E(A, B)$ means go from E to A, through B. The cost to go from A to B is 8, then to go from B to A, is better to pass trough C, D, E and then reach A, rather than go from B to A directly. This can form a loop because when we arrive at E, the packet go again to B. This is not important.



Distance Table: example



Routing © see page 2

From DT to RT

Cost to .. Via ...				
D ^E ()	A	B	D	
A	1	14	5	
B	7	8	5	
C	6	9	4	
D	4	11	2	

Destination Table

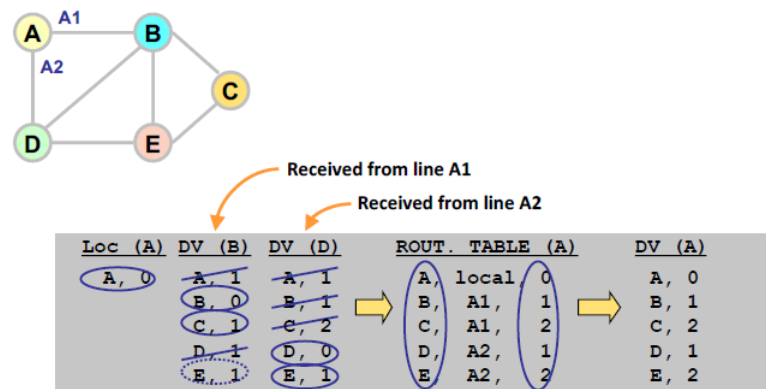
Cost to .. Via ...				
D ^E ()	A	B	D	
A	1	14	5	
B	7	8	5	
C	6	9	4	
D	4	11	2	

Destination Table

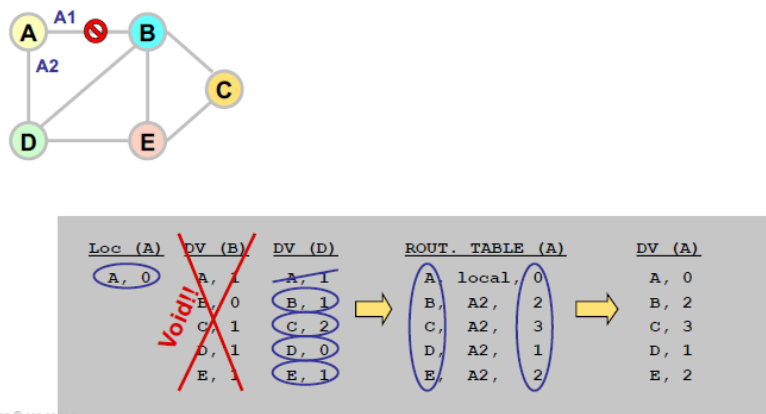
Next Hop, Cost		
A	A	1
B	D	5
C	D	4
D	D	2

Routing Table

Sample scenario: the neighbours do not tell how they reach a node, but only the cost to reach it.



If topology change:



ting © see page 2

While simple, Distance Vector routing suffers from slow convergence and problems such as routing loops, count-to-infinity, black holes, and bouncing effects, mainly because routers lack global topology knowledge.

→ A node implementing a Distance Vector algorithm increases the cost to reach the destination when it receives a DT with cost increase from the neighbour used to reach the destination.

In Distance Vector routing, a routing table is derived from the distance table by selecting the shortest path per destination.

Path Vector

Today we looked at an evolution of the distance vector algorithm known as the path vector algorithm. The idea behind it is to compensate—at least partially—for the limitations of the distance vector approach. It does this by including, within the advertised distance information, the entire path that a router uses to reach a destination. Hence the name path vector.

- Nodes exchange dynamically-updated path information
- Easy to detect loops in the vector
- Increased overhead



A:

Loc (A)	PV (B)	PV (C)	RT (A)	PV (A)
A, 0	A, 1, [B]	A, 1 [C]	A, loc, 0	A, 0, [-]
	B, 0, [-]	B, 1, [B]	B, A1, 1	B, 1, [A]
	C, 1, [B]	C, 0, [-]	C, A2, 1	C, 1, [A]
	D, 2, [B,C]	D, 1, [D]	D, A2, 2	D, 2, [A,C]

The mechanism is simple: each neighbour advertises not only the destinations it can reach, but also the ordered list of routers that must be traversed to get there. Because every update contains the full path, routing loops become easy to detect. If a router appears twice or more in the path, a loop clearly exists. The router detecting this inconsistency can then avoid using that information and prevent traffic from being routed along a looping path.

Of course, this improvement introduces additional overhead, which is why path vector techniques must be used judiciously.

Distributed Routing with Global Information: Link State algorithms

Distance vector and path vector algorithms rely on partial information exchange since routers only talk to their neighbours. In contrast, link state algorithms use **global information** and adopt a different strategy.

With link state, each router communicates with all other routers in the topology, not only its neighbours. Every node sends cost info (status) of each link in broadcast to all other nodes. Every router broadcasts information describing:

- how many ports it has,
- to which routers these ports are connected,
- and the cost or state of each link.

From this, every router can independently reconstruct the entire network topology and also detect inconsistencies, knowing all link costs. Every node compute minimum cost path (paths) toward every other node.

For example, if router A claims to be connected to router F on a certain port, but F does not list any link to A, the information is inconsistent and should not be used.

Routers share this information by using **multicast addresses** among routers, that only routers subscribe to. This avoids wasteful broadcasting. Through this controlled form of flooding—limited only to routers—the entire network's link state information is distributed efficiently.

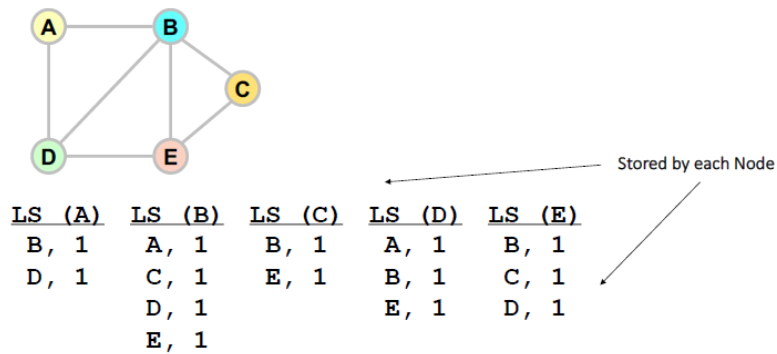
Once this exchange is complete, every router possesses the same global view of the network. They then run the same algorithm—Dijkstra's shortest path algorithm—to compute optimal paths. Each router builds a link state database containing the link state information received from every other router.

Consider a simple scenario in which every link has cost 1:

- Router A's link state lists neighbours C, B, and D.
- Router B lists A, C, D, and E.

These reports are consistent because A lists B and B lists A.

After running Dijkstra's algorithm, each router determines its own shortest-path tree and compiles its routing table. Because all routers have the same information, their routing tables remain consistent.

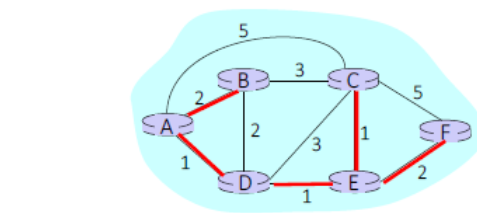


→ In a Link State Algorithm, the connectivity (i.e., the number of links) of another node is known when a link state advertisement is received from that node through flooded multicast.

Dijkstra Algorithm

- Iterative algorithm: after k iterations the minimum cost path toward k destinations are obtained
- Only works for positive costs
- Low complexity: $L * \log(N)$
 - o L: number of links
 - o N: number of nodes
- Also known as Shortest Path First
 - o The next node "nearest" to the root is identified
 - o Its next hop is inserted into the routing table

Step	start N	D(B),p(B)	D(C),p(C)	D(D),p(D)	D(E),p(E)	D(F),p(F)
→ 0	A	2,A	5,A	1,A	infinity	infinity
→ 1	AD	2,A	4,D		2,D	infinity
→ 2	ADE	2,A	3,E			4,E
→ 3	ADEB		3,E			4,E
→ 4	ADEBC					4,E
5	ADEBCF					



see page 2

destination	Next hop, cost
B	B,2
C	D,3
D	D,1
E	D,2
F	D,4

Link state routing offers several clear benefits:

- Fast convergence → All routers know the network topology, so changes (LSs) propagate rapidly, without intermediate processing.
- Limited Routing Traffic and Storage
 - o Link states are small
 - o Fast and efficient neighbor greeting
- Controlled flooding → Although link state uses flooding, it is multicast-based and restricted to routers that subscribe to the specific multicast group.
- Small advertisements → Link state packets only contain a router's direct links, which are usually few. By contrast, distance vector messages can grow large because they must list all known destinations.
- Rare occurrence of loops → The global topology and consistent databases make loop formation unlikely.
- Simple to understand and troubleshoot → Since all routers work with the same information, debugging inconsistencies is easier. All nodes have identical databases.

The main complexity in link state algorithms lies in implementing **selective flooding** correctly. It took years before the relevant RFCs fully standardized these mechanisms. Today they are stable and widely deployed.

Conceptually, link states should be regenerated only when topology changes. In real protocols, link states are also periodically refreshed: this increases reliability and ensures databases remain synchronized even without explicit topology changes.

Internet Routing Architecture

Routing protocols

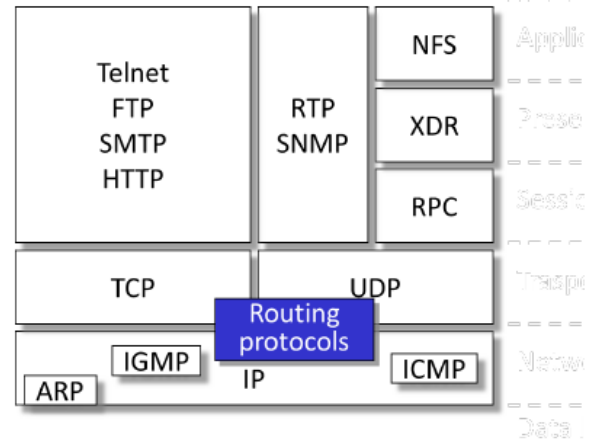
Protocols that allow routers to exchange information on the network and determine the best route to each destination uses routing algorithms.

Routing protocols operate in the control plane and serve two primary functions:

1. Implement the algorithm that selects the best path.
2. Collect and distribute routing information across the network.

A routing protocol specifies:

- which metrics are used and how they are encoded in packets,
- specific timing on how often updates are sent (periodically or event-driven),
- and the scope of its operation.



Problem: what is the operational domain of the protocol? A subnet? An operator's network? A country? The Internet at large?

When a network grows to the scale of the global Internet—with potentially billions of reachable destinations—routing faces major scalability challenges.

Routers cannot simply store an individual entry for every possible destination; the routing tables would become impossibly large. Likewise, exchanging complete routing information among all routers would generate so much traffic that the links would quickly become saturated.

The routing model described so far is therefore an idealized one:

- all routers cooperate equally,
- the network is viewed as a single, flat structure,
- and every router has the same role and responsibility.

In reality, the Internet does not operate as a flat network. It is composed of many separate organizations, each running its own network with its own policies. This leads to the notion of administrative autonomy.

Autonomous System (AS)

The Internet is fundamentally a network of networks. Each network is operated by an independent administrative entity—public or private—that manages its own routers and links. These entities are called Autonomous Systems (AS).

AS is a set of subnets grouped based on topology and organizational criteria e.g., the subnets of a large internet service provider. Each AS:

- **Autonomous internal routing choices** → manages its own internal routing,
- **Negotiated external routing choices** → maintains connectivity to the external Internet, and cooperates with other ASes so the global Internet remains reachable.

Cooperation is driven not by coercion but by mutual benefit. If an ISP could only reach destinations within one country, its service would be unattractive. Universal reachability gives the Internet its value.

An AS is identified by a 2-byte AS number, assigned by IANA, allowing for around 65,000 ASes under the traditional scheme.

To the outside world, an autonomous system appears as a single cloud. Other ASes don't know or care about its internal structure—only that it can accept incoming traffic and potentially forward it to certain destinations →

Scalability

Internet approach to scalable routing

Each AS is typically operated by a single administrative entity—such as an ISP, a company, or a university—and expresses its autonomy in two fundamental ways.

Intra-AS routing (IGP – Interior Gateway Protocol)

Intra-AS routing refers to routing within a single Autonomous System. All hosts and routers that belong to the same AS exchange routing information using a common, agreed-upon routing protocol.

A key requirement is that every router in the AS must run the same intra-domain routing protocol, ensuring internal consistency and smooth operation.

Different ASes, however, are free to use different internal protocols. For example, one AS might use OSPF while another uses IS-IS or RIP. The choice of protocol is entirely up to the AS operator and does not affect other ASes.

Routers located at the boundaries of the AS are known as gateway routers. These devices connect the AS to neighboring ASes and therefore participate both in the internal routing of their own AS and in routing across AS boundaries.

Inter-AS routing (EGP – Exterior Gateway Protocol)

Inter-AS routing governs how different Autonomous Systems exchange information about reachable destinations. Unlike intra-AS routing, which operates entirely within a single AS, inter-AS routing coordinates traffic flow between autonomous systems across the global Internet.

This role is handled by gateway routers, which sit at the boundary of their AS. These routers communicate with gateways in neighboring ASes to determine how traffic should be forwarded beyond their local domain. Because they operate at the intersection of multiple networks, they participate in both intra-domain routing (inside their AS) and inter-domain routing (between ASes), making them essential to end-to-end connectivity.

Only the edge routers of an AS run BGP. These gateway routers have a dual function:

- Internally, they use the AS's intra-AS routing protocol to reach all internal networks.
- Externally, they run BGP to exchange reachability information and apply routing policies with neighboring autonomous systems.

This combination enables the Internet to scale globally while respecting the autonomy of each administrative domain.

Intra-AS Routing

Interconnected ASes

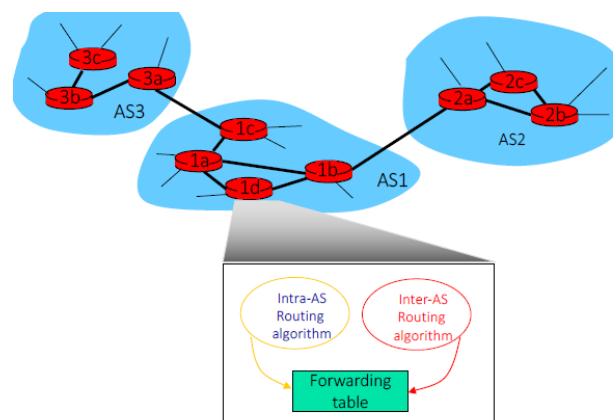
A forwarding table of a router is configured by both intra- and inter-AS routing algorithm

- intra-AS routing determine entries for destinations within AS
- inter-AS & intra-AS determine entries for external destinations

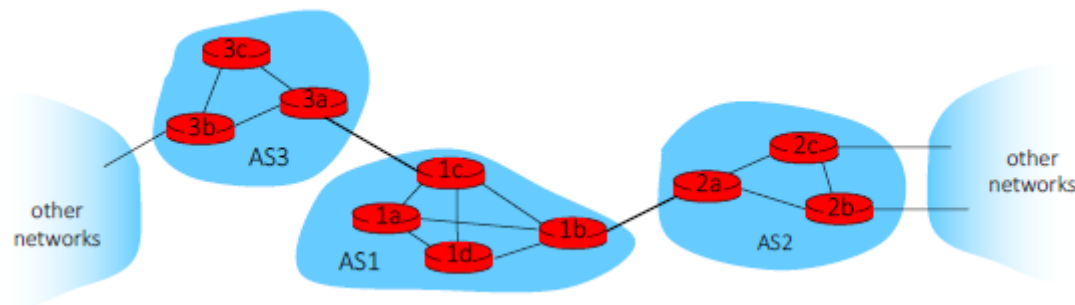
Consider an AS (AS1) situated between two other systems (AS2 and AS3). AS1 is a transit AS, meaning traffic may enter from AS2 and exit toward AS3.

Routers inside AS1 must determine how to reach internal destinations (via intra-AS routing), and how to reach external prefixes advertised by AS2 or AS3 (via inter-AS routing).

Suppose a router in AS1 receives a datagram whose destination lies outside AS1. The router knows it must forward the packet to a gateway router—one of the routers connecting AS1 to another AS. But which gateway should it choose? For this, AS1 needs to learn which external



destinations are reachable through AS2 and which are reachable through AS3 and distribute this reachability information to all internal routers so that every router can choose the correct gateway. This is precisely the role of **inter-AS routing**.



Intra-AS routing - also known as **Interior Gateway Protocols (IGPs)** - governs routing within a single Autonomous System. All routers inside the AS must run the same IGP for consistency. Common intra-AS routing protocols include:

- OSPF (Open Shortest Path First) – link state
- IS-IS – link state
- EIGRP (originally Cisco-proprietary, now public)
- RIP (distance vector, now obsolete)

OSPF and IS-IS rely on link state advertisements using multicast and incorporate strong authentication mechanisms to prevent malicious router announcements or black-hole attacks. OSPF also supports multipath routing.

OSPF (Open Shortest Path First)

OSPF is called “open” because its specification is publicly available. It is a link-state protocol, meaning every router:

- floods link-state advertisements (LSAs) throughout the AS,
- builds a complete topology map of the AS, and
- computes routes using Dijkstra’s shortest-path algorithm.

Some key features of OSPF:

- Link-state advertisements are distributed to all routers in the AS using controlled flooding.
- OSPF packets are carried directly over IP, not over TCP or UDP.
- For each link, the router advertises its state and associated cost.
- The IS-IS routing protocol operates in a nearly identical manner.

OSPF “advanced” features

OSPF includes several advanced capabilities that make it more powerful than simpler IGPs like RIP:

- Security: All OSPF messages can be authenticated to prevent malicious updates.
- Equal-cost multipath (ECMP): Multiple paths with the same cost may be used simultaneously. (RIP allows only one.)
- Multiple cost metrics per link: A link may have different costs depending on the Type of Service (ToS). For example, a satellite link might be given a low cost for best-effort traffic but a high cost for real-time traffic.
- Integrated unicast and multicast routing:
 - o Multicast OSPF (MOSPF) extends OSPF to support multicast using the same topology database.
- Hierarchical organization: OSPF supports network hierarchies to scale efficiently in large ASes.

Hierarchical OSPF

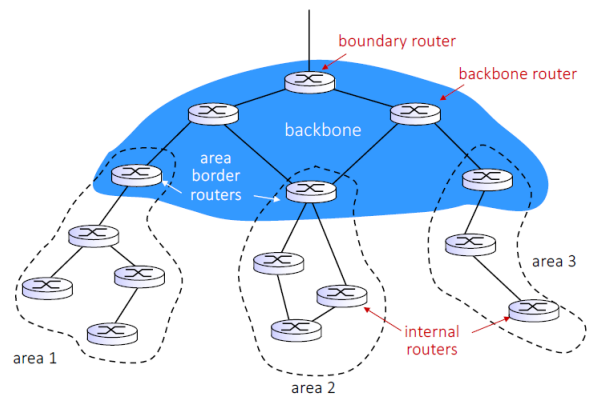
To improve scalability, OSPF organizes large networks into a two-level hierarchy:

1. **Areas** – local regions where detailed link-state information is exchanged.
2. **Backbone** (Area 0) – the core area that connects all other areas.

Key aspects of this hierarchy:

- Link-state advertisements remain within an area, reducing overhead.

- Routers maintain full topological knowledge of their own area but only know summary routes for other areas.
- Area Border Routers (ABRs): summarize the networks within their area and advertise this information to other areas.
- Backbone routers: participate only in OSPF routing within the backbone.
- Boundary routers: connect the AS to external Autonomous Systems.



This hierarchical structure reduces routing traffic and allows OSPF to support very large networks efficiently

Inter-AS Routing

Over the years, numerous issues and misconfigurations have demonstrated how fragile global routing can be. A recent example involved a Cloudflare outage: although the root cause was an internal misconfiguration, the real impact came from how this error influenced the BGP advertisements associated with Cloudflare's services. Because BGP determines how networks across the world reach those services, a faulty announcement had widespread consequences.

For this reason, many people view BGP as a double-edged sword. On one hand, it is indispensable: without BGP, the Internet as we know it simply would not exist. On the other hand, it is also considered a potential liability—a weak point—because there have been several cases in which BGP malfunctioned or was misused, forcing operators to devise numerous countermeasures to keep routing stable.

Inter-AS routing must consider not only technical metrics—such as hop count or delay—but also routing policies. These policies reflect business relationships and administrative decisions, such as whether two ASes are willing to exchange traffic or which paths should be preferred or avoided.

BGP

In practice, there is only one inter-AS routing protocol in use on the Internet: BGP (Border Gateway Protocol). At its core, BGP provides each Autonomous System with a way to learn which IP prefixes are reachable through neighboring ASes. This exchange of reachability information between different ASes is known as External BGP (eBGP).

Once an AS receives reachability information from its neighbors, it must propagate this information internally. This is handled by Internal BGP (iBGP), which distributes the learned prefixes to all routers inside the AS so they can make consistent forwarding decisions. In essence, BGP is built on cooperation between internal routing protocols (such as OSPF or IS-IS) and external routing, which governs communication between ASes.

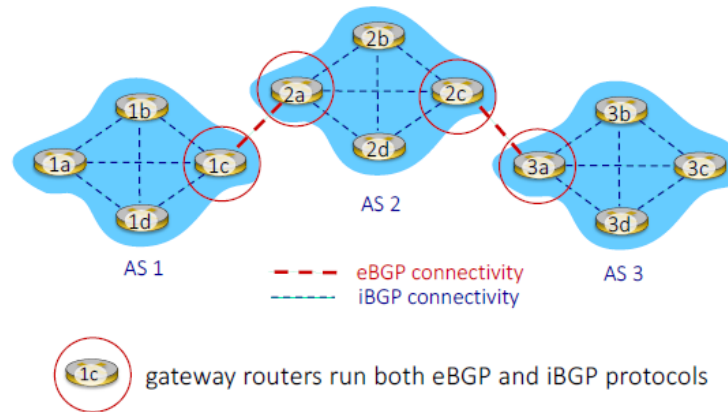
After collecting reachability data, each AS selects the best routes, that is not necessarily the best path but it is based on both reachability and policy. BGP prioritizes policy compliance and operational stability, meaning the chosen path is the one that preserves reachability while supporting the AS's economic or administrative goals (Reflect agreements among ASs).

BGP also relies heavily on prefix aggregation. Autonomous Systems typically advertise the shortest possible prefixes in order to reduce the size of global routing tables. It allows subnet to advertise its existence to rest of Internet: "I am here".

However, if a more specific (longer) prefix is advertised—for example, a /24 inside a /16—BGP will prefer the more specific one. This behavior ensures precision in routing: aggregated prefixes reduce complexity, but specific prefixes still override them when necessary.

Destinations can be aggregated: 195.1.2.0/24 and 195.1.3.0/24 can be announced as 195.1.2.0/23

eBGP, iBGP connections



Consider a network composed of three Autonomous Systems—AS1, AS2, and AS3. In this scenario, AS2 acts as a transit AS, forwarding traffic between AS1 and AS3. Routers 2A and 2C are the edge routers of AS2.

In such a topology:

- **eBGP sessions** (highlighted in red) operate on the links connecting AS2 to AS1 and AS3. These sessions exchange reachability information between ASes.
- **iBGP sessions** (shown in black) distribute the received route advertisements to other routers inside AS2 so that all internal routers share the same view of external destinations.

It is important not to confuse iBGP with an intra-AS routing protocol. While intra-AS protocols (like OSPF or IS-IS) create the actual connectivity inside an AS, iBGP merely propagates external reachability information within the AS. The two types of protocols coexist and complement each other but serve distinct purposes.

The purpose of iBGP is to take advantage of the connectivity already established by the intra-AS routing protocol. Its job is to distribute reachability information that was originally learned through eBGP. In other words, eBGP allows the edge routers of an autonomous system to discover which external networks are reachable, while iBGP propagates that information internally.

However, iBGP alone cannot operate if the autonomous system has not already organized itself using an intra-AS routing protocol. iBGP does not tell routers how to reach one another inside the AS. It simply distributes reachability information, relying on the existing internal routing protocol—which has already created full internal connectivity—to forward packets.

A key point is that iBGP connectivity is not the same as intra-AS routing. Internal routing (e.g., OSPF or IS-IS) builds topology knowledge and forwarding paths inside the AS; iBGP only injects external reachability into that structure.

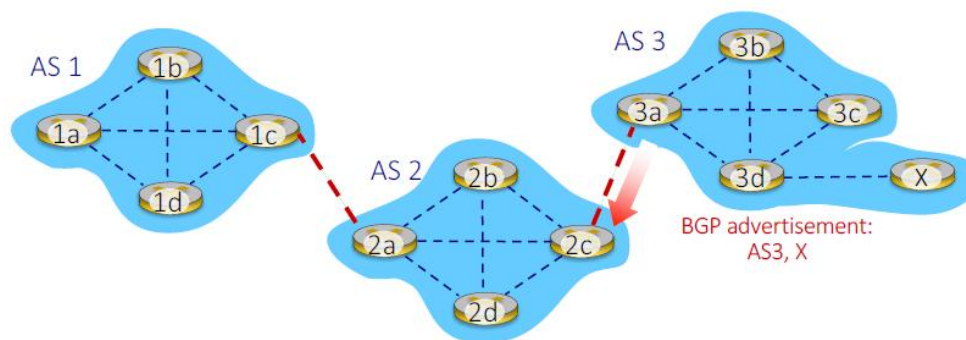
Gateway routers therefore run both eBGP and iBGP at the same time.

BGP basics

- **BGP session:** two BGP routers (“peers”) exchange BGP messages over semi-permanent
- **TCP connection:** advertising paths to different destination network prefixes

BGP operates by establishing persistent sessions between routers running BGP. These sessions are carried over TCP, which means that BGP behaves like an application-layer protocol that performs routing functions. BGP messages are encapsulated inside TCP, then inside IP, and finally inside whatever Layer-2 protocol the network is using.

Because BGP relies on long-lived TCP connections, these sessions serve as the channel over which routers exchange advertisements about reachable destinations.



Suppose AS3 hosts a network prefix X. Its edge router, 3A, advertises reachability for X to AS2's edge router, 2C, using eBGP. Once 2C receives this information, it distributes it inside AS2 to other edge routers—such as 2A and 2D—using iBGP over the TCP sessions established between all edge routers.

If AS2's policy allows it, AS2 may also propagate that same advertisement to AS1 via eBGP. In this case, AS1 learns that to reach X, it should go through AS2 and then AS3. However, this depends entirely on policy: if AS2 does not want AS1 to route traffic to X through it, the advertisement simply will not be forwarded.

This is because any eBGP advertisement acts as a commitment: once an AS advertises a prefix externally, it is promising that it will accept traffic for that prefix. If an AS does not want to carry that traffic, it must not advertise the prefix.

Inside AS2, 2C continues to propagate the information via iBGP to all routers so that they know how to reach X, even if the information is not exported further by eBGP.

Path attributes and BGP routes

Advertised prefix includes BGP attributes → **prefix + attributes = "route"**

BGP advertisements contain attributes that enrich the reachability information. The most important ones are:

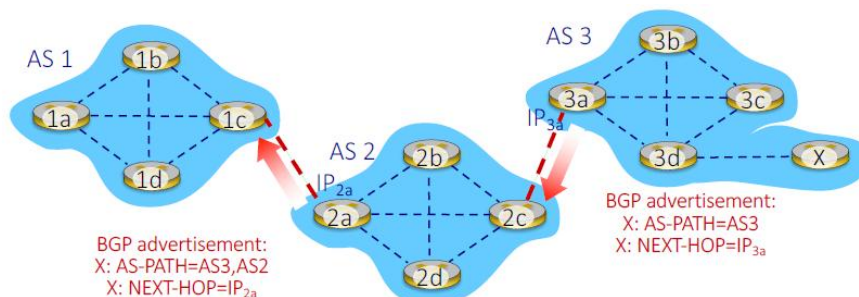
- **AS Path:** list of ASes through which prefix advertisement has passed. This makes BGP a path vector protocol.
- **Next Hop:** the IP address of the internal-AS router that should be used as the next hop toward the destination.

→ Conventionally, BGP identifies as NEXT-HOP the IP address of the first router interface in the first AS of the AS-PATH

Policy-based routing:

- gateway receiving route advertisement uses import policy to accept/decline path (e.g., never route through AS Y).
- AS policy also determines whether to advertise path to other neighboring ASes

For example, if AS3 advertises X, the initial AS Path is simply "AS3," and the next hop is the IP of 3A. When AS2 propagates this internally, the next hop remains 3A's address. When AS2 advertises X to AS1, AS1 sees a path "AS3 AS2," but the next hop becomes the IP address of router 2A, since 3A is not directly reachable from AS1.



BGP messages

Routers that run BGP are called **peers**. Each peer maintains a TCP session with all other peers it must exchange information with. Edge routers usually create:

- one TCP connection to every other edge router in the same AS (for iBGP), and

- one TCP connection to each neighboring AS that they exchange routes with (for eBGP).

Over these TCP sessions, routers exchange standard BGP message types:

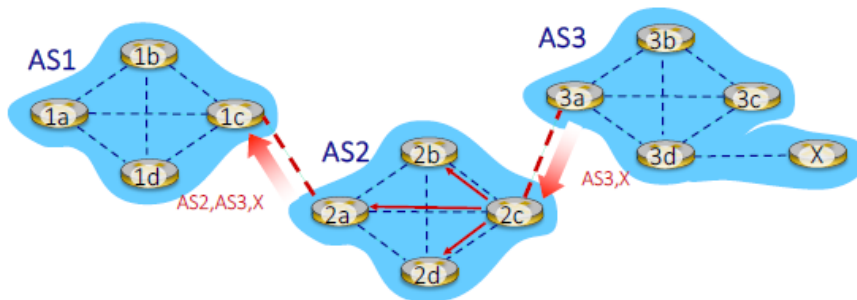
- OPEN: opens TCP connection to remote BGP peer and authenticates sending BGP peer
- UPDATE: advertises new path (or withdraws old)
- KEEPALIVE: keeps connection alive in absence of UPDATES; also ACKs OPEN request
- NOTIFICATION: reports errors in previous msg; also used to close connection

Example:

AS2 router 2c receives path advertisement AS3,X (via eBGP) from AS3 router 3a

Based on AS2 policy, AS2 router 2c accepts path AS3,X, propagates (via iBGP) to all AS2 routers

Based on AS2 policy, AS2 router 2a advertises (via eBGP) path AS2, AS3, X to AS1 router 1c



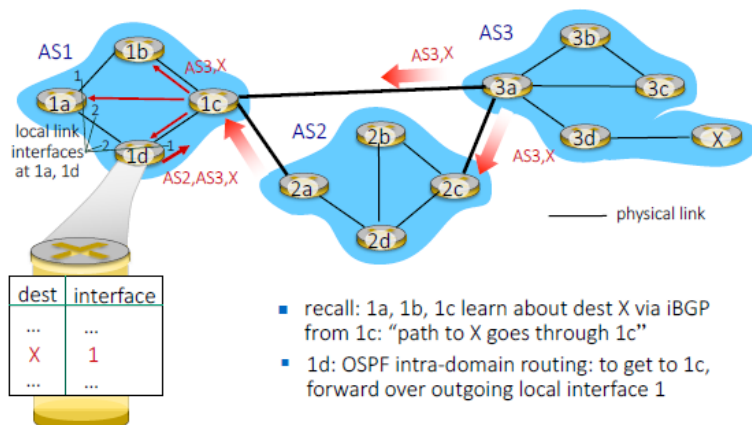
BGP, OSPF, forwarding table entries

Consider an AS where routers learn about prefix X via iBGP from router 1C. The internal routers—1A, 1B, and 1D—add forwarding entries for X in their tables.

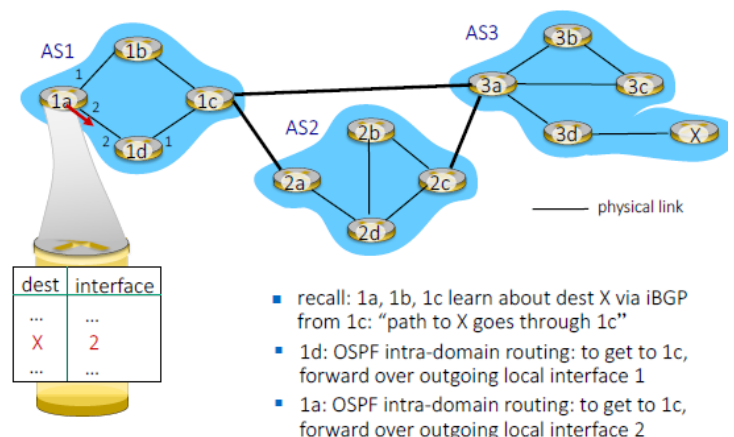
For example, router 1D will record that X is reachable through an internal interface that leads to 1C. Meanwhile, 1A also learns that X is reachable through 1C, but since 1A is not directly connected to 1C, it consults the intra-AS routing protocol (e.g., OSPF) to determine how to reach 1C. This cooperation between eBGP, iBGP, and the IGP is fundamental:

- eBGP imports external routes into the AS
- iBGP distributes those routes to all routers in the AS
- the intra-AS protocol determines how to physically reach the BGP next hop

Q: how does router set forwarding table entry to distant prefix?



Q: how does router set forwarding table entry to distant prefix?



Router may learn about more than one route to destination AS, selects route based on:

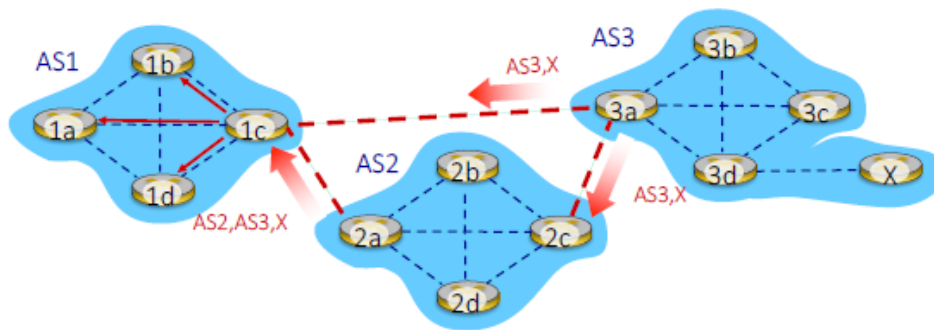
1. local preference value attribute: policy decision
2. shortest AS-PATH
3. closest NEXT-HOP router: hot potato routing

4. additional criteria

Example: router 1C might learn two paths to X:

- AS1 gateway router 1c learns path AS2,AS3,X from 2a
- AS1 gateway router 1c learns path AS3,X from 3a

The choice is entirely policy-based. A network administrator might prefer AS2 simply because AS2 offers a cheaper transit agreement, even if the direct link to AS3 is shorter, and advertises path within AS1 via iBGP



Problems: BGP Hijacking

A major weakness of BGP is that it is based entirely on trust. When one AS advertises a prefix, other ASes trust that advertisement with no built-in verification mechanism. A BGP hijack happens when an AS falsely originates someone else's IP prefix: other networks believe that AS is the "right" path for that traffic. The attacker (or misconfigured AS) announces a prefix, often more specific than the legitimate one (e.g., /24 vs /23), and BGP's "longest prefix match" makes most of the Internet send traffic to the wrong place.

Common causes of BGP issues, like route hijacking, include misconfigurations (such as announcing wrong prefixes or missing filters), compromised routing accounts, and deliberate attacks aiming to intercept or block traffic.

A well-known incident occurred in 2008, when the Pakistani government attempted to censor YouTube. Local telecom operators implemented the block by advertising a more specific prefix for YouTube's network, ensuring that all local traffic was redirected away from YouTube. Unfortunately, this advertisement leaked to the global Internet, and because BGP prefers more specific prefixes, much of the world began routing YouTube traffic toward Pakistan unintentionally.

To prevent this, network operators use measures like prefix filtering, origin validation, limits on prefix announcements, and better security practices at ISPs and internet exchange points. These steps help keep global routing reliable and secure.

Hot Potato Routing

Hot potato routing is a routing strategy commonly used in inter-domain routing with BGP. The main idea is simple: an Autonomous System (AS) tries to forward traffic out of its own network as quickly as possible, minimizing the use of its internal resources.

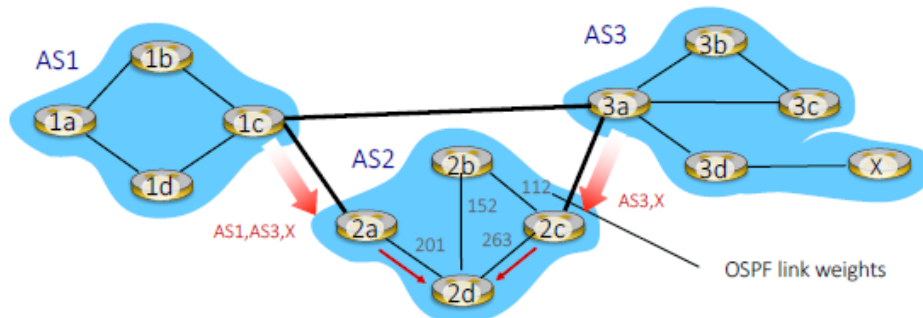
Consider a router, such as 2d, that learns via iBGP that it can reach a destination X through two possible exit points, 2a or 2c. Both gateways provide a valid route to X, but they differ in the intra-domain cost, that is, the cost of reaching the gateway within the AS.

With hot potato routing, router 2d chooses the gateway with the lowest intra-domain cost, for example 2a, because it is closer in terms of internal network metrics. This decision is made even if exiting through 2a results in more inter-domain (AS-level) hops to reach X compared to exiting through 2c. The guiding principle is: "get the traffic out of the AS as fast as possible", without worrying about what happens afterward in other ASes.

This behavior can lead to suboptimal end-to-end routing. In fact, according to standard BGP decision criteria, a route such as (AS3, X) might be preferred based on higher-level attributes (for example, fewer AS hops). However, once

multiple routes are considered equivalent with respect to those attributes, BGP relies on the closest exit rule, which triggers hot potato routing.

As a result, although BGP may initially identify a path that looks better from an inter-domain perspective, the final forwarding decision favors minimizing internal cost. This highlights a key limitation of hot potato routing: it optimizes for the local AS, not for global routing optimality.



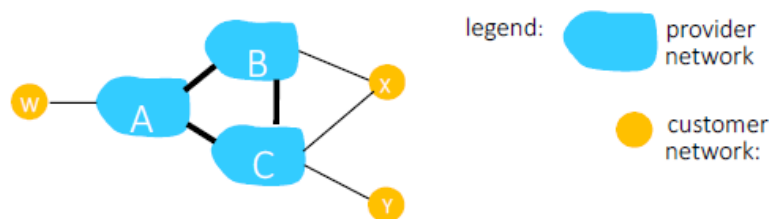
BGP Connection Types

There are two types of BGP interconnections:

- **Transit:** an ISP can provide reachability to the entire Internet for another endpoint (e.g., enterprise, content or application provider, residential broadband provider, etc.). Accepts traffic that do not originate inside the autonomous system and it is not destined inside the AS, so it is just goes through the AS. The endpoint entity pays the ISP to carry traffic to and from the Internet
- **Peering:** The networks interconnect to exchange only traffic that originates or terminates within their networks (including the networks of their customers, in the case of carriers and Tier 1 networks)
 - o **Public peering:** through an internet exchange point (IXP) -> Peering with multiple networks
 - o **Private peering:** networks interconnect to exchange only traffic that originates or terminates within their networks

→ A network Z with a public BGP peering will not carry traffic does not have source or destination in Z

Consider three provider networks—A, B, and C—and three customer networks—W, X, and Y. Suppose X is dual-homed to B and C but wishes to remain only a peer, not a transit provider. X will therefore avoid advertising routes that would allow B and C to use X as a shortcut between them.



Let us now look at a case of Peering Agreements among ISPs

- A advertises path Aw to B and to C
- B chooses not to advertise BAw to C:
- B gets no “revenue” for routing CBAw, since none of C, A, w are B’s customers
- C does not learn about CBAw path
- C will route CAw (not using B) to get to w

Rule of thumb: traffic flowing across an ISP’s backbone network must have either a source or a destination (or both) in a network that is a customer of that ISP.

Multimedia Networking and Quality of Service

Multimedia traffic includes all forms of non-textual digital content, such as audio and video, as well as immersive environments like virtual reality. These applications typically require tight performance guarantees, because users are very sensitive to impairments in delivery. Typical multimedia applications include audio–video communication platforms (such as WhatsApp, FaceTime, Zoom), virtual reality systems, teleconferencing, and streaming audio and video services.

Streaming refers to the continuous delivery of audio or video data over time, and it differs fundamentally from browsing, messaging, or file downloads. In most cases, streaming is one-way and non-interactive. Streaming applications can be divided into:

- **Real-time streaming**, for example watching a live sports event on a smartphone.
- **Non-real-time streaming**, such as video-on-demand (VoD) services like Netflix, Amazon Prime, or Disney+.

Real-time streaming is particularly sensitive to delay and, even more importantly, to delay variation (jitter). Some amount of delay is unavoidable in real networks, but as long as the delay is relatively constant, the user experience remains acceptable. When delay fluctuates significantly, packets may arrive later than expected, causing playback gaps, audio/video desynchronization, and visible interruptions. In contrast, VoD applications are more throughput-sensitive, since buffering can compensate for delay but not for sustained bandwidth shortages.

Traditionally, the networking community describes performance in terms of Quality of Service (QoS), which measures how well the network meets application requirements such as delay, jitter, packet loss, and throughput. However, the Internet—both IPv4 and IPv6—operates under a best-effort service model, meaning that IP provides no guarantees on bandwidth, delay, or loss.

For this reason, operators increasingly focus on **Quality of Experience (QoE)**, which reflects the user’s perceived satisfaction rather than raw network metrics. Delays may exist, but if playback appears smooth and uninterrupted, the QoE is considered good. For example, watching a football match without knowing the result may feel “live” even if it happens later, as long as playback quality is high. Applications can also trade off video quality against playback stalls to optimize QoE.

Engineering QoS for multimedia

Two broad strategies exist in real networks:

- **Overprovisioning** in the core and CDNs (Content Distribution Networks) close to users: shorter paths and fewer bottlenecks.
This approach simply increases network capacity—more bandwidth, more fiber, faster switches and routers—so that congestion and performance degradation rarely occur. By throwing bandwidth at the problem, the network remains so well-provisioned that even best-effort service performs acceptably.
- **Application-layer techniques**: HTTP adaptive
Applications such as Netflix perform smart buffering. They assess the connection quality and decide how much data must be preloaded before playback begins. The buffer is large enough to ensure smooth playback but not so large that the user must wait excessively. As playback proceeds, more data is continuously downloaded. Interruptions occur only during major network outages or drastic throughput reduction.

What about network-layer mechanisms?

Streaming Apps Requirements

For conversational multimedia, especially when voice is involved

- Very low **one-way delay (<150 ms)**, to keep conversation natural and controls responsive. Beyond that threshold, users begin to notice inconsistencies in conversational flow.
- **Delay variation** (jitter) must be small and smoothed by short playout buffers, otherwise audio and video become choppy or desynchronized. Large variation cannot be masked with buffering, because buffering would itself introduce extra delay, which conversational applications cannot tolerate.

- **Packet loss** must be limited, ideally below 1–2% for real-time media, because too many missing packets cause artifacts, frozen frames and audio glitches. Packet loss is somewhat more tolerable in multimedia. Loss rates of 1–2% might cause small glitches or brief interruptions but usually do not severely impact user perception.
- Available **throughput** must exceed the media bitrate with some margin, so the application can adapt quality without frequent stalls. Throughput requirements depend entirely on the media bitrate, which varies widely—from low-resolution smartphone video to high-quality 4K or 5K streams on large displays. Throughput must be high enough to accommodate the bitrate plus some margin to tolerate temporary degradations.
- For non-interactive streaming (VoD, live events with a few seconds delay), larger buffers and slightly higher delay are acceptable, but loss and throughput still directly affect stall frequency and resolution. If throughput drops too much, playback stalls or quality drops.

Countermeasures in the Network

Three key mechanisms were introduced to support QoS:

1. Traffic classification
2. Packet scheduling algorithms, WFQ, RR, WRR, CBQ
3. Control on traffic entering the network at various levels

These mechanisms aim to compensate for IP's best-effort model by adding intelligence to routers.

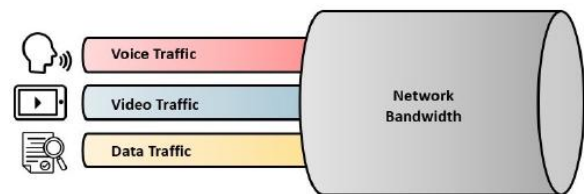
In other words:

- Limit the number of packets arriving at network nodes
- Handle appropriately packets that need specific QoS

Traffic Classification

Traffic classification is the process of distinguishing different traffic types, even though IP itself treats all packets alike. Applications differ fundamentally: messaging, videoconferencing, streaming, email, control traffic—and each type has different requirements.

To classify traffic accurately, routers must inspect more than the IP header. Several layers can help, based on implementation contained in the IP and TCP/UDP headers (5-tuple):



- Layer 3: Source and destination IP addresses (e.g., traffic from a known CDN)
- Layer 4: Transport protocol (UDP vs. TCP)
- Layer 4 ports: Source and destination port numbers (well-known ports allow identification of HTTP or other services)

By examining these fields, routers can infer the type of traffic and decide whether it requires special handling.

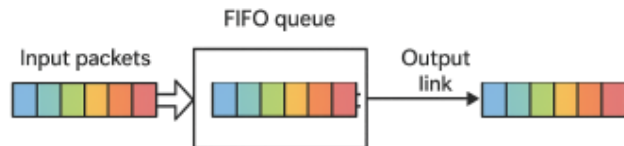
However, classification at high link rates (e.g., 100 Gbps) requires specialized hardware implementations because software inspection would be too slow:

- **ASIC**: Application Specific Integrated Circuit
- **CAM**: Content Addressable Memory

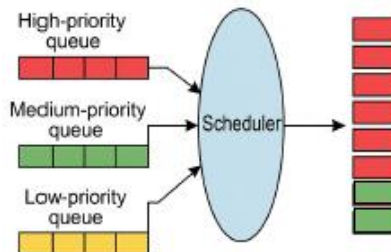
Packet Scheduling Algorithms

Once traffic is classified, routers must decide which packet to send next on each outgoing link (who goes first). Multiple queues exist, and different scheduling algorithms manage how packets from these queues are selected.

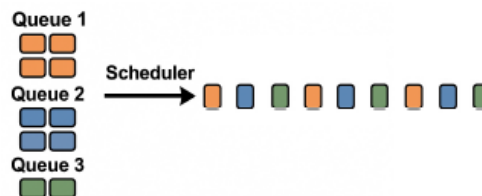
1. **FIFO (First-In, First-Out)**, packets leave in the same order in which they arrived. No differentiation between traffic types.



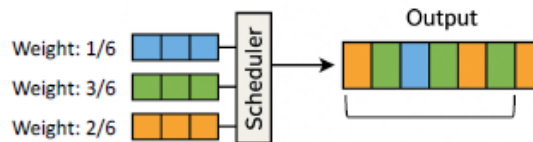
2. **Priority Queuing**, traffic is divided into priority classes. The highest priority queue is always served first, which guarantees the best treatment for that class but risks starving lower-priority traffic. Multimedia would perform well, but messaging or email could be delayed indefinitely.



3. **Round Robin**, each queue is served in turn, one packet at a time. All queues are treated fairly and equally, with no prioritization. This avoids starvation but cannot provide preferential treatment to multimedia flows.



4. **Weighted Fair Queuing (WFQ)**, this method combines the ideas of priority and fairness. Each queue is assigned a weight, representing its relative importance. The scheduler serves queues in a statistical round-robin fashion, giving higher-weight queues a larger share of the bandwidth without completely starving others. This provides balanced, controlled preferential treatment.

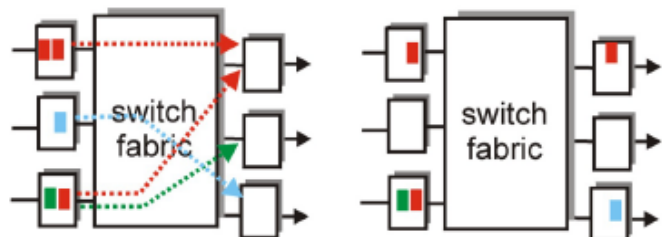


Note: switching capacity

Whenever packet scheduling is implemented inside a router, an efficient switching fabric is required. The switching fabric is the internal subsystem that transfers packets from input ports to output ports based on their destination. It represents the router's internal processing and interconnection capability.

Inside each router, multiple switching paths exist, and the capacity of the switching fabric must be higher than the external link bitrates. If the switching fabric could only forward packets at the same rate as a single input line, it would not be able to handle situations where multiple packets arrive simultaneously from different input ports and are destined for the same output port. In that case, internal contention would occur, leading to internal queues, increased delay inside the router, and potentially packet loss.

To guarantee immediate switching without internal blocking, routers require speed-up in the switching fabric. Speed-up means that the internal transfer rate of the fabric is greater than the line rate of each input port. With sufficient speed-up, multiple inputs can send packets to the same output at the same time, and the switching fabric can serialize them within a single external time slot.



Without adequate speed-up, output contention causes internal buffering, increased latency, and possible cell or packet loss. This requirement becomes especially critical at very high-capacity links, such as 100 Gb/s or 400 Gb/s, where even small internal inefficiencies rapidly translate into large buffers and severe QoS degradation.

Control on Traffic entering the network

Once traffic has been classified and intelligent queueing policies have been implemented inside the network, it is also necessary to control and shape traffic at the network boundaries. This is particularly important for network operators that have strict service-level agreements (SLAs) with large content and media providers. Traffic cannot simply be injected freely into the operator's network; instead, it must respect specific traffic profiles.

This is the domain of traffic engineering, a complex process that determines which traffic is allowed to enter the network and how it is handled internally using classification, queueing, and scheduling mechanisms.

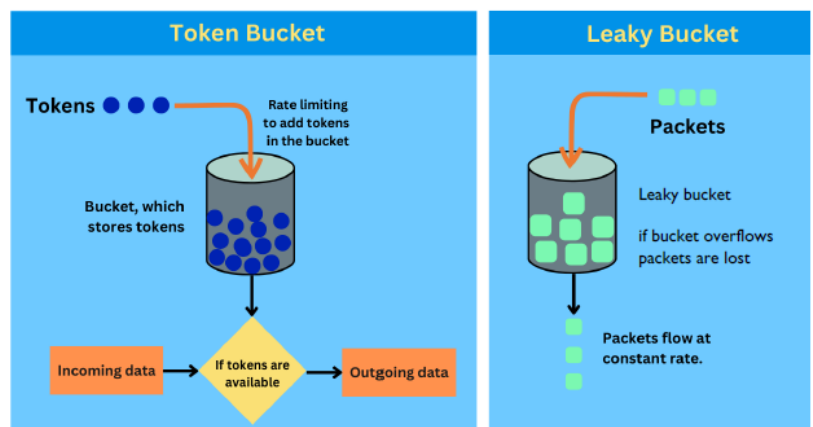
Traffic control can be applied at different levels:

- **Packet level**, through policing and shaping mechanisms
- **A priori**, through network engineering decisions such as dimensioning and admission policies
- **User level**, by limiting the number of active users
- **Network-wide**, by distributing traffic in a controlled manner across the infrastructure

Policing and shaping traffic

Traffic policing and shaping regulate how traffic flows use bandwidth, ensuring compliance with QoS requirements and SLAs.

The token bucket mechanism allows controlled bursts. Tokens are generated at a rate R and stored in a bucket with capacity B . A packet is transmitted (or marked as in-profile) only if enough tokens are available. This ensures that the average throughput does not exceed R , while still allowing short bursts up to B . In other words, the long-term offered traffic rate is limited by the token generation rate.



The leaky bucket mechanism smooths traffic by draining packets (or credits) at a constant rate R . If packets arrive faster than this rate, they are queued or dropped, resulting in an output stream with nearly constant bitrate.

The two mechanisms are often used together. Even if many tokens are available, traffic is not transmitted instantaneously at a very high rate, which would cause congestion.

In practice:

- Token bucket is typically used for policing, especially at network edges. It classifies packets as in-profile or out-of-profile and may drop or remark packets that exceed the agreed traffic contract.
- Leaky bucket is typically used for shaping, delaying packets in a buffer so that traffic conforms to a configured rate even when the source sends bursts.

In practice, many routers implement shapers as a token bucket followed by a leak bucket, with tokens controlling dequeue rate and the queue absorbing bursts, combining the flexibility of token bucket with the smoothing effect of a leaky bucket.

Network Engineering/Traffic Engineering

Network operators adopt preventive strategies to maintain performance:

- The network is dimensioned for near worst-case scenarios, assuming that all users may transmit close to their maximum allowed rate.

- Traffic statistics are collected and analyzed.
- A traffic matrix is computed to understand traffic distribution across the network.

During operation:

- The network state is continuously monitored.
- Network dimensioning and the traffic matrix can be adjusted dynamically if conditions change.

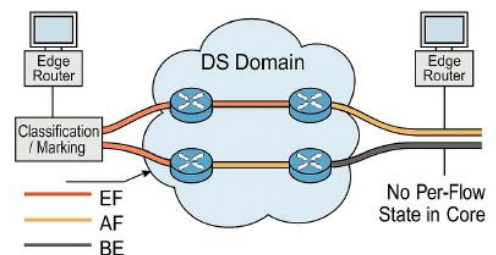
Internet's Ambitious Solution: Integrated Services (IntServ)

- uses the Resource Reservation Protocol (RSVP) to signal QoS requirements along the end-to-end path, ensuring that each router can reserve the necessary resources
- maintains per-flow state information in routers, allowing for precise control and management of individual traffic flows
- admission control ensures that new flows are only admitted if sufficient resources are available, preventing resource contention and maintaining QoS guarantees
- offers multiple service classes, including guaranteed service for strict latency and bandwidth requirements,

Too much overhead! – Not used

Lowering Ambitions: Differentiated Services (DiffServ)

- DiffServ uses an aggregate approach to QoS, classifying packets into broad categories rather than managing individual flows -> simpler network management
- When entering the DiffServ domain, packets are marked with a Differentiated Services Code Point (DSCP) using Type of Service in IPv4 and Type of Class in IPv6 headers
- Routers apply predefined per-hop behaviors (PHBs) based on these markings (via queueing/priority scheduling/WFQ)



Lighter and more manageable

DiffServ Today

Inside ISP cores networks: packets are marked with a small set of DSCP values, mapped to a few behavior aggregates (e.g., EF for voice, AF classes for premium / assured traffic, BE for best effort)

In enterprise and campus networks:

- QoS policies for voice (VoIP), video, critical business apps.
- Access switches mark/re-mark DSCP; WAN edge routers use them to prioritize during congestion.

At the boundaries with other technologies:

- Wi-Fi QoS (WMM) usually maps DSCP to access categories (different wait times before channel sensing, that depends on the priority of the packet)
- Mobile networks (LTE/5G) often map IP DSCP to specific bearers, i.e., they use DSCP as input to internal QoS classes

Virtual Private Network

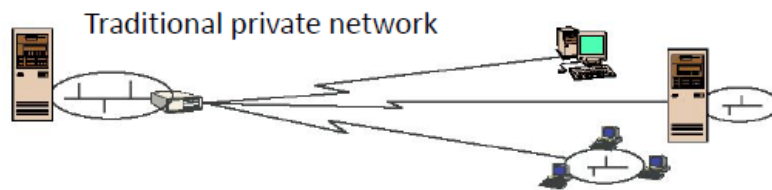
A Virtual Private Network (VPN) provides connectivity across a shared infrastructure, such as the Internet, while maintaining the same policies, control, and security as a private network. Essentially, it allows organizations to connect multiple remote sites or users securely over public networks, as if they were part of the same private infrastructure.

Shared infrastructure: private/public network e.g., the one of an Internet Service Provider, IP, frame Relay, ATM, the Internet

Policies: security, Quality of Service (QoS), reliability, addressing, etc.

Motivation and Purpose

Imagine a large company with several branches located in different cities or even different countries. These branches need to communicate and exchange data regularly. Before VPNs existed, this was done by installing physical cables or leased lines between each branch and the main headquarters.



While technically feasible, this approach was not scalable and extremely expensive—especially for companies spread over vast areas, such as across the United States. VPNs were therefore introduced to reduce costs and enhance security, providing a flexible way to connect distant offices over the Internet without exposing their internal traffic to outsiders.

Motivations

VPNs enable cutting costs with respect to expensive connectivity solutions.

Private Networks are based on:

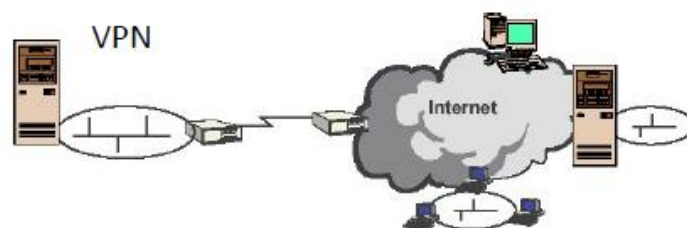
- private leased lines;
- long distance dial-up solutions;

VPN enables selective and flexible access to corporate network (services).

- Limited services available to external users:
 - o high security;
 - o few services allowed through firewall;
- All intranet functionalities available to corporate users accessing from the Internet:
 - o VPN connection allowed through firewall;
 - o services available as if connected directly to the corporate network;

How It Works

The idea behind a VPN is to create a secure “tunnel” over the Internet, a shared and potentially untrusted infrastructure, connecting two or more endpoints. For example, you might have a VPN server at the headquarters and a VPN gateway at a branch office. These two endpoints communicate through the Internet, but all data is encrypted and encapsulated within the VPN tunnel. This means that, although the packets travel across public networks, they remain protected, confidential, and authenticated, preventing third parties from intercepting or altering the data.



Practical Example

Let's say a company's headquarters are in New York and one of its branches is in Chicago. Without a VPN, the company would have to pay for a dedicated private line between the two sites. The cost to realize the direct link is higher compared to the cost an ISP requires to realize a VPN. For instance:

- A T1 connection between San Francisco and New York could cost around \$10,000 per month.
- Dial-in access from Denver or Chicago to San Francisco might cost \$600 per month. Over three years, using VPNs instead of dedicated



lines could result in savings of about \$237,600, even after accounting for VPN equipment costs of around \$7,800.

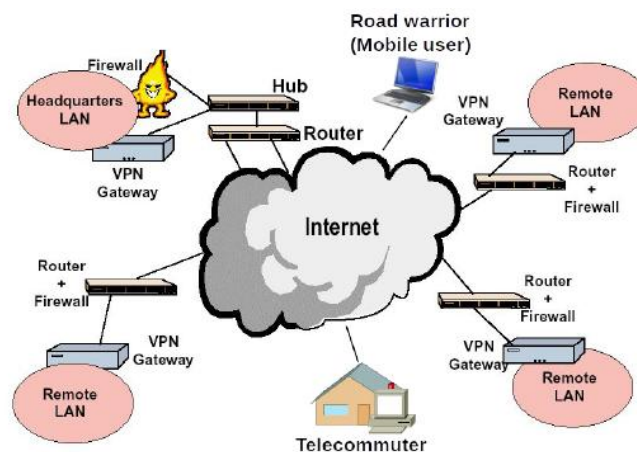
This clearly shows how VPNs provide a cost-effective alternative to traditional private networks.

VPNs in Modern Work Environments

Today, VPNs are not only used to interconnect branch offices but also to support remote work and mobility.

- Telecommuters are employees who work remotely, typically from home. They connect to the company's internal network via VPN, gaining access to the same services and privileges they would have inside the office LAN.
- Road Warriors, on the other hand, are mobile workers who connect from various locations, such as hotels, client offices, or even from their cars. Since they often switch networks (and therefore IP addresses), VPNs must handle mobility seamlessly, ensuring continuous connectivity even as the user moves between networks.

In both cases, VPNs guarantee secure access to the corporate network and ensure that authorized users retain the same rights, privileges, and protections as if they were physically present within the organization's infrastructure.



Key elements

1. Tunnel

- The VPN tunnel provides secure encapsulation of corporate traffic while it travels over shared networks like the Internet.
- In some VPN solutions, this tunnel also encrypts the traffic, ensuring confidentiality and integrity.
- However, not all VPN implementations include encryption; in certain lightweight configurations, the tunnel only handles routing or encapsulation without full security guarantees.
- Not present in some solutions

2. VPN Gateway

- The VPN gateway is a termination device on the corporate network.
- It can serve as the tunnel endpoint, opening and closing VPN connections.
- Depending on the architecture, it may be more or less complex—and thus more or less expensive—depending on the level of security, scalability, and functionality required.

Basic Terminology and Scenarios

When dealing with network security and VPNs, several core concepts are essential to understand:

- End-point authentication: ensures that the communicating entity (source or destination) really is who or what it claims to be.
- Data integrity: guarantees that transmitted data has not been modified or tampered with during transit (including coming from declared source).

- Data confidentiality: data cannot be accessed/read by anyone else than intended destination
- Data separation: achieved through tunneling, which isolates one user's traffic from others on shared networks.

Authentication and Integrity through Digests

Authentication and integrity can be achieved using a mechanism known as a digest. A digest is generated through a hash function, which takes a block of data and produces a much smaller, fixed-size output string, often called a hash value or message digest. While it may resemble compression, hashing is different: the result is not reversible, and even the slightest modification in the original data produces a completely different digest.

To add security, hash functions can be combined with cryptographic keys:

- In symmetric cryptography, the same key is shared between sender and receiver.
- In asymmetric cryptography, two keys are used: a private key (known only to the sender) and a public key (shared with others).

Here's how this works in practice:

1. The sender computes a digest of the message using a key (private or shared).
2. This digest is attached to the message and sent over the Internet.
3. The receiver, who knows the correct key, recomputes the digest from the received data.
4. If the new digest matches the one sent by the transmitter, two things are confirmed:
 - a. The data integrity is preserved — no alteration occurred, since any small change would produce a different digest.
 - b. The authentication is valid — only the legitimate sender, who holds the correct key, could have generated that matching digest.

This mechanism ensures that data arrives unaltered and from an authenticated source.

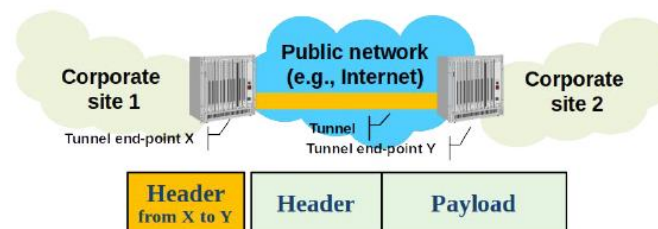
Tunnelling

In a VPN, tunneling is the process that allows packets from one private site (say, Site X) to reach another private site (Site Y) through a public network, such as the Internet.

The idea is simple but powerful: each packet from the private network is encapsulated inside another packet that can travel through the public infrastructure. The original header and payload become the payload of a larger packet with a new outer header, which contains public source and destination addresses.

This encapsulation allows two important things:

1. Communication between private networks even if they use public IP addresses.
2. Secure and isolated transmission of corporate data over an untrusted public medium.



Think of a tunnel as a virtual channel connecting two VPN gateways, one at the headquarters and another at a branch office. Network administrators configure these gateways so that they create a secure tunnel between them. From a logical standpoint, it looks like a direct, private link between the two sites, even though the data physically travels across the Internet. This encapsulation process is what makes the network appear as a single Virtual Private Network.

How Encapsulation Works (Step by Step)

1. Inside the private network, a host sends its packets as usual, with Layer 2, 3, and 4 headers and its data.
2. The packet is delivered to the local VPN gateway.

3. The gateway adds an extra outer header, effectively wrapping the entire packet inside another one.
4. This new outer header contains public IP addresses belonging to the two VPN gateways (X and Y).
5. Routers on the Internet only read the outer header, so they simply forward the packet based on the public addresses of the gateways.
6. When the packet reaches the remote VPN gateway, it recognizes the encapsulation, removes the outer header, and forwards the inner packet to the destination host within the remote private network.

From the routers' point of view, the communication seems to be happening only between two public IPs (the gateways). But from the company's perspective, it's as if both private networks were directly connected, a virtual extension of the LAN across the Internet.

This is the essence of a VPN: extending private networks securely and transparently over public infrastructure.

Why Two VPN Gateways Are Needed

Suppose you have a laptop with a private IP address (e.g., 10.0.0.1) and you try to reach an external service like Google without a VPN. You can send packets toward the Internet, but when Google tries to reply, it won't know how to reach your private address — private IPs are not routable on the public Internet. That's why VPNs always require two gateways:

- One on the sender's side to encapsulate outgoing traffic.
- One on the receiver's side to decapsulate and forward it correctly.

VPN Flavors (Types)

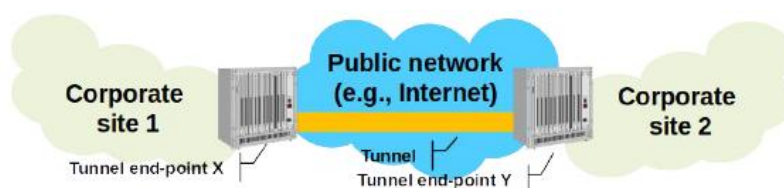
There are several types of VPNs, depending on what they connect and how they're used:

1. **Site-to-Site VPN**
 - a. Connects entire remote networks (e.g., branch offices to headquarters).
 - b. Acts as a virtual leased line between the sites.
2. **End-to-End VPN**
 - a. Connects individual hosts directly (e.g., two specific servers).
 - b. Also virtualizes a leased line but on a smaller scale.
3. **Access VPN (Remote or Virtual Dial-In VPN)**
 - a. Connects a single user (like a teleworker) to a remote corporate network.
 - b. Virtualizes the traditional dial-up access connection, enabling remote employees to work securely from anywhere.

s2s VPN Tunneling

In a site-to-site VPN, the tunnel is terminated on the gateways, usually the border routers of each company site. These routers are responsible for opening and closing the VPN tunnel between locations.

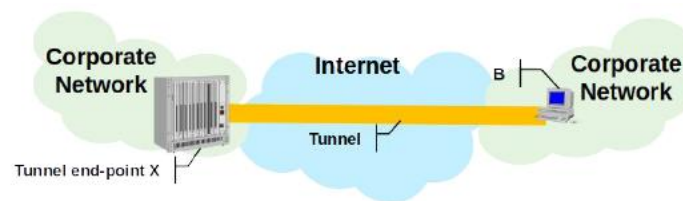
- Tunnel endpoints: The tunnel is established between the border gateways of different company sites.
- Connection: Once the tunnel is active, many-to-many communication is possible — all hosts belonging to the connected sites can communicate through the same single tunnel.
- Cost: Since only one tunnel is maintained for the entire inter-site connection, the computational load and network overhead remain low, making this solution cheap and efficient.
- Security: The main limitation is that data traveling within each local LAN, before reaching the gateway, is not yet encrypted. This means that local traffic (between hosts and their site's gateway) can still be intercepted or sniffed before the encryption process begins.



E2e VPN Tunneling

In an end-to-end VPN, the tunnel is terminated directly on the communicating end systems — meaning the two hosts themselves perform the tunneling.

- Tunnel endpoints: The endpoints of the tunnel are the two communicating hosts.
- Connection: Each pair of communicating hosts requires its own dedicated tunnel, so a new tunnel is created for every host-to-host communication.
- Cost: Because of this one-tunnel-per-connection design, the resource consumption can become highly expensive and unsustainable in large networks. The hosts themselves must be capable of handling the encryption and encapsulation tasks, which may exceed the capabilities of some devices.
- Security: The great advantage of this setup is that data is encrypted from the very beginning to the very end of the communication path. Since tunneling starts at the host level, even local LAN traffic is protected, making this the only solution that guarantees true end-to-end security.



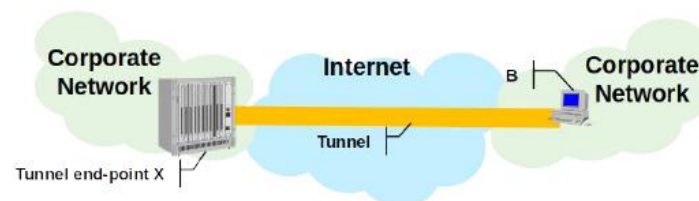
Conceptually, in this case, there are no external VPN gateways — instead, the hosts themselves run software that acts as a virtual VPN gateway. This setup is rarely used in practice, as it would require creating a separate tunnel for every pair of hosts across sites — which quickly becomes unmanageable. However, E2E VPNs are often adopted for highly sensitive communications, for example, when two servers or users must exchange confidential or encrypted information directly.

In some advanced scenarios, both site-to-site and end-to-end VPNs may coexist: you can have a secure site-to-site tunnel connecting two branches, and on top of it, end-to-end tunnels between specific hosts that require extra confidentiality.

Remote VPN Tunneling

A remote VPN (also called remote access VPN or virtual dial-in) is the most common and historically the first VPN model. Here, the tunnel is terminated between a VPN gateway and a remote host (such as a traveling employee's laptop or smartphone).

- Tunnel endpoints: The tunnel is established between the end system (the remote device) and the VPN gateway of the corporate network.
- Connection: Multiple users can connect simultaneously to the same VPN gateway. However, each external user must create their own tunnel to access the corporate network.
- Cost: This model sits in the middle ground — it's more resource-intensive than site-to-site, but far less than end-to-end VPNs.
- Security: While the external connection is encrypted, traffic inside the gateway's LAN could still be exposed to sniffing before encryption.



A practical example: suppose you're in Italy and want to access a service only available in the US, such as a region-locked streaming platform. When you connect through a VPN server located in the US, your packets are encapsulated, sent through the tunnel, and decapsulated within the US network. This makes your traffic appear as if it originated there, allowing access to geo-restricted content.

VPN Deployment Scenarios

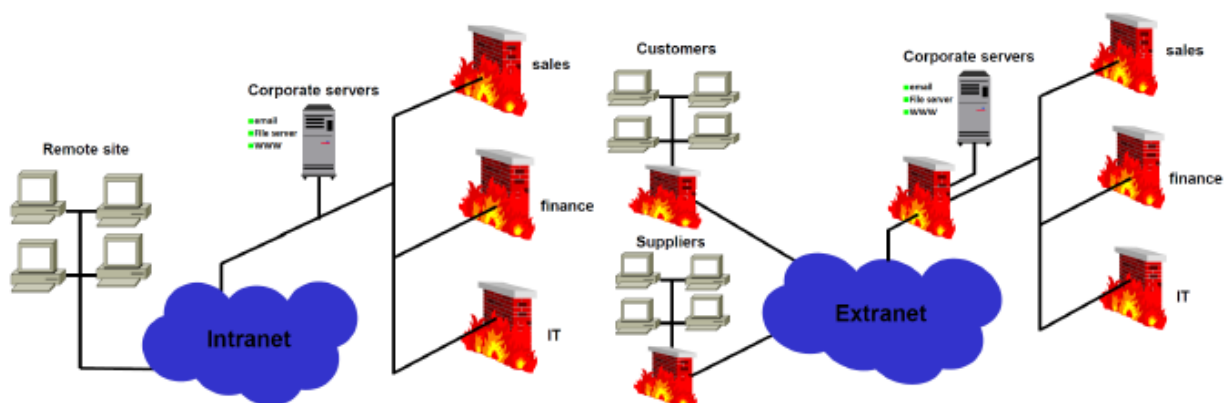
VPNs can be deployed in different organizational contexts. The two main categories are Intranet VPNs and Extranet VPNs.

1. Intranet VPN

This type interconnects the various branches and departments of the same organization, for example, the headquarters, remote offices, or telecommuters.

- Purpose: To securely connect all sites of a single company.
- Access control: Internal firewalls can be added to ensure that employees only access the services they're authorized for.
 - o For instance, administrative staff shouldn't access the research database, and researchers shouldn't access payroll data.

This setup ensures a secure internal communication channel within the company, even when employees work remotely or from different sites.



2. Extranet VPN

An extranet VPN interconnects customers, suppliers, partners, or communities of interest to a corporate Intranet.

- Purpose: To provide controlled, secure communication between independent networks.
- Access control: Firewalls are usually stricter here, ensuring that external users only reach the intended services.
 - o Example: Customers can access corporate servers, while suppliers are limited to the finance or logistics systems.
- Overlapping address spaces: Private IP ranges (such as 10.0.0.0/24 or 192.168.0.x/24) are often reused by multiple organizations. When two networks with overlapping private IPs want to communicate, the VPN gateway must use NAT to translate addresses. This ensures that each host has a unique, routable IP within the VPN, avoiding conflicts and enabling seamless communication.
- Traffic control: Firewalls and VPN gateways also ensure that partner traffic doesn't overload or compromise the internal corporate network.

Overall, Extranet VPNs demand more sophisticated configuration and coordination — both for security (firewalls, authentication) and address management (NAT, routing rules).

Issue	Description / Solution
Restricted access to network resources from interconnected networks	Enforced via firewalls at VPN gateways
Overlapping address spaces	Resolved with NAT
Open, standard-based solution	Enables interoperability among different organizations
Traffic control	Avoid that partner traffic compromises performance on corporate network

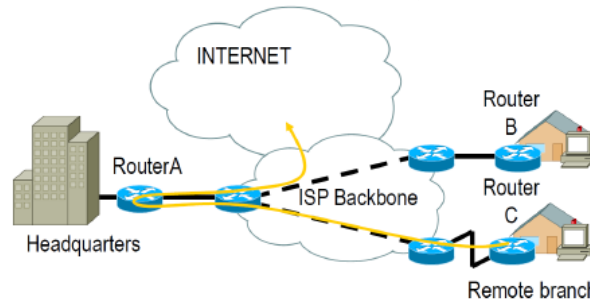
Internet Access via VPN

When discussing VPNs, one important topic is how remote branches or users access the Internet. There are two main approaches: centralized and distributed Internet access.

1. Centralized Internet Access

In a centralized setup:

- Remote branches or users only use the public IP network to reach the headquarters.
- Internet access is forced through the headquarters. In other words, all Internet traffic goes through the corporate VPN.
- The VPN carries both corporate traffic and Internet traffic, ensuring that all connections are encrypted.
- Centralized access control is enforced via a firewall at the corporate site.



Example scenario: imagine a European branch connected to the company's headquarters via a VPN tunnel. If a user at the branch wants to access the Internet (e.g., open a website or stream a video), all traffic must first pass through the VPN tunnel to the headquarters. From there, the headquarters routes the traffic to the Internet and back.

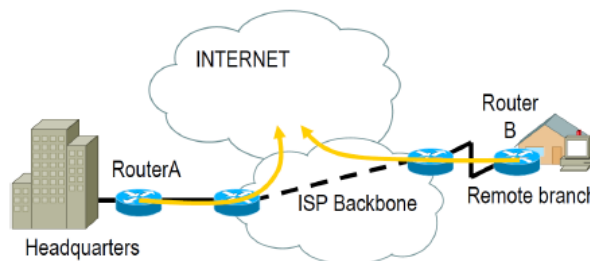
- This method ensures strong security, as all traffic is encrypted and filtered.
- Drawback: The path is longer, increasing latency. For example, even a simple request to access a website goes through the tunnel, then through the corporate Internet gateway, and finally back to the user.
- This model does not scale well for large organizations with heavy Internet use, as the VPN and headquarters router links can become overloaded.

Centralized Internet access is ideal when security is the primary concern, for example, when connecting over public Wi-Fi networks (like airports) or untrusted networks.

2. Distributed Internet Access

In a distributed setup:

- Remote branches or users access the Internet directly through their local ISP, rather than routing all traffic through the headquarters.
- The VPN is used only for corporate traffic, while Internet traffic bypasses the VPN.
- Costs are lower for the company, as employees pay for their own Internet service.



Example scenario:

If the branch has a VPN tunnel to headquarters for corporate services, the employee can still access websites or streaming services directly using their local connection. Only traffic destined for the company network goes through the VPN.

Advantage: Reduced latency and faster Internet access. Ideal for activities like streaming video or video calls.

Trade-off: Security for Internet traffic is not guaranteed. Malware or malicious traffic on the local network could potentially compromise the user's device or corporate VPN traffic.

Example — Accessing Netflix USA from Italy

Let's consider what happens when you try to access Netflix USA from Italy.

- Without a VPN: when your device sends a request to Netflix, the packet carries your private IP address as its source. Routers on the Internet can deliver the packet to Netflix's servers, but when Netflix tries to reply, it sends the response back to that same private address. Since private IP addresses (like those in the ranges 192.168.x.x or 10.x.x.x) are not routable on the public Internet, the response never reaches you, it gets discarded
In practice, your ISP usually performs Network Address Translation (NAT) to replace your private address with a public one. However, the key point is that your real, public location is still visible, in this case, Italy, so Netflix will redirect you to the Italian version of the service.
- With a VPN using centralized Internet access: your Internet traffic is first sent through an encrypted VPN tunnel to the company's US-based corporate gateway. Once the packet reaches the gateway, the VPN decapsulates it, meaning it removes the VPN wrapper, and then forwards it to Netflix as if the request originated from within the US.
From Netflix's perspective, your request now seems to come from an American IP address. When Netflix replies, the response is sent back to the US gateway, which re-encapsulates it and securely sends it through the VPN tunnel to your device in Italy.
This setup ensures both security (because all traffic is encrypted end-to-end) and geolocation control (you appear to be in the US). The downside is that the communication path is longer, introducing extra latency, your packets must travel from Italy → US → Netflix → US → Italy.
- With distributed Internet access: in this case, only your corporate traffic (e.g., accessing internal servers) goes through the VPN. Regular Internet traffic, such as Netflix, travels directly through your local ISP in Italy.
This means your connection is faster and more efficient, since the packets don't need to travel through the VPN tunnel and back. However, it also means your traffic is no longer encrypted or filtered by the company's firewall, and your geolocation is your actual location, Italy. Netflix will detect this and only show you the Italian catalog.

(Virtual) VPN Topologies

When designing a Virtual Private Network (VPN), the topology — that is, how the different sites are connected — plays a key role in determining performance, scalability, and ease of management. The two most common VPN topologies are Hub-and-Spoke and Mesh.

1. Hub and spoke

In this setup, all branches (the spokes) are connected to a single central site — the headquarters (the hub). Each branch communicates directly with the headquarters, not with other branches.

This model matches the data flow of many companies, especially those with a centralized data center or mainframe, where most resources are hosted.

Routing is simple but suboptimal, since communication between two branches must pass through the hub.

There is a small number of tunnels, since each branch only needs one tunnel — the one connecting it to the headquarters.

However, the hub can become a bottleneck, as all traffic passes through it.

Example: Imagine a company with one main office in New York and three branches in Chicago, Los Angeles, and Boston. In a hub-and-spoke VPN, each branch sets up one secure tunnel directly to New York. If the Chicago branch needs to communicate with Los Angeles, the traffic must go through the New York headquarters first — adding extra latency and load on the hub.

Advantages:

- Simple to configure and maintain
- Works well for centralized applications (e.g., all services hosted at HQ)

Drawbacks:

- Higher latency for branch-to-branch communication
- Possible performance bottleneck at the hub

2. Mesh

In a mesh VPN, tunnels are established between all sites, allowing direct communication between branches without having to pass through the headquarters.

Each branch can communicate directly or indirectly with any other branch.

Routing is optimized, as data can take the shortest path.

However, this setup requires a large number of tunnels, especially as the number of sites grows.

Configuration and management become more complex for the network administrator.

Example: if the same company (New York, Chicago, Los Angeles, Boston) used a mesh VPN, each site would have a tunnel to every other site — so Chicago could talk directly to Los Angeles without involving New York. This improves performance but increases setup complexity.

Advantages:

- Efficient routing and low latency
- No single point of failure (no central bottleneck)

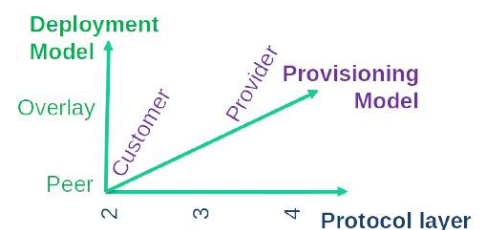
Drawbacks:

- Difficult to manually configure for large networks
- High maintenance overhead

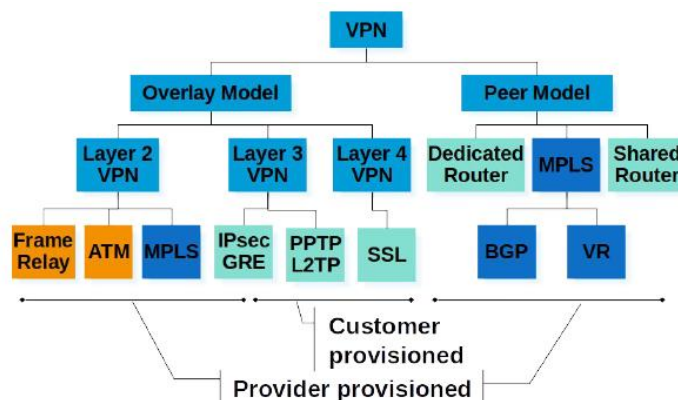
Deployment Models, Provisioning Models & VPN Protocol Layers

When designing or implementing a VPN, three main aspects must be considered:

1. Deployment model – How the VPN is created
2. Provisioning model – Who is responsible for creating and managing the VPN
3. Protocol layer – Which network layer(s) the VPN operates on



Different combinations of these dimensions and protocols can be used depending on the organization's needs for performance, control, and security.



Deployment models

Overlay Model

In the overlay model, the public network (e.g., the Internet) does not participate in the creation of the VPN and is unaware of VPN destinations. Its only role is to provide basic connectivity between VPN gateways.

- Each VPN gateway must maintain a connection with every other VPN gateway (often forming a highly meshed structure).
- Routing is handled entirely by the VPN gateways.
- The service provider is not involved in VPN creation or routing.

Suppose you have three corporate branches connected via VPN tunnels over the Internet. In the overlay model, each branch's VPN gateway is directly responsible for creating tunnels to the other sites. This means that even though there appears to be a direct virtual connection between two VPN gateways, the underlying physical route may not be optimal — the routers along the path are unaware that the traffic belongs to a VPN. As a result, security features such as encryption and authentication are guaranteed, but other properties like latency or reliability are not necessarily optimized.

Advantages:

- Strong control over VPN configuration
- Works without requiring ISP support
- Strong end-to-end encryption and security

Drawbacks:

- Suboptimal routing paths
- Scalability issues for large networks
- Performance (latency, reliability) not guaranteed

Peer Model

In the peer model, each VPN gateway interacts directly with a public router, known as its peer. Here, the service provider participates in VPN creation and routing.

- VPN gateways exchange routing information with their public peers.
- The service provider's network disseminates routing data between all VPN sites.
- The public network routes traffic between VPN gateways belonging to the same VPN.
- This model is typically used when the VPN is provisioned by the service provider.

In this case, the VPN gateway does not create a tunnel directly to another gateway. Instead, it establishes a tunnel with the first public IP router, and the service provider's network handles routing between sites.

The provider is aware that you are requesting a VPN service, and it can route packets according to specific requirements agreed upon with you (for example, bandwidth, latency, or security guarantees). Thus, one tunnel is set up between your VPN gateway and your ISP's router, and another tunnel exists between routers on the provider's side to reach the destination gateway.

Advantages:

- Optimized and guaranteed routing
- Easier scalability through provider support
- Service-level guarantees (QoS, latency, availability)

Drawbacks:

- Less control for the customer
- Depends on the service provider's configuration
- May require additional costs for managed VPN services

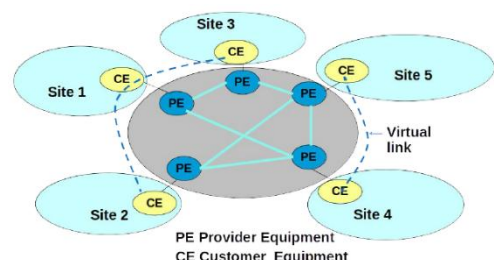
Provisioning models

When designing VPNs, it's important to distinguish who is responsible for implementing and managing the VPN, which leads to two main provisioning models: Customer Provisioned VPN and Provider Provisioned VPN.

Customer Provisioned VPN

The customer is responsible for implementing the VPN solution. This involves more than just having a host; the customer must own, configure, and manage the devices that handle VPN functionalities. The network provider is not aware that the traffic is part of a VPN because all VPN features are implemented on the customer's devices.

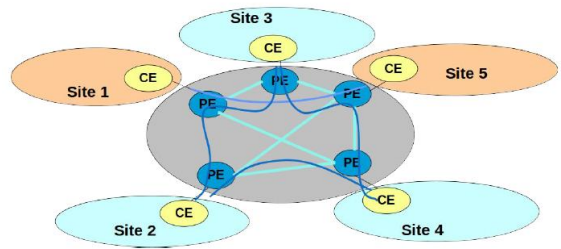
- Customer Equipment (CE): Terminates the VPN tunnels and manages VPN features.
- VPN Gateways: Hardware owned by the customer, often used by agencies to connect multiple branch offices.
- Provider Equipment: Simply forwards packets; the provider does not control the VPN.
- Logic Tunnels: Exist only between the customer devices. The packets may traverse provider equipment, but the provider is unaware of the VPN. This is essentially an overlay model.



Provider Provisioned VPN

Provider implements VPN solution, owns, configures, manages devices implementing VPN functionalities

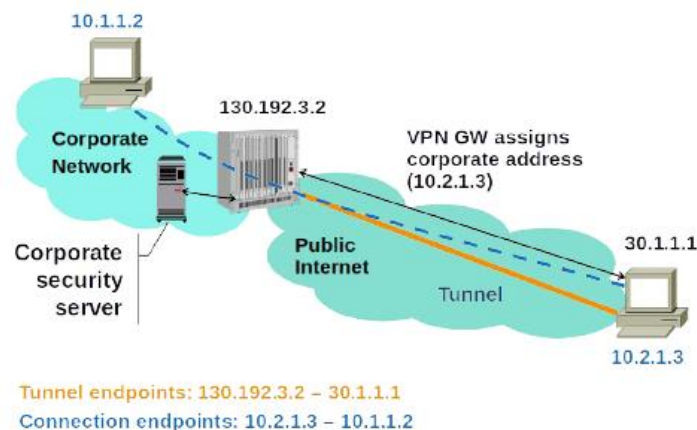
- Agencies negotiate specific features with the provider to obtain a dedicated VPN channel.
- Tunnels are created and maintained on provider devices, providing guaranteed VPN features without relying on customer equipment.
- Provider Equipment (PE): Terminates tunnels and separates traffic for different VPNs.
- Customer Equipment (CE): Appears as if it is connected to a private network, while the provider ensures VPN state and traffic separation.



Access VPN Customer Provisioned

A Remote Access VPN example illustrates a customer-provisioned model:

1. Suppose a remote host wants to connect to a corporate server:
 - o Host private IP: 30.1.1.1
 - o Target server private IP: 10.1.1.2 (e.g., at a university or corporate site)
2. VPN Setup:
 - o A VPN application is installed on the host (acting as a VPN gateway).
 - o The VPN gateway contacts the corporate server's public IP (e.g., 120.192.3.2) to establish the tunnel.
 - o The corporate security server may assign another private IP via DHCP, so the host ends up with two private IPs.
 - o The host encapsulates internal packets (with the source IP of the host and the destination IP of the target server) inside external packets (with the source IP of the VPN gateway and the destination IP of the tunnel endpoint).



Effect:

- Internally, the host appears to be on the same local network as the target server, allowing seamless access.
- Externally, traffic is routed securely through the tunnel.

Packet Structure Example:

- Inner packet:
 - o Source: 10.2.1.3 (host private IP in corporate network)
 - o Destination: 10.1.1.2 (server IP)
- External packet:
 - o Source: 30.1.1.1 (host public IP)
 - o Destination: 130.192.3.2 (VPN gateway/server endpoint)

This allows a remote host to appear local to the corporate network, extending the LAN over the Internet.

N.B. The DHCP server sit on the same LAN of the host. The corporate security server act like they are in the same server, but they are not.

Customer VS Provider Provisioned

In a Customer-Provisioned VPN, the remote host always has two addresses:

- One ISP-assigned for general Internet access.
- One corporate address for accessing the VPN.

The remote host itself terminates the VPN tunnel, meaning the VPN is established directly on the host. The user must manually activate the VPN tunnel. If the tunnel is not active, the host can still operate normally on the Internet.

Flexibility: Can be used from any Internet connection (ISP), since the tunnel is managed entirely by the host.

In a Provider-Provisioned VPN, you may not have a tunnel between your own equipment and the first provider device. Instead, all VPN tunnels are managed by the service provider, so the remote host only has one corporate IP address.

- The VPN tunnel is terminated by the provider's NAS (Network Access Server).
- The remote host is always connected via VPN.
- Internet access is centralized through the provider, and you may need to use a specific ISP to connect.

Layer N VPN

The concept of a Layer N VPN depends on the protocol used inside the encapsulated data. In general, the tunneling or transport of packets can be provided either:

- By the Layer N protocol itself, or
- As a Layer N service offered by the provider.

Layer N depends on which header is on the external packet, we do not care of the inner packet.

Layer 2

Layer 2 VPNs typically work best in a customer-provisioned model, because they extend the LAN across a public network. Here's why: in a Layer 2 VPN, the MAC addresses of devices are preserved, so it appears as if the remote network segment is part of your LAN. If the packet needs to be forwarded to the service provider's router, it requires an IP address. Therefore, the provider must be aware of the Layer 2 traffic and know how to route it in advance. In a purely customer-provisioned scenario, this is feasible because the provider is configured for it.

Examples of Layer 2 VPN services:

- Virtual Private LAN Service
 - o Emulates functionalities of LANs
 - o Can be used to connect LAN segments
 - Works as single LAN through the public network
 - o VPN solution emulates learning bridges
 - o Routing based on MAC addresses
- Virtual Private Wire Service
 - o Emulates a leased line
 - o Any protocol can be carried
- IP-Only LAN-like Service
 - o CEs are IP routers or IP hosts (not Ethernet switches)
 - o Only IP (plus ICMP and ARP) packets travel through the VPN

Layer 3

Layer 3 VPNs work at the IP level. Here, the external packet structure is Layer 3, and packets are forwarded across the public network based on Layer 3 addresses (routing based on layer 3 addresses).

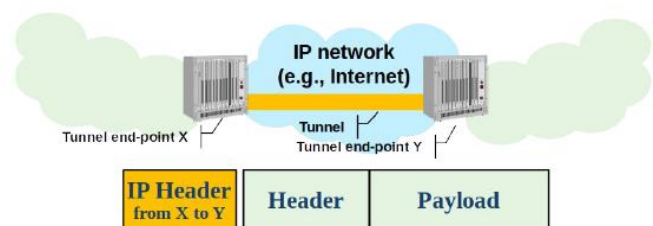
CE devices can be either IP routers or IP hosts.

Two main types of addressing:

- Peer: VPN/corporate/customer addresses
- Overlay: backbone addresses

Tunneling in Layer 3 VPN:

- A packet (or frame) is carried through an IP network within an IP packet
- Common protocols include:
 - o IP-in-IP (GRE, IPsec): Managed with



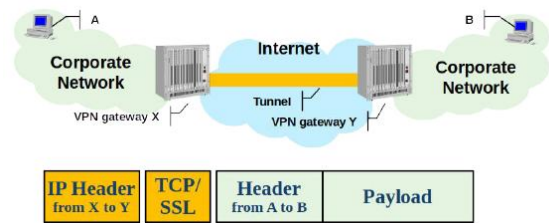
Layer 3 IP addresses.

- Layer 2 frames inside IP packets (L2TP, PPTP over GRE): Can carry Layer 2 or Layer 3 addresses, essentially extending the LAN over IP, preserving MAC addressing.

Layer 4 VPN Tunneling: s2s

Layer 4 VPNs use TCP connections to establish tunnels.

- Security is provided using SSL/TLS, which adds a Layer 4 header to the communication.
- The VPN still requires Layer 3 addressing, but Layer 4 is used for:
 - Secure communication over the Internet
 - Packet integrity and arrival control
 - Extended TCP-based features



This approach is commonly used for secure, end-to-end connections between sites while leveraging existing TCP/IP infrastructure.

GRE – Generic Routing Encapsulation

Packet Format

GRE (Generic Routing Encapsulation) is a protocol used to encapsulate one network protocol inside another. In simpler terms, it lets you take a packet (for example, an IP packet) and “wrap” it inside another IP packet. This process is called tunnelling, it allows packets to travel across networks that might not normally support the original protocol.

When we say inner packet, we mean the original packet that is being encapsulated inside the GRE tunnel. The inner packet could be anything — an IPv4 packet, an IPv6 packet, or even something from another protocol (like IPX or AppleTalk, in older systems). GRE adds its own header around that inner packet, and then the whole thing is placed inside a new outer IP header.

[Outer IP Header] + [GRE Header] + [Inner Packet (original data)]

The outer header is used for routing through the tunnel (from one GRE endpoint to another, e.g. VPN gateways), while the inner packet is the actual data that will reach the final destination after the tunnel ends.

Unlike protocols like IPsec, GRE itself does not provide encryption or security — it’s mainly about encapsulation and routing flexibility. However, it can be combined with other protocols (like IPsec) to add reliability and security end-to-end.

GRE can be used in two ways:

1. Pure tunneling: communication happens only between two GRE endpoints (e.g., routers or VPN gateways). The data inside the tunnel is just forwarded, with no end-to-end reliability.
2. Tunneling + End-to-End connection: the GRE tunnel encapsulates another protocol that already provides reliability (e.g., TCP).

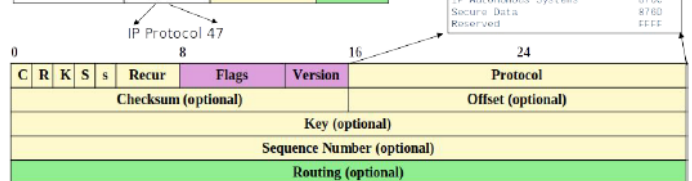
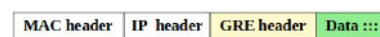
When GRE carries a Layer 3 protocol that already ensures end-to-end delivery (like IP with TCP inside), the connection becomes slower because GRE adds extra headers (encapsulation overhead). Both GRE and the inner protocol may perform control operations (like flow management). This causes higher latency and extra processing at both tunnel endpoints.

Version 0

The basic GRE header (version 0) contains fields that control optional behaviors.

ation (tunneling) of any protocol
g IP) into IP

ersion 0



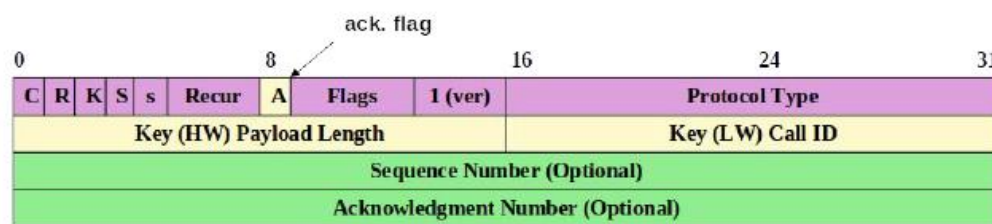
Protocol Family	PTYPE
Reserved	0000
SNMP	0004
OSI network layer	00FE
PPP	0200
SNMP	0600
IP	0800
Chaos	0804
RPC, RDP, ARP	0806
Frame Relay ARP	0808
VTNETS	08AD
VINES Echo	08AE
VINES Loopback	08AF
DECnet (Phase IV)	0803
Transparent Ethernet Bridging	0558
Raw Frame Relay	0559
AppleLink DSSSL	8019
EtherTalk (Appletalk)	800B
Novell IPX	8137
RPC 1144 TCP/IP compression	8768
IP Autonomous Systems	876C
Secure Data	876D
Reserved	FFFF

- C, R, K, S: Flags indicating the presence/absence of optional fields
- S: Strict source routing flag. If the destination is not reached when the source route list end, the packet is dropped
- Recur: Max. number of additional encapsulation permitted (must be 0)
- Protocol: ID of the payload protocol, so identifies the protocol type of the inner packet (for example, IPv4 = 0x0800).
- Routing: contains a list/sequence of router IP addresses or ASs for source routing

Although basic GRE can support optional features such as source routing, in practice it provides **no reliability, no flow control, and no congestion control**. Packets are simply wrapped and forwarded, and any loss, duplication, or reordering is handled (or not) by higher-layer protocols.

Version 1

GRE version 1, often called **Enhanced GRE**, extends the basic protocol by adding session management and limited reliability mechanisms. It is used in protocols such as PPTP.



Advanced functionalities:

- Key (high 16 bit)→ Payload length: no. of bytes excluding GRE header
- Key (low 16 bit)→ Call ID: session ID for this packet
- Sequence number →For ordered delivery, error detection and correction; used to detect lost or out-of-order packets.
- Acknowledge number →Indicates the highest sequence number successfully received (like TCP ACKs). Together, these two fields enable ordered delivery and basic reliability.

Enhanced GRE introduces:

- a Call ID, which identifies the session,
- Sequence numbers, used to detect lost or out-of-order packets,
- Acknowledgment numbers, which confirm successful reception.

Together, these fields allow GRE to provide ordered delivery and basic flow control. Enhanced GRE uses a sliding window mechanism to regulate transmission and dynamically adapts timeout values based on received acknowledgments.

However, unlike TCP, GRE does not retransmit packets when losses occur. Out-of-order packets are discarded, and congestion is handled only by adjusting the sending window. This design keeps GRE lightweight and efficient but makes it less reliable than TCP.

Layer 2 frame, within an IP packet

Layer 2 tunneling protocols allow Layer 2 frames to cross networks that only understand IP. Access VPNs are used when a remote user wants to securely connect to a corporate network over the Internet.

There are two main tunneling protocols for this purpose:

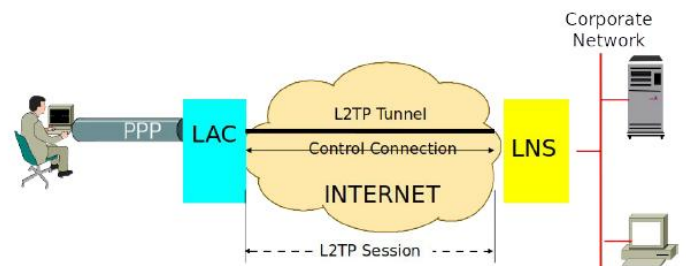
- **L2TP (Layer 2 Tunneling Protocol)**.
 - o Originally proposed by Internet Service Providers (ISPs) for provider-managed VPNs. At first, it wasn't widely supported on user devices (terminals)

- Independent of layer 2 protocol used by the host (it can encapsulate different data link protocols).
- Security is not built-in, so it's usually combined with IPsec for encryption and authentication. → The secure version is called L2TP/IPsec.
- Provides strong security, but it's complex to configure because it involves two protocols (L2TP + IPsec).
- **PPTP (Point-to-Point Tunneling Protocol)**
 - Customer provisioner
 - Originally proposed by Microsoft, Apple, ...
 - Weak encryption and authentication
 - Proprietary key management

L2TP - Layer 2 Tunneling Protocol

L2TP creates a tunnel between a public access point (like an ISP or access provider) and a corporate network (the VPN gateway). Main components:

- LAC (L2TP Access Concentrator): Network access device supporting L2TP
 - A device (or software) that provides access to the tunnel from the client side.
 - Usually part of a Network Access Server (NAS).
- LNS (L2TP Network Server):
 - The VPN gateway on the corporate side.
 - Terminates the tunnel and authenticates the connection.



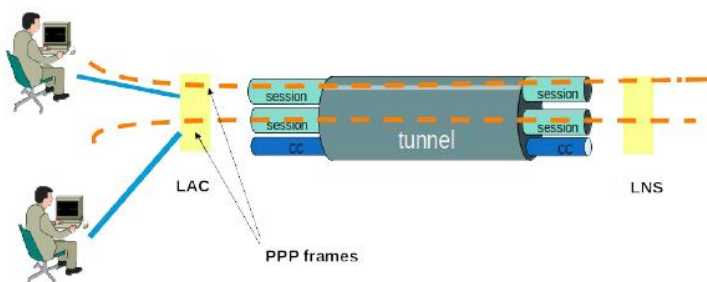
Provider provisioned deployment mode

L2TP can be used in two main scenarios:

1. Site-to-Site VPNs: Used to connect two fixed locations — for example, a branch office and the headquarters. In this case, both sites have a VPN gateway, and the tunnel is created between them.
 - a. The gateway at the headquarters is called the LNS (L2TP Network Server).
 - b. The gateway at the branch office is the LAC (L2TP Access Concentrator).

Today, the LAC can also be implemented as software, not only as hardware.

2. Remote Access VPNs: Used when individual users connect remotely to the corporate network. In this case, the user's computer or device acts as the LAC, and the corporate VPN gateway acts as the LNS. Because this involves individual user connections, PPP (Point-to-Point Protocol) is used inside the L2TP tunnel to manage authentication and session setup.



The tunnel is the connection between the LAC and the LNS. It is responsible for transporting packets. Inside a single tunnel, there can be multiple sessions, each session corresponds to one user or one logical link. Multiple tunnels can also exist between the same LAC and LNS (or different LNSs).

So one tunnel can contain many sessions, and each session carries Layer 2 frames (like PPP).

To control all of this, L2TP defines a control channel that manages the creation, maintenance, and termination of tunnels and sessions.

L2TP Operation:

- Establish a control connection for a tunnel between LAC and LNS
- Establish one or more sessions triggered by a call request
- The control connection must be established before a connection request is generated
- A session must be established before tunnelling PPP frames

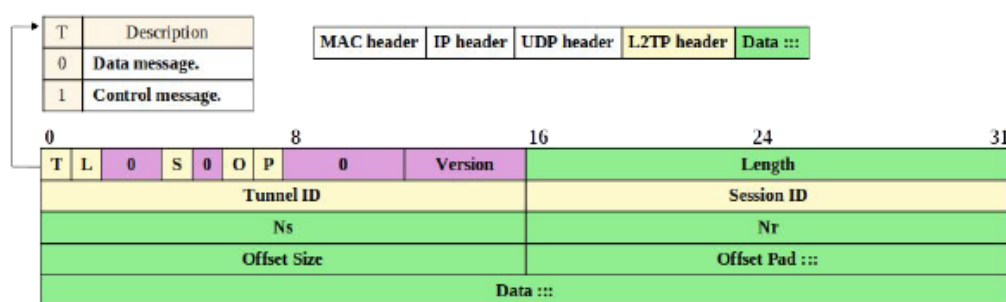
Authentication is a key feature of VPNs. Peer can be authenticated. A shared secret must exist between LAC and LNS. L2TP uses CHAP (**Challenge-Handshake Authentication Protocol**) to authenticate peers. The goal is to avoid sharing sensitive information like passwords directly.

It uses a symmetric secret (a shared password) known by both peers. Authentication happens through a challenge–response process, not by sending the password itself. The process:

- The authenticator (server) sends a challenge — a random number.
- The peer (client) combines this challenge with the shared secret and runs a hash function (usually MD5, Cryptographic algorithm used to create the answer)
- The peer sends back the hash result (the response).
- The server performs the same hash and compares the result — if they match, authentication succeeds.
- The tunnel endpoints exchange the local ID (tunnel ID and session ID) attributed to the tunnel, which identify the connection

Authentication happens between the tunnel endpoints (LAC and LNS) — not directly between the end hosts.

L2TP Header



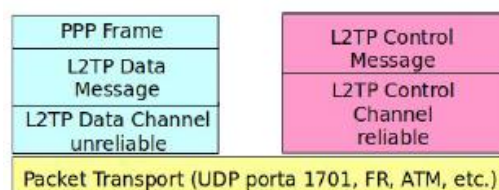
There is a UDP header added for compatibility reasons, not for transport-layer functionality. It was chosen because UDP was already used in other existing network protocols, so it helped with interoperability and NAT traversal. However, this does not mean the protocol is actually operating at Layer 4 (the transport layer). The UDP header is only a wrapper, used to make the tunneled traffic look like a standard UDP flow. The real job (control, session management, tunneling) is done at Layer 2 or Layer 3, above UDP. It doesn't provide typical Layer 4 features such as: connection establishment (like TCP's 3-way handshake), reliable delivery or retransmission, flow or congestion control.

Header fields:

- L, S, O: flags indicating whether the fields length, Ns & Nr and offset are present. For control messages L=S=1 and O=0
- P: Priority flag, if set, the priority is high
- Ver: version, must be 2
- Tunnel ID: Recipient's ID of the control connection (local meaning)
- Session ID: Recipient's ID of the session within the same tunnel (local meaning)
- Ns: sequence number of the data or control message
- Nr: Sequence number of the next control message to be received (i.e. last Ns received in order +1 modulus 216)
- Offset: Number of bytes, past the header, where the payload data starts

For data messages, sequence numbers are used only to detect out-of-order packets. There are no acknowledgments and no retransmissions, because reliability is assumed to be provided by the underlying Layer 2 protocol (such as HDLC). Packet loss is therefore tolerated.

For control messages, sequence numbers play a stronger role. Control traffic uses acknowledgments and retransmissions based on a Selective Repeat mechanism, with transmission and reception



windows that can scale up to 32,768 packets. This ensures reliable management of tunnels and sessions.

Authentication between tunnel endpoints occurs only during the tunnel establishment phase. After that, a malicious user who can observe the traffic may be able to inject packets into an existing session. To reduce this risk, tunnel and session identifiers should be random and unpredictable, rather than sequential.

Packet-level security (encryption, authentication, and integrity protection) is not provided by the tunneling protocol itself and must be handled by an external mechanism such as IPsec.

Similarly, end-to-end authentication is also delegated to the underlying security protocol

PPTP – Point-to-Point Tunneling Protocol

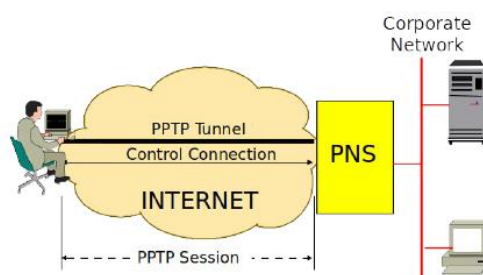
PPTP, or Point-to-Point Tunneling Protocol, was officially standardized by the IETF in RFC 2637. It was designed to allow PPP (Point-to-Point Protocol) frames—which are normally used for direct dial-up connections—to be tunneled over packet-switched networks, such as the Internet. In simpler terms, it lets users create a secure “virtual” point-to-point link over a shared public network.

To ensure data protection and user authentication, PPTP relies on Microsoft technologies:

- For encryption, it uses MPPE (Microsoft Point-to-Point Encryption), which secures the data being transmitted through the tunnel.
- For authentication, it uses MS-CHAP (Microsoft Challenge Handshake Authentication Protocol), allowing the server to verify the identity of the user without transmitting the actual password in clear text.

The PPTP architecture involves two main components:

- The PPTP Network Server (PNS), which acts as the corporate VPN gateway. It is usually implemented in software on the organization’s host system rather than as a separate hardware device.
- The PPTP Access Concentrator (PAC), which is typically used in provider-managed deployments. The PAC is maintained by the Internet or access service provider to connect remote users to the PNS through the public network.



Customer provisioned deployment mode

PPTP Connections

PPTP uses two separate channels, each with its own header — Control and Data — unlike L2TP, which uses a single tunnel for both.

The control channel requires reliability and security because it manages the tunnel setup, maintenance, and termination. If a control packet is lost, the entire VPN tunnel could fail.

The data channel (used for user traffic) can tolerate occasional packet loss, since it doesn’t impact tunnel integrity.

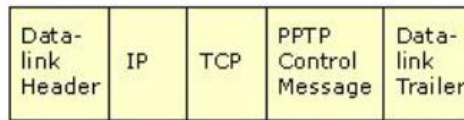
PPTP Data Tunneling: Encapsulates PPP frames inside GRE (Generic Routing Encapsulation), which in turn runs over IP

- PPP tunneling
- GRE (of PPP over IP) provides a lightweight mechanism for transporting PPP packets across an IP network.

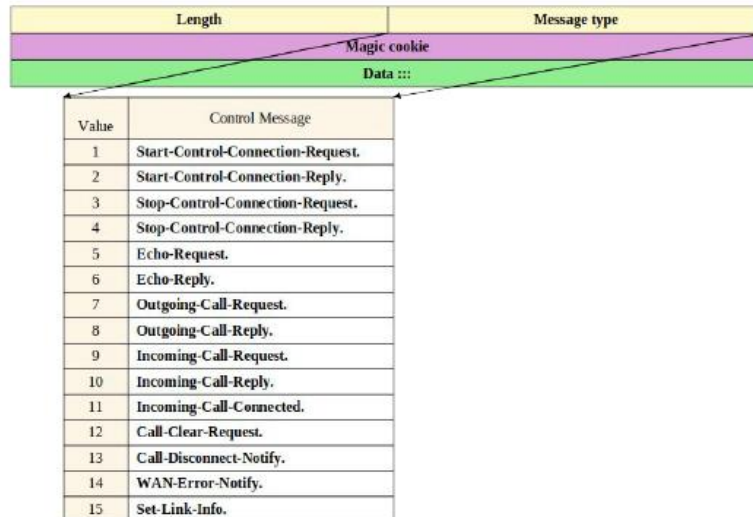
Data-link Header	IP Header	GRE Header	PPP Header	Encrypted PPP Payload (IP Datagram, IPX Datagram, NetBEUI Frame)	Data-link Trailer
------------------	-----------	------------	------------	--	-------------------

Control Connection:

- responsible for:
 - o Tunnel setup, management, and tear-down.
- Exchanging control messages between the PAC and PNS.
- Uses TCP encapsulation, providing reliable delivery for control data.
 - o Default port: TCP 1723 (on the PNS).



PPTP Header



IP sec VPN

IPsec is the primary Layer 3 solution for creating secure VPNs. While tunneling protocols like L2TP manage the encapsulation of traffic, they often rely on IPsec to provide essential security features such as encryption and authentication.

IPsec operates directly at the IP layer, protecting communications between two network devices—like gateways or hosts—regardless of the applications or transport protocols running above it. It defines two main security protocols:

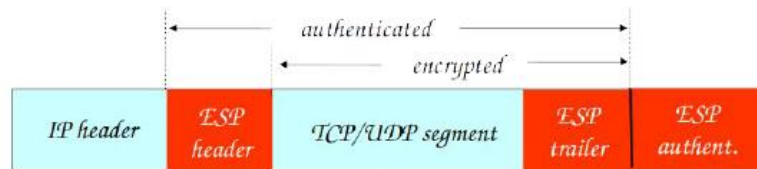
- **Authentication Header (AH)** → AH ensures data integrity and authenticates the sender, meaning it verifies that the data has not been altered during transmission and confirms the source's identity. However, AH does not encrypt the data, so confidentiality is not provided. The AH header is inserted between the IP header and the payload and uses protocol number **51**. While routers can forward AH-protected packets normally, AH is incompatible with Network Address Translation (NAT) because NAT modifies IP addresses, which breaks AH's integrity checks.

Key fields in AH include:

- o Security Parameters Index (SPI): Identifies the security session and specifies the cryptographic algorithm and keys used.
- o Authentication Data: Contains the cryptographic signature to verify authenticity.
- o Next Header Field: Indicates the protocol (e.g., TCP, UDP) that follows the AH header.
- o This allows flexibility in security levels and verification methods.



- **Encapsulating Security Payload (ESP)** → ESP focuses on confidentiality by encrypting the packet payload so only authorized parties can access the data. It also optionally provides authentication and integrity fields similar to AH. ESP uses protocol number **50**. Unlike AH, ESP does not authenticate the original IP header, only the payload inside the packet. When both header authentication and payload encryption are needed, AH and ESP can be used together.

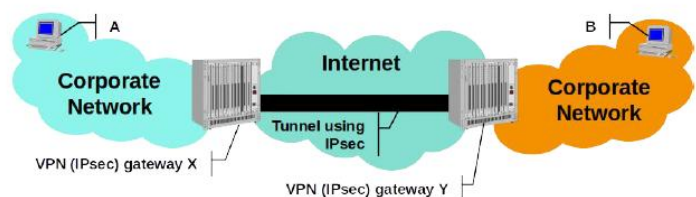


Data confidentiality means that both the actual data being sent and the **ESP trailer** are encrypted, ensuring that only authorized parties can access the information.

The **Next Header field**, which is part of the ESP trailer, indicates what kind of protocol is being carried inside the encrypted payload, helping the receiver understand how to process the data.

IPsec VPNs establish secure tunnels between VPN gateways—either hardware or software devices located at the edges of networks. These tunnels provide:

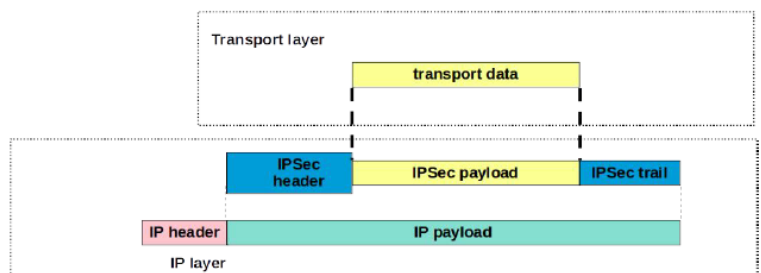
- Encryption to protect data confidentiality,
- Authentication to verify the identity of communicating parties,
- Encapsulation to wrap and protect the original packets.



IPsec supports two main modes for securing packets: Transport Mode and Tunnel Mode, which differ in the extent of packet protection and how routing is handled.

Transport Mode

In Transport Mode, only the payload of the IP packet (the actual data) is encrypted and protected, while the original IP header remains mostly unchanged. The host creates and sends the packets normally, but it is the VPN gateway, not the host, that adds the IPsec headers for encryption and security.



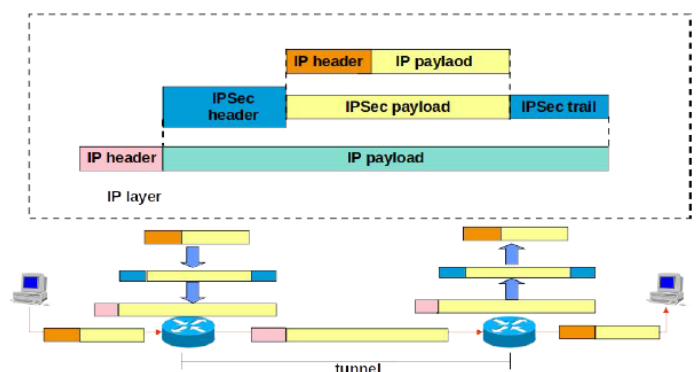
When packets travel between VPN gateways, routers use the original IP header for routing, so they can see the source and destination IP addresses, but they cannot access the encrypted payload.

Tunnel mode

The entire IP packet (header + payload) is protected and encapsulated inside a new IP packet with a new outer header.

This new header contains the IP addresses of the VPN gateways, not the end hosts. As a result, no sensitive information about the internal communication is exposed.

This is the mode typically used for site-to-site VPNs, where two networks connect securely through the Internet.



The Optical Internet

The concept of the Optical Internet represents a fundamental technological step in building high-speed core networks. At its heart, this approach concerns the physical layer of communication systems — that is, how signals are transmitted efficiently within the core of a network using optical technology rather than electrical transmission.

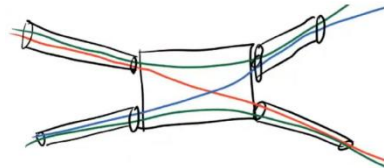
Optical signals are physical waveforms that propagate as light waves through fiber optic cables. A wavelength, often represented by the Greek letter λ (lambda), corresponds to a specific frequency of light — the two are inversely related ($\lambda = c/f$, where c is the speed of light). In essence, a wavelength identifies the sinusoidal form of an optical signal, and by exploiting multiple wavelengths, we can transmit multiple data channels simultaneously through the same optical fiber.

WDM: Wavelength Division Multiplexing

In traditional radio communications, Frequency Division Multiplexing (FDM) is used to separate signals based on their frequencies. In optical networks, a similar concept is applied using light instead of radio waves, known as Wavelength Division Multiplexing (WDM).

WDM (Wavelength Division Multiplexing) allows multiple light signals, each with a different wavelength, to be transmitted simultaneously through the same strand of optical fiber. A single fiber can thus carry multiple distinct data streams, each associated with a specific λ , dramatically increasing the transmission capacity without needing to lay additional fibers.

To generate and receive these light signals, the system uses laser transmitters and receivers capable of emitting and detecting different wavelengths accurately.



There are two primary types of WDM, differing mainly in wavelength spacing and system cost:

- **DWDM (Dense Wavelength Division Multiplexing):** a more sophisticated and expensive system that packs many closely spaced wavelengths together, achieving extremely high data capacity per fiber.
- **CWDM (Coarse Wavelength Division Multiplexing):** a simpler, lower-cost alternative with fewer channels and wider spacing between wavelengths. It is more economical but offers less total capacity.

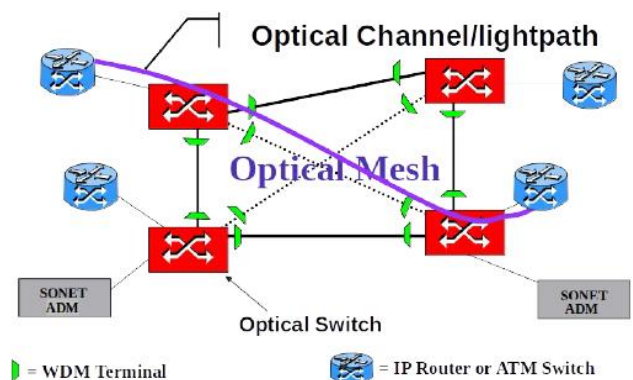
Motivation for Optical Networking

Telecommunication service providers sought to shift to optical transmission within the network core because of its massive bandwidth potential. Compared with copper-based or purely electronic transmission systems, fiber optics allow parallel propagation of multiple light waves, effectively multiplying available bandwidth with minimal additional infrastructure cost.

This leads to:

- A significant increase in transmission capacity of existing fiber infrastructure.
- Higher trunk bandwidth for backbone connections.
- Improved return on investment (ROI), as more data can be transmitted using the same physical medium.

While WDM enables multiple wavelengths to coexist within a single fiber, the challenge arises when these optical channels must be dynamically routed between different network paths. This requires devices that can redirect or manage light signals without converting them to electrical form, known as optical switches, lambda switches, or wavelength switches.



These switches allow optical signals to travel from one input port to different output ports, enabling point-to-point or mesh connectivity and dramatically increasing overall network flexibility and throughput. At the receiver side, demultiplexers separate the combined wavelengths into individual channels again.

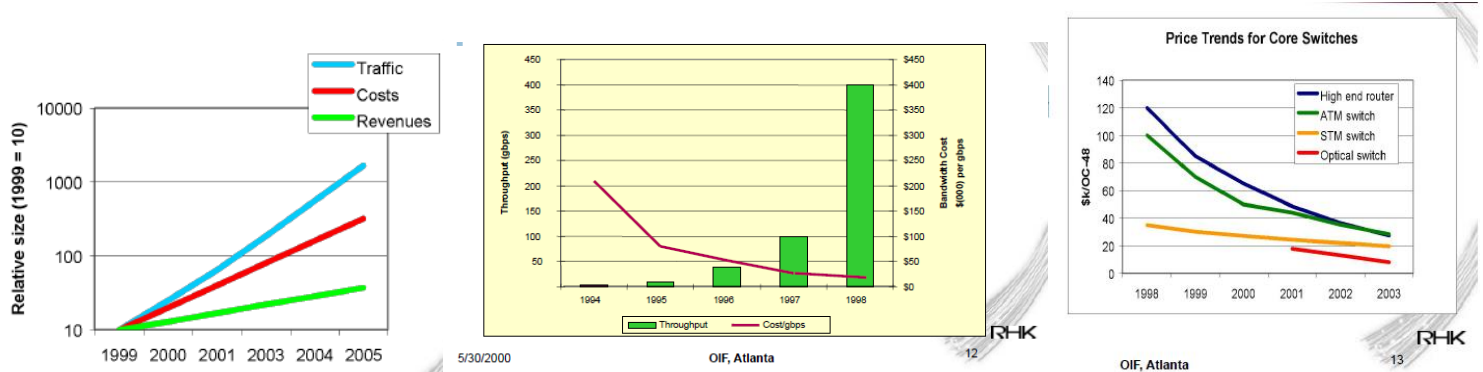
Why Optical switching?

Optical switches are typically deployed in the network core, where bandwidth demands are extremely high. A single optical channel can carry data rates of 2.4 Gb/s or higher, making these systems ideal for backbone interconnections between major network routers.

Optical switching offers several key advantages:

- Simplicity of design — potentially enabling very low cost per switched bit.
- High scalability — as the switching fabric can handle enormous aggregate capacities.
- Power efficiency — since early designs relied purely on optical mechanisms without requiring constant electrical power for data path operation.

Historically, around the year 2000, Internet traffic was growing at explosive rates, while network operational costs were rising even faster than revenues. Optical technologies promised to reverse this trend by increasing traffic capacity while reducing cost per bit, reshaping the economics of large-scale data transport.



The introduction of DWDM systems drastically reduced the cost of trunk bandwidth, allowing much more data to be transmitted over the same physical fiber. Around 2002, optical switching technologies began to offer the highest bandwidth per dollar, particularly for high-capacity systems operating at 10 Gbps and beyond (compared to older OC-48 systems at 2.5 Gbps).

However, despite the initial enthusiasm, practical challenges emerged — especially in the implementation and reliability of optical switches. These issues slowed down the widespread adoption that had been predicted.

Types and Architectures of Optical Switches

Optical switches exist in several variations, depending on their internal mechanisms and operational complexity. They can differ in:

- Optical vs. Electronic cores (whether the switching decision is made in the optical or electrical domain)
- Cross-connects vs. switches (full reconfigurable matrix vs. port-to-port routing)
- Wavelength conversion capabilities
- Complexity and flexibility levels

Several technologies have been explored for building all-optical switching cores:

1. **MEMS (Micro-Electro-Mechanical Systems):** These switches use tiny tilting mirrors made of silicon that can deflect incoming light beams from one fiber to another. The mirrors are voltage-operated and can physically change orientation to steer light paths based on control signals.
2. **Holographic Reflecting Surfaces:** These rely on voltage-operated crystalline materials that change their reflective properties when stimulated electrically. They are highly resistant to vibration but expensive, as they use rare materials. Over time, however, these crystals can degrade after many deflection cycles.
3. **Thermo-Optic or Electro-Optic Materials:** Certain materials can change their refractive index when exposed to variations in heat, pressure, voltage, or current, allowing dynamic light routing through physical property changes rather than mechanical movement.

In some systems, optical-to-electrical converters are used to temporarily transform the optical signal into a digital form, perform electronic processing or routing, and then convert it back into optical form — a hybrid approach combining both worlds, though at the expense of additional power and latency.

Optical Core Properties

An optical core network is based on the use of optical switching and light-based transmission, without constant conversion to electrical form. This approach comes with distinct advantages and disadvantages compared to traditional electrical switching architectures.

Advantages:

- **Low capital cost (CAPEX potential)** → Optical cores can eventually be inexpensive once the technology matures. The components themselves — such as fibers, lenses, and passive optical devices — can be made from low-cost materials and rely on simple, replicable manufacturing processes. After large-scale deployment, this leads to economies of scale and reduced setup costs.
- **Bit-rate and signal independence** → Optical transmission is format-agnostic. The light signal does not depend on any specific data rate or protocol standard — whether it carries 10 Gbps, 40 Gbps, or 400 Gbps — because the optical layer deals only with wavelengths, not digital encoding. This gives virtually unlimited scalability and multi-standard compatibility, allowing the same infrastructure to support evolving technologies without redesign.
- **Low operational cost (OPEX)** → Once installed, optical systems consume very little power compared to electronic switching. There is minimal electrical-to-optical conversion, so energy consumption, heat dissipation, and maintenance costs are much lower than in traditional routers or electronic cross-connects.

Disadvantages:

- **High production costs and immature technology** → Although the long-term cost potential is low, current manufacturing processes for optical switches and converters remain expensive due to immature fabrication techniques and specialized materials. These systems are still developing compared to well-established electrical counterparts.
- **Signal degradation and no regeneration** → Optical signals experience attenuation and dispersion as they travel long distances. Unlike electrical systems that can easily regenerate the digital signal, a purely optical core cannot “refresh” the light signal unless it is converted back into electrical form. Without regeneration, optical transmission distances are limited.

Electrical Core

An electrical core performs switching in the electrical domain. It converts incoming optical signals into electrical signals, processes them digitally, and then converts them back to optical form at the output.



Characteristics:

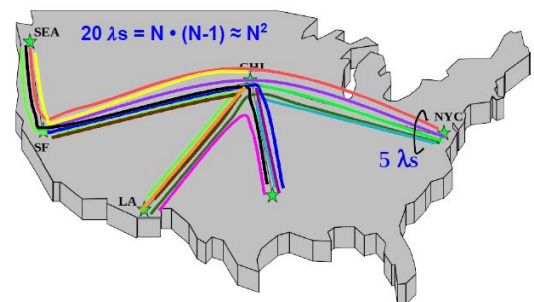
- The optical-to-electrical (O/E) conversion allows the switch to receive and interpret bits directly, making routing and packet handling simpler.
- However, this approach loses all the intrinsic advantages of an optical core: independence from bit rate and signal type, extremely low power consumption, very low per-bit transmission cost

Despite this, at the current technological stage, electrical switching is still cheaper and less complex to implement than full optical packet switching. It remains more mature, reliable, and practical for commercial deployments.

Wavelength Conversion and the N^2 Problem switching

In an optical network, each light signal is transmitted on a particular wavelength, represented by λ (lambda). However, when data travels across multiple switches and fibers, several issues arise.

Imagine a source and destination connected through several intermediate optical switches. Each link in the path uses certain wavelengths. To maintain a continuous end-to-end optical path, the same wavelength must be available and reserved along the entire route. If two connections require the same wavelength on any shared link, a conflict occurs. Checking and reserving these wavelengths for every possible source–destination pair leads to a quadratic scalability problem (N^2 combinations).



To avoid this complexity in practice, optical systems do not preserve the same wavelength end-to-end; instead, they allow wavelength conversion at intermediate nodes, the signal can change color (λ) at each hop. This simplifies wavelength assignment but introduces conversion overhead.

However, wavelength conversion, especially if done optically, is technically difficult and expensive. For this reason, fully optical end-to-end switching is rare in commercial system

Optical-Electrical-Optical (OEO) Conversion

A practical solution is to use electrical cores for wavelength conversion. When an optical signal arrives:

1. It is converted into electrical form (O/E).
2. The signal can be regenerated, errors corrected, and its wavelength changed.
3. It is then re-transmitted as an optical signal (E/O).

This process allows signal regeneration and wavelength flexibility but OEO systems have several drawbacks:

- They are complex and expensive, requiring precise optical-electronic interfaces.
- The process is not data-transparent — the system must interpret the data format, which limits scalability across technologies.
- They depend on physical device properties, such as optical resonance chambers or amplifiers, which make them fragile and costly.
- The technology is still immature, meaning production and maintenance remain expensive.

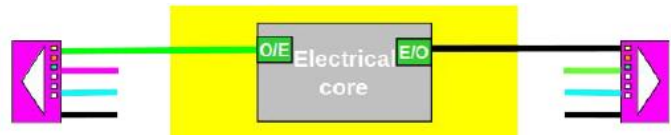
However, OEO-based systems avoid the wavelength assignment problem and eliminate the N^2 scalability issue by allowing flexible per-link wavelength usage

Common Architectures and Combinations

Wavelength Conversion

Wavelength conversion is the process by which one or more wavelengths arriving on an input fiber are mapped to one or more (possibly different) wavelengths on an output fiber. Conversion can be done in several ways, and it has a number of practical advantages:

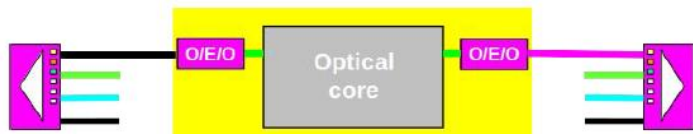
- Flexibility: converting wavelengths at intermediate nodes removes the need to reserve the same wavelength for an entire end-to-end path, which simplifies resource allocation.
- Electrical assistance: conversion is frequently implemented using an electrical core (O/E/O), where the optical signal is converted to electrical form, processed or regenerated, and then re-transmitted on a different λ .
- Monitoring and maintenance: when signals are converted to the electrical domain it becomes much easier to monitor signal quality, measure BER, and apply corrective techniques.
- Error control: electrical processing enables the use of Forward Error Correction (FEC) and other digital techniques to reduce the Bit Error Ratio (BER) before the signal is retransmitted optically.



Optical core with OEO (optical-electrical-optical) conversion

An optical core that incorporates OEO conversion does more than route light: it also regenerates and restores optical signals. Key properties:

- Signal regeneration: OEO allows amplifying, retiming, and reshaping the data: attenuated or distorted optical channels can be recovered to a clean digital representation and then re-emitted.
- Wavelength flexibility: because the signal is in the electrical domain during processing, you can change its wavelength before re-transmission.
- Tradeoffs: OEO gives control and reliability at the cost of higher complexity, greater power consumption, and potentially added latency.



Dynamic Optical Switching

Dynamic optical switching is essential to efficiently route packets in an optical network. However, it brings several challenges, especially in routing protocols: how do you send a packet exactly where you want it? How do you create the path through the network? This requires resource allocation, control signaling, and precise path management.

There are various types of wavelength switches that either do or do not support wavelength conversion. These switches can change their configurations dynamically to adapt to traffic demands. Configuration changes might be managed through different mechanisms such as:

- Network management systems
- Scheduled changes based on time of day
- Signaling from end systems
- Potentially even on a per-packet basis

Two important paradigms in optical switching include:

- **Optical Packet Switching**, where individual packets are switched optically without conversion.
- **Optical Burst Switching**, where bursts of packets are sent and switched dynamically.

Deployment

An optical network is expected to provision and protect lightpaths end-to-end, meaning it should reliably set up dedicated optical channels between source and destination nodes. Client equipment, such as routers, must be capable of controlling the provisioning of these optical layer lightpaths, using appropriate signaling mechanisms.

The network should also enable cost-effective deployment of flexible and scalable infrastructures, adaptable to varying traffic patterns and user demands.

Provisioning

A core challenge in provisioning is reserving resources for specific users or IP routers within the optical network. This involves ensuring that one user's data can be routed reliably and efficiently to another user without interference or resource conflicts.

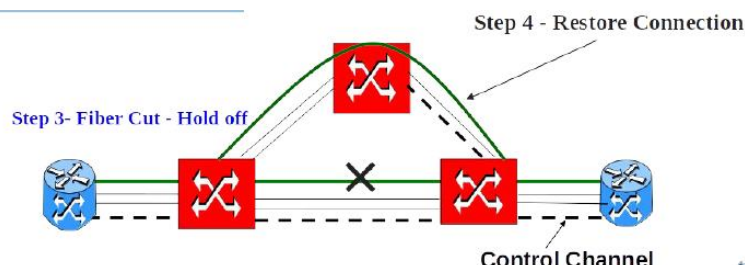


Protection/Restoration

Protection and restoration are critical for maintaining network reliability in case of failures.

- Protection refers to predetermined, pre-planned actions such as backup paths reserved in advance. Although this ensures quick recovery, it can lead to suboptimal resource usage because backup paths may remain idle until needed.
- Restoration involves dynamically determining alternative routes or resources after a failure occurs. This optimizes resource utilization but requires more complex control and longer recovery times.

For example, if a fiber link is cut, the network must have backup resources ready to restore the connection and maintain fault tolerance. This might involve providing additional capacity or backup buses specifically for fault tolerance purposes.



Protection and restoration mechanisms can operate at multiple layers:

- Layer 1 optical, e.g. SONET-like
- Layer 2 data link bundle
- Layer 2.5 protected MPLS Label Switched Paths (LSPs)
- Layer 3 Network routing protocols manage failover.

Different layers may trigger restoration processes on different time scales, from very fast physical layer switching to slower routing layer reconvergence. It is crucial to avoid issues such as unnecessary traffic shifting, packet loss or reordering, control plane instability (churn), pathological feedback loops, and non-self-stabilizing behaviors that degrade network performance during failures.

Control Plane

A significant challenge is enabling communication between IP routers and optical switches, which typically speak different protocols. The IP router must be able to request and control optical paths, but it often does not have full knowledge of the optical network topology.

Optical switches require the control plane to support several functions: resource discovery, topology, access points and node identification, resource usage, connection management/signaling, lightpath setup, lightpath take down, lightpath modification, distributed routing algorithms, mesh/ring network protection and recovery, establishment of protection service classes

Similarly, users of the optical network (such as IP routers or client equipment) need: resource discovery, address of users reachable through the optical network, manage lightpaths, lightpath setup, lightpath take down, lightpath modification, negotiate protection service classes, protected, unprotected, best effort lightpaths.

This architecture bears similarity to ATM networks, where resource reservation and signaling were also key.

Routing in Optical Networks

In the context of an Optical Internet, the users are routers themselves, connected via the optical network which provides the underlying connectivity.

Two main routing models exist:

- **Overlay Model:** the optical network acts as a black box that simply connects routers. Routers might receive reachability information but don't know the detailed optical topology.
- **Peer Model:** routers and optical switches participate in the same routing protocols. Routers have full knowledge of the optical network topology and can select preferred lightpaths for their traffic, optimizing routes end-to-end.

Control and Signaling Protocols

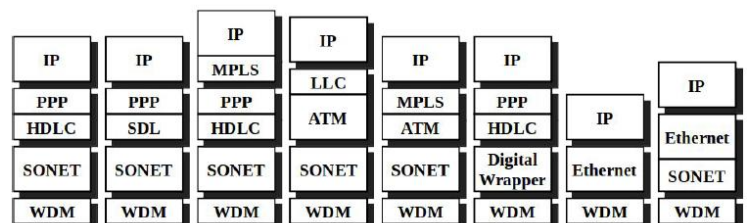
Control of the optical network can be performed at Layer 3, using common protocols such as MPLS with Label Distribution Protocol (LDP) or Resource Reservation Protocol (RSVP), which allow Label Switched Paths (LSPs) to be mapped over optical wavelengths.

Standard routing protocols like OSPF or BGP4 might be extended or new protocols defined to support the unique needs of optical networks. Control channels may be implemented either out-of-band or in-band, sometimes over Ethernet.

Multi-Protocol Lambda Switching (MP λ S) is a promising approach for controlling optical networks. It combines traditional MPLS signaling and routing protocols (OSPF, IS-IS, BGP) for resource discovery with RSVP/LDP for signaling path setups.

Data Transport and Protocol Stack

A Layer 2 protocol is essential to mark the start and end of data frames and to handle congestion control within the network. Historically, many protocols existed to manage this, but nowadays MPLS is commonly used due to its flexibility and scalability in handling diverse traffic types over optical networks.



MPLS - Multi-protocol label switching

Introduction

MPLS is not an end-user protocol like many others. Instead, it is a technology primarily used by telecommunications operators. Its purpose is to efficiently support traffic engineering and high-performance forwarding in the core of large networks.

Because MPLS operates inside service-provider backbones, regular users rarely interact with it directly, even though their packets often traverse MPLS networks without them realizing it.

History

Today, a large portion of the packets that travel across the Internet cross an MPLS network at some point. MPLS has become one of the fundamental technologies used in the backbone of modern communication systems.

The origins of MPLS date back to the late 1990s. At that time, a group of developers proposed an idea intended to enhance the performance of the then-dominant switching technology known as **ATM** (Asynchronous Transfer Mode). Their proposal focused on improving how IP traffic could be transported over ATM networks, introducing a mechanism that conceptually resembled label switching.

Initially, the proposal was presented to Cisco. Since it was designed exclusively for ATM, Cisco rejected it. However, as often happens in the industry, Cisco later revisited the core idea, generalized it, and developed what became MPLS.

The key innovation was extending label-switching concepts to all Layer-3 protocols—not just IP over ATM. This is why the name Multi-Protocol Label Switching emphasizes the term multi-protocol: in the 1990s multiple Layer-3 protocols coexisted, and MPLS was originally designed to support all of them. Today, the Internet has consolidated around only IPv4 and IPv6, but the name remains.

In the 1990s, the term ICT (Information and Communication Technology) did not yet exist. Instead, there were two largely independent domains:

- Communication Technology (CT) world
 - Telcos operators, telephone calls
 - Based on **circuit-switching** mechanism
 - A dedicated end-to-end circuit is established before the call begins
 - Main protocols: **SONET/SDH**
- Information Technology (IT) world
 - Internet, Computers, www, etc.
 - Based on **packet-switching** mechanism
 - Packets are sent independently
 - Forwarding decisions are made hop-by-hop
 - Main protocols: **IP**

By the 1990s, these two worlds—telecommunications and information technology—coexisted but were built on incompatible principles. This created a huge challenge for telecom operators: they had invested billions in circuit-switched systems, yet the new packet-based paradigm required completely different hardware and a different way of thinking. Replacing everything would have been financially devastating.

MPLS offered a solution by enabling operators to reuse existing infrastructure while efficiently supporting IP traffic with carrier-grade reliability and Quality of Service (QoS). This bridged the gap between telecom and IT worlds, allowing a smooth transition to modern services without huge costs.

MPLS played a crucial role in bridging these two previously separate worlds. It brought carrier-grade reliability and traffic engineering capabilities from the CT domain into IP-based networks, enabling the convergence that eventually created modern Information and Communication Technology (ICT).

MPLS goal

MPLS was introduced with a very clear goal: to become the enabling technology for a new generation of broadband public networks. Although it was originally designed to work with any Layer-3 protocol, in practice it quickly became associated with IP, both in the late 1990s and today, since IP has become the dominant protocol.

At that time, IP could already be transported over other technologies such as ATM or Frame Relay. These solutions worked, but they were expensive, inefficient, and difficult to scale. As the Internet grew rapidly, it became obvious that these older technologies could not support the performance and scalability that IP would require in the future.

MPLS positioned itself as the answer to this problem. It offered a more efficient, flexible, and cost-effective way to carry IP traffic across large public networks, while also improving performance. This is one of the key reasons why MPLS was adopted so quickly by operators worldwide: it delivered better results at lower cost and provided a clear path forward for networks that needed to support the explosive growth of IP traffic.

- **Private Network** → Polito network carries Polito traffic only
- **Public Network** → Telecom Italia, Fastweb, etc. networks carries traffic of everyone and makes money over this. It is "public" because it serves the whole public, not because it is free or government-owned.

For example, in Italy the national infrastructure was once owned by Telecom Italia, and today it is owned by FiberCop. All service providers—Vodafone, Fastweb, TIM, and others—pay FiberCop to use this infrastructure. These operators then deliver services to end users and charge them accordingly. Part of what you pay covers the cost of accessing the shared infrastructure, and part covers the service provided by your operator.

IP has no QoS guaranteed (best effort). This means there is no guarantee that the data you transmit will actually be received. Back in the 1990s, when packet switching was first introduced, this lack of reliability was one of the main challenges for applications and the broader adoption of packet-switched networks.

IP + MPLS → made existing public IP network suitable for selling services with requirements

This allowed service providers to use IP in a packet-switched environment while also providing guarantees on delivery and overall quality of service. As a result, providers could confidently offer these services to users, who in turn could pay for them knowing their traffic would be handled with certain performance assurances.

From ATM to MPLS

CT protocols were worldwide established. To understand why MPLS was adopted so quickly, it helps to look not only at the needs of the IT world but also at the motivations of the traditional Communication Technology (CT) world.

By the late 1990s, telecom operators had already invested enormous amounts of money in circuit-switched infrastructure. Their networks relied on ATM—an advanced, automated version of virtual-circuit switching—which had been the state of the art for decades. ATM worked extremely well for voice traffic, and its routers were reliable but very expensive.

When IP began to rise as the dominant protocol for data at the time. To avoid replacing the expensive ATM infrastructure, operators tried to run IP traffic over ATM networks (IP over ATM), often through an additional layer like Frame Relay. It allowed reusing existing hardware while supporting the new IP protocol. Technically, this solution worked, but it introduced significant drawbacks:

- Great overhead needed to construct the virtual circuit → made IP extremely slow
- ATM routers very complex and very expensive
- The more IP got spread, the more ATM issues become relevant, the more everyone realized ATM was not the solution

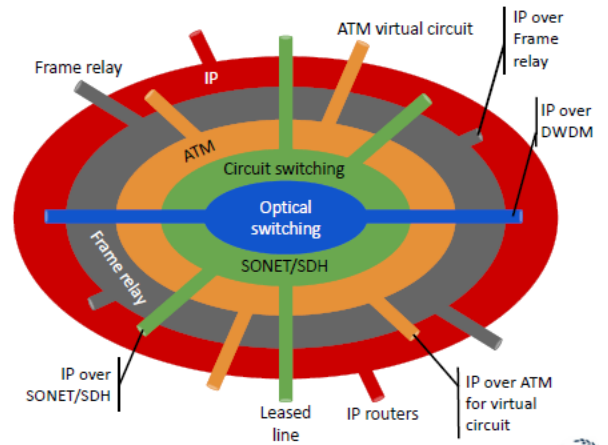
Telecom operators realized that IP and packet switching were disruptive and that relying solely on ATM would force costly network rebuilds. MPLS was designed to work with both IP and ATM, allowing operators to keep their ATM hardware and upgrade software to support MPLS routers, modernizing networks at low cost. For the traditional communication technology (CT) sector, MPLS preserved investments and avoided financial loss, while for IT, it offered better efficiency, performance, and scalability than IP-over-ATM.

MPLS was adopted worldwide in just a few years, an extraordinary speed compared with other transitions, such as the slow migration from IPv4 to IPv6.

The “onion” that made telcos “cry”

Before MPLS emerged, the networking landscape of the 1990s was extremely fragmented. Multiple technologies coexisted, and telecom operators combined them in many different ways. This created what engineers jokingly called “the onion”—a deeply layered and often painful set of protocols stacked one on top of another.

- Optical switches for PHY layer, which by then had become optical.
 - o Telecom operators wanted to use optical fibers and optical switches, although implementing fully optical switching was difficult and expensive. A common compromise was to use optical transmission combined with electrical switching at the core. Interestingly, MPLS later proved to be highly compatible with this hybrid approach.
- Circuit switching + ATM for creating a virtual circuit for calls
 - o ATM introduced a more flexible model: still based on virtual circuits, but using fixed-size cells rather than packets. ATM was somewhat of a hybrid: not fully packet-switched, not fully circuit-switched, but something in between.
- Frame relay to reuse the same structure for packet switching
- IP for packet switching



Because ATM was widespread, operators tried to use it to carry IP traffic as well. They experimented with several combinations:

- IP directly over ATM (functional but inefficient)
- IP over Frame Relay over ATM
- IP over Frame Relay over ATM over optical networks
- plus the entire parallel stack required to maintain traditional voice networks

As a result, telecom infrastructures became a patchwork of possible configurations. One operator might use one subset of the onion, while its neighbor used a completely different subset. This made interoperability extremely challenging. Engineers needed to understand all these protocols in detail, and operators had to maintain many overlapping solutions simultaneously. The overall system was complex, expensive, and difficult to scale.

MPLS eventually emerged as the technology capable of simplifying and integrating these layers.

MPLS architecture

One major advantage for the telecom (CT) world was the ability to eliminate many legacy protocols while still maintaining strong Quality of Service. MPLS was designed not only to work with IP, but also to enhance it with guaranteed QoS—something pure IP could not provide at that time.

Consider a user in the 1990s who wanted to share data. Data services required Layer 3 protocols like IP, and IP networks typically existed at the edge of the infrastructure. But the telecom operators in the core relied heavily on ATM. The issue was straightforward: the IP router at the edge (call it X) knew the IP destination address (D). It could run an IP routing protocol and determine the path—up to the border of the ATM network. But inside the ATM cloud, everything used a different protocol and a completely different forwarding logic. The IP router at the boundary could not communicate with ATM routers; they simply did not speak the same “language.”

This forced operators to use expensive edge devices capable of translating between IP and ATM (sometimes with Frame Relay in between). These devices were complex because they had to interpret both worlds.

Inside the ATM network, another router (call it R) received the converted information and had to establish an ATM virtual circuit toward the far edge. Only once this circuit was created could the data flow through. When the traffic exited the ATM cloud, IP routing resumed on the other side.

This architecture technically worked, but it introduced significant overhead:

- IP routing up to the ATM edge
- a separate ATM signaling procedure to create virtual circuits
- total inability to use IP routing inside the ATM core
- full translation steps again upon exiting

It was an operational patchwork—functional but not scalable. As IP traffic demand grew, it became clear this solution could not keep up.

Despite its limitations, ATM did offer something valuable. By creating dedicated virtual circuits between endpoints, it provided optimal resource allocation. Even today, circuit-based approaches remain the only method to guarantee truly reserved bandwidth end-to-end.

This insight inspired one of MPLS's core ideas: MPLS is a packet-switching technology, but it emulates the beneficial behavior of virtual circuits. It brings the predictability and resource allocation of ATM into an IP-based environment.

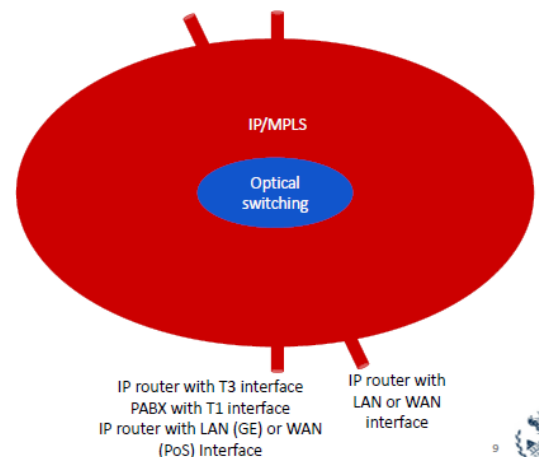
Now imagine the same scenario, but with MPLS in the core instead of ATM. The key difference is that routers inside the MPLS network and those outside it speak the same fundamental “language.” MPLS's control plane is fully compatible with IP routing protocols. This means the IP router at the edge can use its standard routing mechanisms to establish a direct logical connection through the MPLS core to the destination.

There is no translation step, no incompatible protocol boundary, and no need to wait for ATM-style circuit creation. Instead:

- MPLS routers and IP routers share the same routing information
- resource allocation is established quickly
- forwarding is efficient and predictable
- overhead is dramatically reduced

In other words, MPLS preserves the efficiency of packet switching while recreating the advantages of virtual circuits—without the weight and complexity of ATM.

MPLS keep optical switching as PHY layer.



The idea

When a router receives a packet, it needs to decide where to forward it next. To do this, the router performs an operation called longest prefix matching on the packet's destination IP address. LPM discriminate on IP dst addr (single users) or aggregation on them (geographical areas or institutions)



For IPv4, this involves checking the 32-bit destination address; for IPv6, the full 128-bit address. The router looks up its routing table to find the entry with the longest matching prefix to determine the correct subnet or next hop for the packet. This lookup process can be very complex and resource-intensive. The longer and larger the routing table, the more CPU and memory the router needs to perform these lookups. Back in the 1990s, this was a significant bottleneck because routers had limited processing power and memory.

Even today, although modern routers use specialized hardware like TCAMs (Ternary Content-Addressable Memories) to speed up these lookups, the underlying operation remains computationally expensive. TCAMs help by allowing parallel searches for prefixes, which accelerates routing decisions.

Despite advances in hardware, many telecom operators continue to use MPLS. The main reason is cost efficiency: MPLS is fundamentally a software-based solution that doesn't require replacing all the existing hardware infrastructure.

Newer technologies might offer better performance, but they often demand expensive hardware upgrades. Telecom companies prefer to upgrade incrementally, replacing hardware only when absolutely necessary, to avoid large upfront costs.

MPLS idea is to add a label in front of the packet (**short fixed-length labels**)

- 20 bits-long label
- MPLS labels is not associated to anything related to the IP packet, so it can discriminate on anything

MPLS uses labels for packet forwarding instead of IP dst addr

Instead of searching a large routing table for the longest matching prefix, MPLS routers simply use the label as an index to quickly find the next hop (One-to-one match with routing tables). For example, if a packet's label is 5, the router directly accesses the fifth entry in its forwarding table (called the MPLS forwarding table) to determine where to send the packet next.

This reduces the routing decision to a simple, constant-time operation — essentially $O(1)$ complexity — which is much faster and less CPU-intensive than traditional IP routing.

Labels can be associated to anything and this allows Traffic Engineering.

Beyond enabling the same routing process as IP, MPLS adds the ability to discriminate traffic in ways that traditional IP routing cannot. With standard IP routing, decisions are based solely on the destination IP address, sometimes aggregated by geography or institution, like grouping all traffic from a university campus under one prefix.

MPLS labels allow for much finer-grained control over routing decisions. If you consider how traffic is distributed over the Internet, it's clear that not all hosts are equal. Different hosts or servers on the same network can generate very different types and amounts of traffic.

Back when the Internet was first designed in the 1990s, it wasn't obvious how this traffic would concentrate. The assumption was that traffic would be more evenly distributed across many destinations because there were no dominant platforms like Google or Facebook back then. Today, however, the Internet is becoming increasingly centralized, with a few major services handling most of the traffic. This creates challenges: traditional IP routing tends to send all traffic along the shortest path to these popular destinations. As a result, some network links become heavily congested, while others are underutilized. This leads to problems like packet loss and retransmissions, affecting overall performance.

MPLS's traffic engineering capabilities allow operators to spread traffic across multiple paths even if they are not the shortest. This can balance network load, reduce congestion, and improve reliability.

The problem with routing algorithms like Dijkstra's shortest path algorithm is that all routers along the shortest path make the same decision, tunneling traffic down the same links. This is undesirable for high-traffic destinations like Google, where you want to distribute traffic more evenly—even if some packets take slightly longer routes.

This approach improves network performance significantly, especially for large-scale services. Although TCP can handle packets arriving out of order, performance degrades when this happens frequently. So, traffic engineering to avoid congestion and out-of-order delivery is important for maintaining quality.

MPLS essentially introduces a **connection-oriented paradigm in IP networks**. While IP itself remains connectionless (Connections are on layer 4), MPLS establishes **virtual paths** or label-switched paths that behave like dedicated circuits, similar to traditional circuit switching.

These paths allocate resources between source and destination, enabling guarantees on quality of service (QoS) on the public network.

For managers of large services—like Google or Netflix—this means they can offer paid services with guaranteed quality. MPLS traffic engineering helps ensure those guarantees by controlling how and where traffic flows in the network, avoiding bottlenecks and ensuring sufficient bandwidth.

In summary, MPLS combines the flexibility of IP routing with the predictability and control of circuit-switching, enabling efficient traffic management and quality of service even on a connectionless IP infrastructure.

MPLS network

MPLS is used only in the core network → User devices do not use MPLS!!

When you purchase a device designed to distribute signals across different parts of a network, one key characteristic is that it must operate without disrupting the existing infrastructure. This means, for instance, that such devices don't necessarily have their own IP addresses like traditional routers do. Instead, routers operate at Layer 3 of the OSI model, and a fundamental feature of routers is their programmability. As a network operator, when you buy a router, you need to configure it to handle traffic according to your network policies.

Routers also segment the network into multiple subnets. MPLS introduces elements that behave like routers but operate differently. These MPLS elements, often called **Label Switching Routers (LSRs)**, still work at Layer 3, they are programmable and disrupt subnets in the sense that they segment the network, but their core functionality is based on label switching rather than IP routing. Instead of reading the IP address at every hop, the router looks at the label, which makes forwarding decisions simpler and faster.

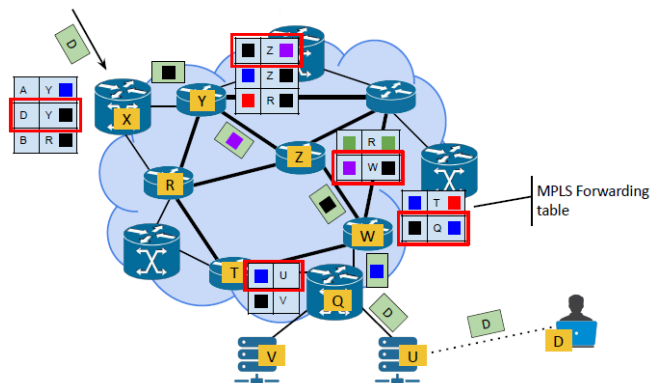
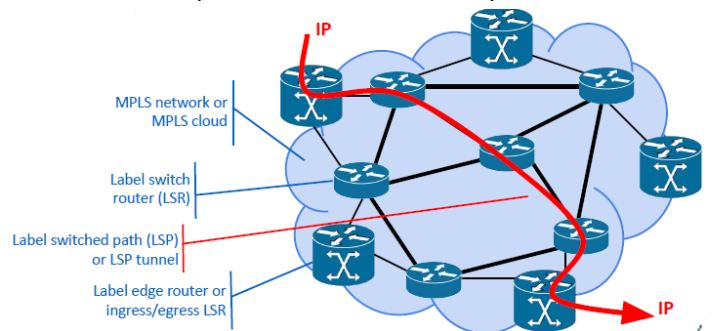
Each packet carries a label that tells each router how to forward it. These labels are unique only on each link, not globally across the network. This is somewhat similar to the concept in optical networks where wavelength division multiplexing assigns unique wavelengths per link to avoid conflicts.

An LSR will switch the label on each hop—changing the label it received to a new one before forwarding the packet onward. Occasionally, two different routers might assign the same label to packets, but this is coincidental and generally, labels are different at each hop.

In the MPLS core network, LSRs perform this label switching. At the network edge, however, there are more sophisticated devices called **Label Edge Routers (LERs)**. These are responsible for interfacing with external IP networks. An ingress LER receives normal IP packets from outside the MPLS domain, encapsulates them with MPLS labels, and forwards them into the MPLS core, ignoring the IP header until the packet reaches the egress LER. Conversely, an egress LER strips off the MPLS labels and forwards the original IP packets out to their final destination.

Label Switched Paths (LSPs) are predefined routes through the MPLS network. These LSPs act like dedicated tunnels with allocated resources, ensuring packets follow a specific path. This contrasts with normal IP routing, which dynamically selects paths based on shortest distance or other criteria.

A critical concept in MPLS is the **Forwarding Equivalence Class (FEC)**. FECs group packets that can be treated identically and forwarded along the same LSP. This aggregation is important because it prevents the network from managing every single packet individually, which would be inefficient.



MPLS offers much finer granularity. For example, labels can be assigned based on specific parameters such as the receiver and transmitter ports, source IP address, destination IP address, or combinations of these.

MPLS key elements

MPLS “header” contains the **MPLS label**.

The label selection and distribution process are handled by dedicated signaling protocols that coordinate label assignments across the MPLS routers. This signaling protocols allows LSR to create the LSP. These protocols ensure that labels are unique where needed and maintain consistency along LSPs, enabling features like traffic engineering.

Speaking of traffic engineering, MPLS supports enhanced routing control by allowing constraints on path selection. Instead of simply choosing the shortest path, the routing protocols used with MPLS can consider additional parameters, such as link utilization, minimum capacity, or independence between links. This allows operators to optimize resource use, balance loads, and improve fault recovery mechanisms.

Other MPLS versions

Over the years, many variations of MPLS have been proposed. For instance, MPλS (Multi-Protocol Lambda Switching) for optical switching decide which frequency to use is similar to MPLS problem of what label to use.

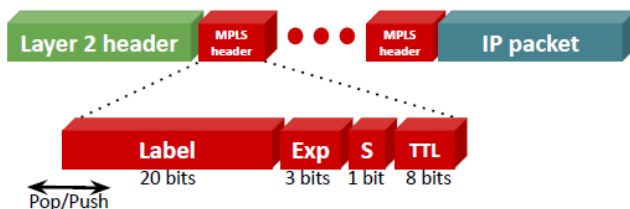
More generalized frameworks have extended MPLS concepts to other technologies demonstrating the flexibility of MPLS as a multiprotocol solution.

G-MPLS (Generalized MPLS) suited for:

- Packet switching → IP
- Cell switching → ATM
- Circuit switching → SONET/SDH
- Lambda switching → Optical routing
- “Anything” switching due to a control plane compatible with multiple protocols

MPLS header

The MPLS header itself, known as the shim header, sits between the Layer 2 and Layer 3 headers. It contains a 20-bit label, a 3-bit Quality of Service (QoS) field to prioritize traffic, a 1-bit flag indicating if it is the bottom of the LIFO stack (since MPLS supports multiple labels stacked), and an 8-bit Time To Live (TTL) field that ensures that packets don’t loop indefinitely inside the network.



If layer 2 is ATM or Frame relay, MPLS label can be put inside a layer 2 field. This choice made old ATM and Frame relay routers compatible with MPLS!!

LSP setup: label and path selection

MPLS’s control plane, responsible for signaling and routing decisions, is separate from the data plane, which handles the actual forwarding of packets. The control plane manages the complex task of label distribution, path computation, and routing table updates, while the data plane executes fast, efficient forwarding based on the label lookups.

LSP setup

Forwarding Equivalence Class (FEC) is a set of packets which are treated the same way by each LSR. They will follow the same path in the MPLS network (with the same direction) and they will have the same labels across the path. These flows maintain consistent label levels along the path—not necessarily the exact same label from start to finish, but the labels are switched hop by hop at each router consistently for every packet in the flow.

MPLS can also defines more fine-grained parameters for the FEC (RX/TX LSR port, src IP addr, etc.)

LSP setup is performed in 3 steps:

1. Label binding

2. Label mapping
3. Label distribution

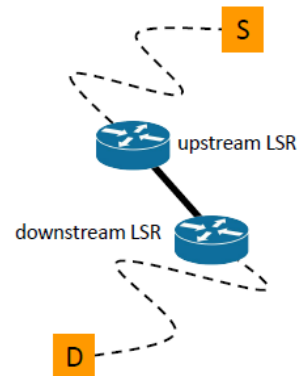
Label binding

Label binding → choosing a label for a FEC

- It is always done by downstream LSRs. In a link between two LSRs, the LSR closest to the destination is the downstream LSR. Conversely, the upstream router is the one closer to the source.
- The downstream LSR performs label binding → choose the label that is unique and convenient for its own forwarding table operations (perform the lookup in the table)

Routers inside the MPLS network use the same IP routing protocols to learn about the network topology and routing paths as standard IP routers do. So, they know who is upstream and downstream based on routing information.

Why does the downstream router decide the label? Because the lookup happens at the downstream router. Unlike IP routing where the destination IP remains constant along the path, labels can change hop-by-hop. Since the downstream router needs to efficiently perform the lookup and forwarding operation, it makes sense that it assigns the label.



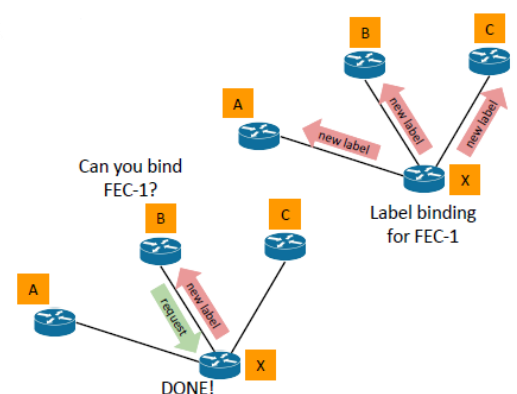
Label distribution

Notification from a LSR to another that a label was bound. Performed from a downstream LSR to its neighbours.

Once a downstream router selects a label, this information must be shared with its neighbors. This distribution ensures that upstream routers know which labels to use when forwarding traffic. There are two methods:

- **Unsolicited:** an LSR independently decides to bind a label to a FEC
- **On-demand:** someone ask the LSR to bind a label to a FEC

On-demand approach is the most widely used today because it offers greater efficiency and reduces unnecessary label exchanges.

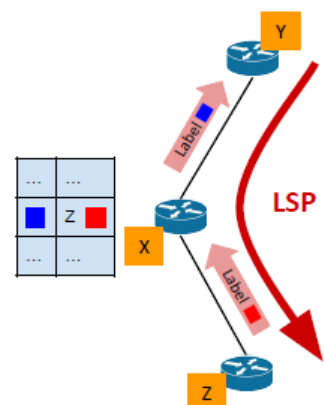


Label mapping

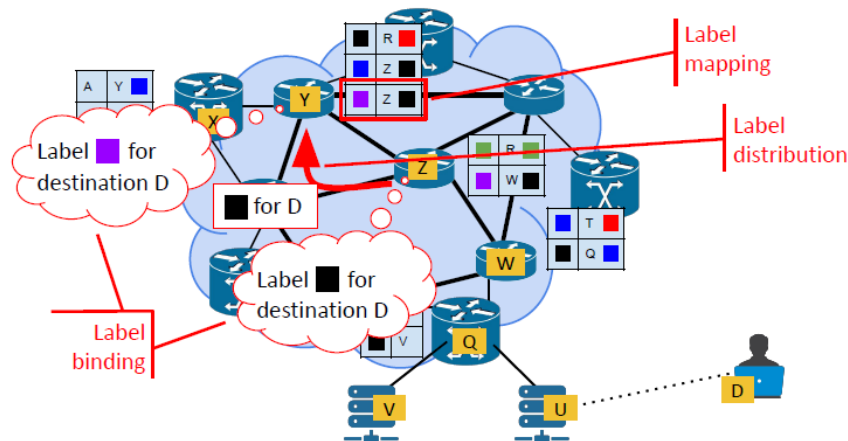
This is the final step where all the pieces come together inside the forwarding table. Label mapping matches incoming labels to outgoing labels and next hops, forming the actual forwarding rules.

It combines the input label generated during label binding, the output label received from label distribution, and the next hop identified through routing. This combination creates the entries in the MPLS forwarding table and ultimately defines the LSP.

For example, if router X receives a packet with label Blue on the link from router Y, it knows from its label binding and distribution steps how to forward the packet — by swapping the label (say, to Red) and sending it to the next router (say, router Z), which it knows from the routing protocols. This mapping is repeated for every router and link along the LSP.



Sample LSP setup



Static/dynamic label binding

Label binding (and mapping) can be decided statically or dynamically.

In the **static** approach, all bindings are manually configured by the network administrator during the setup phase. This method is conceptually simple but highly impractical: static bindings cannot adapt to network changes, they do not scale, and they require full knowledge of traffic patterns in advance. It is not scalable.

Static MPLS paths remain fixed for the entire lifetime of the network configuration. For this reason, static label binding and mapping are almost never used, except perhaps in very small networks where traffic is perfectly predictable.

Because real networks are large, dynamic, and subject to continuous changes, MPLS typically relies on **dynamic** label binding. Dynamic binding can follow different strategies, depending on how the need for a new label is detected. There are two major categories: data-driven and control-driven mechanisms.

- In **data-driven** binding, the process begins when a router receives a packet destined for an address it has never encountered before. Since the router does not yet know how to reach that destination, or which label to associate with it, it must invoke routing protocols to discover the path and then initiate label binding, label distribution, and label mapping for this new FEC.
Although possible, this approach is not common: ideally, all LSPs should already exist before data traffic arrives. If a router receives a packet for an unknown destination, it often indicates an abnormal situation, because routing protocols normally run before MPLS bindings are set up.
- In **control-driven** the binding is triggered by control messages rather than by unexpected data packets. There are two variants:
 - o Control-driven and **topology-based (unsolicited)** → I discovered a new next hop via routing protocols, then I start the label binding for that next hop. Similar to IP and not good for traffic engineering because the LSP follows whatever shortest-path the routing protocol selects.
 - o Control-driven and **explicit (on-demand)** → I receive a control message to signal the binding started by the ingress LSR (which has the broadest view of the incoming traffic) for all the LSRs of the path. The ingress LSR decides to establish a new path—for example because it detects a new destination, because it was explicitly requested by an operator, or because of traffic-engineering needs—and then contacts each LSR along the intended route, asking them to bind a label.
 - Each LSR then performs the binding, distributes the label upstream (this time using on-demand distribution), and finally maps the labels to form the LSP.
 - This mechanism resembles the creation of a virtual circuit, similar to what occurs in connection-oriented technologies like ATM. While not the only option, it is often preferred because service providers may have different requirements and may wish to exert fine-grained control over path creation.
 - Similar to connection-oriented protocols.

Label distribution protocols

Once a label is bound, it must be distributed to the upstream LSRs. This distribution requires specific protocols, and there are three mutually incompatible options:

- **Routing protocol** (e.g., Border Gateway Protocol – BGP)
 - o We are going to use BGP anyway for routing, so we can use it also for label distribution
 - o Can be used only for content-driven and topology-based binding because it operates during the routing-discovery phase.
- **Label Distribution Protocol (LDP)**
 - o Made on purpose for MPLS by Cisco (proprietary protocol), the same company that originally proposed MPLS. MPLS itself later became a standard accessible to all vendors, but LDP remained proprietary. Cisco's idea was straightforward: MPLS could be freely adopted, but effective deployment would require LDP, giving Cisco a commercial advantage.
Over time, however, vendors realized that relying on a proprietary protocol was not ideal. Although Cisco devices still support and use LDP today, most other vendors have moved away from it.
 - o Nowadays used only on Cisco routers
- **Resource reservation protocol (RSVP)**
 - o RSVP is a generic, non-proprietary protocol originally designed for resource reservation and QoS in IP networks. Because it can also reserve network resources, RSVP naturally fits into MPLS for label distribution. Unlike LDP, RSVP is open, and many vendors gradually adopted it.
Both LDP and RSVP eventually reached "rough consensus and running code"—the guiding principle for many standardization bodies: once multiple vendors agree and at least one working implementation exists, the protocol is ready for standardization.
After both were standardized, the market was allowed to decide freely. In the long term, most router vendors converged on RSVP due to its open nature and broader capabilities, although Cisco-aligned environments still commonly deploy LDP.

MPLS routing protocol

In MPLS networks, routing protocols continue to play a crucial role, even though packet forwarding itself is no longer based on traditional routing tables. MPLS forwards packets according to Label Switched Paths (LSPs), but these LSPs must first be constructed—and routing protocols are essential for this process.

During the label mapping phase, each entry in the MPLS forwarding table is defined by three components: the input label, the next hop, and the output label. While labels determine how packets are handled, the next hop is still computed using standard IP routing protocols. This means MPLS does not use routing tables for forwarding packets, but it absolutely relies on them when building its forwarding structures.

Routing protocols therefore directly influence the label mapping phase. The choice of routing protocol, as well as the specific configuration options used, determines the structure of the routing table and consequently affects how next hops are selected. Since next-hop selection is part of label mapping, routing protocols ultimately shape how MPLS forwarding tables are built and how LSPs are established.

In MPLS we can reuse (re-adapt) some routing protocols existing for IP. Not all, but just the more scalable which are suited for very large networks

- Open Shortest Path First (OSPF)
- Intermediate System to Intermediate System (IS-IS)
- Border Gateway Protocol version 4 (BGP-4)

They all carry topological information (about the links, the routers, links speed, etc.)

MPLS was originally designed to integrate smoothly with IP, allowing IP traffic to be transported efficiently over existing carrier infrastructures, which at the time were largely based on circuit switching technologies. For this reason, MPLS reuses the same routing protocols familiar from IP networks. However, in large-scale core networks—typically

national or continental in size—not all routing protocols are appropriate. Only those capable of supporting very large and complex topologies are used.

In MPLS we re-adapt the same IP protocols, enhancing them. We use them to carry information to constraint routing decision (constraint data).

New enhanced routing protocols for traffic engineering:

- OSPF-TE (OSPF supporting Traffic Engineering)
- IS-IS-TE (IS-IS supporting Traffic Engineering)

These protocols distribute detailed topological information, including link characteristics, capacities, and speeds, which is crucial for building accurate next-hop information.

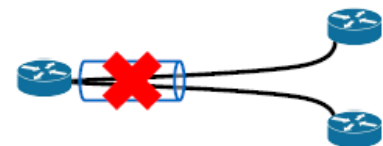
Beyond simple routing reuse, MPLS also offers enhanced routing capabilities. The goal is not only to transport IP traffic across existing infrastructures, but also to improve overall performance. A key aspect of this enhancement is the introduction of constraint-based routing. Traditional IP routing generally selects paths based solely on shortest-path metrics. In contrast, constraint-based routing allows MPLS to compute paths that meet additional requirements.

→ Not just shortest path

- “Find a shortest path whose links are used at most 50%” → Link utilization
- “Find a shortest path whose links have a minimum capacity” → Link capacity
- “Find a shortest path whose links are independent from links of another path” → Link dependencies for fault recovery

This concept becomes particularly important for Traffic Engineering. MPLS Traffic Engineering distributes network traffic more efficiently and deliberately, avoiding congestion and improving resource utilization. Constraint-based routing enables the network to compute LSPs that satisfy engineering objectives rather than simply following the default shortest path.

Fault recovery provides another strong motivation for constraint-based routing. Historically, circuit-switched technologies like ATM offered extremely fast protection mechanisms and high availability. Telecommunication operators were expected to guarantee service continuity with stringent requirements, such as the well-known “four nines” (99.99%) reliability, and to keep end-to-end latency for voice traffic below 100 ms. These constraints continue to influence modern networks, and MPLS inherits this legacy. Today, MPLS is capable of recovering from failures in less than 50 ms—another reason why carefully engineered, constraint-aware LSPs are essential.



To support constraint-based routing, enhanced versions of traditional routing protocols are used. For example, OSPF-TE and IS-IS-TE extend their base protocols to carry additional link attributes required for Traffic Engineering, enabling routers to compute paths that satisfy the defined constraints. Classic shortest-path algorithms like Dijkstra cannot handle these constraints alone, since they assume uniform link treatment. TE extensions overcome this limitation by providing the extra information needed to compute constraint-based paths.

Routing modes

In MPLS there are two routing modes:

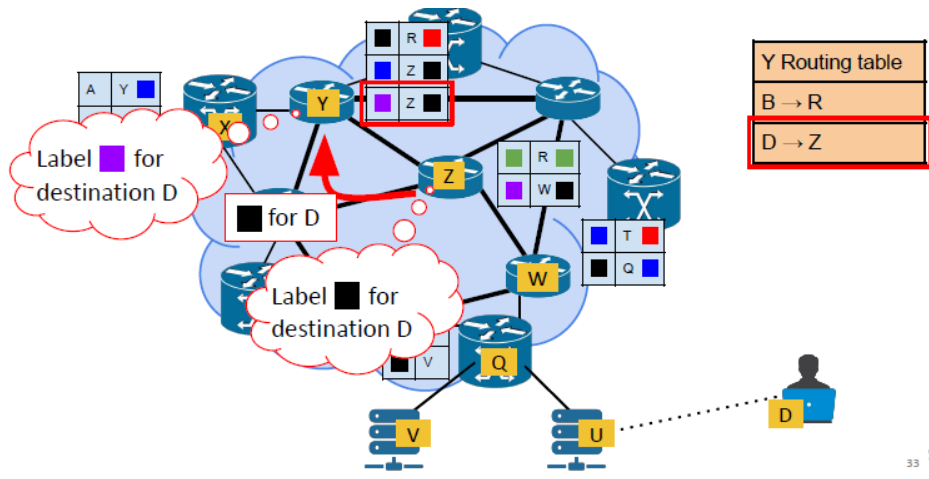
- Hop-by-hop routing → every LSR independently looks at its routing table and picks the next hop to the destination; here is no global decision for the LSP.
- Explicit routing → one LSR decides the whole path.

Hop-by-hop routing

In this mode, each Label Switching Router (LSR), When has performed label binding and has received the output label from the neighbours LSRs via label distribution, independently consults its IP routing table to determine the next hop toward the destination. There is no centralized or global decision for the LSP. After an LSR has received labels from its neighbors through the label distribution process, it looks up the next hop in its routing table, creates its label mapping, and installs the corresponding entry in its MPLS forwarding table.

This process essentially reproduces the behavior of classical IP routing. MPLS in this mode offers only marginal advantages—mainly faster lookup thanks to the use of labels—but it does not enable Traffic Engineering. Since each LSR makes its decision independently, no global optimization is possible.

Hop-by-hop routing aligns closely with the mechanics of traditional distributed routing. Each LSR independently builds its routing table, independently determines the next hop, and independently binds labels. Because the decision process is decentralized, each router may choose paths that differ from the globally optimal path, simply because it lacks a complete end-to-end view.



Explicit routing

Explicit routing is required for Traffic Engineering because constraint-based paths cannot be achieved through independent hop-by-hop decisions. Constraint-based routing must be centralized; otherwise, local decisions taken by different routers could conflict with one another, resulting in paths that violate the required constraints.

In explicit routing, only one LSR decides the entire LSP path. Typically, this role is taken by the ingress LSR, since it is the first router receiving the traffic and the one initiating label binding for that flow. The ingress LSR computes the complete hop-by-hop sequence based on the constraints and then informs all downstream LSRs of the chosen path.

To support this mechanism, enhanced versions of label distribution protocols are required. Traditional label distribution protocols do not carry the additional information necessary for constraint-based routing. Therefore, MPLS uses **CR-LDP (Constraint-Based Routing LDP)** or **RSVP-TE (Resource Reservation Protocol for Traffic Engineering)**, both of which allow the ingress LSR to communicate the full path and the associated constraints to all routers in the LSP.

Explicit routing is inherently slower than hop-by-hop routing, since the ingress LSR must first distribute the path information to all routers before label binding can begin. However, it enables capabilities that are impossible in hop-by-hop mode.

Advantages of explicit routing

Explicit routing makes Traffic Engineering possible.

Guarantees on QoS (theoretically) not yet supported today in MPLS. Only way to guarantee QoS is to reserve resources on all the links, similar to what was done in ATM networks. While such full reservation is technically possible with MPLS TE, it is rarely implemented in large-scale packet-switched networks because service providers cannot reserve dedicated resources for every user.

Resource reservation is therefore generally limited to special cases, such as large corporations—like Stellantis—purchasing dedicated contracts to guarantee QoS between geographically distributed sites. These agreements allow link-by-link reservation along the chosen paths but represent exceptions rather than common practice.

For regular customers, service providers instead rely on **Classes of Service (CoS)**. CoS does not reserve per-flow resources but assigns different priorities to different categories of traffic. For example, VoIP traffic typically receives

higher priority than background data transfers because voice communication must meet strict latency requirements. Empirical studies have shown that end-to-end latency must remain below approximately 100 ms for voice quality to remain acceptable; thus, priority handling is dictated by the intrinsic nature of the traffic, not by user preference.

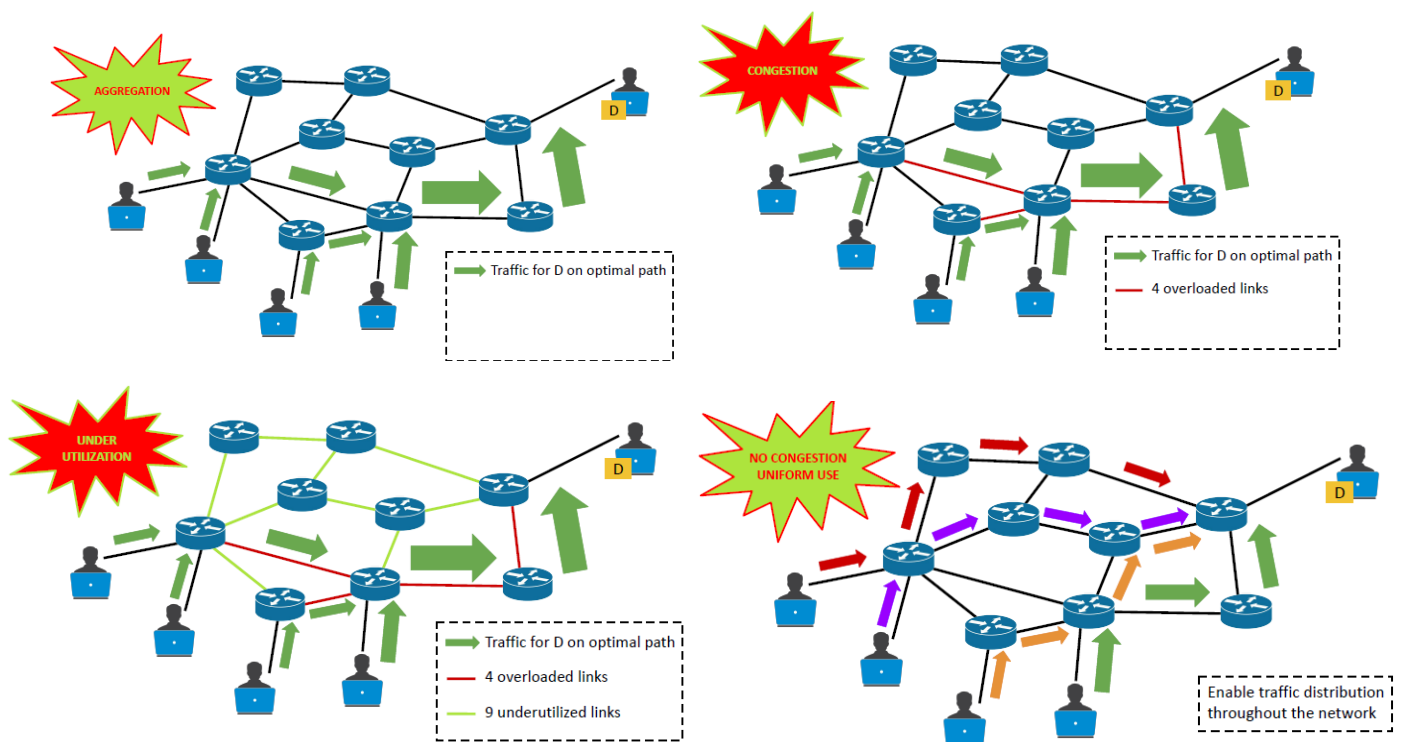
Explicit routing also makes **fast fault recovery** possible. MPLS networks can recover from failures in under 50 ms, a performance level unattainable with traditional IP routing protocols. This rapid recovery is enabled by backup LSPs computed through constraint-based routing and installed proactively.

Traffic Engineering

Traffic engineering is one of the most important advantages MPLS offers for managing networks efficiently. To appreciate why, imagine a typical core network without MPLS.

In the early internet design of the late 1990s, service providers envisioned a fully distributed network, where any device could communicate with any other. While this remains true, traffic has become increasingly centralized over time. This shift is mainly due to a few dominant content providers—like Google, Netflix, and Amazon—hosting services accessed by nearly everyone. As a result, traffic patterns show many users sending data toward a limited number of destination servers.

In traditional IP routing, traffic from multiple sources merges over the same physical links as it approaches the destination. This happens because routing protocols, like those based on Dijkstra's algorithm, choose the shortest path. Since neighboring routers often share the same shortest path, many flows end up converging along identical routes. This aggregation can cause some links to become congested while others remain underused, wasting costly network resources.



For service providers, this imbalance is problematic: expensive backbone infrastructure is overloaded in some areas, leading to higher latency, packet loss, and degraded user experience. Ideally, traffic should be spread evenly across all available paths, preventing bottlenecks and maximizing capacity.

MPLS enables exactly this kind of optimized traffic distribution. Instead of strictly following the shortest path, MPLS lets providers define multiple Label Switched Paths (LSPs) with specific constraints—such as link capacity or current utilization—to balance the load. For example, if four users send traffic to the same destination, MPLS can distribute their flows over different routes, avoiding congestion and improving network efficiency.

Although some aggregation still occurs (since traffic naturally converges near the destination), it is controlled and limited to an acceptable level. This uniform traffic distribution leads to better network efficiency and improved user experience.

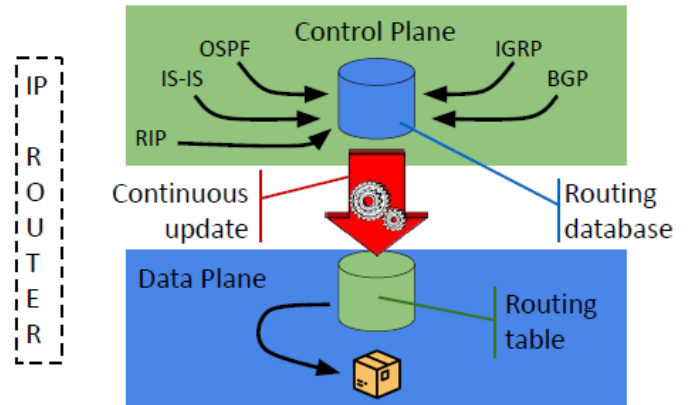
The key innovation is MPLS's ability to create multiple distinct LSPs to the same endpoint, unlike traditional IP routing which uses only the shortest path. MPLS achieves this by assigning labels to packets that determine their route independently of shortest-path calculations. This "constraint-based routing" takes into account network factors like bandwidth and link usage, enabling dynamic and flexible traffic engineering. For instance, one LSP (green) might take a different path than another (red), both reaching the same destination but balancing network load differently.

IP router Control Plane & Data Plane

To understand why this is not possible in normal IP routing, it's helpful to look at the architecture of routers themselves.

All routers are composed by a Control Plane and a Data Plane:

- **Control Plane** → This is the software component responsible for routing decisions. It runs routing protocols (like OSPF, BGP, IS-IS), builds a routing database, and determines the best paths to each destination based on network topology.
- **Data Plane** → This hardware-driven component handles the actual forwarding of packets according to the routing tables generated by the control plane.



Routing protocols run periodically, updating the routing database and tables as network conditions change (such as discovering new destinations or link failures). These updates propagate from the control plane to the data plane.

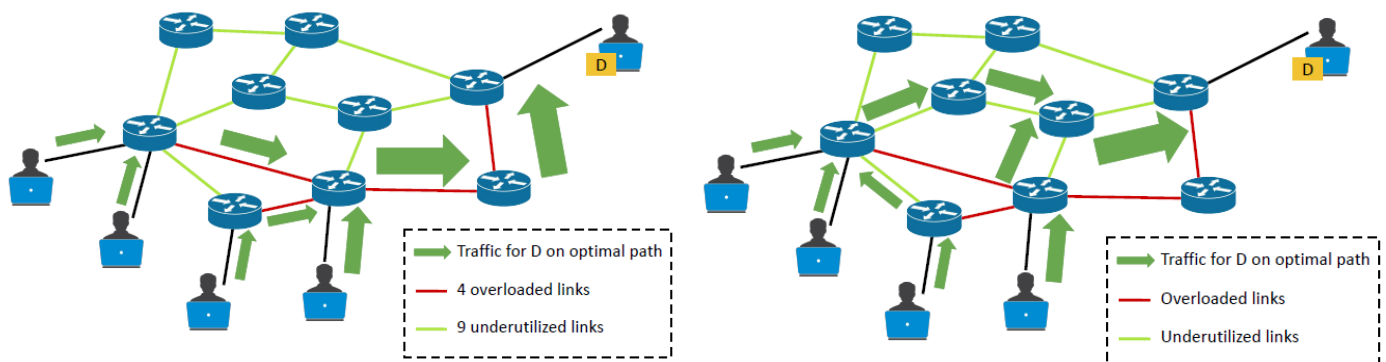
Traditional IP routing hardware typically forwards packets based on the longest prefix match in the routing table. Although some hardware acceleration exists for these operations, the forwarding behavior is essentially fixed.

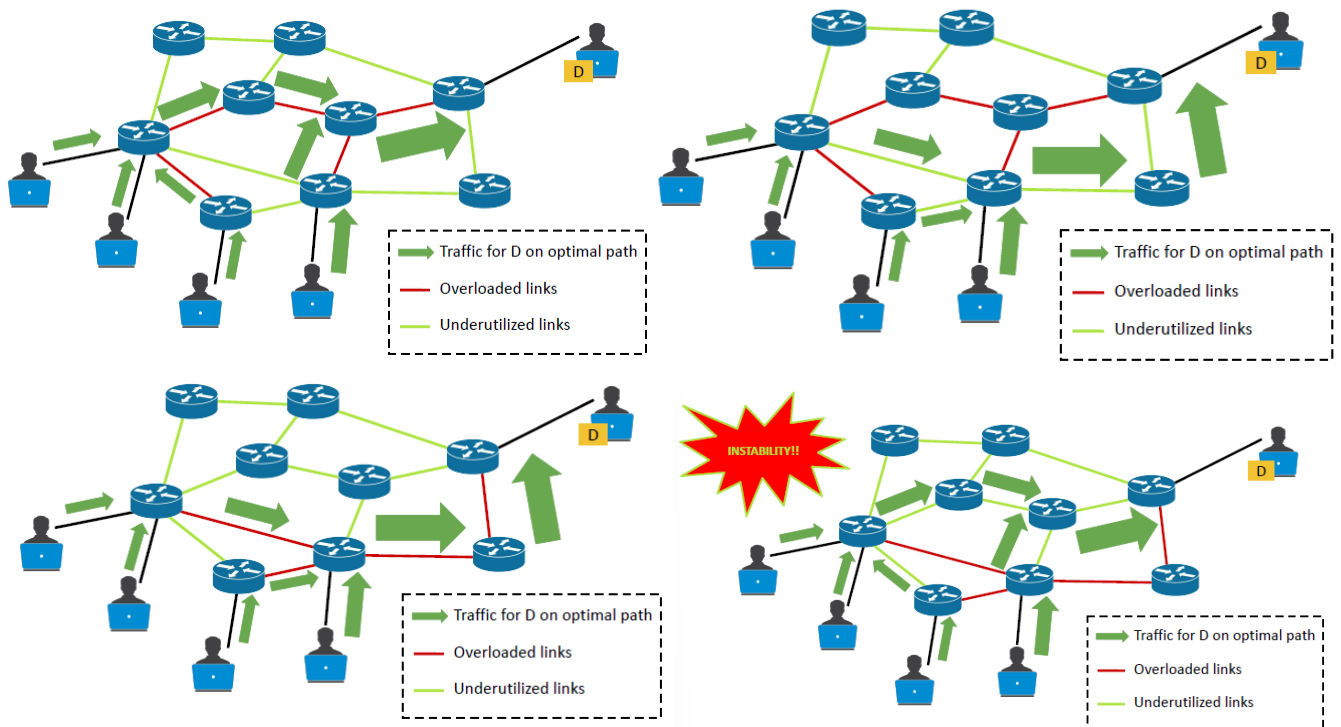
In IP routers every time there is a new or better destination, the routing table is recomputed.

Can we do the same with IP routing?

When a network admin tries to avoid congestion by configuring routers to ignore certain paths, traffic just shifts to other shortest paths, causing instability. This "route flapping" means traffic oscillates between paths, degrading performance and complicating network management. Furthermore, continuous routing table recalculations overload router CPUs, increasing power consumption and operational costs—undesirable outcomes for providers.

Although IP routers support policy-based routing that can differentiate traffic flows and direct them along specific paths, this approach requires manual configuration on every router, making it impractical at large scales in dynamic networks.



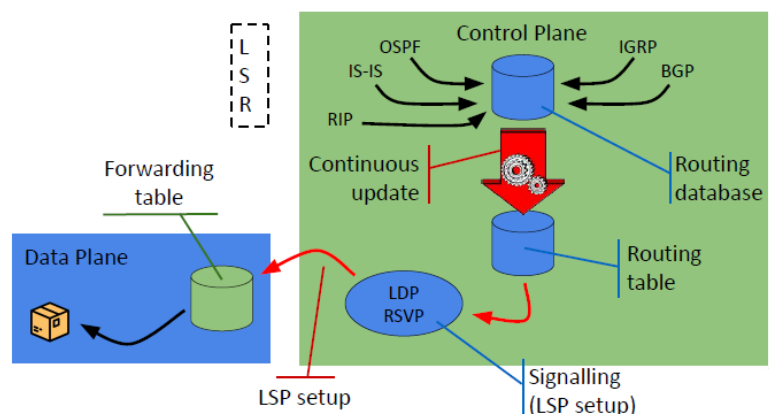


LSR Control Plane & Data Plane

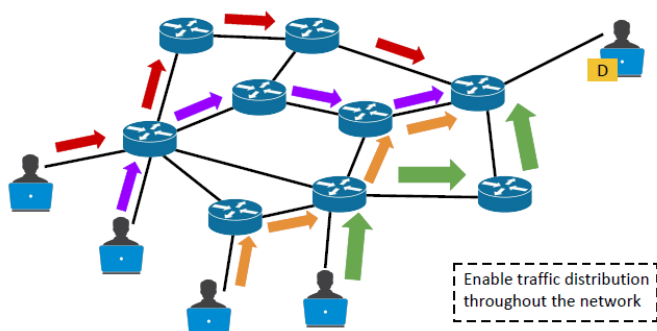
MPLS overcomes these issues by separating forwarding decisions from shortest-path constraints. It uses predetermined LSPs optimized for load balancing, resource allocation, and fault tolerance, resulting in a more stable, efficient network.

The key difference lies in how MPLS routers operate, particularly how the control plane and forwarding plane are separated. The control plane (the software part) still uses routing protocols—similar to those used in IP, sometimes enhanced—but routing tables in MPLS serve a different purpose.

In MPLS, routing tables are primarily used during the Label Switched Path (LSP) setup, specifically in the label mapping phase. Here, the routing table helps determine the next hop for forwarding decisions. However, once the LSP and forwarding tables are established, packet forwarding is no longer directly handled by routing tables. Instead, forwarding relies on the MPLS forwarding table, which uses label switching.



This means that when network topology changes, routing table updates do not immediately disrupt forwarding. Packets continue to flow through established LSPs until forwarding tables are updated smoothly, avoiding instability.



Thanks to this design, MPLS can create multiple labeled paths that distribute traffic efficiently and stably. This scalability and stability are not achievable with IP's policy-based routing.

Traffic engineering wasn't entirely new before MPLS. For example, ATM networks supported it, but in a

static and manual way. ATM required setting up dedicated virtual circuits, which had to be configured manually and could not adapt dynamically to network changes. This made ATM traffic engineering complex and not scalable.

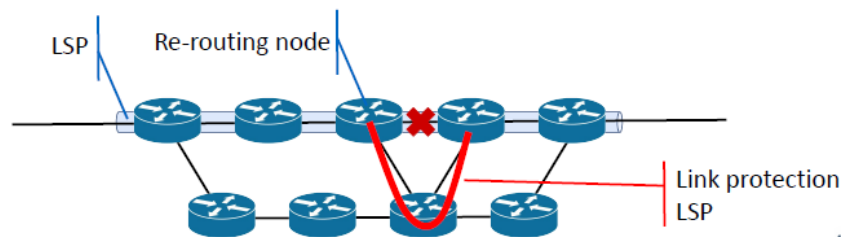
MPLS advances this by automating traffic engineering in a packet-switched, IP-aware environment. It dynamically adapts routes and forwarding tables based on network conditions, combining the flexibility of IP with the performance of circuit switching.

Fault recovery

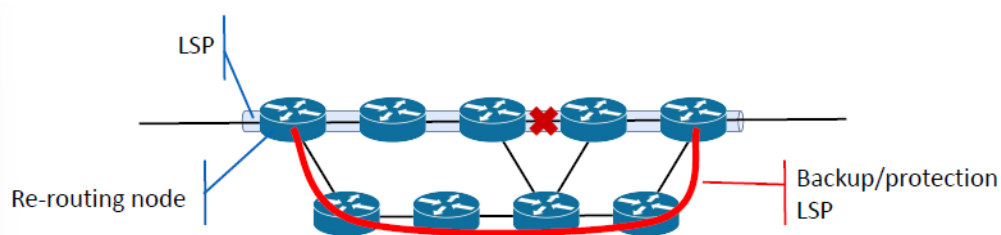
Fault recovery, like traffic engineering, existed before MPLS, notably in circuit-switched networks such as ATM, which guaranteed very high reliability (up to 99.99% uptime) and had strict recovery time policies—often within 100 milliseconds. MPLS aims to match or improve this by enabling automatic fault recovery typically under 50 milliseconds, essential for modern network demands.

There are two main MPLS fault recovery methods, both requiring careful planning and manual configuration by the provider to ensure redundancy:

- **Link re-routing:** This method protects specific physical links by creating backup Label Switched Paths (LSPs) for link redundancy. In case of link fault, we will use the link protection LSP. For instance, if the primary link between routers X and Y fails, traffic is instantly rerouted via a secondary LSP through another router Z. Recovery is very fast because the ingress router detects the fault (e.g., loss of signal) and pushes an extra label (not swapped) to switch the traffic onto the backup path. The failover is transparent to downstream routers, thanks to MPLS label stacking: the added outer label directs packets along the backup route, while the original inner label remains intact, preserving end-to-end forwarding. The external label is pushed at X and popped at Z, ensuring that Y receives the packet as if it came directly from X. → Very fast recovery



- **Edge-to-edge re-routing** → This approach protects the entire LSP rather than just a link. It works well with explicitly routed LSPs and involves fault notifications sent backward to a re-routing node. That node then switches traffic to a pre-established backup LSP. This method is slower than link rerouting but faster than traditional IP rerouting, balancing speed and flexibility.



Label hierarchy

LSPs can be set up on demand with explicit routing, where the Label Distribution Protocol (LDP) coordinates label assignment only among routers involved in the path.

For example, when you assign a label—say, a purple label—to a particular LSP, the label distribution protocol ensures that only routers within that LSP receive it. If you want to add a redundant LSP for fault tolerance, this must be done manually by the network administrator. They need to select the same label or possibly a different one for the redundant path.

In practice, this means that a router Y will receive the label it expects to bind and distribute, whether this was decided by Y itself or assigned differently for redundancy across other routers like X or Z. This setup allows routers to know which labels to expect and forward accordingly.

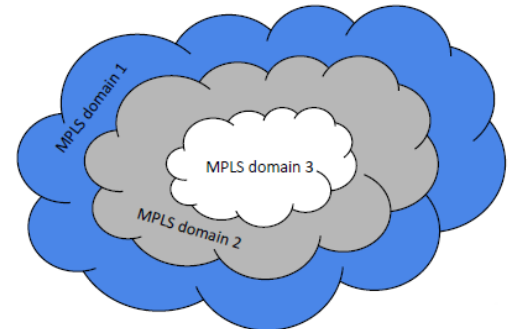
Routers maintain consistent inner labels that identify the LSP end-to-end, while outer labels are swapped hop-by-hop, directing packets along the correct route. This **label stacking mechanism** allows MPLS to efficiently manage complex paths without overwhelming routing and forwarding tables.

When a fault occurs, the router's software updates forwarding tables, replacing the current forwarding entry with an alternative path. For example, router X switches traffic from Y to Z using a new outer label (e.g., an orange label). This dynamic update happens in milliseconds through carefully designed MPLS data structures, enabling near-instantaneous failover.



→ **MPLS supports stacking multiple labels, creating a hierarchy that improves scalability.**

Inner MPLS domain can ignore the LSPs outside and treat them as a single inner LSP with an additional label pushed.



From a scalability perspective, MPLS supports hierarchical label stacks to manage large networks efficiently. Multiple MPLS domains managed by different service providers can coexist, each with their own labels and routing within their domain. For example, in Italy, the infrastructure owner (formerly Telecom Italia, now FiberCop) manages the physical network, while multiple service providers like Vodafone and Fastweb use that infrastructure and operate their own MPLS domains on top of it.

This hierarchy allows for separation of responsibilities among providers, improves scalability, and simplifies management by localizing routing tables and forwarding information within each domain. The labels are stacked accordingly to reflect domain boundaries and are pushed or popped as packets cross these borders.

Penultimate Hop Popping (PHP)

In MPLS, there is a technique called **Penultimate Hop Popping (PHP)**, where the label is removed not by the last router (egress LSR), but by the second-to-last router (penultimate LSR). Since the egress router doesn't need the label to forward the packet, popping it earlier reduces its processing load and optimizes overall performance.

For example, if router Z is the penultimate hop before the egress router Y, Z will pop the outer MPLS label, sending a regular IP packet to Y. This simplifies forwarding at the egress and speeds up processing. The egress router informs the upstream router that it is the final hop, enabling PHP.

There are two types of PHP:

- **Implicit PHP** → the penultimate router signals the final router to pop the label by assigning a special "pop" label (label 3). The last router then removes the label upon receipt.
 - Egress LSR decides input label 3
 - Label 3 means pop the label, so one to last LSR of the LSP will pop the label
- **Explicit PHP** → the penultimate router pops the label itself.
 - Used when the egress LSR wants to look not at the label, but to the other fields of the MPLS header (like Class of Service) before forwarding the packet
 - Egress LSR decides input label 0 which is meaningless for MPLS protocol

Peer VPN and MPLS-based solutions

VPNs can be established through various methods and protocols, which were classified into three categories: overlay models, customer-provisioned models, and provider-provisioned models, or alternatively by the layer at which the VPN operates.

MPLS-based VPNs appear under provider-provisioned VPNs because MPLS is a protocol designed to run on the core or backbone of a network, primarily managed by service providers rather than end users. Therefore, VPN solutions built on MPLS inherently involve the service provider in their setup.

IP-based peer VPNs

There are two main approaches to IP-based peer VPNs offered by service providers:

- **Dedicated Routers:** In this model, the provider operates a network of routers exclusively dedicated to a single customer. This solution offers high isolation and control but is typically only feasible for very large clients due to its cost and complexity.
 - **Shared or Virtual Routers:** More commonly, providers create multiple virtual router instances within a single physical router. Modern high-end routers can support hundreds of these virtual routers, each with its own routing table and routing protocols. This allows efficient resource use while maintaining logical separation between customers. Traffic between these virtual routers is usually secured and tunneled using IPsec or GRE protocols. Hardware support, including ASICs and operating system capabilities, ensures that performance remains high even with many virtual routers running concurrently.
-

To recap the difference between customer-provisioned and provider-provisioned VPNs: in customer-provisioned VPNs, the customer sets up the VPN themselves, often for remote access or direct end-to-end connectivity. In provider-provisioned VPNs, the service provider actively participates in establishing the VPN, which is typical for organizations requiring secure connectivity between multiple remote branches spread over wide geographic areas.

For example, imagine a large agency with several offices across a country or continent. This organization would want to securely connect these offices over a VPN to protect sensitive information such as non-disclosure agreements or intellectual property. Instead of setting up client-side VPNs at each location, the company contracts a telecommunications provider to establish the VPN across the network backbone, ensuring security and managed service.

There are two main approaches to achieve this with MPLS VPNs. First, the Layer 2 VPN solution, which uses the overlay model, and second, the Layer 3 VPN solution, which follows the peer model. The fundamental difference lies in whether the service provider actively controls the Label Switched Path (LSP) setup or not.

In the overlay model (Layer 2 VPN), the customer has some involvement in setting up the LSPs, and the service provider is not fully aware of the detailed internal tunnel construction. Conversely, in the peer model (Layer 3 VPN), the control planes of the customer and service provider interact directly, allowing the service provider to automatically set up the VPN tunnels.

Another key point is the difference in the encapsulation layer: the Layer 2 VPN carries traffic with an outer Layer 2 header, while the Layer 3 VPN encapsulates packets with an outer Layer 3 header. This distinction affects how the VPN handles the inner data packets.

MPLS-based Layer 2 VPNs

One prominent Layer 2 VPN solution is **Pseudo Wire Emulation End-to-End (PWE3)**, which allows multiple types of services—such as IP, leased lines, Frame Relay, ATM, and Ethernet—to coexist on the same MPLS network.

The goal is to create a virtual “wire” connecting two remote sites of an organization. Imagine two sites of the same agency connected through an MPLS network. Each site’s local network has a device at the edge—called the Customer Edge (CE)—which could be a router or switch, depending on the implementation.

In this setup, **customer edge** (CE) devices connect to **provider edge** (PE) devices, managed by the service provider, with traffic carried through Label Switched Paths (LSPs) between these points, via Provider (P) routers.

To summarize the terminology:

- **CE (Customer Edge):** Device at the edge of the customer’s network (could be a router or switch).
- **PE (Provider Edge):** Device at the edge of the service provider’s MPLS network.
- **P (Provider):** Routers within the core MPLS network.

In the PWE3 solution, the customer edge devices at the two sites are connected via VPN tunnels established through the provider edge devices. This creates an end-to-end Layer 2 connection as if a physical wire connected the two sites directly—hence the term “wire emulation.” The internal structure of the packets in this connection does not matter; the key point is that the outer header uses Layer 2 encapsulation.

Each packet is labeled twice:

- An external label used for routing within the MPLS core network, identifying the network access point.
- An internal label that multiplexes multiple users or services at the same access point.

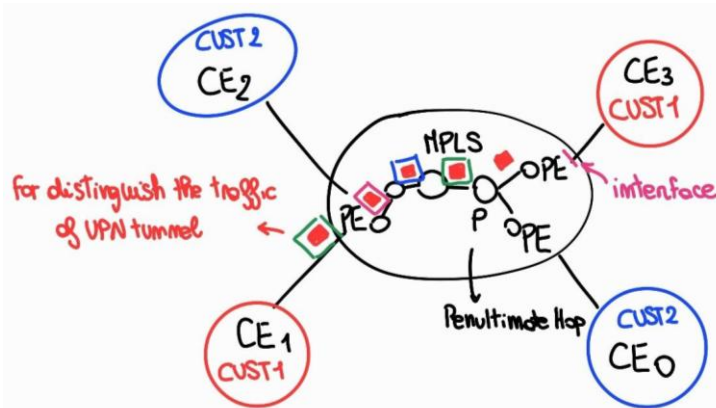
Within the provider’s network, there may be aggregation devices (for example, ATM switches) that manage traffic between different users. Typically, the setup of these LSPs is manual, though protocols like LDP and BGP are being explored to automate the process.

An important aspect of this Layer 2 VPN is that the customer networks on both sides may simply be Ethernet networks, not necessarily IP-based. The CE devices could be standard Ethernet switches, which cannot communicate control information directly with the provider’s routers. This means the control planes of the CE (switch) and PE (router) devices are fundamentally different, so they cannot directly negotiate the VPN tunnels.

This situation is similar to the older example of IP-over-ATM networks, where IP routers had to communicate through an ATM core network. The control planes of IP and ATM were incompatible, so the connections had to be manually and statically configured by network administrators, resulting in complexity and requiring expert intervention.

Similarly, in PWE3, the connection between customer edge and provider edge devices requires manual (and static) configuration by the network operator. The VPN tunnel is set up from PE to PE, but it logically connects CE devices on either end. The CE devices specify the destination they want to reach, and the PE devices establish the tunnel accordingly.

In summary, the PWE3 Layer 2 VPN extends the customer’s Layer 2 domain across the MPLS backbone, allowing remote Ethernet networks to behave as if they were directly connected.



MPLS-based Layer 3 VPNs

Agencies having CE devices not being normal switches but routers, so they can directly connect to the ingress and egress (PE). You can reuse some functionalities of some IP protocols for routing, for example BGP. It is suited for large scale network and it is used for AS-to-AS.

Layer 3 VPNs operate with full provider involvement, meaning the service provider manages all VPN policies and configurations. Customers do not need specialized expertise, making these solutions highly scalable and suitable for large deployments.

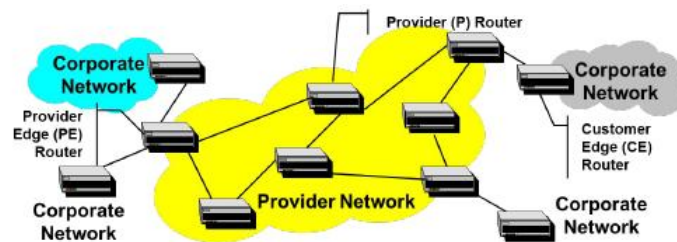
Two widely used approaches exist:

- **RFC 2547bis (BGP-based VPNs):** Originally developed and supported by Cisco, this method is now the most widely adopted for Layer 3 MPLS VPNs. It leverages BGP to distribute VPN routes securely and efficiently.
- **RFC 2917 (Virtual Router):** Supported initially by Nortel and Lucent, this approach uses virtual routers to isolate VPN instances.

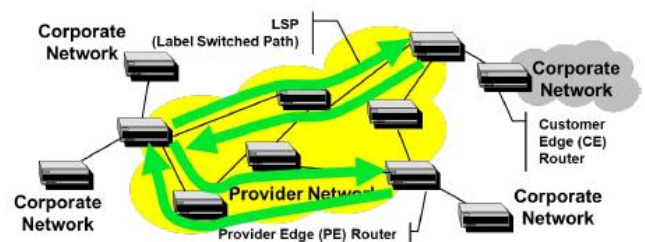
MPLS-based VPN Components

In an MPLS-based VPN, the Customer Edge (CE) router establishes adjacency with the Provider Edge (PE) router. This means the CE router:

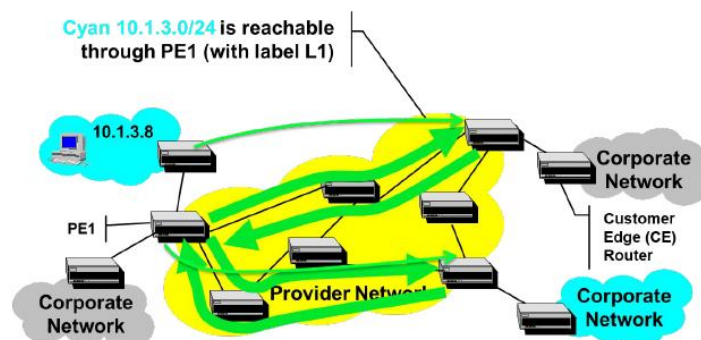
- Advertises its own destination networks to the PE.
- Receives advertisements about other VPN destinations from the PE.
- Typically uses static routing or an Interior Gateway Protocol (IGP) like OSPF or RIP to manage this communication.



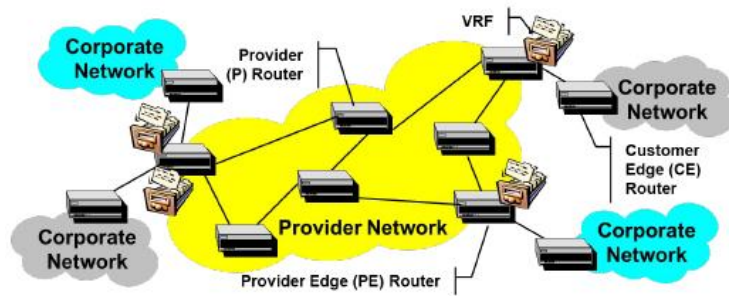
Within the provider's network, the routers (PE and P routers) maintain routes only to PE routers, not to customer destinations. PE routers set up Label Switched Paths (LSPs) between themselves using protocols such as LDP, RSVP, or sometimes Interior BGP (I-BGP), allowing label binding based on network topology.



PE routers exchange routing information exclusively among themselves. This is done via I-BGP for BGP-based VPN solutions or IGP in Virtual Router (VR) solutions. Each PE router only keeps routing information for the VPNs directly connected to it, maintaining efficiency and isolation between VPNs.



A **VRF (VPN Routing and Forwarding)** table is a crucial component in MPLS VPNs. Each VRF is associated with one or more physical ports that are not MPLS-enabled themselves (i.e., regular interfaces connected to customer networks). The VRF holds forwarding information specifically used to route traffic arriving through these ports, ensuring that packets are handled according to the correct VPN context. This mechanism effectively isolates traffic between different VPNs by maintaining separate routing and forwarding tables for each VPN instance within the provider's network.



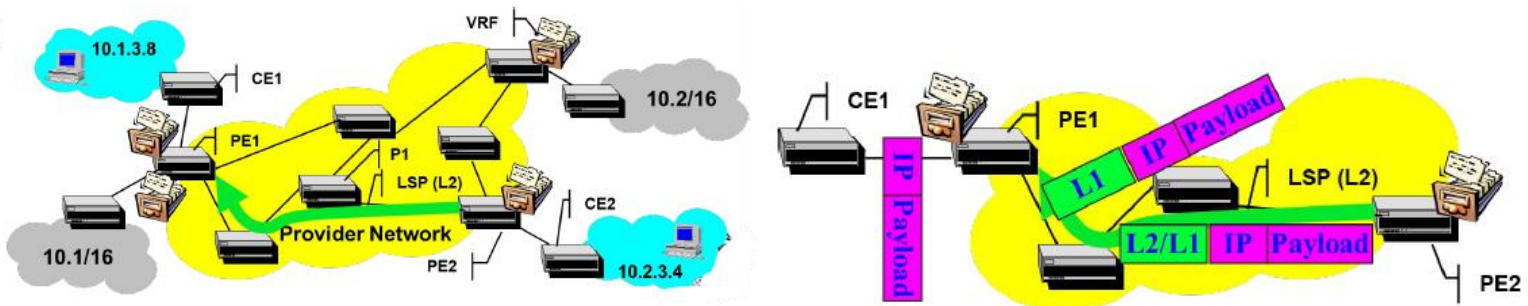
Packet routing

Consider a packet originating from IP address 10.2.3.4 (in the “cyan” VPN) destined for 10.1.3.8:

1. The packet’s default gateway is PE router 2 (PE2).
2. PE2 looks up 10.1.3.8 in the “cyan” VRF (VPN Routing and Forwarding table).
3. The next hop is PE router 1 (PE1), and PE2 pushes label L1, assigned by PE1 for the cyan subnet 10.1.3.0/24.
4. PE2 then looks up PE1 in the main routing table and finds the next hop is P router 1 (P1), pushing label L2, which represents the LSP from PE2 to PE1.
5. The label stack on the packet is now [L2, L1].

P routers forward the packet toward PE1 using label L2. The penultimate router before PE1 removes (pops) label L2 (this is Penultimate Hop Popping, PHP), so PE1 receives the packet with only label L1.

PE1 then pops label L1, revealing the plain IP packet, and routes it to the correct output interface based on that information.



Benefits

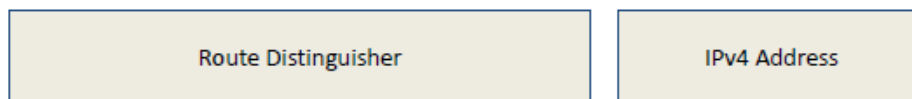
- No strict requirements on the IP addressing plan; addresses need only be unique within each VPN.
- CE routers do not directly exchange routing information with each other.
- Customers don’t manage or see the backbone network.
- Providers don’t need to create separate virtual backbones for each customer.
- VPNs can span multiple providers seamlessly.
- Security is comparable to traditional Frame Relay or ATM networks through traffic isolation.
- No cryptographic mechanisms are required to ensure confidentiality.
- Quality of Service (QoS) can be supported via experimental bits in the MPLS header.

Routing in MPLS-BGP VPNs

In MPLS-BGP VPNs, the inner label plays a crucial role in differentiating traffic belonging to various VPNs. The Provider Edge (PE) router uses this label to determine the correct outgoing interface for forwarding the packet.

At the network edges, routing information is exchanged using Multi-Protocol BGP (MP-BGP), which supports multiple address families and allows for sophisticated route filtering. PE routers use MP-BGP to decide which routes to install in each Virtual Routing and Forwarding (VRF) instance. This setup enables overlapping IP address spaces across different VPNs by combining a unique Route Distinguisher (RD) with the standard IPv4 address, effectively creating a distinct VPN-IPv4 address for each VPN.

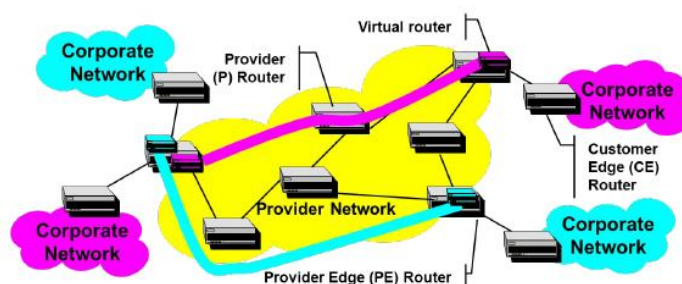
This mechanism supports different types of tunnels and traffic—whether external BGP routes or interior routing protocols—ensuring traffic is properly isolated and routed within the provider’s MPLS infrastructure.



MPLS-Virtual Router VPN

Some MPLS VPN architectures adopt a more advanced approach by running multiple virtual router (VR) instances on each PE router—one for every VPN it supports. Each VR maintains its own independent data structures and routing tables. VRs belonging to the same VPN communicate via Label Switched Paths (LSPs), preserving logical separation and improving scalability.

Although this architecture offers fine-grained control and isolation, it is relatively uncommon in practice due to its complexity and overhead.



Transport IPv6 traffic in the core of the (MPLS) network

DS-Lite and IPv6 backbones

Transporting IPv6 traffic across an MPLS core raises several architectural questions, especially in networks that were originally designed for IPv4. One approach often discussed is DS-Lite, which assumes that the backbone already supports native IPv6 transport. Under this model, IPv4 is not required within the core at all. Instead, IPv4 traffic is encapsulated and tunneled toward a specific element called the Address Family Transition Router (AFTR). Because the core only needs to forward IPv6 packets, IPv4 routing becomes irrelevant.

This immediately raises a practical question: would any realistic provider willingly convert their entire backbone into an IPv6-only network? Replacing a mature, stable, and well-tested IPv4 infrastructure with a completely new IPv6 one—still relatively less experienced and less validated—seems unlikely for most operators.

6PE

A more pragmatic solution for providers with an MPLS core is 6PE (IPv6 Provider Edge). This mechanism offers a way to introduce IPv6 connectivity without modifying the existing MPLS backbone. The idea is straightforward: keep the MPLS core exactly as it is—at least initially—while adding IPv6 support only at the network edge. Provider Edge (PE) routers that connect customers can be upgraded to handle IPv6 traffic, and IPv6 routing information is then distributed across the MPLS/BGP control plane in much the same way MPLS VPN routes are handled today.

This approach is formally described in RFC 4798, Connecting IPv6 Islands over IPv4 MPLS Using IPv6 Provider Edge Routers (6PE).

Possible options for IPv6 over MPLS networks

When enabling IPv6 over MPLS, providers have three main options:

1. **Native IPv6 over MPLS (full upgrade).** In this case, IPv6 and IPv4 traffic are treated identically inside the core. However, this requires a complete control-plane upgrade: routers must run IPv6 routing protocols internally and must support IPv6 label distribution. If both IPv4 and IPv6 services are offered, operators end up maintaining a dual control plane, which adds operational complexity.

2. **Native IPv6 over MPLS using Layer 2 encapsulation.** Here, Layer 2 frames—such as Ethernet frames or ATM cells—are simply encapsulated into MPLS and transported across the network. The advantage is that provider routers require no software or configuration changes. The drawback is scalability, especially in environments with highly meshed topologies where a large number of L2 tunnels is needed.
3. **6PE.** Technically similar to MPLS VPNs, this method leverages existing MPLS capabilities and BGP extensions to carry IPv6 routes. The complexity and operational model are comparable to VPN provisioning, making it familiar and manageable for providers already running MPLS services.

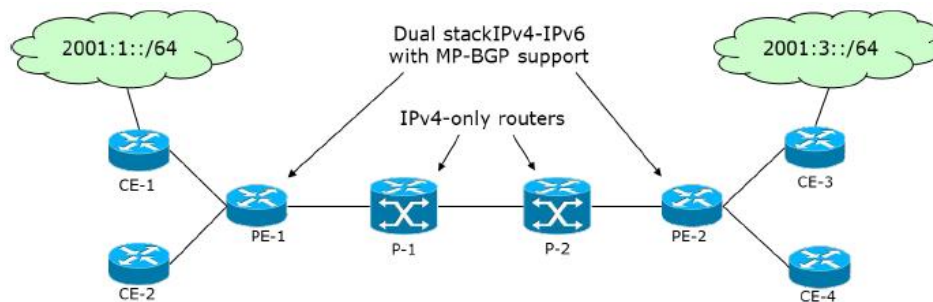
6PE: applicability

The applicability of 6PE is clear: it is an excellent solution for ISPs that already have an MPLS backbone and perhaps already deploy MPLS VPN or related services. It is particularly attractive when IPv6 demand is still low or moderate. When the number of IPv6 customers increases to the point where nearly all access routers need to become 6PE-enabled, it may be time to consider a full network-wide IPv6 upgrade.

6PE is especially useful for providers who want to avoid both a complete core upgrade and the operational burden of deploying IPv6-over-IPv4 tunnels.

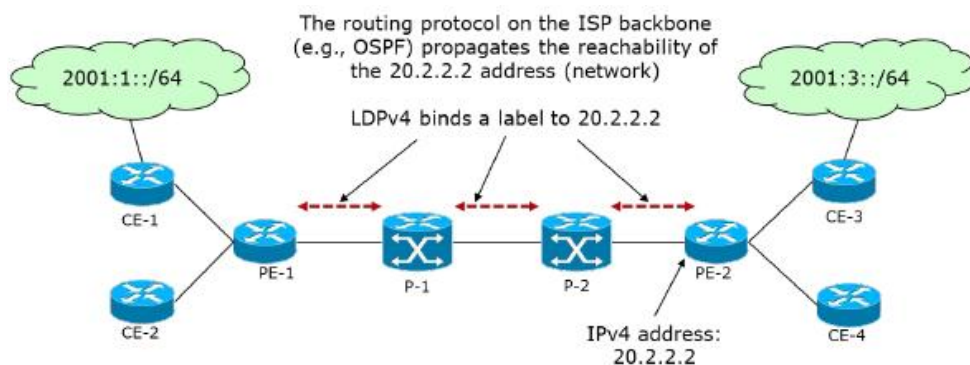
Requirements

To deploy 6PE, the requirements are minimal. Provider Edge routers must be upgraded to support IPv6 and MP-BGP, as they are responsible for learning and distributing IPv6 routes. However, core (P) routers do not require any modification at all—neither in configuration nor software—since they simply forward labeled packets without needing IPv6 awareness.



Announcing IPv6 networks

Finally, the process of announcing IPv6 networks is integrated with existing MPLS mechanisms. Each 6PE router advertises its IPv4 address (or prefix) into the IGP of the IPv4/MPLS core. As these routes propagate, every MPLS router in the domain assigns a label corresponding to the path to each 6PE router. This labeling allows the network to carry IPv6 traffic encapsulated within MPLS just as it would with MPLS VPNs, without requiring IPv6 capabilities in the core.



Distributing MPLS labels

In an MPLS network that supports IPv6 through 6PE, label distribution works differently for IPv4 and IPv6 routes.

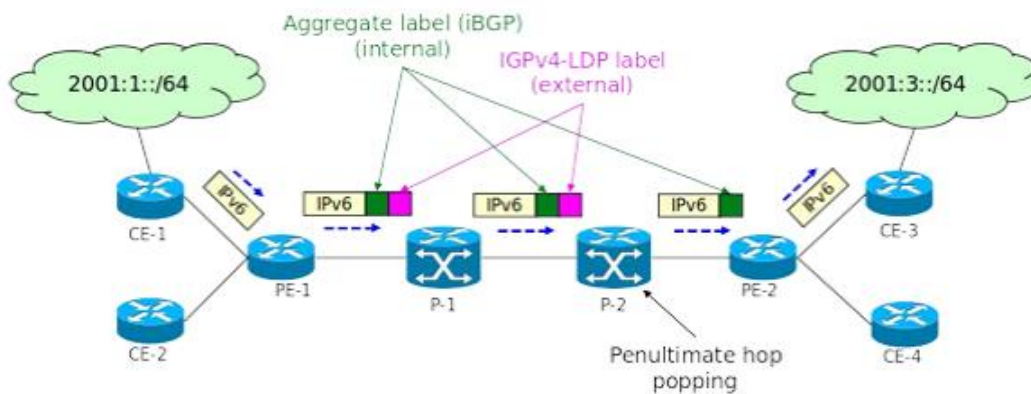
Within the core, IPv4 labels are exchanged using LDPv4, which is the standard mechanism P routers rely on for establishing label bindings.

IPv6, however, follows a different path: IPv6 labels are distributed using MP-iBGPv4 among the PE routers. This separation reflects the fact that P routers are not required to understand or process IPv6 routes at all—they simply forward labeled packets through the MPLS core.

Forwarding IPv6 traffic

When a Customer Edge (CE) router sends an IPv6 packet, it is delivered to the ingress 6PE router (PE-1). PE-1 must determine how to reach the remote 6PE router responsible for the packet's destination IPv6 prefix.

This is done by tunneling the IPv6 packet over an LSP toward the egress 6PE router. The egress 6PE is identified using its IPv4 address, derived from the IPv4-mapped IPv6 address that appears in the BGP Next Hop field for the advertised IPv6 prefix.

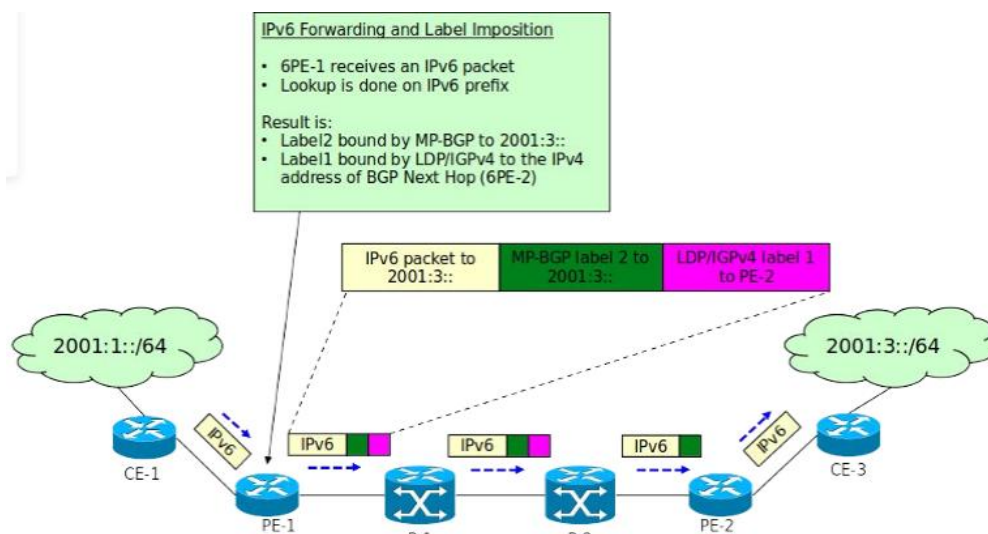


To make this possible, 6PE uses two MPLS labels:

- **Inner label** ("aggregated IPv6 label") → This label corresponds to each advertised IPv6 destination prefix. It identifies which IPv6 route the packet belongs to once it reaches the destination PE.
- **Outer label** → This label identifies the IPv4 address of the egress 6PE router and allows the packet to be transported across the MPLS core just like any other labeled packet.

Technically, the inner label is not strictly necessary for forwarding to work. However, including it keeps the encapsulation model consistent with other MPLS tunneling mechanisms such as VPNs.

Without the inner label, the penultimate hop router would need to forward a plain IPv6 packet directly to the egress PE—something the core is not designed to handle



CDN - Content Delivery Network

Introduction

Unlike protocols such as MPLS, IPv4, or IPv6 that we have studied previously, a CDN is not a protocol but rather a structural framework for managing modern core network infrastructure. As users, you might not even be aware of the existence of CDNs, yet they play a fundamental role in delivering internet content efficiently on a large scale.

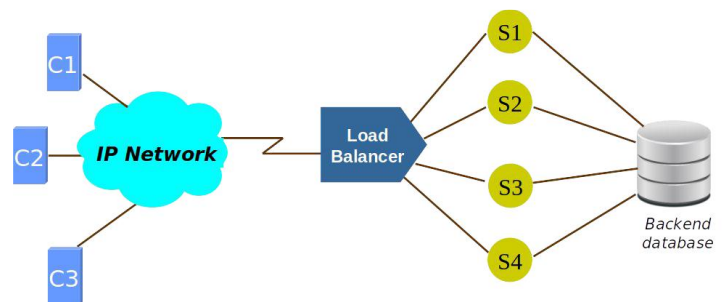
Unlike protocols such as IP, VPN protocols like GRE or IPsec, or MPLS—which are standardized by formal bodies with fixed specifications—CDNs are not standardized protocols. Instead, they are more of a methodology widely used on the Internet by content providers to improve user experience when retrieving content from servers.

How to deploy large-scale web servers

CDNs are crucial tools especially for large content providers—companies like Google, Amazon, Netflix, and others—that need to deliver their services worldwide. These providers cannot rely on a single server to serve the entire globe. Instead, they operate numerous large-scale data centers, often called **server farms**, strategically distributed around the world. Each farm contains hundreds or even thousands of servers.

Efficiently deploying and managing these server farms involves several key components:

- Front-end load balancers distribute incoming client requests evenly across multiple servers.
- Backend databases are accessed by all servers to maintain consistent data.



Managing traffic to these server farms is a significant challenge. Typically, in front of these servers, there is a **load balancer**—sometimes also called a **scheduler**. The primary function of a load balancer is to distribute incoming client requests across multiple servers in an efficient way. The term “load” here refers to various server resources such as CPU usage, RAM, disk input/output, or network bandwidth. Ideally, the load balancer ensures no server is overwhelmed while others are idle—for example, it would be inefficient to have one server running at 90% CPU while the others are barely used. Instead, the goal might be to have all servers operating around 50% capacity for balanced utilization.

However, load balancing can have other important objectives beyond equal distribution. One significant consideration is energy consumption. Large data centers consume vast amounts of power, and reducing this cost is a priority. Interestingly, studies have shown that server power usage remains relatively constant except when CPU load is below 20% or above 90%. This means operating servers mostly in the middle range of utilization doesn’t save much power. Instead, a power-saving strategy might aim to keep most servers under 20% load and turn off or reduce the power of unused servers, which conflicts with the goal of balanced load.

Thus, managing load balancing is a complex optimization problem, often requiring sophisticated algorithms tailored to specific goals—whether that’s balancing traffic, minimizing power consumption, or a combination of both. Load balancers are advanced software components capable of dynamically scheduling traffic to meet these diverse requirements.

From the network perspective, consider the case of a client wanting to access Google’s servers. The client queries the DNS, which resolves the domain to an IP address. This IP typically belongs to a load balancer or a server within a farm. There are different ways to design this:

- One approach is to have the load balancer hold the public IP address, and all client requests are directed to it. The load balancer then forwards requests to individual servers with private IPs. This requires the load balancer to rewrite the destination IP in packets and to modify source IPs on responses, effectively acting as a network “middleman.” While this centralizes traffic control, it can create a bottleneck as all traffic funnels through the load balancer.

- An alternative design is to assign the same public IP to all servers, allowing the DNS to direct traffic accordingly. In this scenario, the load balancer's role is simplified to selecting a server for each request, based on load or energy considerations. This reduces the bottleneck but requires more distributed intelligence.

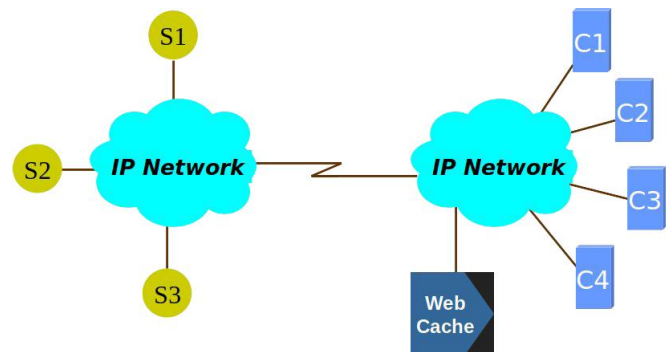
While load balancing manages traffic within server farms, there is a bigger challenge: controlling how traffic reaches the data centers globally. Large providers often have multiple server farms spread across continents. For example, Google might have farms in the US and Europe. It is preferable for a user in Italy to access the European farm rather than the US one, minimizing latency and network congestion.

DNS plays a key role here by directing users to the nearest or best-performing data center. However, smaller content providers might have only a few servers in one location, so managing global traffic efficiently becomes even more challenging.

Old days: web Caches

Before CDNs, traffic reduction and local content delivery were often handled using **web caches**. A web cache is a server placed close to the user's network, storing copies of frequently requested web content. When a user requests a page, the web cache intercepts the request and, if it has the content stored, serves it directly, reducing the need to fetch data from the remote server farm repeatedly.

If the requested content is not in the cache, the web cache forwards the request to the origin server, receives the response, stores a copy, and then serves it to the user. This way, subsequent requests for the same content are served locally. Web caches are sometimes called **proxy servers** because they act as intermediaries between clients and servers.



A Web Cache is a device that stores a local copy of more recently required HTTP objects and reacts as proxy server to clients' requests

Why caches?

1. Keep traffic local, decreasing the pressure (i.e., cost) on Internet transit links
 - a. Connection points between ISP and Internet backbones may be more overloaded than intra-ISP links
 - b. Cost of transferring traffic over Internet backbones is usually more expensive than intra-ISP links (the former may also depend on the volume of traffic, while the latter is usually "flat")
2. Better Quality of Experience for end users
 - a. Decrease latency, hence more prompt answer from remote web server (e.g., limits the impact of the three-way handshake)
 - b. Improves throughput (TCP throughput is proportional to WND/RTT)
3. Enables localized content (e.g., advertisement of local shops)

The benefits of web caches include reducing traffic over wide area networks and decreasing latency by keeping content "local." This also has economic benefits, as it reduces the load on expensive backbone network links. However, traditional web caching has limitations and is no longer widely used in its original form due to the evolving nature of internet traffic and content delivery.

There are service providers managing the autonomous systems that lie between networks. For instance, companies like Fiber Coppo in Italy or similar providers across Europe operate these networks. These service providers charge fees based on the volume of traffic that passes through their infrastructure.

Therefore, it's not only about reducing the overall traffic load but also about minimizing costs by keeping the traffic as local as possible. If you can prevent requests from traversing multiple autonomous systems or zones, you save money because every time traffic passes through a link owned by a service provider, you incur costs. So, maintaining local traffic helps reduce both network congestion and expenses.

To clarify this with an example related to improving user experience: by localizing content—meaning storing it closer to the user—you significantly reduce latency. This proximity not only speeds up service delivery but also allows better management of quality of service (QoS) since traffic is controlled within a single network. If the data must traverse multiple autonomous systems, ensuring consistent QoS across all these independent systems becomes difficult, if not impossible.

Another important function of web caches is content localization. By storing content closer to the user, it becomes possible to customize what the user sees based on their geographical location. For example, accessing the Google homepage from Italy might display Italian advertisements, while a user accessing Netflix from the U.S. sees a different content catalog than one accessing it from Italy. This geo-based customization hinges on knowing the user's location at the time of the request.

Additionally, web caches contain metadata that indicates how “fresh” the stored content is. For instance, a cache storing a news homepage like “La Stampa” will need frequent updates because new articles or advertisements appear regularly—sometimes every half hour. This means caches must have mechanisms to verify that the stored content is up-to-date, not just originally fetched from the source.

HTTP Caching

- Clients often cache documents
 - o When should origin be checked for changes?
 - o Every time? Every session? Date?

HTTP protocols provide ways to manage this freshness. HTTP/1.0 uses an expiration date header, while HTTP/1.1 offers more sophisticated mechanisms like ETags (entity tags) that mark the version of the content. When the cache determines its content has expired, it sends a conditional request to the origin server to check if the content has changed. If the content remains the same, the server responds with a “not modified” status instead of resending the full content. This strategy significantly reduces network traffic by avoiding unnecessary data transfers.

- HTTP includes caching information in headers
 - o HTTP/1.0 used: “Expires: <date>”; “Pragma: no-cache”
 - o HTTP/1.1 has “Cache-Control”
 - “No-Cache”, “Private”, “Max-age: <seconds>”
 - “E-tag: <opaque value>”
- If not expired, use cached only. If expired, use condition GET request to origin
 - o “If-Modified-Since: <date>”, “If-None-Match: <etag>”
 - o 304 (“Not Modified”) or 200 (“OK”) response

Historical perspective

From a historical perspective, **web caching** emerged in the 1990s as a method to reduce bandwidth consumption and improve user-perceived performance. Early web caches had to be explicitly configured by end-users, which immediately revealed a classic problem: users might forget to configure them, enter incorrect parameters, or simply refuse to bother. Over time, this led to the adoption of **transparent proxies**, which intercept and cache traffic automatically, without requiring any user action. Transparent TCP/HTTP proxies became widely deployed and helped improve efficiency across many networks.

However, the arrival of **SSL/TLS** fundamentally changed this model. Encrypted connections were marketed as offering stronger security—only the final endpoint (e.g., Google) can inspect the user's traffic. But in practice this also means that **nobody else along the path, including the ISP, can see or interpret the content**. Since web proxies need to inspect HTTP requests to determine whether content can be served from cache, widespread encryption effectively makes ISPs “blind,” preventing transparent proxies from performing their traditional role. A proxy cannot cache what it cannot read. Breaking encryption would undermine security and privacy, so encrypted traffic must be forwarded untouched.

Because of this shift, ISPs and Over-The-Top content providers (like Google or Netflix) increasingly had to establish **explicit agreements** to deploy caches directly within the provider backbone. Yet even these approaches have significant limitations:

- Server farms placed by content providers in ISP networks do not solve general network congestion or reduce latency beyond their own content.
- Caching proxies only serve the clients within their domain; they provide no benefit to the wider Internet.
- Content providers cannot rely on the presence or correct behavior of third-party caching proxies.
- Accounting becomes problematic. For example, a site like cnn.com needs accurate hit counts for ads, which is hard to track when proxies serve cached pages.
- More than 50% of HTTP objects are inherently uncacheable, meaning proxies provide limited benefit even when they work correctly

Adding to this, most modern Internet traffic uses HTTPS, which makes the traditional caching model nearly obsolete. Since proxies cannot inspect encrypted payloads, they cannot determine cacheability, verify freshness, or serve cached responses. They are reduced to simple forwarders.

At the same time, content providers increasingly want full control over distribution, analytics, and revenue models. If a third-party proxy serves a cached object, the provider loses visibility into access patterns, undermining both business intelligence and monetization. This misalignment of incentives between ISPs (trying to reduce costs) and content providers (protecting control and data) became a major structural problem.

These pressures and limitations ultimately drove the transition toward Content Delivery Networks (CDNs), where content providers deploy and manage their own distributed caches at the edge of the network, fully under their control.

Content Delivery Networks

A **Content Delivery Network** (CDN) is an overlaid network of Caches, a.k.a. Content Servers, a.k.a. Delivery Nodes, a.k.a. Replicas.

CDNs are infrastructures owned by content providers themselves rather than service providers. Their goal is to replicate the content closer to users to improve latency, reduce network traffic, and maintain control over content distribution.

Conceptually, CDNs operate similarly to web proxies, they cache content near users, but the key difference lies in ownership and control. Web proxies are managed by ISPs or telcos, focusing on cost reduction by localizing traffic, whereas CDNs are managed by content providers prioritizing user experience and content control.

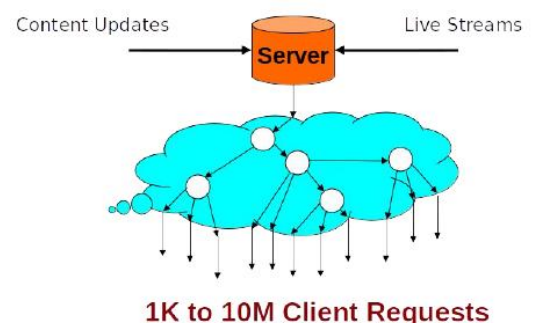
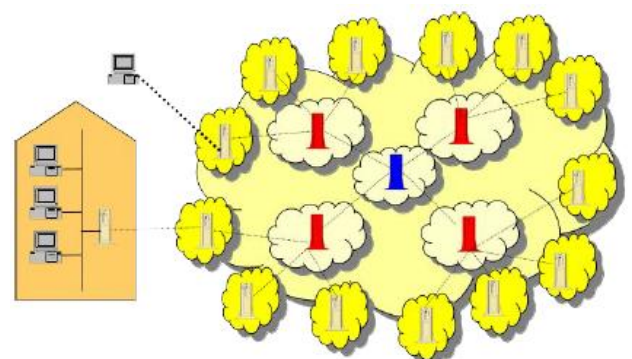
CDNs also address scalability better than web proxies. Since they are directly managed by content providers, they can proactively distribute content replicas across multiple servers, independently of user requests. This contrasts with reactive web caches, which only store content after a user requests it.

Usage Examples for CDNs:

- Distributed Web Hosting
- Video-On-Demand
- Scalable Live Streaming
- Dynamic Content
- Conditional-Access Content, Advertisements

Example of a CDN used for Live Streaming

A practical example is DAZN, a streaming service for football matches. When DAZN first entered the Italian market, the streaming quality was



poor because it relied on service providers' infrastructure rather than its own CDN. Service providers might lack the resources, budget, or motivation to deploy servers throughout Italy to optimize the streaming experience. In contrast, DAZN's goal was to provide a high-quality user experience by controlling where and how the content was served.

With a CDN, DAZN can have an original server located, for example, in Milan and deploy replica servers distributed across Rome, Naples, and other cities. These replicas hold copies of the original content, reducing latency and improving streaming quality by serving users from nearby servers instead of a single distant origin.

The original server hosts the master copy of the content but does not serve users directly. Instead, its role is to distribute the content to the replica servers, which handle the user requests. This entire infrastructure is managed by the content provider, who controls where the replicas are placed based on demand patterns, and how content is distributed.

For example, if there is a popular live football match between Milan and Naples, the provider will likely place more replicas close to Milan and Naples to serve the users efficiently. The content provider owns the infrastructure and controls content distribution strategies, deciding not only the locations of servers but also the content flow and replication.

When streaming a live event, such as a football game, the content originates from the source — say, a cameraman capturing the match. This live feed goes first to the original server, which holds the authentic stream but doesn't serve end users directly. Instead, this original server pushes the content to distribution nodes, which in turn send it to geographically dispersed replica servers. Users connect to these replicas to access the content

This setup separates the roles: the original server is responsible for content creation and management, while distribution nodes handle disseminating content to replicas that serve the users. The replicas are spread across different geographical locations, ensuring that users access content from a nearby server, reducing latency and improving performance.

The content is continuously updated during a live stream — for example, new frames of the match or news updates are generated by the original server and propagated live to the replicas. This real-time distribution ensures that users always have access to the latest content.

Moreover, CDNs also allow content providers to customize the content delivered based on the user's location. For instance, a user accessing a news website like La Stampa from Turin may receive news more relevant to that area, while a user in Naples will see different local updates. This geographic tailoring enhances user experience by providing localized content.

Content providers also control how long content remains available on replica servers. For example, a live football match may be stored on replicas for a few days after the event, allowing users to watch replays, after which the content is removed to free up resources for new content.

CDNs also enable content providers to differentiate between types of users, such as offering free access to some content while providing premium services to paying customers. This fine-grained control over access and distribution is an important business feature.

An alternative approach to improving performance on the Internet is to deploy caches strategically across the network. This helps enhance web performance, especially for static web pages or non-live streaming media, which do not change frequently. Since many server farms are geographically far from users, bringing content closer reduces round-trip time (RTT), improving user experience. Caching also helps save bandwidth, lowers the likelihood of encountering bottlenecks, and improves service availability by distributing load. During traffic spikes, this distribution is crucial, as it prevents origin servers from becoming overloaded and crashing.

CDN vs Caching Proxies

Over time, the limitations of traditional caching led to the rise of Content Delivery Networks (CDNs). While both caching proxies and CDNs aim to improve performance, their goals and operating models differ significantly:

- ISPs use caching proxies mainly to reduce bandwidth consumption. In contrast, CDNs are deployed by content providers to offer better performance and reliability to users.
 - o Comparing CDNs to traditional caches, both aim to improve service quality for users, but they differ significantly. First, the ownership is different: CDNs are usually owned by large companies with significant economic resources, such as Amazon, Google, or Netflix, capable of managing global infrastructures. On the other hand, caches may be managed by different entities with smaller scale and resources.
- Caching is reactive: content is stored only after a user requests it. If no one asks for a specific content, it won't be cached. CDNs are proactive, pushing content to distributed replicas before demand appears, even if users have not yet requested it, ensuring immediate availability and better performance during high demand.
- Caching proxies serve only the clients of the ISP operating them, whereas CDNs serve both content providers and end users directly.
 - o The ownership difference is crucial: some content providers, like La Stampa or the New York Times, focus on producing content but do not specialize in managing infrastructure or server networks. These providers often contract CDN companies to handle the content distribution network.
- CDNs give content providers tight control over how their content is delivered; caching proxies do not provide this control.
 - o Hence, the roles are typically split: the original server managed by the content provider creates and updates content, while the CDN provider operates the distribution nodes and replica servers. This separation allows content creators to focus on their core business, while CDN providers ensure efficient content delivery.
 - o From the user's perspective, accessing content involves interacting with the CDN's replicas rather than the original server. The original server is primarily responsible for content management and does not handle user requests directly.

Content Replication in CDNs

To operate effectively, a CDN deploys thousands of servers either in large data centers or closer to end users. The CDN then replicates the content of its customers, meaning the content providers, to these distributed nodes. Whenever the content provider updates an object, the update is propagated across the CDN's infrastructure.

When a webpage is distributed via a CDN, all URLs pointing to static content (like images) are rewritten to redirect requests to the CDN's domain rather than the original server.

From a technical standpoint, this requires rewriting URLs within the content. For example, an image originally referenced as:

www.lastampa.com/image1.jpg

is rewritten to something like:

www.cdnprovider.com/image1.jpg

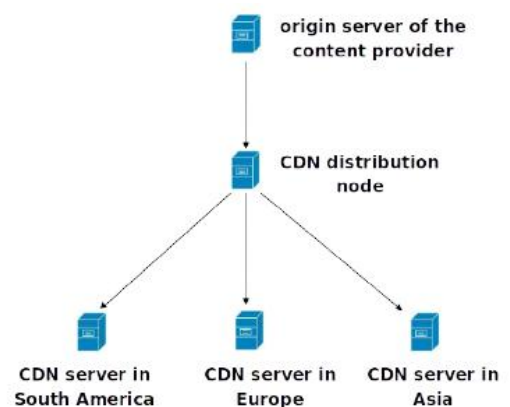
This ensures that requests for static elements are automatically directed to the CDN's nearest replica, making the process completely transparent to users.

However, the practical accomplishment of this idea encompasses multiple challenges: how to replicate content, where to replicate content, how to find replicated content, how to choose among server replicas, how to direct clients towards server replicas...

Server Selection

Once content is replicated, the system must determine which server should serve each client request. Selection criteria may include:

- The server with the lowest load, to balance traffic on servers
- The server offering the best performance (lowest RTT, highest throughput), to improve client performance, even if it's geographically further from the user.



- Any functioning server, to ensure fault tolerance; Relying on a single replica could lead to overload and failures, so distributing requests among multiple replicas ensures better availability.

Metrics such as geography, latency, load, and network conditions can be considered.

Directing clients to replicas can be achieved through several mechanisms:

- As part of routing: anycast, cluster load balancing
- As part of application: HTTP redirect
- As part of naming: DNS

Because CDNs are not defined by a single standard, and because content providers usually implement them in proprietary ways, there is a wide variety of approaches. Each content provider designs its own CDN architecture, which makes it impossible to delve into all the implementation details, as many are proprietary and core to their business models. Some CDN providers even build their entire business around these implementations, so they understandably keep their methods confidential.

There are different technical methods to direct clients to replicas:

- **Anycast routing:** This method selects one replica server from many possible ones, somewhat similar to load balancing across servers with the same IP in a server farm, but this involves having multiple servers share the same IP address, and the network routes the client request to the nearest or best server. The selection can be from all servers or just a subset. It is simple from a routing perspective and transparent to the client but offers little control to the content provider about which exact replica is serving the user.
 - o Pros: Transparent to clients, circumvents many routing issues
 - o Cons: Little control, complex, scalability
- **HTTP Redirects:** In this method, instead of serving content directly, the original server replies with a redirect message, telling the client to fetch the content from another replica server by providing the IP address. The client then requests content from the designated replica. From the client's perspective, this is automatic — the browser follows the redirect without user intervention. This approach introduces additional latency because it requires two HTTP requests, but it gives the content provider precise control over which replica serves each client, allowing for more detailed management.
 - o Pros: Application-level, fine-grained control
 - o Cons: Additional load and RTTs, hard to cache
- **DNS-based redirection:** This is the most commonly used method today. When a client requests the IP address of a domain, the DNS server directs the client to the IP address of the most appropriate replica based on the client's location or other criteria. This reduces latency because the client connects directly to the best replica from the start. However, the downside is that content providers typically have limited control over DNS servers, which are often managed by third parties, limiting fine-grained traffic management.
 - o Pros: Well-suited for caching, reduce RTTs
 - o Cons: Request by resolver not client, request for domain not URL, hidden load factor of resolver's population

DNS redirection allows broad geographic distribution decisions — for example, routing Italian users to servers in Italy and French users to servers in France — but less precision within regions.

DNS-based CDN

DNS servers are distributed hierarchically across the Internet. When a user (really, the browser on behalf of the user) wants to access a content provider's server, for example `www.google.com`, they need to translate that domain name into an IP address.

The browser first queries a nearby DNS resolver, typically located in the user's access network. If the resolver doesn't have the answer cached, it queries other DNS servers higher up the hierarchy, moving closer to the content provider's authoritative DNS server. The authoritative DNS server manages the IP addresses for the content provider's servers and finally returns the IP address to the resolver, which then answers the browser's request.

The core idea behind modern Content Delivery Networks is the introduction of a new DNS server architecture. In this architecture, DNS is no longer used only as a simple name-to-IP translation service, but it becomes an active component in traffic management.

Host names are exploited to redirect client traffic toward the most appropriate replica server. The selection of the replica does not happen at the application layer, but at the moment when the domain name is translated into an IP address. In other words, the decision of which server the client will contact is embedded directly in the DNS resolution process.

In this model, the authoritative DNS servers act not only as domain name resolvers but also as traffic routers, redirecting requests to replica servers that hold the desired content. These servers are sometimes called content routers because they route client requests based on internal algorithms that consider network topology and other metrics, though these algorithms are typically proprietary.

In a traditional DNS lookup, the input to the DNS query is just the domain name, and the output is the IP address of the server to contact. In DNS-based CDNs, this model is enhanced by adding the source IP address of the requester to the input of the DNS query. This allows the DNS server to make more informed decisions about which replica server IP to return, aiming to reduce latency by directing the user to a geographically or topologically closer server.

However, because of the hierarchical nature of DNS, the "source IP" seen by the authoritative DNS is often not the client's original IP, but the IP of the DNS resolver. This limits the granularity of the decision, making it more of a coarse-grained selection based on the resolver's location rather than the exact user location or the specific content requested.

Typical metrics include round-trip time (RTT), server load, response time, and other Layer 3 network metrics. Based on these measurements, the DNS server computes and continuously updates a replica routing table, which maps DNS names and source IP addresses to destination IP addresses of replica servers. Conceptually, this can be represented as a mapping of the form:

$$\{ \text{DNS name, source IP address} \} \rightarrow \text{destination IP address}$$

However, accurately measuring these metrics is not trivial. Network conditions change dynamically, measurements can be noisy, and the DNS server often sees only partial information about the client's actual network path. Moreover, relying solely on Layer 3 metrics, such as IP-level routing information, is often insufficient. These metrics do not fully capture application-level performance or user experience and therefore are not particularly meaningful when used in isolation.

There is a trade-off: the more detailed the input metrics used for server selection, the more complex the decision process becomes, which can increase DNS query latency. Often, a less precise but faster decision is preferable to a highly precise but slower one, because users prioritize quick webpage loading over perfectly optimal server selection.

For this reason, CDN providers typically combine multiple metrics and heuristics to approximate the best possible replica selection, balancing accuracy, scalability, and response time.

To illustrate the difference between traditional DNS and DNS with CDN, consider the hierarchical network structure.

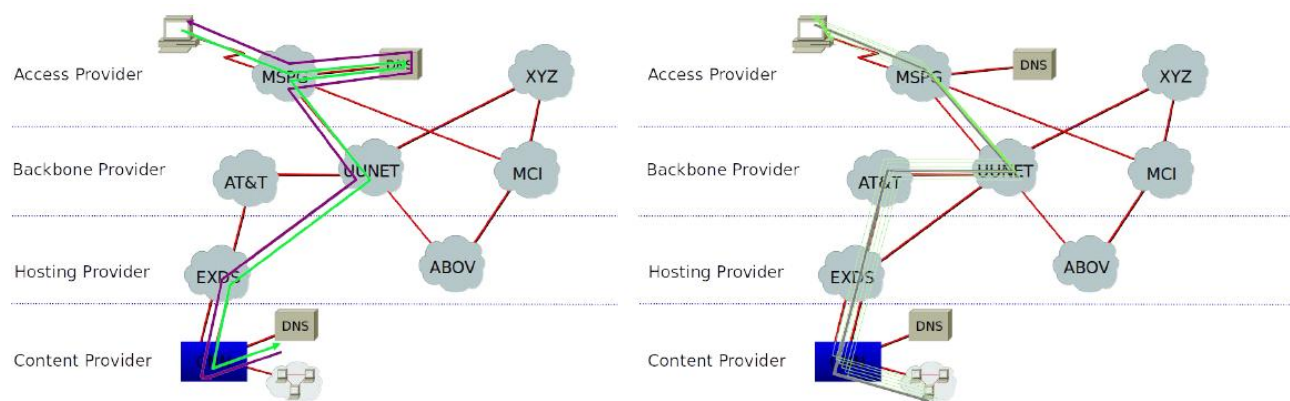
When you make a request for content, such as a web page, the process involves the DNS hierarchy. At the top of this hierarchy, there is the authoritative DNS server, which typically resides within the content provider's network. This authoritative DNS server is responsible for managing and merging the IP addresses of the content provider's servers, as stored in their server files.

Let's take an example: imagine you want to access the homepage of [cnn.com](http://www.cnn.com), the well-known American news broadcaster. When you type "www.cnn.com" in your browser, your request first reaches your access network. The initial step is contacting the DNS resolver, which is usually located within your access network. Since your device doesn't know the IP address corresponding to www.cnn.com, it asks this DNS resolver to translate the domain name into an IP address.

Most of the time, the DNS resolver does not have the answer cached, so it queries up the hierarchical DNS system until it reaches the authoritative DNS server associated with the content provider's network—in this case, CNN's network. However, often this authoritative DNS is not directly managed by CNN but by a CDN (Content Delivery Network) provider or a general network provider close to the server farm of CNN.

This authoritative DNS server in the CDN or network provider's system knows which IPs belong to the various servers within the content provider's server farm. It may also act somewhat like a load balancer, selecting the best server IP to respond to the request based on various metrics such as load, proximity, and network conditions.

Once the authoritative DNS selects the appropriate server IP, it sends this IP back down the chain to the DNS resolver, which finally replies to your browser with the resolved IP address. Your browser can then directly contact the server to retrieve the requested content.

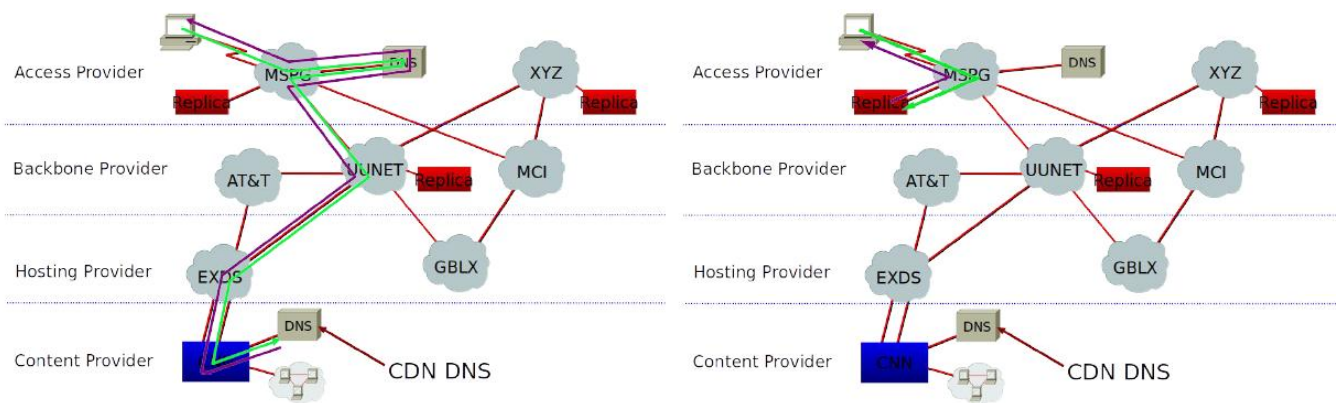


When your browser sends the request to the server, it doesn't usually receive a single response. Instead, it gets multiple responses because web pages are composed of many elements. The main file you receive is typically an "index.html" file, which serves as the backbone of the web page structure. This HTML file includes the layout, text, and references to various other content pieces.

Within this HTML file, you'll find references (URLs) to dynamic content elements such as logos, banners, videos, advertisements, and other images. Static elements like text and page structure form a smaller portion of the web page, while the dynamic content constitutes the bulk of what you download.

After receiving the main HTML file, your browser issues multiple GET requests to retrieve each piece of dynamic content separately. This is why you may notice that browsers first load the static page structure and text, and then progressively load images, videos, and other media, allowing you to see the page quickly while other elements are still loading.

An important metric in web performance is the total response time — the duration from your initial request for the HTML page until the last dynamic content finishes loading. For complex pages with many videos or images, this can take significantly longer.



Now, let's look at how CDNs change this process.

With CDNs, the DNS request path remains mostly the same, but the content of the DNS queries and responses changes. When your resolver asks for the IP address, the authoritative DNS (now implemented by the CDN) includes additional information, such as your source IP and various network metrics. This allows the CDN to decide which replica server you should contact — ideally, one geographically or network-topologically close to you.

Instead of pointing your browser to a server inside the original server farm, the response points to a replica server managed by the CDN provider, which has a copy of the content. Your browser then contacts this replica server directly, which serves the content more quickly due to proximity and load distribution. This reduces traffic to the original content provider's server farm, decreasing the load on the origin servers and improving user experience with faster content delivery.

For example, consider CNN's homepage again. The page consists of dynamic elements such as the logo, images, videos, banners, and advertisements, all referenced inside the index.html file. CDNs focus their caching and delivery efforts primarily on this dynamic content, which accounts for the majority of traffic and user data consumption.

DNS-based CDNs Limitations

Despite their widespread use, DNS-based CDNs have intrinsic limitations:

- Redirection granularity is limited to the host name, not the full URL, so fine-grained decisions based on specific objects or URLs are not possible.
- Large websites cannot easily split content into multiple caches using the same host name.
- Using the same domain for both static and dynamic content is challenging.

Akamai

Akamai implements its CDN strategy through the use of so-called "Akamaiized URLs", which are URLs specifically rewritten to route requests through Akamai's distributed infrastructure. An example of such a URL is:

<http://a836.g.akamaitech.net/7/836/123/e358f5db0045e/www.cnn.com/logo.gif>

Instead of requesting content directly from the original domain (e.g., www.cnn.com) the browser is instructed to fetch the resource from a domain managed by Akamai (or another CDN provider). These URLs encode the content name and implicitly identify the replica server that hosts the cached content.

To enable this mechanism, Akamai creates new domain names for each client content provider, such as a128.g.akamai.net. Akamai's DNS servers are authoritative for these domains, giving Akamai full control over how requests are resolved and which replica servers are selected.

The content provider collaborates by modifying its web pages so that all embedded resources reference the Akamai-managed domains rather than the original one. For example:

Original URL: <http://cnn.com/images/logo.gif>

CDN-based URL: <http://a128.g.akamai.net/7/23/cnn.com/images/logo.gif>

This URL rewriting process is typically automated using server-side tools such as Apache modules or IIS ISAPI filters, making the transition transparent for both developers and users.

The resulting URLs and server assignments are often obfuscated through hashing, reflecting proprietary algorithms used by CDNs such as Akamai, Cloudflare, and Amazon CloudFront. These algorithms determine the optimal replica server based on factors including geographic location, network load, and request patterns.

Behind the scenes, CDN systems carefully decide which content name, which content domain, and which replica server should be contacted to ensure efficient delivery. Although this complexity is invisible to the user, inspecting modern web pages reveals a structure far more intricate than it appears on the surface.

Content Replication and Domain-Based Routing

The core idea is that content is no longer fetched directly from the origin server. For example, a GIF originally hosted on `sireng.com` will be served via Akamai. There is always at least one copy of the content on the original server, which is required to generate the web page, but multiple additional copies are distributed across replica servers worldwide.

When a user contacts a replica server, the requested content is already cached closer to the user, significantly improving performance and reducing latency.

Using multiple domain names per client allows the CDN to further subdivide content into logical groups. Since DNS resolution only sees the requested domain name, Akamai can route different domains independently. For instance, requests for `www.repubblica.it` may be directed primarily to Italian caches, while requests for `www.elpais.es` may be routed to Spanish caches. This enables geographically and topologically optimized routing decisions, partially compensating for the coarse granularity of DNS-based redirection.

Coarse-Grained Decisions and Localization

An important limitation of DNS-based CDNs is that the CDN only sees the IP address of the DNS resolver, not that of the actual client. Therefore, discrimination is possible only at the level of the access network, not at the level of individual URLs or specific content objects.

For example, a DNS-based CDN cannot determine which movie a Netflix user is requesting based solely on DNS resolution. However, since Netflix operates its own CDN infrastructure, it can rely on additional mechanisms to customize content delivery, such as serving targeted advertisements based on the requested movie.

In general, domain-level discrimination is widely used and effective. Accessing `repubblica.it` from Italy typically results in Italian advertisements, while access from Spain leads to Spanish ads. Although coarse-grained, this approach still enables useful localization and content customization.

Detailed Interaction with Akamai

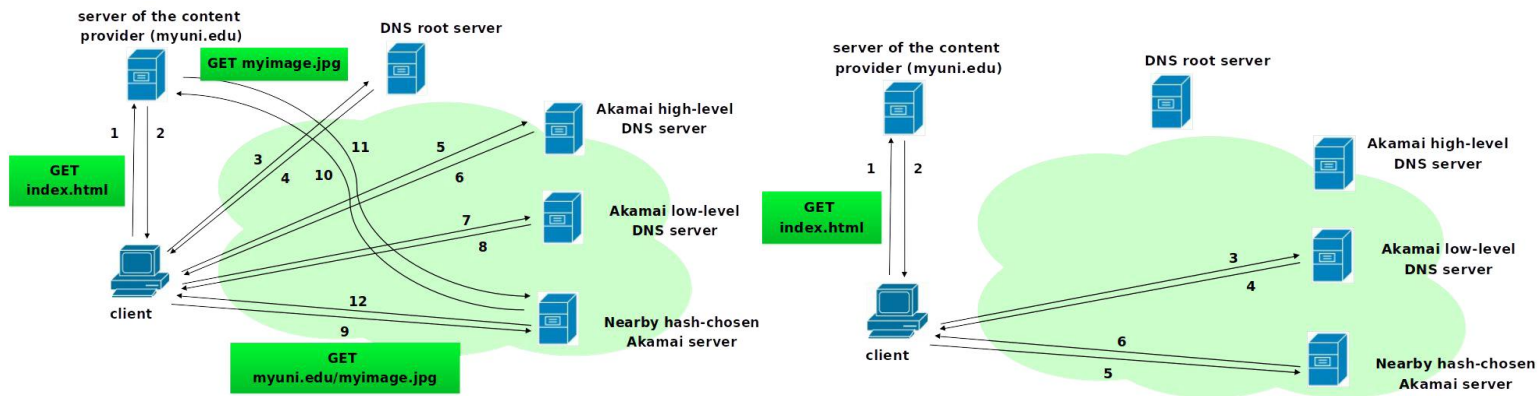
In a typical Akamai-assisted browsing session, the DNS resolver returns the IP address of a server within Akamai's infrastructure rather than the origin server. However, the first HTTP request is usually sent to the original server to retrieve the main HTML file. This does not contradict the goal of reducing origin traffic, because the HTML file is small and mainly contains references to dynamic content.

Once the HTML file is received, it includes many URIs pointing to dynamic objects. These objects are what the CDN caches and serves from replica servers. Subsequent requests are redirected—via DNS—to appropriate Akamai servers.

DNS resolution follows a hierarchical process until reaching the authoritative DNS server. If a replica server does not already have the requested content (a rare event), it retrieves it from the origin server, caches it, and then serves it to the client.

The CDN system is stateful, tracking which contents are cached and where. Caching can also occur at multiple levels, including within the browser itself, reducing the number of requests that must traverse the network.

This behavior can be observed by inspecting network traffic on major websites such as La Stampa. The initial request retrieves the HTML page from the origin server, while many subsequent requests are directed to CDN-managed domains.



Extension to DNS-based CDNs

There are multiple mechanisms for selecting the appropriate replica server. DNS-based CDNs provide a relatively coarse-grained solution, while HTTP redirection allows finer control by inspecting the full URL.

HTTP redirection can be implemented either by allowing every server in the farm to redirect requests or by using a **Server Load Balancer** (SLB). However, this approach always requires two requests per object: one to the original site and one to the actual content URL. This increases latency and can negatively affect user experience, especially for time-sensitive applications.

The Next Step: URL-based CDNs

The next evolution is represented by URL-based CDNs, where URLs are directly used to route traffic to the optimal content server. This requires TCP termination, which is complex and computationally expensive. TCP termination introduces additional delay, and there is typically only one termination point, which must be carefully placed—either close to the client or close to the server.

Performance Evaluation

The concept of a **Cumulative Distribution Function** (CDF) is often used in networking experiments to analyze latency.

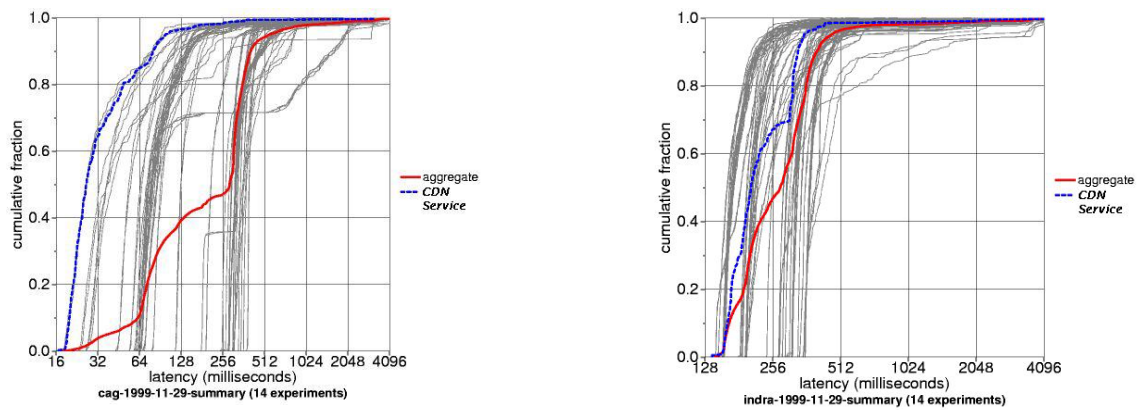
A CDF shows the proportion of measurements that fall below a certain value. For example, if you measure the loading time of a webpage multiple times, a CDF plot can show you not only the average latency but also how latency values are distributed across all measurements. This is important because network measurements vary due to traffic, routing, and other conditions. Relying on a single measurement is not enough, so you run a series of experiments—say 100 or more—and then analyze the results.

For instance, the CDF might show that 20% of samples have latency below 66 ms, and 50% have latency below 300 ms. This gives you a fuller picture than just average and standard deviation.

In one example, loading a page without CDN might have a median latency of 300 ms, while with CDN it could be around 30 ms—showing a tenfold improvement. However, results vary based on conditions, and sometimes the difference is less significant.

Sometime CDNs are very good!

Sometime are not so good!

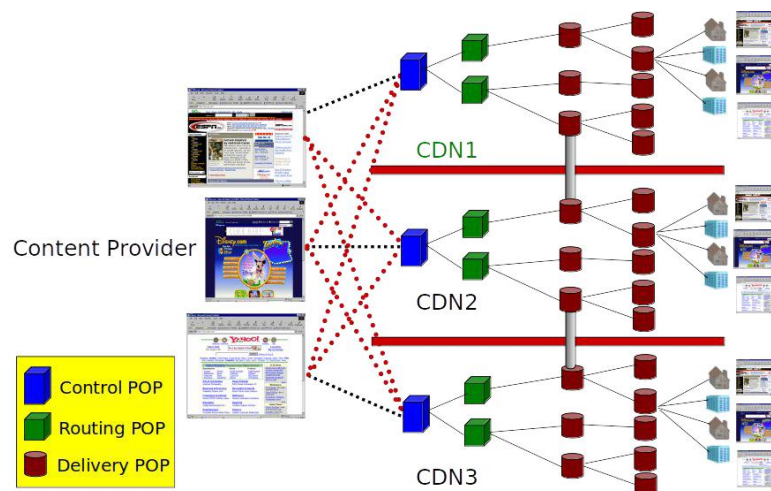


Content Peering

The image illustrates a content peering scenario in a multi-CDN architecture. A single content provider distributes its content to several independent CDNs (CDN1, CDN2, and CDN3) instead of relying on a single delivery infrastructure. Each CDN operates its own distributed system composed of different **Points of Presence**: **control PoPs**, which handle DNS resolution and request redirection; **routing PoPs**, which steer traffic within the CDN; and **delivery PoPs**, which cache and serve the content to end users.

The key idea shown in the figure is that the same content is replicated and made available across multiple CDNs. The red dashed connections represent content-level interactions, such as replication, synchronization, or coordinated delivery among CDNs. This goes beyond traditional network peering, which only concerns packet forwarding, and instead focuses on how and where content is stored and delivered.

Through content peering, requests from users can be served by different CDNs depending on performance, availability, or policy decisions. This improves scalability, resilience, and quality of experience, but also introduces complexity, since multiple autonomous CDNs must coordinate content placement, control, and economic agreements. For this reason, content peering is a powerful yet still partially unresolved aspect of modern content delivery systems.



Conclusions

DNS-based Content Delivery Networks are widely deployed and form a fundamental component of modern Internet infrastructure. They are used not only for web content but also for multimedia streaming. Replica servers may deliver slightly different content, such as localized advertisements.

Despite their success, challenges remain—particularly content peering, which is still an unresolved problem. Major CDN providers include Akamai, Amazon CloudFront, Cloudflare, and Limelight Networks.

Pattern Analysis

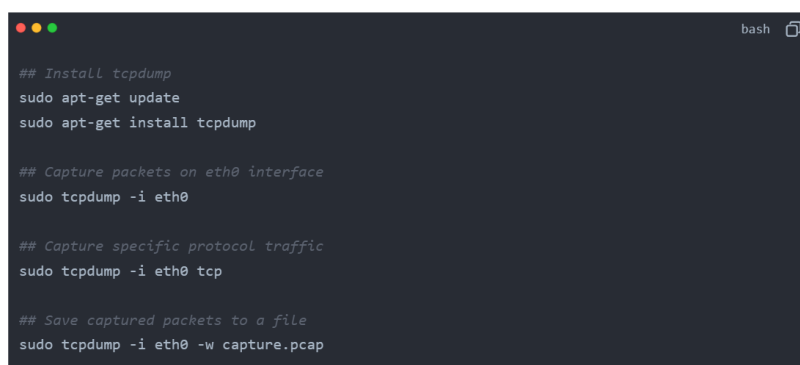
Understanding and analyzing network traffic is a fundamental skill in modern cybersecurity and network management. Observing **how network traffic behaves over time** allows professionals to plan capacity, detect anomalies, optimize performance, and respond to potential security threats. This analysis is crucial for various applications, including forecasting network traffic, detecting and mitigating intrusions, dynamically allocating bandwidth, and fine-tuning system performance while issuing timely alerts when unusual activity is detected.

Network traffic refers to all the data moving across a computer network at a given moment. It encompasses all digital communication between devices, servers, and applications. Traffic can be classified in multiple ways:

- **Inbound traffic:** data entering the network from external sources.
- **Outbound traffic:** data leaving the network toward external destinations.
- **Internal traffic:** communications between devices within the same network.
- **External traffic:** communications that cross network boundaries, often to the Internet.

At a technical level, network traffic is made up of **data packets**, which are small units of data transmitted independently across the network. Each packet contains several essential elements: the **source IP** (origin), the **destination IP** (target), the **protocol** used for communication (e.g., TCP or UDP), and the **payload**, which is the actual data being transmitted.

To analyze traffic, it first must be captured. Tools like **tcpdump**, a powerful command-line packet analyzer, allow capturing and inspecting packets directly from network interfaces.

A terminal window with a dark background and light green text. The window title is 'bash'. It contains several lines of commands and comments for installing and using tcpdump. The commands are: 'sudo apt-get update', 'sudo apt-get install tcpdump', 'sudo tcpdump -i eth0', 'sudo tcpdump -i eth0 tcp', and 'sudo tcpdump -i eth0 -w capture.pcap'. The comments are: '## Install tcpdump', '## Capture packets on eth0 interface', '## Capture specific protocol traffic', and '## Save captured packets to a file'.

```
## Install tcpdump
sudo apt-get update
sudo apt-get install tcpdump

## Capture packets on eth0 interface
sudo tcpdump -i eth0

## Capture specific protocol traffic
sudo tcpdump -i eth0 tcp

## Save captured packets to a file
sudo tcpdump -i eth0 -w capture.pcap
```

Once captured, traffic can be quantified through key metrics such as:

- **Bandwidth:** the total amount of data transferred over a given time.
- **Throughput:** the actual successfully transmitted data.
- **Latency:** the time it takes for data to travel from source to destination.
- **Packet Loss:** the percentage of packets that fail to reach their destination.

These metrics provide both a snapshot of network performance and early indicators of potential problems.

Traffic pattern analysis is a technique used to understand normal network behavior, detect anomalies, and manage resources effectively. A common method is to establish a **baseline**, which includes:

- Typical traffic volumes at different times
- Peak usage periods
- Distribution of protocols used (for instance, TCP vs. UDP percentages)
- Standard bandwidth consumption

Once a baseline is defined, deviations from this normal behavior can indicate network issues or security threats.

Network traffic prediction can be categorized based on the time horizon:

- **Long-period predictions** (hours to years) help estimate future workload and traffic patterns, supporting strategic capacity planning.
- **Short-period predictions** (milliseconds to minutes) are used for dynamic resource allocation, such as routing adjustments or bandwidth adaptation.

Python is often used for practical traffic analysis, for example through the **Scapy library**, which allows packet manipulation and inspection for monitoring, detection, or simulation purposes.

```
python
import scapy.all as scapy
import pandas as pd

def analyze_network_traffic(interface, duration=60):
    packets = scapy.sniff(iface=interface, timeout=duration)

    ## Extract packet details
    packet_data = []
    for packet in packets:
        if packet.haslayer(scapy.IP):
            packet_info = {
                'Source IP': packet[scapy.IP].src,
                'Destination IP': packet[scapy.IP].dst,
                'Protocol': packet[scapy.IP].proto
            }
            packet_data.append(packet_info)

    return pd.DataFrame(packet_data)

## Usage example
traffic_df = analyze_network_traffic('eth0')
print(traffic_df)
```

Anomaly Detection

Anomaly detection in network traffic identifies patterns that deviate from expected behavior. There are three main approaches:

- **Rule-Based Analysis:** Uses predefined rules to flag suspicious activities immediately.
- **Statistical Analysis:** Compares current traffic against historical patterns to detect deviations.
- **Machine Learning:** Models normal traffic behavior and detects anomalies adaptively.

```
python
import socket
import logging
from scapy.all import *

class NetworkSecurityMonitor:
    def __init__(self, interface):
        self.interface = interface
        logging.basicConfig(filename='security_log.txt', level=logging.WARNING)

    def detect_suspicious_traffic(self, packet):
        ## Analyze packet characteristics
        if packet.haslayer(IP):
            src_ip = packet[IP].src
            dst_ip = packet[IP].dst

            ## Check for potential suspicious patterns
            if self._is_suspicious_connection(src_ip, dst_ip):
                self._log_security_event(packet)

    def _is_suspicious_connection(self, src_ip, dst_ip):
        ## Implement custom logic for suspicious connection detection
        suspicious_ips = ['192.168.1.100', '10.0.0.50']
        return src_ip in suspicious_ips or dst_ip in suspicious_ips

    def _log_security_event(self, packet):
        logging.warning(f"Suspicious Packet Detected: {packet.summary()}")

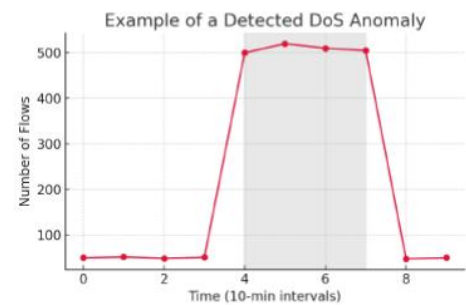
    def start_monitoring(self):
        print("Network Security Monitoring Started...")
        sniff(iface=self.interface, prn=self.detect_suspicious_traffic)

## Usage
monitor = NetworkSecurityMonitor('eth0')
monitor.start_monitoring()
```

Example – DoS Attack: A **Denial-of-Service (DoS)** attack can be identified by certain traffic patterns:

- Very high numbers of flows and packets with minimal data size (e.g., one-packet flows)
- Low variation in destination ports and IPs
- Traffic primarily incoming, often using UDP
- Constant TTL, suggesting a single source

Visualization often shows a sharp increase in flows without a corresponding increase in bytes or destinations, which aligns with known DoS patterns.



Types of Anomalies:

- **Point Anomalies**: A single data point that deviates significantly from others.
 - o **Global**: Outlier compared to the entire dataset
 - o **Contextual**: Outlier within a specific context, but not overall



- **Collective Anomalies**: A sequence of data points that are anomalous together, even if individual points seem normal.
 - o **Seasonal**: Deviates from expected seasonal patterns
 - o **Subsequence**: Contiguous segment deviates from the rest of the series
 - o **Pattern**: Sequence forms an unusual pattern not conforming to known time-series behavior



- **Trend Anomalies (Drift)**: Unexpected changes in trends, such as a sudden shift from growth to decline. These are sometimes called drift anomalies in data science. For example, packet patterns might indicate the onset of a DDoS attack.



It is important to note that trends are application-specific. For example, a drop in traffic on weekends for a service like ChatGPT is normal and not anomalous.

Traffic Modeling

Traffic can be modeled as a **time series**, which captures trends, seasonality, and temporal dependencies. Common modeling techniques include:

- **Holt-Winters Exponential Smoothing**: Good for capturing seasonality.

- **ARIMA (AutoRegressive Integrated Moving Average):** Excellent for stationary time series.
- **Wavelet-based models:** Adapted for highly variable signals.
- **Neural Networks (LSTM, RNN):** Handle complex trends but are more advanced.

Time series models often use **Mean Squared Forecast Error (MSFE)** to minimize cumulative prediction errors. Forecasting is commonly done one step ahead, predicting the next value based on past observations.

Methodology step by step

1. Data are collected, aggregated and filtered. The size of the aggregation interval influences anomaly detection procedures, mainly the training speed of the detection model.
1. Anonymization, e.g., randomly assigning numbers as identifiers for IP addresses, institutions, and institution subnets during database creation.
2. Data preparation: normalization and stationarity checks.
3. Model training, fitting the model to observed data.
4. Forecasting, from 1 to up to 10-steps ahead.
5. Evaluation using error metrics, e.g., RMSE, MAE, SMAPE.

Before modeling, data must be cleaned:

- **Check for missing values:** In Python, `pandas.isnull()` identifies missing entries.
- **Fill missing data:** Options include forward filling (`ffill`), backfilling (`bfill`), or using the mean or other statistical measures.
 - o `bfill` (i.e., backfilling) missing values are replaced by the next valid observation
 - o `ffill` (i.e., forward filling) will replace the missing values with the most recent non-missing value

Stationary means that the distribution of the data doesn't change with time. For a timeseries to be stationary, it must fulfill three criteria:

- The series has zero trends. It isn't growing or shrinking.
- The variance is constant. The average distance of the data points from the zero line isn't changing.
- The autocorrelation is constant. How each value in the time series is related to its neighbors stays the same

For time series models, data must be stationary. The reason for this is that modeling is all about estimating parameters that represent the data; therefore, if the parameters of the data are changing with time, it will be difficult to estimate all the parameters

Before modeling time series data, it is crucial to verify whether the data is stationary. A stationary time series has statistical properties—like mean, variance, and autocorrelation—that remain constant over time. One simple way to get an initial sense of stationarity is by visually inspecting the data. However, the most widely used and reliable method is the Augmented Dickey-Fuller (ADF) test, a statistical test where the null hypothesis assumes that the time series is non-stationary due to trends.

```
from statsmodels.tsa.stattools import adfuller

results = adfuller(data)
print(results)
```

The ADF test produces a tuple of results:

- **Test statistic value:** The more negative this value, the more likely the data is stationary.
- **p-value:** If the p-value is less than 0.05, we reject the null hypothesis and conclude that the time series is stationary.

If the data is found to be non-stationary, it must be transformed into a stationary form before we can apply models like ARIMA. This process is similar to feature engineering in traditional machine learning, where the raw data is preprocessed to highlight patterns or remove unwanted variability.

One common transformation is differencing, where each value in the series is replaced by the difference between itself and the previous value. Other possible transformations include applying a logarithm, square root, or calculating proportional changes to stabilize variance and remove trends.

```
# Calculate the first difference and drop the nans
data_diff = data.diff()
data_diff = data_diff.dropna()

# Run test and print
result_diff = adfuller(data_diff)
print(data_diff)
```

ARIMA – Autoregressive Integrated Moving Average

ARIMA, which stands for AutoRegressive Integrated Moving Average, is a widely used method for modeling stationary time series. These models are particularly effective in domains where a single variable is measured at regular intervals, such as finance, econometrics, or network traffic analysis. In network management, ARIMA can be applied for anomaly detection, load prediction, and traffic pattern recognition.

ARIMA combines three components:

1. **Autoregression (AR):** The AR component models the current value as a function of its previous values. For example, a simple AR(1) model is:

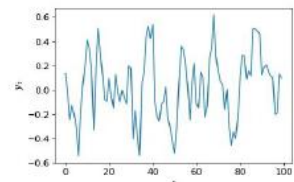
$$y_t = a_1 \cdot y_{t-1} + \epsilon_t$$

Here a_1 is the autoregressive coefficient representing the relationship with the previous value, and ϵ_t is the random noise or shock, which is assumed to be independent across time.

2. **Integration (I):** Integration involves differencing the data to remove trends and achieve stationarity. Each observation is replaced by the difference between itself and its predecessor.
3. **Moving Average (MA):** The MA component accounts for the dependency between an observation and residual errors from previous observations. A simple first-order MA model looks like:

$$y_t = m_1 \cdot \epsilon_{t-1} + \epsilon_t$$

e.g., $y(t) = 0.5 y(t-1) + 0.2 \epsilon(t-1) + \epsilon(t)$ looks like:



Each component in ARIMA functions as a parameter with a standard notation. For ARIMA models, a standard notation would be ARIMA with p, d, and q, where integer values

substitute for the parameters to indicate the type of ARIMA model used. The parameters can be defined as:

- p: the number of lag observations in the model, also known as the lag order. It represents the order of the AR part of ARIMA
- d: the number of times the raw observations are differenced; also known as the degree of differencing and the I part.

- q: the size of the moving average window, also known as the order of the MA part.

D is always an integer, while p and q may either be integers or lists of integers.

The model is often denoted as **ARIMA(p,d,q)**.

A value of zero (0) means that particular component should not be used in the model. This way, the ARIMA model can be constructed to perform the function of an ARMA model, or even simple AR, I, or MA models. For example, to fit an AR model, we can simply use the ARMA class with q equal to zero; to fit an MA model, we set p equal to zero.

Generally, in machine learning, we have a training set on which to fit the model, and a test set on which to test the predictions against. Time series forecasting is just the same, but train-test split is typically done using dates. We use the past values to make future predictions, and so we will need to split the data in time. We train on the data earlier in the time series and test on the data that comes later.

To determine the most appropriate p,d,q, we can use the ACF plot and PACF plots.

A rough rubric for when to use I terms in the model is based on number of times to differentiate to make the data stationary (see DF test).

A rough rubric for when to use AR terms in the model is when:

- ACF plots show autocorrelation decaying towards zero
- PACF plot cuts off quickly towards zero
- ACF of a stationary series shows positive at Lag -1

A rough rubric for when to use MA terms in the model is when:

- Negatively Autocorrelated at Lag -1
- ACF that drops sharply after a few lags
- PACF decreases gradually rather than suddenly

Implementing ARIMA step-by-step

1. **Import Libraries** → First, import the necessary libraries for time series analysis. e.g., numpy, pandas, statsmodel, sklearn
2. **Load Dataset** → Let's download/load and use the dataset of interest.
3. **Extracting data of interest** → With the desired granularity, daily, weekly, monthly data.
4. **Plotting the Obtained Data** → Just to have an idea of the measurements. We can get a rough idea of seasonality, patterns, trends.
5. **Check Stationarity** → Before applying ARIMA, check if your time series data is stationary. Stationarity refers to the statistical properties of a time series remaining constant over time, such as constant mean, constant variance, and constant autocovariance. We can use the Dickey-Fuller (DF) test for this.

```
def check_stationarity(timeseries):
    # Perform the Dickey-Fuller test
    result = adfuller(timeseries, autolag='AIC')
    p_value = result[1]
    print(f'ADF Statistic: {result[0]}')
    print(f'p-value: {p_value}')
    print('Stationary' if p_value < 0.05 else 'Non-Stationary')
```

6. **Identify Model Parameters** → Identify the ARIMA model parameters (p, d, q) using ACF and PACF plots.

ACF Plot: a plot of autocorrelations of the differenced time series. Autocorrelation measures the relationship between a data point and previous data points at different lags. So ACF(1) is the correlation between the time series and a one-step lagged version of itself, ACF(2) is the correlation with the two-step lagged version of itself, and so on.

PACF Plot: a plot of partial autocorrelations of the differenced time series. Partial autocorrelation represents the correlation between a data point and a lag after subtracting the effect of correlation at smaller lags.

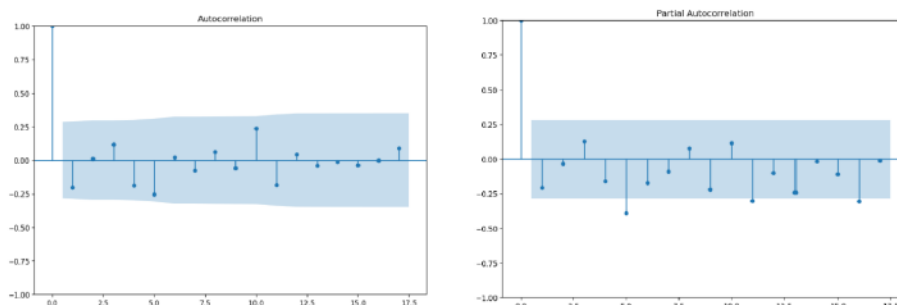
```
from statsmodels.graphics.tsaplots import plot_acf
from statsmodels.graphics.tsaplots import plot_pacf

# plot ACF and PACF

plot_acf(data)
plot_pacf(data)
```

Remarks. The time series must be made stationary before making these plots. If the ACF values are high and tail off very slowly, therefore this is a sign that the data is non-stationary, and it needs to be differenced. On the other hand, if the autocorrelation at lag-1 is very negative this is a sign that we have taken the difference too many times.

ACF and PACF examples



Both plots have on the x-axis the lags, and the y-axis the correlation coefficients ranging from -1 to 1 where -1 means perfect negative correlation, 0 means no correlation and 1 means perfect positive correlation.

There is a blue area in the ACF and PACF plots, which depicts the 95% confidence interval and is an indicator for the significance threshold. That means, anything within the blue area is statistically close to zero and anything outside the blue area is statistically non-zero.

p: Significant at each lag p/ Cuts off after lag p in PACF

q: Significant at each lag q/ Cuts off after lag q in ACF

Rule of thumb:

- The number of lags before the cutoff gives the order
- If neither cuts off clearly → try combinations and choose based on Akaike information criterion (AIC) and Bayesian information criterion (BIC), also recommended for a more formal finding of these values

7. **Fit the ARIM model** → Now, fit the ARIMA model using the identified parameters. The ARIMA function takes the data and the specified order parameters. The fit method is then called to estimate the model's coefficients based on the data.

8. **Generate forecasts** → With the model fitted, generate forecasts for future time periods. The `results.get_forecast(steps=forecast_periods)` function generates the forecast, providing both the predicted mean and a confidence interval for the forecasted values.

```
# Forecast future values
forecast_periods = 12 # Forecast the next 12 months
forecast = results.get_forecast(steps=forecast_periods)
forecast_mean = forecast.predicted_mean
forecast_ci = forecast.conf_int()
```

```
# Plot the forecast
```

9. **Evaluate the model** → Lastly, we can evaluate the forecasted values by comparing them to the observed sales data using three common metrics for this evaluation: Root Mean Squared Error (RMSE), Symmetric Mean Absolute Percentage Error (SMAPE), and R-squared (R^2).

For RMSE and SMAPE, lower values indicate better performance.

The R^2 value ranges from 0 to 1, with 1 indicating that the model perfectly predicts the data and 0 indicating that the model does not explain any of the variance in the data.

In conclusion, ARIMA is a valid approach to forecast network traffic in the future. We don't need LLMs, e.g., ChatGPT and Claude, for everything. Predicting network traffic is helpful to detect anomalies, attacks, or simply to better manage the infrastructure. Although mastering these methods is complex and require some domain knowledge.